ASTO-BOT

AI Streamer Twitch Orchestrator

Benutzerhandbuch

Version 1.1.22 | Entwickelt von CaptainApolloo

Inhaltsverzeichnis

(Bitte in Word: Verweise → Inhaltsverzeichnis → Automatische Tabelle 1 einfügen)

1. Was ist ASTO-BOT?
- 1.1 Woher du ASTO-BOT bekommst
- 1.2 Wie dieses Handbuch aufgebaut ist

2. Installation & Erster Start
- 2.1 Systemvoraussetzungen
- 2.2 Installation
- 2.3 Der erste Start — in dieser Reihenfolge
- 2.4 Updates

3. Das Dashboard
- 3.1 Die Kopfleiste
- 3.2 Karten bedienen
- 3.3 Status & Verbindung
- 3.4 Live-Betrieb
- 3.5 Statistiken
- 3.6 Giveaway, Clips & Medien
- 3.7 Weitere Karten
- 3.8 ASTO-BOT Performance
- 3.9 Live Translator
- 3.10 Voice
- 3.11 Stream Health

4. Settings
- 4.1 AI
- 4.2 Twitch
- 4.3 OBS
- 4.4 Discord
- 4.5 SpeakerBot
- 4.6 Sounds
- 4.7 Websocket
- 4.8 System Check
- 4.9 Queues
- 4.10 Variables
- 4.11 Import / Export
- 4.12 About
- 4.13 Backup & Load Backup

5. Events
- 5.1 Aufbau der Events-Seite
- 5.2 Die Kopfzeile eines Events
- 5.3 Rewards: Kanalpunkte und Power-ups
- 5.4 Trigger — wann feuert das Event?
- 5.5 Die Action Sequence
- 5.6 Die Aktionsgruppen
- 5.7 IF/ELSE — Bedingungen
- 5.8 Chance — der Zufall
- 5.9 Ein Event von vorne bis hinten

6. Command
- 6.1 Aufbau der Seite
- 6.2 Die Command-Karte
- 6.3 Gruppen

6.4 Die mitgelieferten Commands
6.5 Vom Command zum Event
7. Giveaway & Punkte
7.1 Das Punkte-System
7.2 Giveaways anlegen
7.3 Ticket oder Currency — der wichtigste Schalter
7.4 Linked Command — wie Zuschauer beitreten
7.5 Draw, Finish und Reset
7.6 Den Gewinner im Stream feiern
8. Clips
8.1 Streamer und Chat-Befehle
8.2 Clips laden und auswählen
8.3 Clip Playback — die OBS-Quelle einrichten
8.4 Overlay Action — das Drumherum
8.5 Die Clip Playlist
9. Overlay
9.1 Die Kopfleiste
9.2 Wie das Overlay nach OBS kommt
9.3 Elemente
9.4 Die Eigenschaften eines Bildes
9.5 Die Step-Timeline
9.6 Text-Elemente
9.7 Ein Overlay in Aktion
9.8 Das Browser Overlay
9.9 Untertitel-Overlay
9.10 Chat-Overlay
10. Photo Mode
10.1 Die Kopfleiste
10.2 Hintergründe und Texte
10.3 Die drei Bereiche im Bild
10.4 Die Chat-Befehle
10.5 Wo die Fotos landen
11. Scripts — ASTOSCRIPT
11.1 Skripte anlegen und ordnen
11.2 Die Knöpfe oben rechts
11.3 Der Editor hilft mit
11.4 Fehler finden
11.5 Der Debug-Modus
11.6 Das Tutorial-Fenster
11.7 Skripte aus Events starten
11.8 Zwei Skripte aus der Praxis
11.9 Praxisbeispiel: die YouTube-Song-Warteschlange
12. Logs
12.1 Der Live Log Viewer
12.2 Der Event Inspector
13. Wenn etwas nicht funktioniert
13.1 Chat, KI und TTS
13.2 Events und Commands
13.3 Rewards und Giveaways
13.4 OBS, Clips und Overlays
13.5 Musik und Skripte
13.6 Und wenn nichts hilft?

1. Was ist ASTO-BOT?

ASTO-BOT — der *AI Streamer Twitch Orchestrator* — ist ein vollständiges Twitch-Streaming-Werkzeug für Windows. Es verbindet sich direkt mit **Twitch**, **OBS Studio** und verschiedenen **KI-Diensten** und erlaubt dir, auf Ereignisse in deinem Stream automatisch zu reagieren — ganz ohne Programmierkenntnisse.

Mit ASTO-BOT kannst du unter anderem:

- automatisch auf **Follows, Abos, Bits, Raids, Kanalpunkte-Einlösungen** und vieles mehr reagieren,
- **KI-generierte Chat-Antworten** mit eigenem Charakter senden,
- **OBS-Szenen und -Quellen** per Event oder Skript steuern,
- eigene **Overlay-Animationen** erstellen und direkt in OBS einbinden,
- **Clips** aufnehmen, speichern und automatisch abspielen,
- ein **Punkte- und Giveaway-System** für deine Community betreiben,
- eigene Automatisierungen mit der eingebauten Skriptsprache **ASTOSCRIPT** schreiben.

> 💡 ASTO-BOT läuft **vollständig lokal** auf deinem PC. Alle sensiblen Daten wie Tokens und Passwörter werden **verschlüsselt** gespeichert und verlassen deinen Computer nicht.

1.1 Woher du ASTO-BOT bekommst

- **Download:** https://github.com/CaptainApolloo/ASTO-BOT
- **Discord** — Support, Hilfe und Neuigkeiten: https://discord.gg/n2nstVwspk
- **Ko-fi** — wenn du die Entwicklung unterstützen möchtest: https://ko-fi.com/captainapolloo

1.2 Wie dieses Handbuch aufgebaut ist

Die Kapitel folgen den Reitern im Programm. Du musst sie nicht der Reihe nach lesen — aber ein Kapitel solltest du nicht überspringen: **Events**. Dort wird fast alles gesteuert, und die anderen Kapitel verweisen immer wieder darauf zurück.

Im Handbuch findest du zwei Arten von Kästen. Die **cyanfarbenen** enthalten Tipps und Kniffe aus der Praxis. Die **orangefarbenen** sind Hinweise, die du wirklich lesen solltest — dort steht, was schiefgehen kann.

2. Installation & Erster Start

2.1 Systemvoraussetzungen

- **Windows 10** oder **Windows 11** (64-Bit)
- eine **Internetverbindung** — für die Twitch-Anmeldung und die KI-Funktionen
- **OBS Studio 28** oder neuer — optional, aber nötig für Overlays und Szenensteuerung

Für die KI-Antworten brauchst du außerdem den API-Schlüssel eines KI-Anbieters. Wo der eingetragen wird und welche Anbieter unterstützt werden, steht im Kapitel Settings.

2.2 Installation

ASTO-BOT wird über einen Installer eingerichtet.

- Führe die Datei **ASTO-BOT_Installer.exe** aus.
- Wähle beim ersten Mal einen Installationsordner, zum Beispiel `C:\Program Files\ASTO-BOT\`.
- Folge den Anweisungen des Installers.
- Nach der Installation wird ASTO-BOT **automatisch gestartet**.
- Beim ersten Start legt ASTO-BOT alle benötigten Unterordner selbst an.

> **Deinstalliere ASTO-BOT ausschließlich** über **Systemsteuerung → Programme → ASTO-BOT deinstallieren** oder über die mitgelieferte **uninstall-asto.exe** im Installationsordner. Lösche den Ordner **niemals von Hand** — sonst bleiben Einträge in der Windows-Registrierung zurück.

2.3 Der erste Start — in dieser Reihenfolge

Beim ersten Start ist ASTO-BOT noch mit nichts verbunden. Am schnellsten bist du fertig, wenn du dich an diese Reihenfolge hältst. Alle Schritte finden in den **Settings** statt (Kapitel 4).

- **Twitch verbinden.** Melde **beide** Konten an: dein **Streamer-Konto** und dein **Bot-Konto**. Ohne diesen Schritt geht praktisch nichts.
- **KI einrichten.** Anbieter wählen, API-Schlüssel eintragen, Bot-Namen und Charakter festlegen.
- **OBS verbinden.** Websocket in OBS aktivieren, Verbindungsdaten in ASTO-BOT eintragen, danach die Quellen für Overlay, Clips und Photo Mode anlegen.
- **Sounds und SpeakerBot** — falls du beides nutzen willst.
- **System Check** ausführen. Er prüft Konfiguration, KI, Twitch und OBS und sagt dir, was noch fehlt.

> 💡 Wenn du später Berechtigungen hinzunimmst, verlangt Twitch eine **komplette Neuanmeldung beider Konten**. Das ist normal und dauert zwei Minuten.

Danach lohnt sich ein Blick ins Kapitel **Events**: Die mitgelieferte Standard-Gruppe bringt fertige Beispiel-Events mit — Follow, Resub, Raid und andere. Nimm dir eines davon, drücke **Simulate** und sieh dir an, was passiert. Das ist der kürzeste Weg, ASTO-BOT zu verstehen.

2.4 Updates

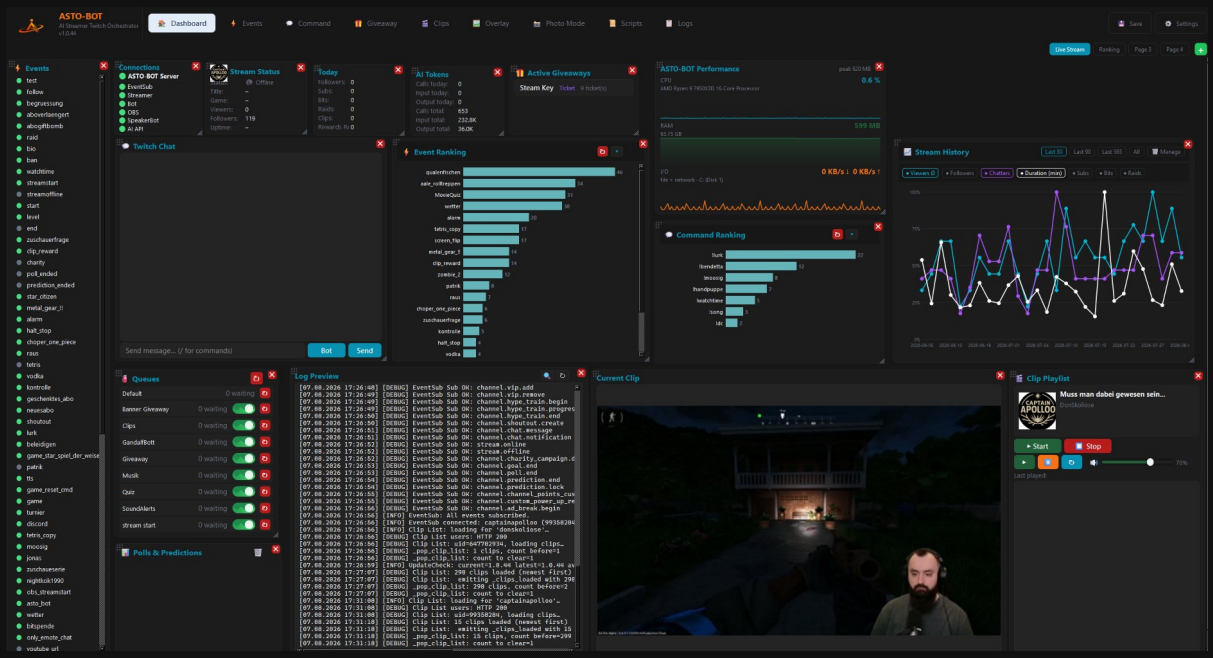
Ist eine neue Version verfügbar, meldet sich ASTO-BOT von selbst — oben in der Kopfleiste — Der Installer überschreibt dabei **nur die Programmdateien**. Deine Konfiguration, Sounds, Overlays und Prompts bleiben vollständig erhalten.

Vor dem Update legt ASTO-BOT außerdem eine **Sicherung der laufenden Version** an. Sollte mit der neuen Version etwas nicht stimmen, kommst du damit jederzeit zurück — wie das geht, steht in Abschnitt 4.12.

> Während das Update läuft, darf **nichts abgebrochen** werden. Warte, bis ASTO-BOT von sich aus **neu startet**.

3. Das Dashboard

Das Dashboard ist die Startseite von ASTO-BOT und deine Kommandozentrale während des Streams. Es besteht aus einzelnen **Karten**, die jeweils einen Bereich abdecken — Verbindungen, Stream-Status, Chat, Warteschlangen, Statistiken, Clips und mehr. Du entscheidest selbst, welche Karten du siehst und wo sie liegen.



Das ASTO-BOT Dashboard — alle Karten in der Übersicht

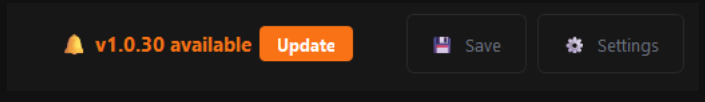
3.1 Die Kopfleiste

Ganz oben findest du das ASTO-BOT Logo mit der installierten **Versionsnummer** und daneben die Reiter, über die du zwischen den Bereichen wechselst:

- **Dashboard** — die Übersichtsseite, die in diesem Kapitel beschrieben wird
- **Events** — automatische Reaktionen auf Twitch-Ereignisse
- **Command** — Chat-Befehle
- **Giveaway** — Verlosungen und Punkte-System
- **Clips** — Clip-Verwaltung und Wiedergabe
- **Overlay** — der Overlay-Editor
- **Photo Mode** — Foto-Modus
- **Scripts** — ASTOSCRIPT-Editor
- **Logs** — das vollständige Protokoll

Rechts oben liegen die Knöpfe **Save** und **Settings**. **Save** speichert alles, **Settings** führt in die Einstellungen, in denen alle Verbindungen und Grundeinstellungen vorgenommen werden (siehe Kapitel 4).

Sobald eine neue Version von ASTO-BOT verfügbar ist, erscheint links neben dem Save-Knopf zusätzlich ein Hinweis mit einem **Update**-Knopf.



Ist ein Update verfügbar, erscheint der Hinweis direkt in der Kopfleiste

3.2 Karten bedienen

Alle Karten auf dem Dashboard lassen sich frei anordnen. Du kannst dir dein Dashboard also genau so einrichten, wie du es beim Streamen brauchst:

- **Verschieben:** Karte mit der Maus greifen und an die gewünschte Stelle ziehen.
- **Größe ändern:** Karte an der Kante bzw. Ecke anfassen und größer oder kleiner ziehen.
- **Schließen:** Auf das rote **X** in der oberen rechten Ecke der Karte klicken. Die Karte verschwindet vom Dashboard.
- **Zurückholen:** Der grüne **+** Knopf ganz oben rechts holt geschlossene Karten wieder auf das Dashboard zurück.

> 💡 Tipp: Karten, die du nicht brauchst, kannst du bedenkenlos schließen — sie sind über den grünen + Knopf jederzeit wieder da. So bleibt Platz für die Karten, die du während des Streams wirklich im Blick haben willst.

Vier Seiten statt einer

Das Dashboard hat **vier Seiten**. Über den Reitern oben rechts — direkt neben dem grünen **+** — springst du mit einem Klick auf die gewünschte Seite. So verteilst du deine Karten, statt sie auf einer Fläche zusammenzuquetschen.

- **Karte verschieben:** Rechtsklick auf die Karte, dann **Move to Page X**. Bei Karten mit eigenem Rechtsklick-Menü — etwa der Twitch-Chat-Karte — triffst du den Titelbereich.
- **Seite umbenennen:** Rechtsklick auf einen Reiter. Aus *Page 2* wird so *Übersetzer* oder *Debug*.
- **Von außen umschalten:** Über den Websocket, zum Beispiel per Stream Deck. Die Befehle stehen in Kapitel 4.7.

> Karten auf anderen Seiten laufen ganz normal weiter. Sie hören nur auf, ihre Anzeige zu aktualisieren, solange du sie nicht siehst — der Übersetzer übersetzt weiter, Overlays laufen weiter, Events feuern weiter.

> 💡 Das rote **X** bedeutet weiterhin "Karte geschlossen", unabhängig von der Seite. Eine Karte, die auf einer anderen Seite liegt, ist nicht geschlossen — sie ist nur woanders.

3.3 Status & Verbindung

Connections

Diese Karte zeigt, was gerade mit ASTO-BOT verbunden ist. Ein grüner Punkt bedeutet: Verbindung steht.

- **ASTO-BOT Server** — der interne Server des Programms
- **EventSub** — die Verbindung zu Twitch, über die alle Ereignisse hereinkommen
- **Streamer** — dein Twitch-Hauptkonto
- **OBS** — die Verbindung zu OBS Studio
- **Bot** — das separate Bot-Konto für Chat-Nachrichten
- **SpeakerBot** — Anbindung für Sprachausgabe
- **AI API** — die Verbindung zur KI

> Wichtig: **ASTO-BOT Server**, **EventSub**, **Streamer** und **OBS** sind die vier Verbindungen, die für den normalen Betrieb entscheidend sind. **Bot**, **AI API** und **SpeakerBot** sind optional — ASTO-BOT läuft auch ohne sie.

Stream Status

Zeigt auf einen Blick, ob du gerade live bist:

- **Status** — Live oder Offline
- **Title** — der aktuelle Stream-Titel
- **Game** — die aktuell gesetzte Kategorie bzw. das Spiel
- **Viewers** — aktuelle Zuschauerzahl
- **Followers** — Gesamtzahl deiner Follower
- **Uptime** — wie lange der Stream schon läuft

Today

Diese Karte zählt, was **im laufenden Stream** dazugekommen ist: Followers, Subs, Bits, Raids, Clips und **Rewards Redeemed** (wie viele Kanalpunkte-Belohnungen eingelöst wurden).

> 💡 Die Karte wird bei jedem neuen Stream automatisch zurückgesetzt — du siehst also immer die Zahlen des aktuellen Streams, nicht die Gesamtsumme.

AI Tokens

Eine Übersicht darüber, wie viel die KI verbraucht:

- **Calls today / Input today / Output today** — Verbrauch der **laufenden Sitzung**
- **Calls total / Input total / Output total** — der Gesamtverbrauch über alle Sitzungen hinweg

> Achtung: Die drei "today"-Werte gelten nur so lange, wie ASTO-BOT geöffnet ist. Schließt du das Programm und startest es neu, stehen sie wieder auf 0. Die "total"-Werte bleiben dagegen erhalten.

3.4 Live-Betrieb

Events

Die Events-Karte listet alle angelegten Events auf. Vor jedem Eintrag sitzt ein Lämpchen:

- **Grün** — das Event ist aktiv und wird ausgelöst
- **Grau** — das Event ist inaktiv

Ein Klick auf das Lämpchen schaltet das Event an oder aus. So kannst du einzelne Events während des Streams schnell stummschalten, ohne sie zu löschen.

Queues

Die Queues sind die Warteschlangen, in denen Aktionen der Reihe nach abgearbeitet werden. Die Karte zeigt für jede Queue, **wie viele Events gerade warten**.

- **Ein-/Ausschalten:** Jede Queue kann einzeln pausiert werden — mit **Ausnahme der Default-Queue**, die immer aktiv bleiben muss.
- **Zurücksetzen:** Jede Queue hat einen eigenen Reset-Knopf, auch die Default-Queue. Zusätzlich gibt es oben in der Karte einen Knopf, der **alle Queues auf einmal** zurücksetzt.

Wird eine Queue deaktiviert, ist sie **pausiert**: Die Events gehen nicht verloren, sondern warten. Sie werden abgearbeitet, sobald die Queue wieder aktiviert wird — oder sie verschwinden, wenn du die Queue zurücksetzt.

> 💡 Tipp: Praktisch, wenn du z. B. während einer ruhigen Szene keine Sound-Alerts willst: Queue kurz deaktivieren, später wieder an — alles Wartende läuft dann nach.

Twitch Chat

Hier schreibst du direkt in deinen Twitch-Chat, ohne das Programm zu verlassen.

- Der Knopf **Bot** neben dem Eingabefeld schaltet zwischen **Streamer-Konto** und **Bot-Konto** um — du bestimmst also, in wessen Namen die Nachricht erscheint.
- Twitch-Chatbefehle funktionieren ebenfalls, einschließlich **Polls** und **Predictions**.
- **Emotes und GIFs** werden als Bild angezeigt und bewegen sich, wenn Twitch sie animiert ausliefert — statt nur als Name dazustehen.
- Befehle laufen über das Konto, das der **Bot**-Schalter gerade zeigt. Darf dein Bot etwas nicht — weil er im Kanal kein Moderator ist —, führt ASTO-BOT den Befehl still über dein Streamer-Konto aus, statt ihn scheitern zu lassen.
- Tippst du einen Befehl, erscheint eine Vorschlagsliste. Das erste **Enter** schreibt den Befehl ins Eingabefeld, erst das zweite führt ihn aus — so kannst du vorher noch etwas dahinter schreiben.

> **/clear** und **/color** funktionieren jetzt. **/color** braucht eine einmalige Neuanmeldung deines Kontos unter **Settings** → **Twitch**, sonst fehlt ASTO-BOT die Berechtigung dafür.

Polls & Predictions

Zeigt die aktuell laufenden Umfragen (Polls) und Vorhersagen (Predictions) an.

Log Preview

Eine kompakte Live-Ansicht des Protokolls. Praktisch, um im Blick zu behalten, was ASTO-BOT gerade tut. Das vollständige Protokoll findest du im Reiter **Logs**.

3.5 Statistiken

Event Ranking

Eine Rangliste der Events — sie zeigt, wie oft welches Event ausgelöst wurde.

> Wichtig: Es werden **nur Events gezählt, die über Kanalpunkte-Belohnungen oder Powerups ausgelöst werden**. Events mit anderen Triggern tauchen in dieser Rangliste nicht auf.

Über den **Pfeil nach unten** im Kartenkopf siehst du, welche Events gerade gewertet werden. Klickst du dort auf einen Eintrag, wird dessen Statistik zurückgesetzt — du kannst also jedes Event einzeln auf null stellen.

Command Ranking

Dasselbe Prinzip für die Chat-Befehle: eine Rangliste, wie oft welcher Command benutzt wurde. Auch hier zeigt der **Pfeil nach unten** die gewerteten Commands, und ein Klick auf einen Eintrag setzt dessen Zähler zurück.

Stream History

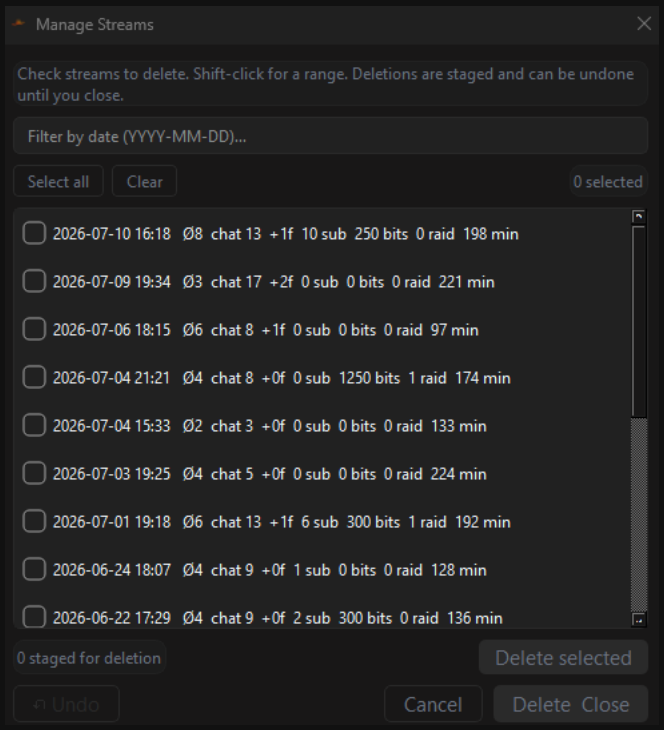
Die Stream History sammelt die Daten deiner vergangenen Streams. Angezeigt werden:

- **Viewers Ø** — durchschnittliche Zuschauerzahl
- **Followers** — gewonnene Follower
- **Chatters** — aktive Chatter
- **Duration (min)** — Streamdauer in Minuten
- **Subs**, **Bits**, **Raids**

Über die Knöpfe oben rechts stellst du den Zeitraum um: **Last 30**, **Last 90**, **Last 365** oder **All**. Ab 365 Tagen wird die Darstellung in **Wochen** zusammengefasst, damit der Verlauf lesbar bleibt.

Manage Streams — Einträge löschen

Der Knopf **Manage** in der Stream-History-Karte öffnet ein Fenster, in dem du einzelne Streams aus der Statistik entfernen kannst.



Manage Streams — Einträge markieren, löschen und bis zum Schließen rückgängig machen

- **Einzeln:** Häkchen bei dem Datum setzen, das raus soll.
- **Bereich:** Datum A anklicken, dann mit **Shift + linke Maustaste** auf Datum B — alles dazwischen wird markiert, egal wie weit die beiden auseinanderliegen.
- **Filter:** Über das Feld oben kannst du nach einem Datum suchen (Format JJJJ-MM-TT).

> Achtung: Solange das Fenster offen ist, lässt sich eine Löschung noch **rückgängig** machen. Sobald du das Fenster schließt, sind die Einträge endgültig weg — es gibt dann kein Zurück mehr.

3.6 Giveaway, Clips & Medien

Active Giveaways

Zeigt, welche Giveaways gerade laufen.

Current Clip

Zeigt das Vorschaubild des aktuell ausgewählten Clips.

Clip Playlist

Über diese Karte steuerst du den Clip-Playlist-Player:

- **Start / Stop** — Playlist starten und beenden
- **Play / Pause** — ein laufender Clip kann jederzeit angehalten und wieder fortgesetzt werden
- **Wiederholen** — spielt den zuletzt gespielten Clip noch einmal ab
- **Lautstärke** — Regler für die Wiedergabe-Lautstärke

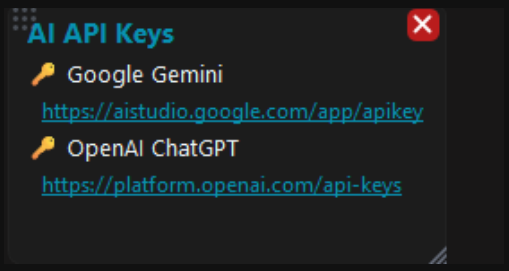
Darunter wird unter **Last played** eine Liste der zuletzt gespielten Clips aufgebaut.

3.7 Weitere Karten

Die folgenden beiden Karten sind standardmäßig möglicherweise ausgeblendet — du holst sie über den grünen **+** Knopf oben rechts wieder auf das Dashboard.

AI API Keys

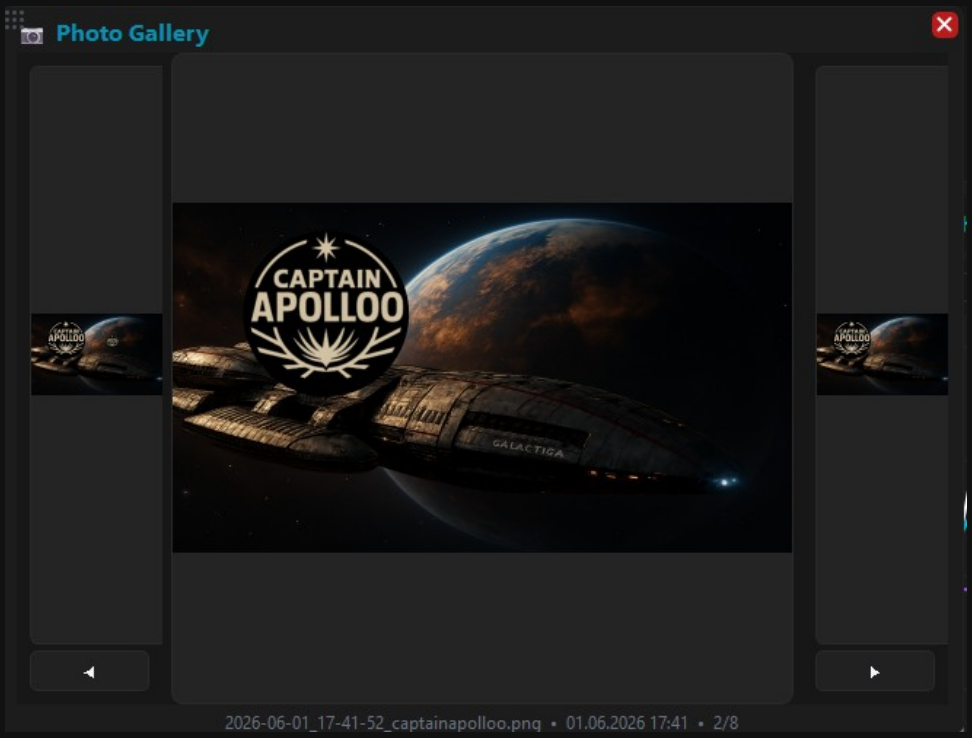
Eine reine Merkhilfe: Die Karte enthält die Links zu den Webseiten, auf denen du deine API-Schlüssel bekommst — Google Gemini und OpenAI ChatGPT. Eingetragen werden die Schlüssel selbst in den **Settings**.



Die Karte AI API Keys mit den Links zu den Anbietern

Photo Gallery

Zeigt die im Photo Mode aufgenommenen Bilder. Mit den Pfeiltasten links und rechts blätterst du vor und zurück; unten stehen Dateiname, Aufnahmezeitpunkt und die Position in der Galerie.

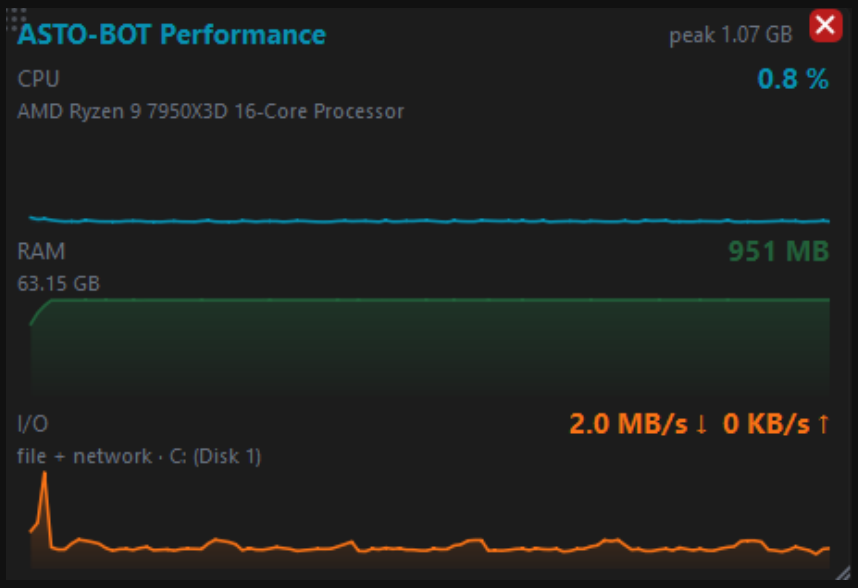


Die Photo Gallery mit Vor- und Zurück-Knöpfen

3.8 ASTO-BOT Performance

Diese Karte zeigt, wie viel Rechenleistung und Arbeitsspeicher ASTO-BOT gerade selbst verbraucht. Sie ist standardmäßig ausgeblendet — du holst sie über den grünen **+** Knopf oben rechts auf das Dashboard.

Solange sie ausgeblendet ist, wird auch nichts gemessen. Sie kostet dich also nichts, wenn du sie nicht brauchst.



Die Performance-Karte mit CPU, RAM und I/O

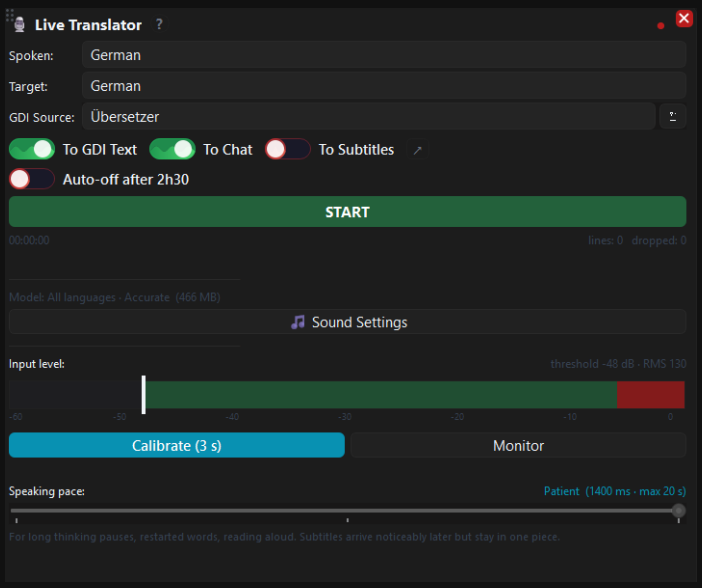
Jeder der drei Werte hat eine Zahl rechts und darunter einen Verlauf über die letzten zwei Minuten. So siehst du nicht nur den Moment, sondern auch, ob etwas über die Zeit ansteigt.

- **CPU** — der Anteil an der gesamten Rechenleistung deines Prozessors, also derselbe Wert, den auch der Task-Manager pro Programm anzeigt. Darunter steht das Modell deiner CPU. Im Leerlauf liegt ASTO-BOT typischerweise unter 1 %; während ein Overlay abgespielt wird, steigt es kurz an.
- **RAM** — der belegte Arbeitsspeicher in MB, darunter der insgesamt verbaute. Rechts oben steht zusätzlich der höchste Wert seit dem Start (**peak**). Wächst der Wert über Stunden immer weiter, ohne wieder zu fallen, lohnt ein genauerer Blick — meist stecken dann sehr große GIFs in einem Overlay.
- **I/O** — Lesen (↓) und Schreiben (↑). Die Beschriftung sagt bewusst **file + network**: Windows fasst Datei- und Netzwerkzugriffe eines Programms in einer Zahl zusammen, eine Aufteilung gibt es dort nicht. Dahinter steht das Laufwerk, auf dem ASTO-BOT arbeitet.

> GPU und Netzwerk getrennt nach Programm bietet Windows nicht auf eine Weise an, die zuverlässig und günstig genug wäre. Deshalb stehen sie hier nicht — lieber drei Werte, denen du trauen kannst, als fünf mit zwei Platzhaltern.

3.9 Live Translator

Diese Karte übersetzt, was du sagst, und schreibt es als Untertitel in deinen Stream. Die Spracherkennung läuft dabei **auf deinem eigenen PC** — es geht kein Ton an einen Dienst, und es entstehen keine Kosten pro Minute.



Die Live-Translator-Karte

Bevor du starten kannst, brauchst du ein Sprachmodell und ein Aufnahmegerät. Beides steht in **Settings → Sounds**; der Knopf **Sound Settings** auf der Karte bringt dich direkt dorthin. Das kleine **?** neben dem Kartentitel führt dich Schritt für Schritt durch die Einrichtung.

- **Spoken** — die Sprache, die du sprichst. Nur bei einem mehrsprachigen Modell wählbar.
- **Target** — die Sprache, in die übersetzt wird. Ist sie dieselbe wie **Spoken**, entstehen reine Untertitel ohne Übersetzung; die KI wird dann gar nicht erst gefragt.
- **GDI Source** — die OBS-Textquelle, in die geschrieben wird.

Wohin der Text geht

Drei Ausgänge, unabhängig voneinander schaltbar — auch während der Übersetzer läuft:

- **To GDI Text** — schreibt in eine Textquelle in OBS. Schrift, Größe und Farbe stellst du dort ein.
- **To Chat** — schickt die Zeilen in deinen Twitch-Chat.
- **To Subtitles** — nutzt ASTO-BOTs eigene Untertitelseite. Die braucht **weder OBS noch ein Twitch-Konto** und ist damit der einzige Weg, der auch ohne beides funktioniert. Der Pfeil daneben springt direkt zur Einrichtung.

Aussteuerung und Sprechtempo

Der Balken **Input level** zeigt deinen Pegel. Die weiße Linie ist die Schwelle, ab der ASTO-BOT Sprache erkennt. **Calibrate (3 s)** misst deinen Hintergrund und setzt die Schwelle passend — miss dabei bei Stille, nicht während Musik läuft. **Monitor** schaltet den Balken ein, ohne zu übersetzen.

Speaking pace entscheidet, wie lange eine Pause sein darf, bevor ein Satz als beendet gilt. Drei Stufen:

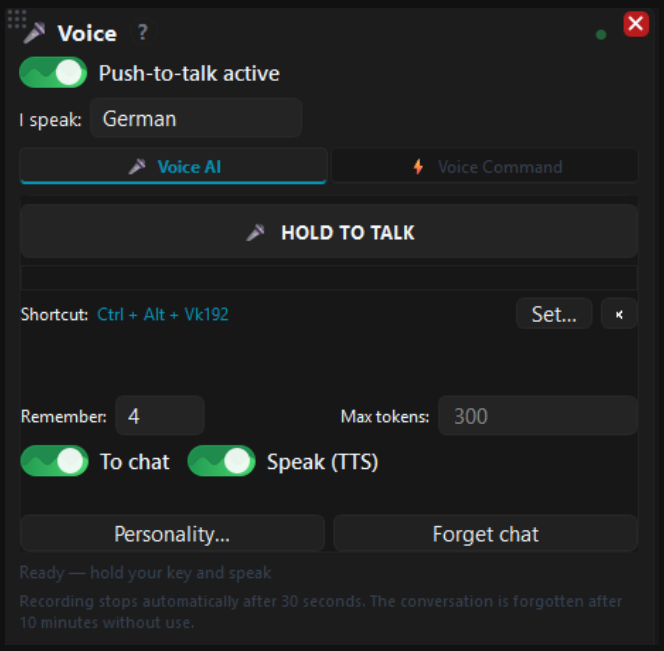
- **Quick** — kurze Pausen, Untertitel erscheinen am schnellsten. Für flüssiges, ununterbrochenes Sprechen.
- **Normal** — verträgt einen Atemzug oder einen kurzen Moment des Nachdenkens mitten im Satz. Passt für die meisten.
- **Patient** — für lange Denkpausen, neu angesetzte Wörter und Vorlesen. Untertitel kommen später, bleiben dafür am Stück.

> 💡 Werden Untertitel mitten im Satz abgeschnitten, stell eine Stufe Richtung **Patient**. Die Erkennung wird deutlich schlechter, wenn Sätze in Bruchstücke zerfallen — der Zusammenhang ist es, aus dem das Programm die Wörter erschließt.

> Aufgenommen wird alles, was auf dem gewählten Gerät liegt. Liegt dein Spielton auf demselben Ausgang, hört die Spracherkennung ihn mit — sie kann deine Stimme nicht davon trennen. Schick auf den Bus, den du aufnimmst, deshalb nur dein Mikrofon.

3.10 Voice

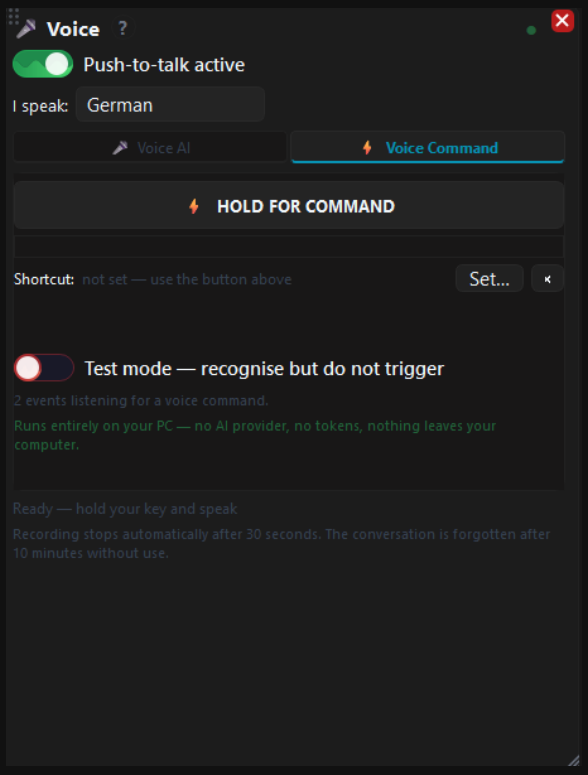
Über diese Karte sprichst du mit ASTO-BOT, statt zu tippen. Zwei Betriebsarten: **Voice AI** schickt das Gesagte an die KI, **Voice Command** löst damit ein Event aus.



Die Voice-Karte mit Push-to-Talk

- **Push-to-talk active** — muss an sein, sonst wird nichts aufgenommen.
- **HOLD TO TALK** — halten, sprechen, loslassen. Genauso funktioniert die zugewiesene Taste.
- **Shortcut** — die Taste, mit der du aufnimmst. Über **Set…** änderst du sie. Eine Taste ist aber kein Muss: Der Knopf auf der Karte, ein Stream Deck und der Websocket tun dasselbe.
- **To chat** und **Speak (TTS)** — ob die Antwort im Chat landet und ob sie vorgelesen wird.

Voice Command ist die zweite Betriebsart. Damit löst du ein Event aus, indem du einen Satz sagst — ohne KI, ohne Internet, alles bleibt auf deinem Rechner.



Die Voice-Karte im Reiter Voice Command

- **HOLD FOR COMMAND** — halten, den Befehl sagen, loslassen. Der Reiter hat eine **eigene** Taste, damit ein Kommando nie versehentlich bei der KI landet.
- **Test mode** — erkennt den Befehl und zeigt an, was ausgelöst *würde*, ohne es zu tun. So probierst du einen neuen Satz im laufenden Stream aus, ohne etwas zu riskieren.
- Unter dem Schalter steht, wie viele Events gerade auf einen Sprachbefehl hören. Steht dort **0**, ist noch bei keinem Event einer eingetragen.

> Den Satz legst du beim Event selbst fest: **Events → Trigger → 🎤 Voice command**. Nimm lieber zwei oder drei Wörter als ein einzelnes — kurze Wörter klingen einander zu ähnlich und werden verwechselt. ASTO-BOT warnt dich, wenn zwei Events zu ähnliche Befehle haben, und löst im Zweifel **gar nichts** aus, statt das Falsche zu treffen.
>
> Voice benutzt **dasselbe Sprachmodell wie der Live Translator**. Ist dort keines gewählt, funktioniert auch Voice nicht — die Karte sagt dir das dann direkt. Das **?** neben dem Titel führt durch die Einrichtung.

Spiele per Sprache steuern

Ein Sprachbefehl kann nicht nur im Chat etwas auslösen, sondern auch **Tasten im Spiel drücken**. Damit steuerst du dein Spiel, ohne die Hände von der Maus zu nehmen — in Star Citizen zum Beispiel Fahrwerk ein- und ausfahren oder die Schilde umschalten.

Der Weg dorthin geht über drei Bausteine, die du schon kennst:

- Ein **Event** mit einem Sprachbefehl unter **Events → Trigger → 🎤 Voice command**, etwa „Fahrwerk einfahren".
- Als Aktion darin **Run Script**.
- Im Script der Tastendruck, also `SENDKEY PRESS "N"` — oder eine Kombination wie `SENDKEY PRESS "STRG+N"`.

> **Wichtig:** Der Tastendruck muss genau der sein, der im Spiel für diese Funktion eingestellt ist. ASTO-BOT drückt die Taste nur — welche Wirkung sie hat, entscheidet allein die Tastaturbelegung des Spiels. Schau also zuerst in den Spieleinstellungen nach und trage denselben Wert ein.

> 💡 Zielt der Tastendruck auf ein bestimmtes Fenster, setz vorher `SENDKEY WINDOW "Star Citizen"`. Ohne diese Zeile geht die Taste an das Fenster, das gerade im Vordergrund ist — im Zweifel also an ASTO-BOT selbst.

> Spiele mit Anti-Cheat nehmen simulierte Tastendrücke nicht immer an. Probier den Befehl deshalb im Ruhigen aus, bevor du dich im Stream darauf verlässt.

3.11 Stream Health

Diese Karte zeigt dir während des Streams, ob dein Bild sauber bei Twitch ankommt — und wenn nicht, **warum**. Sie ist standardmäßig ausgeblendet; du holst sie wie die anderen Karten über den grünen **+** Knopf auf das Dashboard.

Die Werte kommen über die OBS-Verbindung, die ASTO-BOT ohnehin offen hat. Solange OBS streamt, wird alle 5 Sekunden gemessen, sonst schaut die Karte nur alle 15 Sekunden nach, ob ein Stream begonnen hat. Ist sie ausgeblendet, wird gar nichts gemessen.

Was die Karte zeigt

- **Status** oben rechts — ● **OK**, oder die Ursache: **Upload**, **Encoder**, **Rendering**, **Bitrate dip** oder **Reconnecting**. Darunter steht in einem Satz, was zu tun ist.
- **Bitrate** — was OBS gerade sendet, mit Verlauf. Mit Enhanced Broadcasting ist das die Summe aller Qualitätsstufen und deshalb deutlich höher als die eingestellte Bitrate.
- **Dropped · network** — Frames, die OBS verwerfen musste, weil dein Upload nicht hinterherkam.
- **Skipped · encoder** — Frames, die der Encoder nicht rechtzeitig geschafft hat.
- **Missed · rendering** — Frames, die OBS nicht rechtzeitig zeichnen konnte, meist weil das Spiel die Grafikkarte voll auslastet.
- Bei diesen drei steht rechts der Wert der letzten 5 Sekunden, links daneben die Summe seit Streambeginn.
- **OBS CPU**, **FPS** und Renderzeit pro Frame, darunter die **CPU von ASTO-BOT** im selben Moment — so siehst du, dass ASTO-BOT nicht die Ursache ist.
- **Network** — ob über Kabel, WLAN oder Mobilfunk gestreamt wird und wie viel **andere Programme** gerade hochladen.
- **Recent problems** — die letzten sechs Probleme mit Uhrzeit, Höchstwert und Dauer.

Die Diagnose

Die Karte nennt nicht nur das Symptom, sondern wenn möglich auch den Grund:

- Lädt ein anderes Programm gleichzeitig hoch — Cloud-Sync, Steam, Windows Update, ein Speedtest —, steht das samt Menge direkt in der Meldung.
- Streamst du über **WLAN**, weist die Karte darauf hin. Das ist die häufigste Ursache für verworfene Frames.
- Bei Encoder-Problemen mit **x264** schlägt sie einen Hardware-Encoder vor (NVENC, AMF, QuickSync).
- Läuft der PC im **Akkubetrieb**, mit **Energiesparmodus** oder Energiesparplan, steht das dabei — das drosselt CPU und Grafikkarte.
- Bei Render-Problemen sagt sie dir, wenn **OBS nicht als Administrator** läuft. Nur dann bekommt OBS bei voller Grafikkarte Vorrang vor dem Spiel.
- Sendet OBS seit drei Minuten, aber Twitch zeigt dich **nicht als live**, warnt die Karte — meist ein falscher Stream-Key oder ein anderes Konto.

Ein kurzer Render-Ruckler direkt nach einem **Szenenwechsel** ist normal — OBS hängt dabei die Spielaufnahme neu an. Er steht als **Scene switch** in der Liste, löst aber keine Warnung aus. Kleine Ausreißer unter 3 % zählen erst, wenn sie zweimal hintereinander auftreten.

> 💡 Jedes Problem landet außerdem als Marke im Zuschauerverlauf. In **Stream Insights** zeigt der Abschnitt ⚠ **Stream problems** später, ob nach einem Ruckler mehr Zuschauer gegangen sind als in normalen zehn Minuten.

OBS Setup Info

Der Knopf 🪀 **OBS Setup Info** auf der Karte öffnet eine Übersicht aller Einstellungen, die fürs Streamen zählen: Video, Dienst und Server, Encoder samt Preset, Audio, Netzwerk, Aufnahme, allgemeine OBS-Einstellungen und dein PC. Was nicht zu den Richtwerten von Twitch passt, ist orange markiert — etwa ein Keyframe-Intervall ungleich 2 s, keine konstante Bitrate, eine zu hohe oder für die Auflösung zu niedrige Bitrate oder ein eingeschalteter Bandbreitentest.

> Das Fenster **zeigt nur an** — geändert wird in OBS unter **Einstellungen**. Deinen Stream-Key und deine Twitch-Zugangsdaten liest ASTO-BOT dabei nie aus.

Ein Teil der Einstellungen steht nur in den Dateien von OBS, etwa Preset und Keyframe-Intervall im erweiterten Ausgabemodus oder die Browser-Hardwarebeschleunigung. Die kann ASTO-BOT nur lesen, wenn OBS auf **demselben PC** läuft. Eine portable OBS-Installation findet es dabei von selbst.

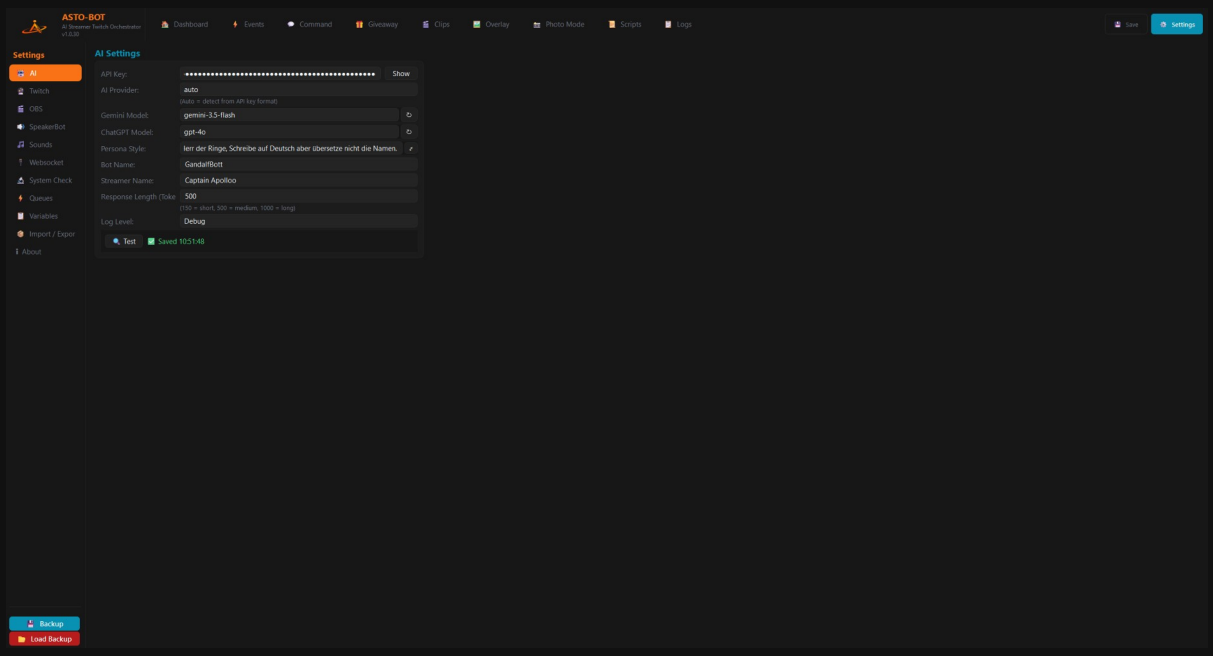
Was die Karte nicht kann

Was bei **Twitch** ankommt, kann kein Programm außerhalb von Twitch abfragen — dafür gibt es keine öffentliche Schnittstelle. Ebenso wenig sieht die Karte, ob ein einzelner Zuschauer puffert. Sie misst alles, was zwischen OBS und deiner Leitung passiert.

4. Settings

In den **Settings** wird alles eingestellt, was ASTO-BOT zum Arbeiten braucht — die KI, die Twitch-Anmeldung, die Verbindung zu OBS, Sounds und mehr. Du erreichst sie über den Knopf **Settings** rechts oben in der Kopfleiste.

Links siehst du die Bereiche, zwischen denen du wechselst. Ganz unten liegen zusätzlich die beiden Knöpfe **Backup** und **Load Backup**.

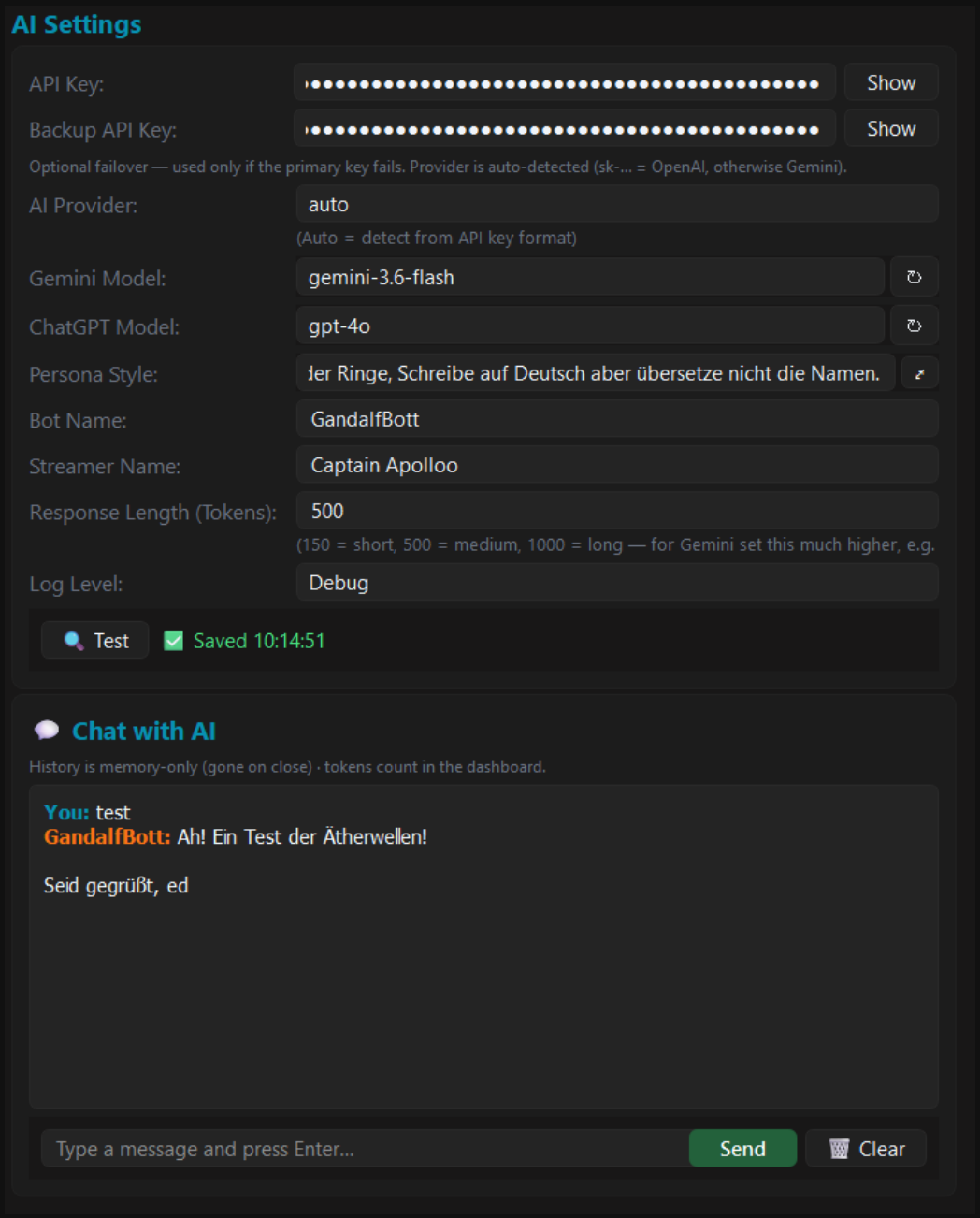


Die Settings mit der Bereichsliste links

> 💡 Zum Einstieg reicht wenig: **AI** (nur wenn du die KI nutzen willst), **Twitch** (Anmeldung) und **OBS** (Verbindung). Alles andere kannst du dir später in Ruhe ansehen.

4.1 AI

Hier wird die Künstliche Intelligenz eingerichtet, mit der ASTO-BOT Chat-Antworten erzeugt.



Die AI Settings

- **API Key** — dein Schlüssel beim KI-Anbieter. ASTO-BOT erkennt selbständig, ob es sich um einen Google-Gemini- oder einen OpenAI-Schlüssel handelt. Mit **Show** machst du den Schlüssel sichtbar.
- **Backup API Key** — optional. Ein zweiter Schlüssel, der **nur einspringt, wenn der erste einen Fehler meldet**. Sein Anbieter wird automatisch am Schlüssel-Format erkannt (`sk-…` = OpenAI, sonst Gemini). Lässt du das Feld leer, verhält sich alles wie mit nur einem Schlüssel.
- **AI Provider** — normalerweise **auto**. Du kannst den Anbieter auch fest auf **Gemini** oder **OpenAI** stellen; das gilt aber nur für den **ersten** Schlüssel. Der Backup-Schlüssel erkennt seinen Anbieter immer selbst.

- **Gemini Model** / **ChatGPT Model** — hier wählst du das Modell. Der kleine Knopf daneben lädt die aktuell verfügbaren Modelle beim Anbieter nach und nimmt dafür automatisch den passenden Schlüssel aus einem der beiden Felder.
- **Persona Style** — der Charakter, den der Bot annehmen soll. Das ist die wichtigste Stellschraube für die Persönlichkeit deines Bots.
- **Bot Name** / **Streamer Name** — unter diesen Namen kennt die KI euch beide.
- **Response Length (Tokens)** — wie lang die Antworten werden dürfen. 150 = kurz, 500 = mittel, 1000 = lang. **Für Gemini deutlich höher stellen** (z. B. 5000): Gemini-Modelle „denken“ intern mit, und diese Denk-Tokens zählen auf dieses Limit — bei einem zu kleinen Wert bricht die Antwort mitten im Satz ab.
- **Log Level** — wie ausführlich protokolliert wird.
- **Test** — prüft **beide** Schlüssel, jeden mit seinem eigenen Ergebnis. Trägt nur der Backup-Schlüssel, steht genau das da — samt Hinweis, wenn **AI Provider** nicht zum eingetragenen Schlüssel passt.

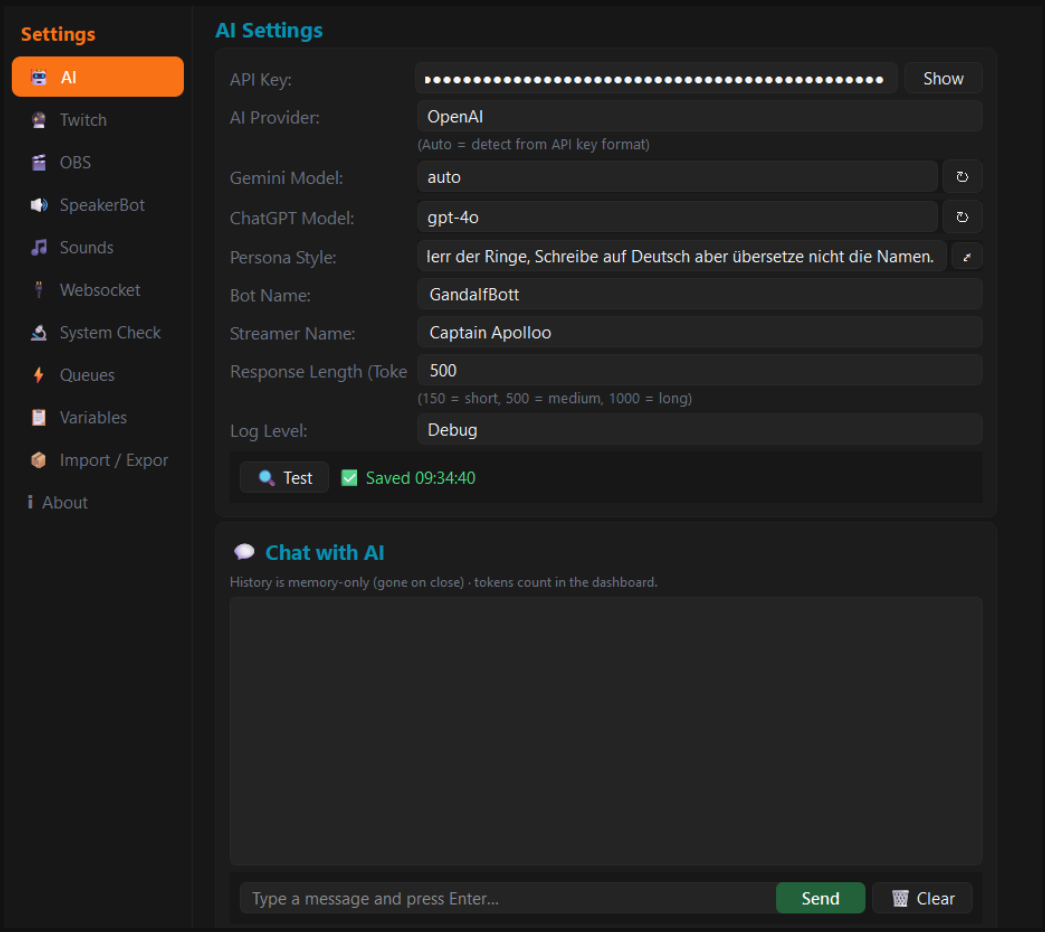
> Zum Nachladen der Modelle sucht ASTO-BOT den passenden Schlüssel in **beiden** Feldern: **Gemini Model** braucht einen Gemini-Schlüssel, **ChatGPT Model** einen OpenAI-Schlüssel. Erst wenn in **keinem** der beiden Felder ein Schlüssel des richtigen Anbieters steht, meldet der Knopf das — das ist dann kein Fehler, sondern der Hinweis, dass der passende Schlüssel fehlt.
>
> Failover: Meldet der erste Schlüssel einen Fehler, springt der Backup-Schlüssel ein und **seine** Antwort geht in den Chat; der Fehler des ersten steht dann nur im **Log**. Schlagen **alle** hinterlegten Schlüssel fehl (oder ist keiner eingetragen), bleibt der Chat still — der Grund steht im Log.

> 💡 Zum **Log Level**: Für den normalen Betrieb ist die Einstellung nebensächlich. Wenn du dem Entwickler bei der Fehlersuche helfen willst, stell auf **Debug** — dann steht im Protokoll alles drin.

> Alle Änderungen in den AI Settings werden **automatisch gespeichert** — bei Texteingaben (Schlüssel, Persona, Namen, Token-Zahl) kurz nach dem Tippen, bei den Auswahlfeldern sofort. Ein kurzes „Saved“ bestätigt es. Einen extra Speichern-Knopf brauchst du hier nicht.

Chat with AI

Unter den AI Settings liegt ein kleines Chatfenster, in dem du **direkt mit der KI schreiben** kannst — praktisch, um deinen Persona-Style oder das gewählte Modell schnell auszuprobieren, ohne dass ein Zuschauer etwas davon mitbekommt.



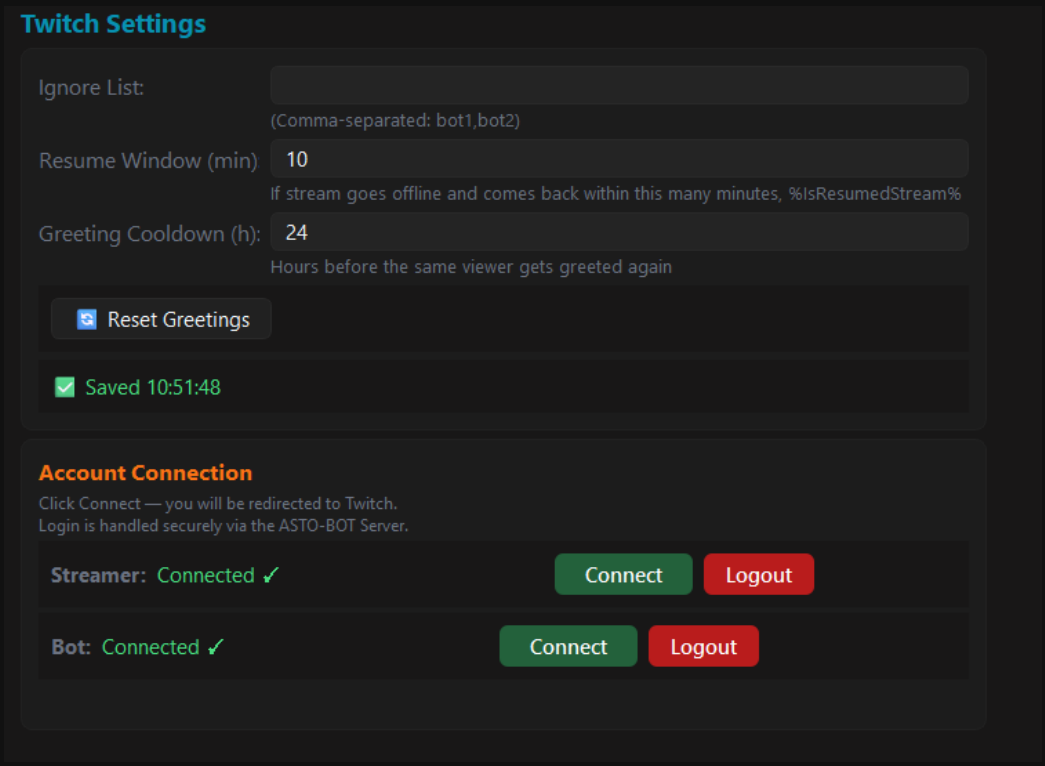
Die AI Settings mit dem Chatfenster darunter

Tipp deine Nachricht unten ein und drück **Enter** oder **Send**. **Clear** leert den Verlauf.

> Der Chat-Verlauf lebt **nur im Arbeitsspeicher** und wird nirgends gespeichert: Sobald du ASTO-BOT schließt, ist er weg. Die verbrauchten Tokens tauchen aber im Dashboard auf.

4.2 Twitch

Hier meldest du dich bei Twitch an und stellst ein, wie sich ASTO-BOT gegenüber deinen Zuschauern verhält.



Die Twitch Settings mit der Account-Anmeldung

Account Connection

Ein Klick auf **Connect** leitet dich zu Twitch weiter, wo du dich anmeldest. Die Anmeldung läuft sicher über den ASTO-BOT Server — dein Passwort bekommt ASTO-BOT nie zu sehen.

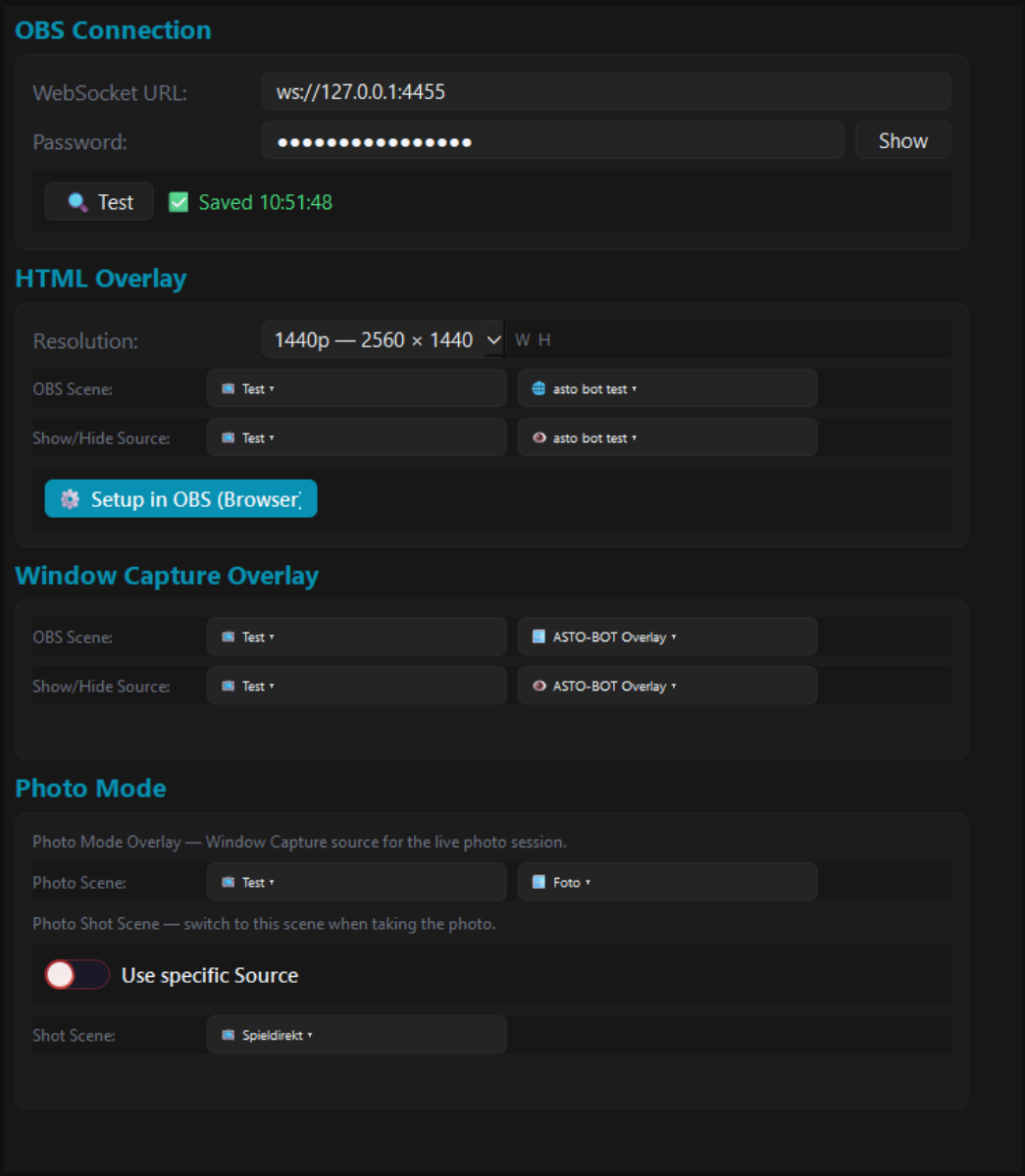
- **Streamer** — dein Hauptkonto. Das ist die Anmeldung, die ASTO-BOT zwingend braucht.
- **Bot** — ein separates Konto, unter dessen Namen der Bot im Chat schreibt. Optional, aber empfehlenswert.
- **Logout** trennt die jeweilige Verbindung wieder.

Einstellungen

- **Ignore List** — Namen, die ASTO-BOT ignorieren soll, durch Komma getrennt. Praktisch für andere Bots in deinem Chat, damit sie keine Events auslösen.
- **Resume Window (min)** — in Minuten. Fällt dein Stream ab, z. B. durch einen Internetausfall, und kommt innerhalb dieser Zeit wieder, wertet ASTO-BOT beides als **einen einzigen Stream** statt als zwei.
- **Greeting Cooldown (h)** — in Stunden. So lange dauert es, bis derselbe Zuschauer wieder begrüßt wird. Das gehört zum Trigger **Greeting** im Events-Bereich.
- **Reset Greetings** — setzt den Zähler zurück. Ab dem Klick wird jeder wieder begrüßt, als wäre er zum ersten Mal da.

4.3 OBS

Dieser Bereich ist der wichtigste — und der, an dem die meisten hängenbleiben. Nimm dir hier ein paar Minuten Zeit, dann funktioniert alles andere später reibungslos.



Die OBS-Einstellungen: Verbindung, HTML Overlay, Window Capture Overlay und Photo Mode

4.3.1 OBS Connection

- **WebSocket URL** — Standard ist `ws://127.0.0.1:4455`. Ändere das nur, wenn du in OBS einen anderen Port eingestellt hast.
- **Password** — das Passwort aus den OBS-WebSocket-Einstellungen.
- **Test** — prüft, ob die Verbindung zu OBS steht.

4.3.2 HTML Overlay

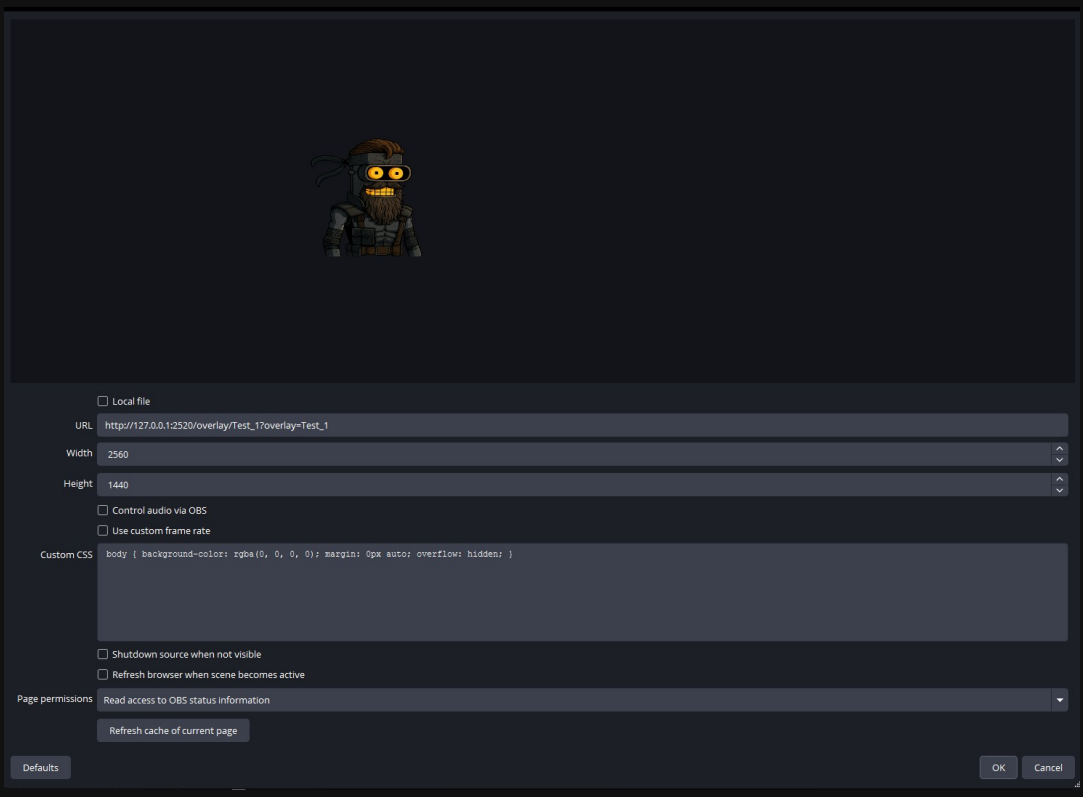
Diese Einstellungen gehören zum Overlay-Bereich. ASTO-BOT muss wissen, **wo** in OBS die HTML-Quelle liegt.

- **Resolution** — sollte mit der Auflösung übereinstimmen, die in OBS als Standard-Canvas eingestellt ist (in der Regel die deines Monitors).
- **OBS Scene** — links wählst du die **Szene**, rechts daneben die **Quelle** darin. Das ist die Browser-Quelle mit dem Overlay.
- **Show/Hide Source** — welche Szene und Quelle ein- und ausgeblendet werden, wenn das Overlay startet. Das ist absichtlich getrennt: Quellen können verschachtelt sein,

und die HTML-Quelle soll sich unter Umständen in einer ganz anderen Szene einblenden.

- **Setup in OBS (Browser)** — aktualisiert die gewählte HTML-Quelle in OBS.

So sieht die Browser-Quelle in OBS aus:



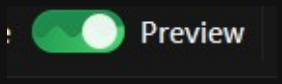
Die Browser-Quelle in OBS — die URL setzt ASTO-BOT selbständig

> Wichtig: Der Haken bei **Local file** darf **nicht** gesetzt sein. Die URL wird von ASTO-BOT automatisch eingetragen (`http://127.0.0.1:2520/overlay/...`).

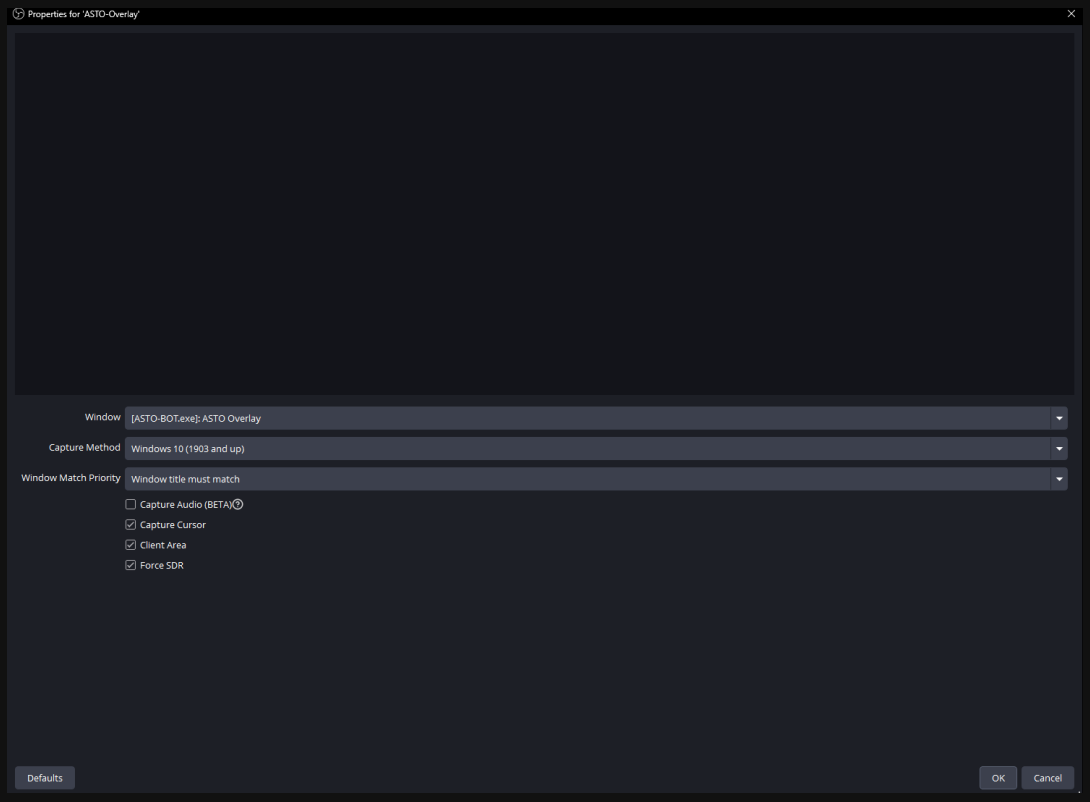
4.3.3 Window Capture Overlay

Die zweite Methode: Statt einer Browser-Quelle nimmt OBS ein Fenster von ASTO-BOT auf. Dafür muss in OBS zwingend eine **Fensteraufnahme** (Window Capture) angelegt sein.

Damit das Fenster in der OBS-Liste überhaupt auftaucht, muss es geöffnet sein: Stell im **Overlay**-Bereich den **Preview**-Schalter an, während du die Quelle in OBS einrichtest.



Der Preview-Schalter im Overlay-Bereich



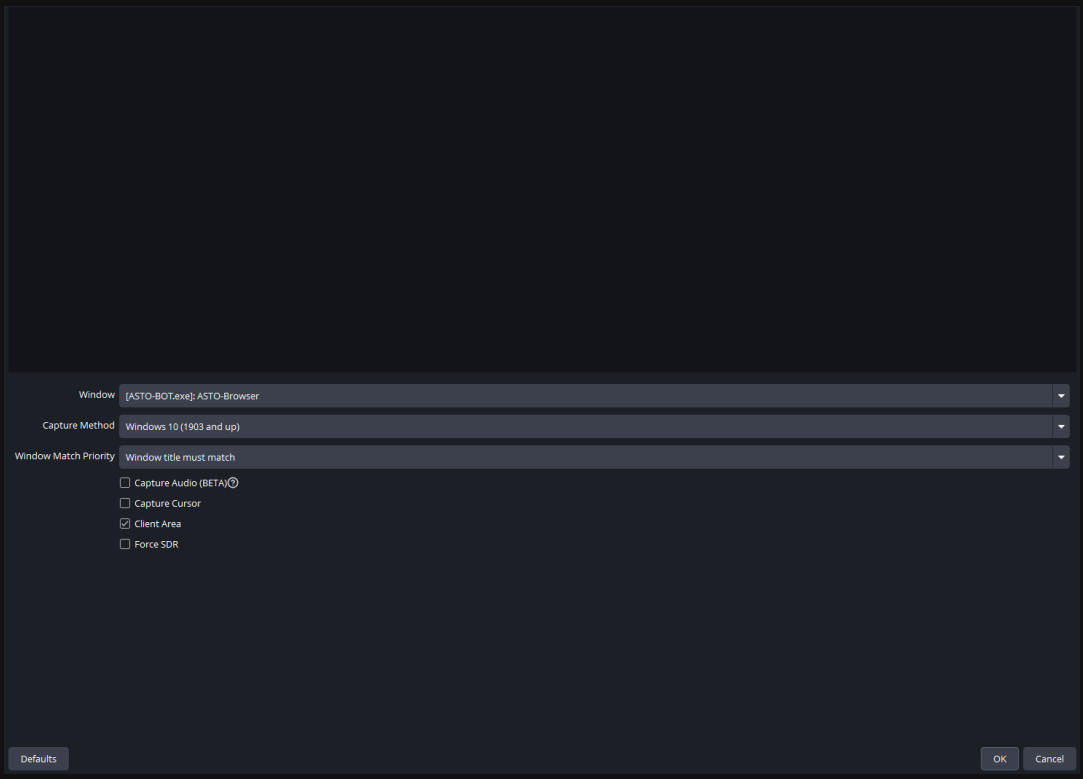
Die Fensteraufnahme für das Overlay in OBS

- **Window** — `[ASTO-BOT.exe]: ASTO Overlay` wählen.
- **Capture Method** — muss auf **Windows 10 (1903 and up)** stehen.
- **Window Match Priority** — auf **Window title must match** stellen, damit OBS nicht versehentlich ein anderes Fenster aufnimmt.
- Alles Weitere ist optional.

In den Settings gibst du dann wieder **OBS Scene** (Szene + Quelle) und **Show/Hide Source** an — genau wie beim HTML Overlay.

4.3.4 Browser Overlay

Das **Browser Overlay** (Kapitel 9.8) ist ein eigenes Fenster von ASTO-BOT und braucht deshalb eine **eigene** OBS-Quelle. Sie wird — genau wie das Window Capture Overlay — als **Fensteraufnahme** (Window Capture) angelegt, nur mit einem anderen Fenster.



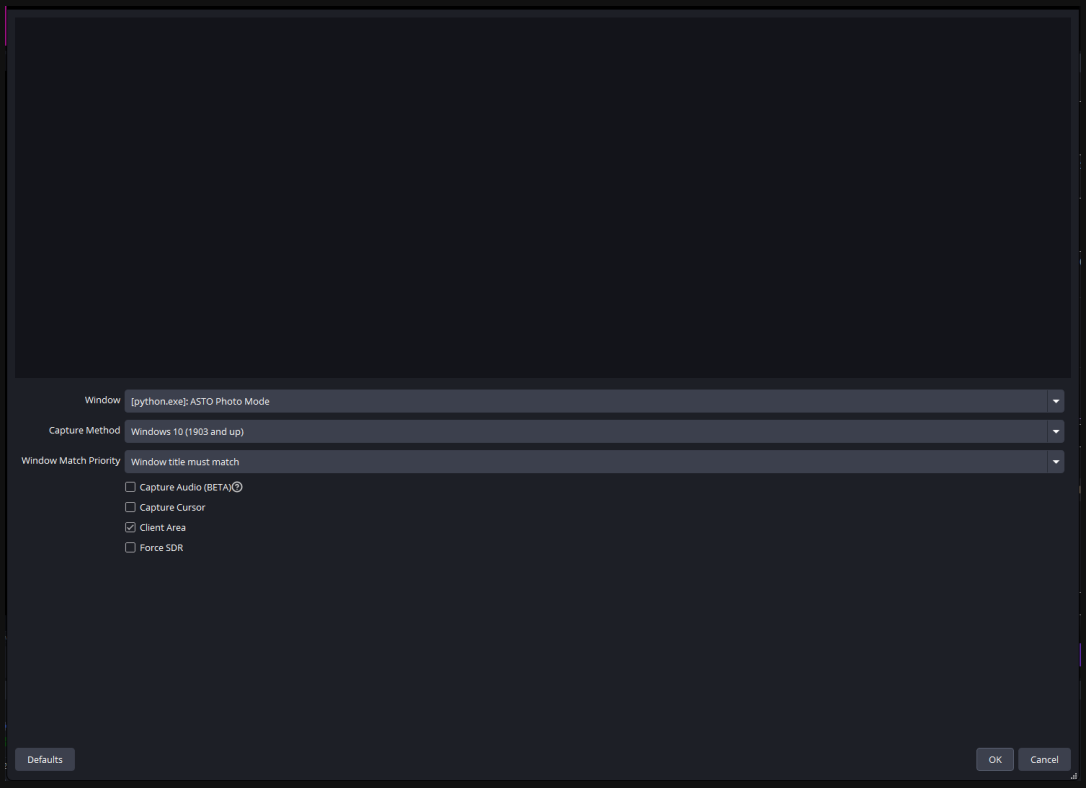
Die Fensteraufnahme für das Browser Overlay in OBS

- **Window** — hier unbedingt `**[ASTO-BOT.exe]: ASTO-Browser**` wählen (nicht ASTO Overlay).
- **Capture Method** — muss auf **Windows 10 (1903 and up)** stehen.
- **Window Match Priority** — auf **Window title must match** stellen.
- **Capture Cursor** — aus, damit der Mauszeiger nicht mit im Bild landet.

> Damit `**[ASTO-BOT.exe]: ASTO-Browser**` in der Fensterliste auftaucht, muss das Browser Overlay laufen: Schalte es im **Overlay**-Bereich mit dem **ON**-Schalter ein, während du die Quelle in OBS einrichtest.

4.3.5 Photo Mode

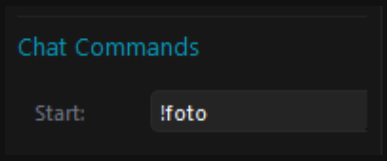
Der Photo Mode wird genauso aufgenommen wie das Window Capture Overlay — auch hier ist es ein Fenster von ASTO-BOT.



Die Fensteraufnahme für den Photo Mode

> Der entscheidende Unterschied: Als **Window** muss `ASTO Photo Mode` gewählt werden. Auch dieses Fenster muss dabei geöffnet sein — starte also im Photo-Mode-Bereich ein Foto, während du die Quelle in OBS einrichtest.

Ausgelöst wird der Photo Mode zum Beispiel über einen Chat-Befehl:



Der Photo Mode, gestartet über den Chat-Befehl !foto

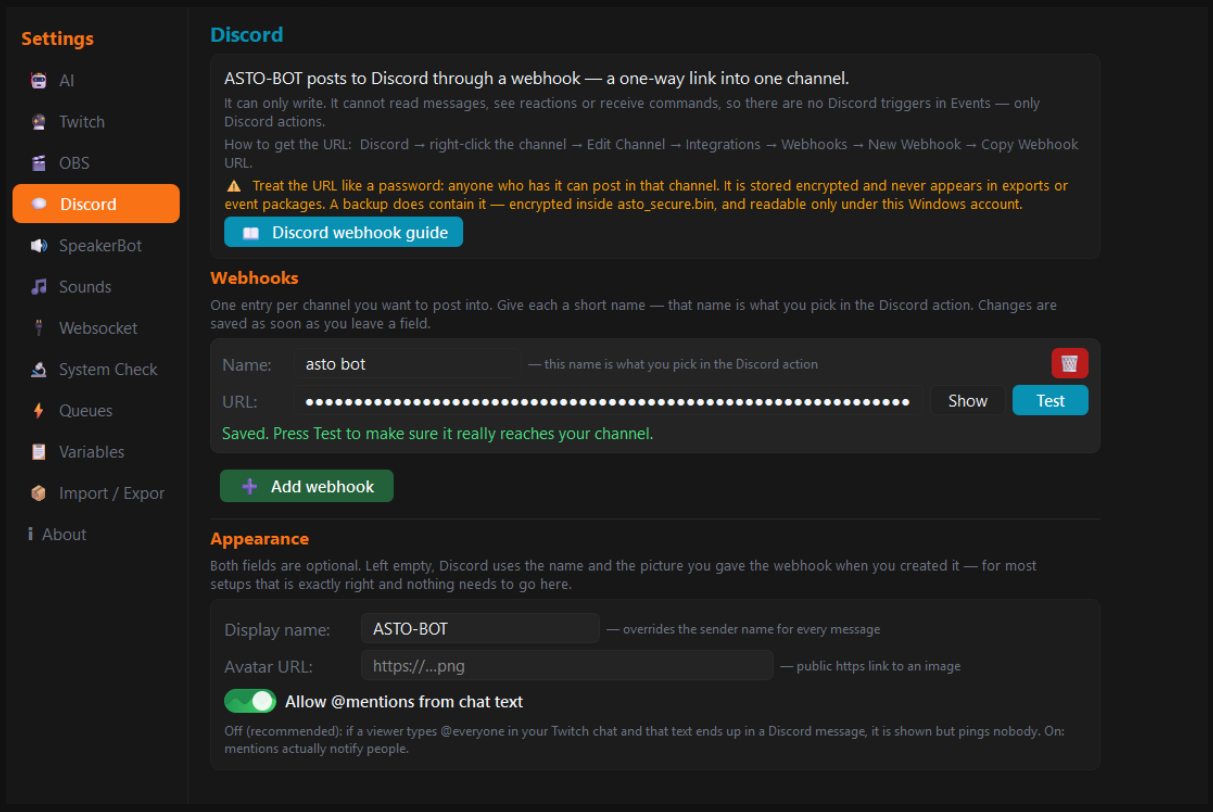
In den Settings stellst du dazu ein:

- **Photo Scene** — Szene und Quelle der Fensteraufnahme für die laufende Foto-Sitzung.
- **Shot Scene** — die Szene, auf die beim Auslösen des Fotos umgeschaltet wird.
- **Use specific Source** — ist der Schalter **aus**, wird auf die **Szene** umgeschaltet. Ist er **an**, wird stattdessen eine einzelne **Quelle** eingeblendet.

4.4 Discord

Über einen Webhook kann ASTO-BOT in deinen Discord schreiben — zum Beispiel eine Ankündigung, sobald dein Stream startet. Ein Webhook ist eine Adresse, die Discord für einen bestimmten Kanal ausstellt: Wer sie hat, darf in diesen Kanal schreiben. Du legst sie in Discord unter **Kanal bearbeiten** → **Integrationen** → **Webhooks** an und kopierst sie hierher.

Du kannst beliebig viele Webhooks anlegen und gibst jedem einen Namen — etwa *Ankündigungen* und *Clips*. In den Discord-Aktionen wählst du später nur noch über diesen Namen aus, wohin die Nachricht geht.



Der Discord-Bereich in den Settings mit einem angelegten Webhook

- **+ Add webhook** — legt einen neuen Eintrag an. Einen je Kanal, in den du schreiben willst.
- **Name** — dein eigener Name für diesen Webhook. Genau dieser Name steht später in den Discord-Aktionen zur Auswahl.
- **URL** — die Adresse aus Discord. Sie steht als Punkte da; **Show** macht sie sichtbar.
- **Test** — schickt eine Testnachricht in den Kanal. Kommt sie an, stimmt alles.
- 📖 **Discord webhook guide** — öffnet die Anleitung, falls du beim Anlegen in Discord nicht weiterkommst.

Gespeichert wird, sobald du ein Feld verlässt — einen Speichern-Knopf gibt es hier nicht.

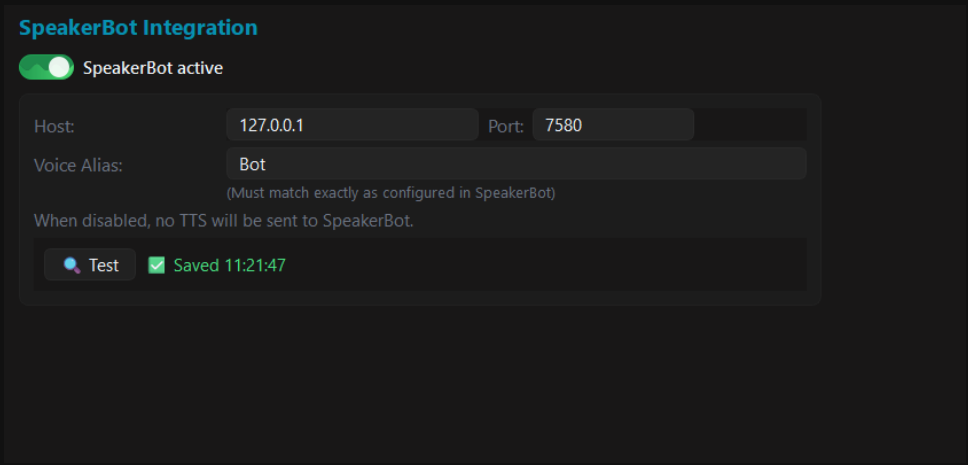
Unter **Appearance** kannst du zwei Dinge überschreiben, musst aber nicht: **Display name** ersetzt den Absendernamen jeder Nachricht, **Avatar URL** das Bild dazu. Lässt du beides leer, nimmt Discord den Namen und das Bild, die du dem Webhook beim Anlegen in Discord gegeben hast — für die meisten Fälle genau richtig.

> Der Schalter **Allow @mentions from chat text** entscheidet, was passiert, wenn ein Zuschauer **@everyone** in deinen Twitch-Chat schreibt und dieser Text in einer Discord-Nachricht landet. **Aus** (empfohlen): Der Text wird angezeigt, benachrichtigt aber niemanden. **An**: Erwähnungen benachrichtigen wirklich.
>
> Die Adresse liegt **verschlüsselt** neben deinen Twitch-Zugängen, nicht im Klartext in einer Einstellungsdatei. Weitergeben solltest du sie trotzdem nie: Wer sie hat, schreibt in deinen Kanal.

> 💡 Nimm für jeden Zweck einen eigenen Webhook. Wird einer missbraucht, löschst du in Discord genau diesen — der Rest läuft weiter.

4.5 SpeakerBot

SpeakerBot übernimmt die Sprachausgabe (TTS). Die Anbindung ist optional — ASTO-BOT läuft auch ohne.

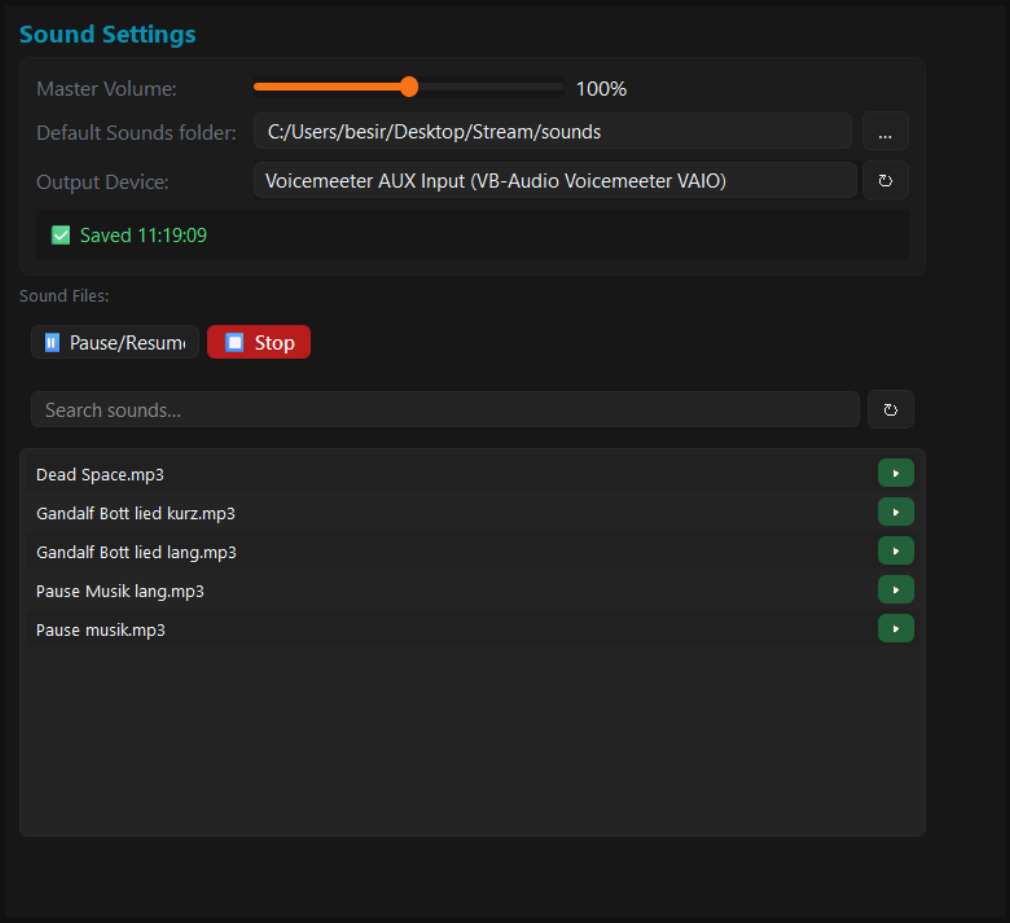


Die SpeakerBot-Integration

- **SpeakerBot active** — schaltet die Anbindung ein und aus. Ist sie aus, wird kein TTS an SpeakerBot geschickt.
- **Host** und **Port** — müssen zu deiner SpeakerBot-Installation passen.
- **Voice Alias** — muss **exakt** so heißen wie in SpeakerBot selbst konfiguriert.
- **Test** — prüft die Verbindung.

4.6 Sounds

Hier stellst du ein, wie ASTO-BOT Sounds abspielt — die Grundlage für alle Sound-Aktionen in Events.



Die Sound Settings mit Ordner, Ausgabegerät und Sound-Liste

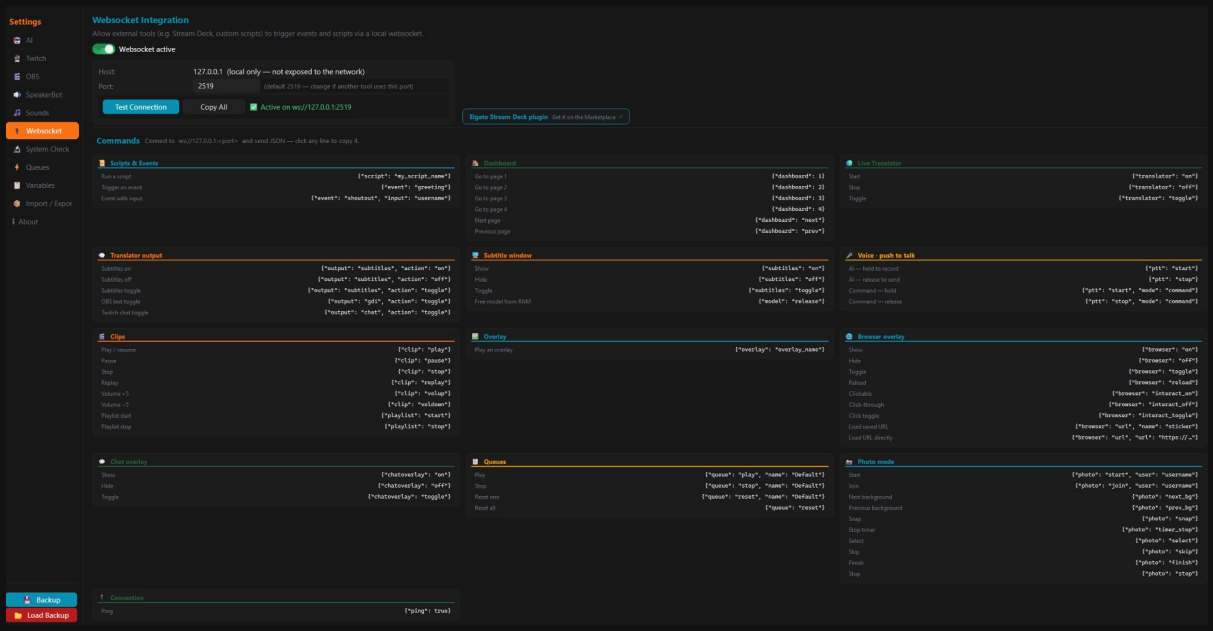
- **Master Volume** — die Gesamtlautstärke.
- **Default Sounds folder** — der Ordner, in dem deine Sounddateien liegen.
- **Output Device** — über welches Gerät die Sounds ausgegeben werden.

Darunter siehst du unter **Sound Files** alle Dateien im gewählten Ordner. Mit dem grünen Knopf rechts spielst du eine Datei direkt ab — praktisch zum Vorhören. Über das Suchfeld findest du in großen Sammlungen schnell die richtige Datei.

> **Pause/Resume** und **Stop** wirken auf den **gerade laufenden** Sound — nicht auf alle. Läuft kein Sound, passiert nichts.

4.7 Websocket

Über den Websocket kann ASTO-BOT von außen gesteuert werden — zum Beispiel von einem Elgato Stream Deck oder eigenen Skripten. Das sieht auf den ersten Blick nach viel aus, ist es aber nicht.



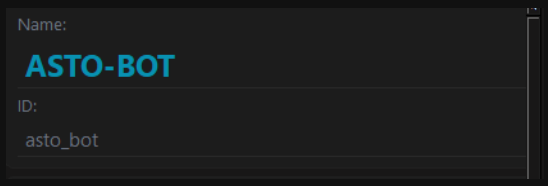
Die Websocket-Integration mit der Befehlsliste

- **Websocket active** — muss eingeschaltet sein, wenn du die Steuerung nutzen willst.
- **Host** — fest `127.0.0.1`. Der Websocket ist **nur lokal** erreichbar und nicht im Netzwerk sichtbar.
- **Port** — Standard ist 2519. Ändere ihn nur, wenn ein anderes Programm diesen Port schon benutzt.
- **Test Connection** — prüft, ob der Websocket läuft.

Darunter steht die Liste aller Befehle. **Ein Klick auf einen Befehl kopiert ihn sofort in die Zwischenablage** — du musst also nichts abtippen. Du ersetzt anschließend nur noch den Platzhalter.

> Sehr wichtig: Bei Events wird die **Event-ID** eingetragen, **nicht der Name**. Bei allen anderen Befehlen — etwa `{"script": "..."}` — wird dagegen der **Name** verwendet.

Die Event-ID findest du im **Events**-Bereich, sie steht immer direkt unter dem Event-Namen. In der Regel entspricht sie dem Namen, nur klein geschrieben und mit Unterstrich _ statt Leerzeichen:



Name und ID eines Events — für den Websocket wird die ID gebraucht

Das Browser Overlay steuern

Für das **Browser Overlay** (Kapitel 9.8) gibt es eigene Befehle. Damit schaltest du es zum Beispiel per Stream Deck ein und aus, lädst es neu oder blendest eine andere URL ein:

```
# Browser Overlay per Websocket
{"browser": "on"}                          # einschalten
{"browser": "off"}                         # ausschalten
{"browser": "toggle"}                      # umschalten
{"browser": "reload"}                      # neu laden
{"browser": "interact_on"}                 # klickbar machen
{"browser": "interact_off"}                # klick-durchlaessig (Standard)
{"browser": "interact_toggle"}             # Klickbarkeit umschalten
{"browser": "url", "name": "sticker"}      # URL aus der Liste per Name
{"browser": "url", "url": "https://..."}   # URL direkt einblenden
```

URL per Name lädt einen Eintrag aus deiner URL-Liste — `name` ist der dort vergebene Name. **URL direkt** blendet eine beliebige Adresse ein.

Das Chat-Overlay steuern

Auch das **Chat-Overlay** (Kapitel 9.10) hat einen eigenen Befehl. **Aus** heißt dabei wirklich aus: Es werden keine neuen Nachrichten mehr geschickt, und die Zeilen, die noch im Bild stehen, werden gelöscht.

```
# Chat-Overlay per Websocket
{"chatoverlay": "on"}                    # einschalten
{"chatoverlay": "off"}                   # ausschalten und leeren
{"chatoverlay": "toggle"}                # umschalten
```

Stream Deck einrichten

Für das Elgato Stream Deck gibt es ein **eigenes ASTO-BOT-Plugin**. Das ist der bequemste Weg: Du ziehst eine Aktion auf eine Taste und wählst aus einer Liste aus — weder Adresse noch JSON-Befehl musst du von Hand eintragen.

Du findest es im Elgato Marketplace:

https://marketplace.elgato.com/product/asto-bot-2db69caf-ee1e-4520-8375-ae8a6a8dbd67

Alternativ suchst du im Stream Deck unter **More Actions** einfach nach *ASTO-BOT*. Und in ASTO-BOT selbst führt im Websocket-Bereich ein Klick auf **Elgato Stream Deck plugin** direkt zur Marketplace-Seite.

Das Plugin bringt sechs Aktionen mit:

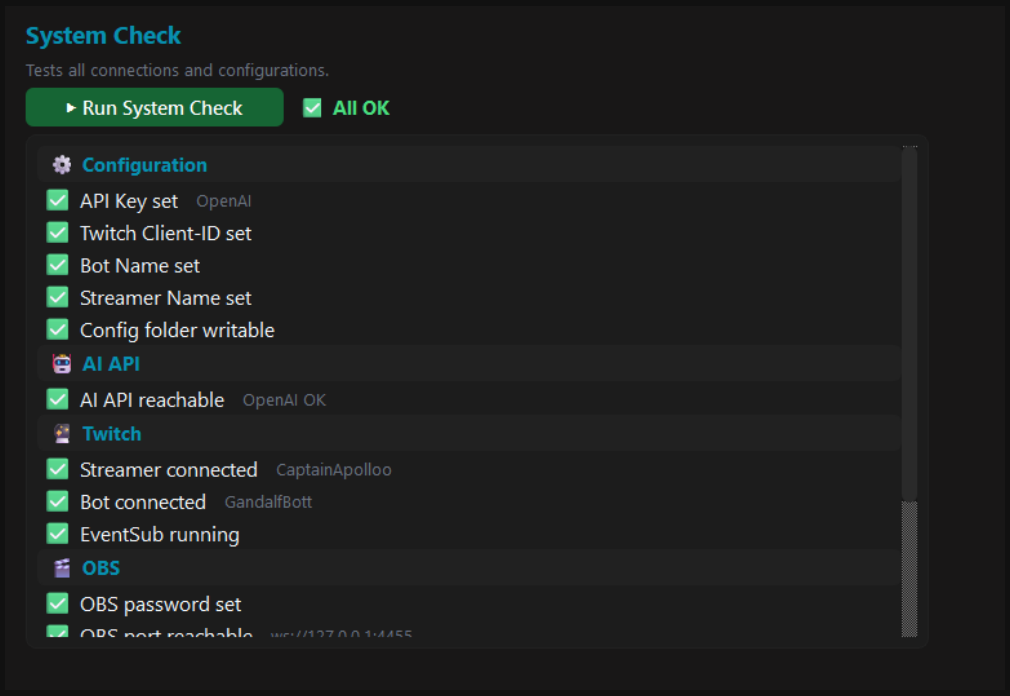
- **Command** — schickt einen beliebigen Websocket-Befehl. Damit erreichst du alles, was in der Befehlsliste weiter oben steht.
- **Run Script** — startet ein Skript. Den Namen wählst du aus einer Liste, die das Plugin live aus ASTO-BOT holt.
- **Trigger Event** — löst ein Event aus, ebenfalls über eine Auswahlliste. Um Name und ID musst du dich hier nicht kümmern.
- **Translator** — startet und stoppt den Live Translator (Kapitel 3.9).
- **Push to Talk** — nimmt auf, solange die Taste gedrückt ist, und schickt beim Loslassen ab (Kapitel 3.10).
- **Reconnect** — baut die Verbindung zu ASTO-BOT neu auf, falls sie einmal abgerissen ist.

Alle Tasten teilen sich **eine** Verbindung zu ASTO-BOT. Reißt sie ab — etwa weil du ASTO-BOT neu startest —, verbindet sich das Plugin von allein wieder.

> Damit das Plugin arbeiten kann, muss **Websocket active** eingeschaltet sein und der **Port** übereinstimmen. Hast du in ASTO-BOT einen anderen Port als 2519 eingestellt, trägst du ihn in den Einstellungen der Taste ebenfalls ein.

4.8 System Check

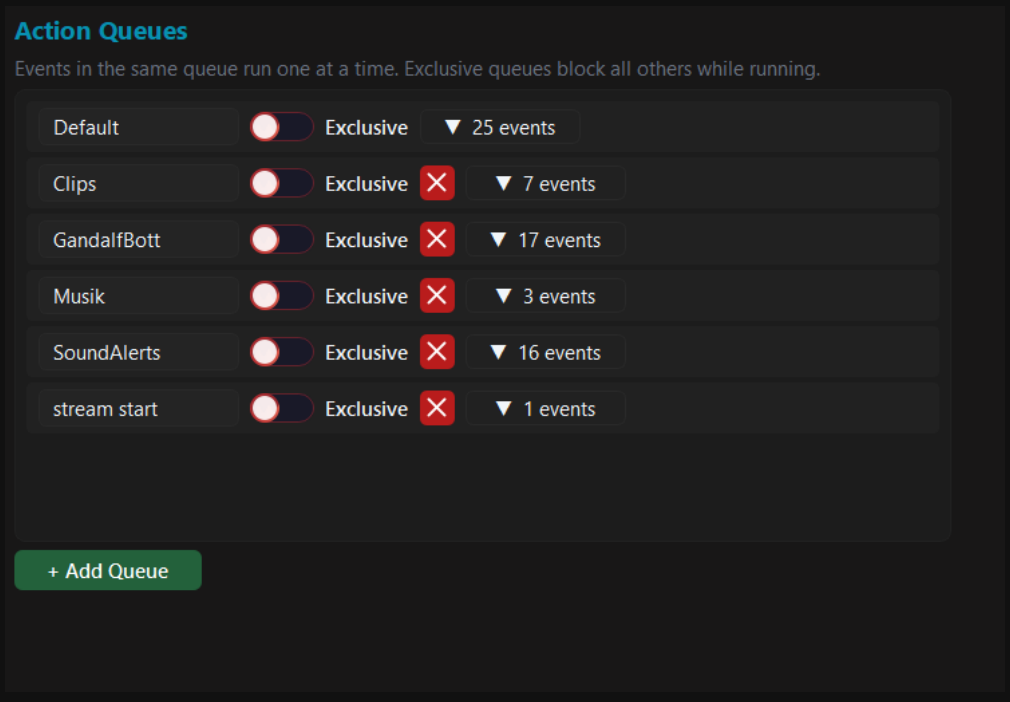
Der System Check prüft auf Knopfdruck alle Verbindungen und Einstellungen durch und zeigt dir, was steht und was nicht. Wenn etwas nicht funktioniert, ist das die erste Stelle, an der du nachsiehst.



Der System Check prüft Konfiguration, KI, Twitch und OBS

4.9 Queues

Hier legst du die Warteschlangen an, denen du später Events zuweist. Events in derselben Queue werden **nacheinander** abgearbeitet — niemals gleichzeitig.



Die Action Queues mit Exclusive-Schalter und Event-Zähler

- **+ Add Queue** — legt eine neue Queue an. Ein Limit gibt es nicht.
- **Exclusive** — eine exklusive Queue **blockiert alle anderen Queues**, solange sie läuft. Erst wenn sie durch ist, geht es überall weiter.

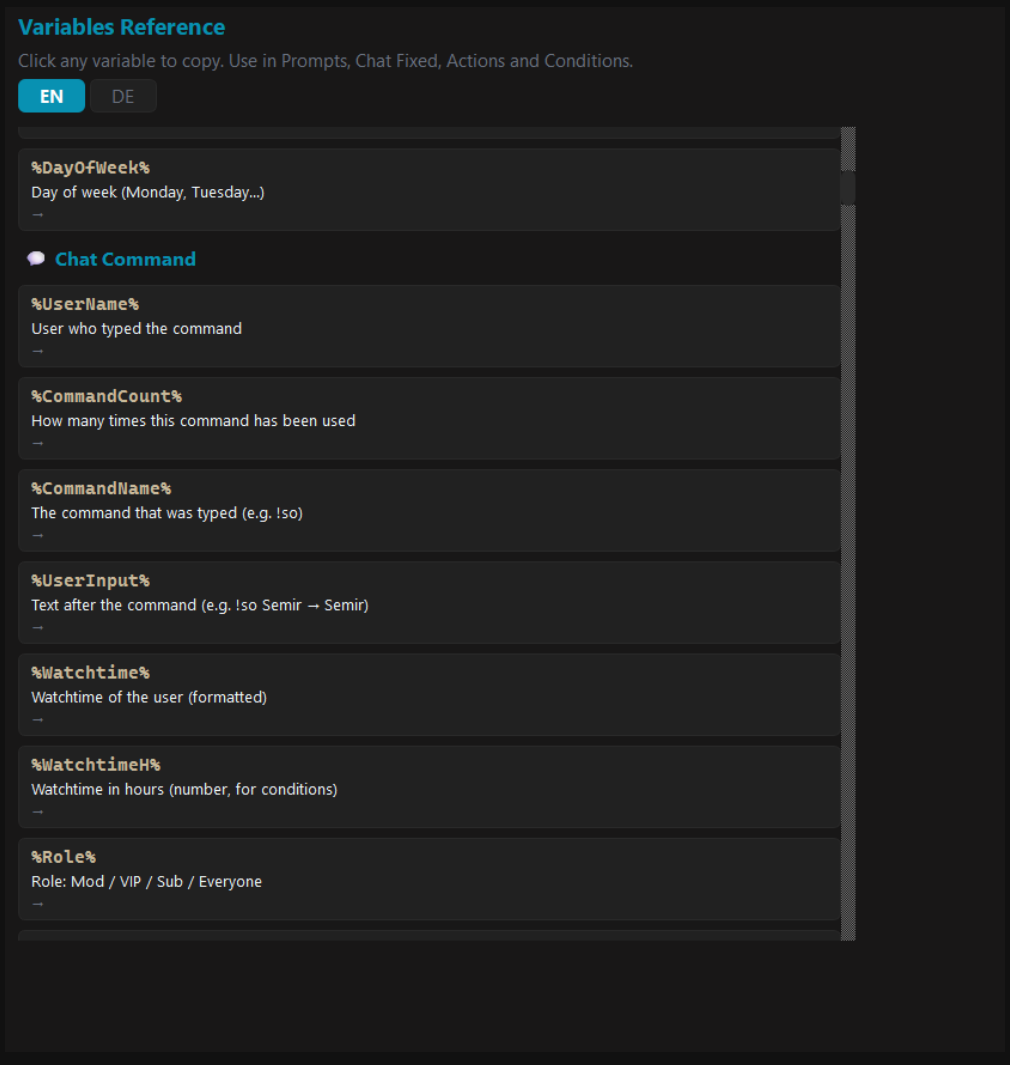
- Der **Pfeil** mit der Zahl (z. B. `▼ 7 events`) zeigt, wie viele Events dieser Queue zugewiesen sind. Klickst du auf ein Event in der Liste, springt ASTO-BOT direkt zu diesem Event.
- Das rote **X** löscht eine Queue.

> Die **Default-Queue** kann und darf **nicht gelöscht** werden — sie hat deshalb als einzige kein rotes X.

> 💡 Eigene Queues lohnen sich, um Dinge zu trennen: Sounds stören sich nicht mit Clips, Musik nicht mit Alerts. **Exclusive** nimmst du für etwas, das die ganze Bühne braucht — zum Beispiel einen Clip, während dessen nichts anderes dazwischenfunken soll.

4.10 Variables

Eine Nachschlageliste aller Variablen, die du in Prompts, Chat-Nachrichten, Aktionen und Bedingungen verwenden kannst. **Ein Klick auf eine Variable kopiert sie** in die Zwischenablage.



Die Variablen-Referenz, nach Bereichen sortiert

Für Datum und Uhrzeit gibt es `%Date%` (TT.MM.JJJJ), `%Time%` (HH:MM:SS) und den Wochentag — `%DayOfWeek%` auf Englisch (*Monday*), `%DayOfWeekDE%` auf Deutsch (*Montag*).

Wenn eine Variable keinen Wert hat

Nicht jede Variable ist in jeder Situation befüllt. Löst du ein Kanalpunkt-Event von Hand aus — über das Stream Deck oder den Websocket —, gab es gar keine Einlösung und damit auch keine Kosten. **Solche Variablen werden entfernt**, bevor der Text im Chat, in einem Overlay oder auf einer GDI-Textquelle landet. Deine Zuschauer sehen also nie ein rohes Prozent-Token.

Soll stattdessen etwas dastehen, schreibst du es mit einem Doppelpunkt direkt in die Variable hinein:

```
%RewardCost:0%       →  die Kosten, sonst 0
%UserName:Jemand%    →  der Name, sonst "Jemand"
%UserInput:–%        →  die Eingabe, sonst ein Strich
%RewardCost%         →  ohne Ersatztext: fällt ersatzlos weg
```

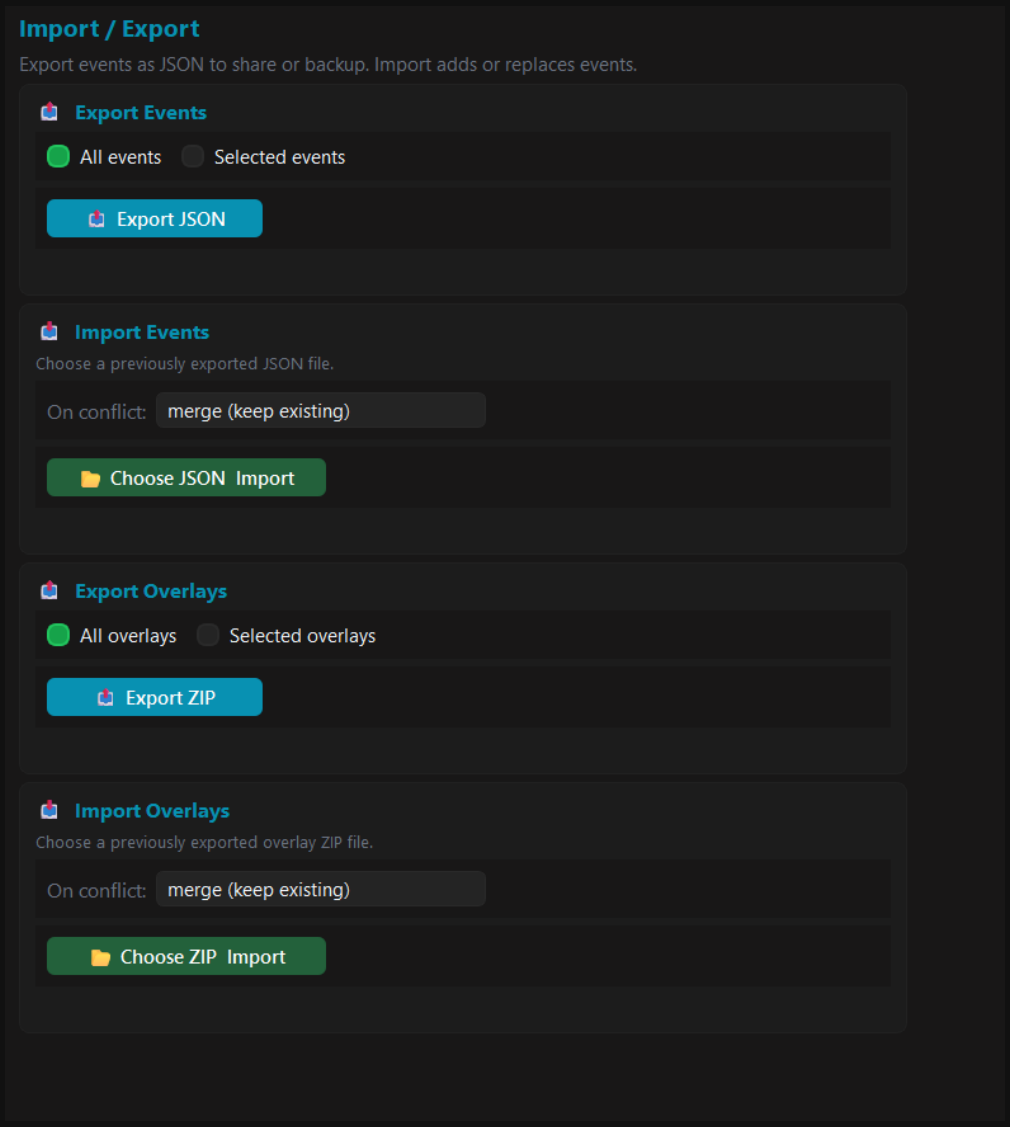
- Der Ersatztext beginnt nach dem **ersten** Doppelpunkt. Er darf also selbst welche enthalten — **%Startzeit:20:15%** ergibt notfalls *20:15*.
- Ein **leerer** Wert zählt als fehlend. Der Ersatztext greift also auch dann, wenn es die Variable zwar gibt, sie aber nichts enthält.
- Ein echter Wert hat immer Vorrang. Der Ersatztext erscheint nur, wenn wirklich nichts da ist.

> Vertippst du dich im Namen, bleibt die Variable **sichtbar** stehen. **%RewardCoast%** kennt ASTO-BOT nicht — und sichtbar ist an dieser Stelle der bessere Hinweis als spurlos verschwunden. Im Log findest du dazu eine Zeile.

> 💡 Das gilt überall, wo Variablen vorkommen: in Chat-Nachrichten, in AI-Prompts, in Overlay-Textelementen, auf GDI-Textquellen und in ASTOSCRIPT.

4.11 Import / Export

Hier gibst du Events und Overlays weiter oder holst sie dir — praktisch, um Dinge mit anderen Streamern zu teilen oder ein Backup einzelner Teile zu haben.



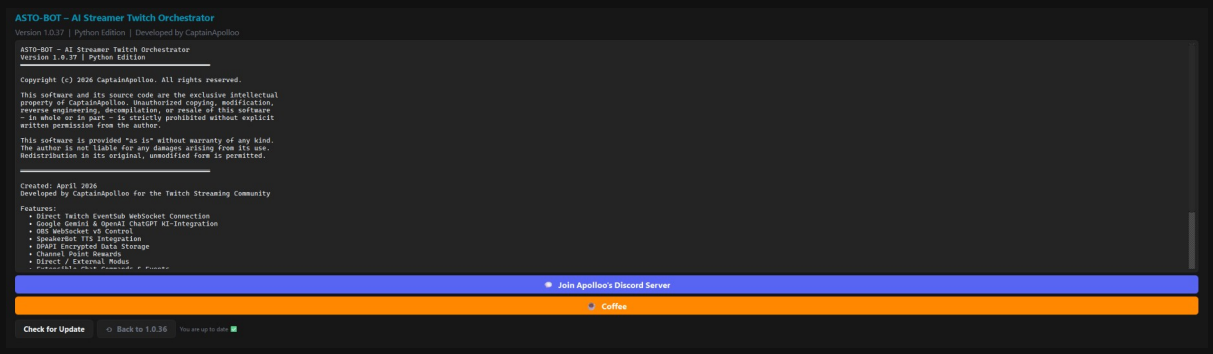
Import und Export von Events und Overlays

- **Export Events** — als JSON-Datei, entweder **alle** Events oder nur **ausgewählte**.
- **Export Overlays** — als ZIP-Datei, ebenfalls alle oder ausgewählte.
- **Import** — die passende JSON- bzw. ZIP-Datei auswählen und einlesen.

> Bei **On conflict** entscheidest du, was passiert, wenn es einen Eintrag schon gibt: **merge (keep existing)** behält das Vorhandene, **overwrite (replace existing)** überschreibt es.

4.12 About

Zeigt die Lizenz und die Programminformationen. Außerdem findest du hier:



Der About-Bereich mit Lizenz, Discord, Kaffee-Spende und Update-Prüfung

- **Join Apolloo's Discord Server** — Support und Neuigkeiten.
- **Coffee** — wenn du die Entwicklung unterstützen möchtest.
- **Check for Update** — sucht nach einer neuen Version.
- □ **Back to …** — stellt die vorherige Version wieder her. Der Knopf nennt die Zielversion direkt und erscheint nur, wenn eine Sicherung vorliegt (siehe unten).

Ist ein Update verfügbar, zeigt ASTO-BOT das außerdem oben in der Kopfleiste an, links neben dem Save-Knopf — du musst also nicht selbst nachsehen.

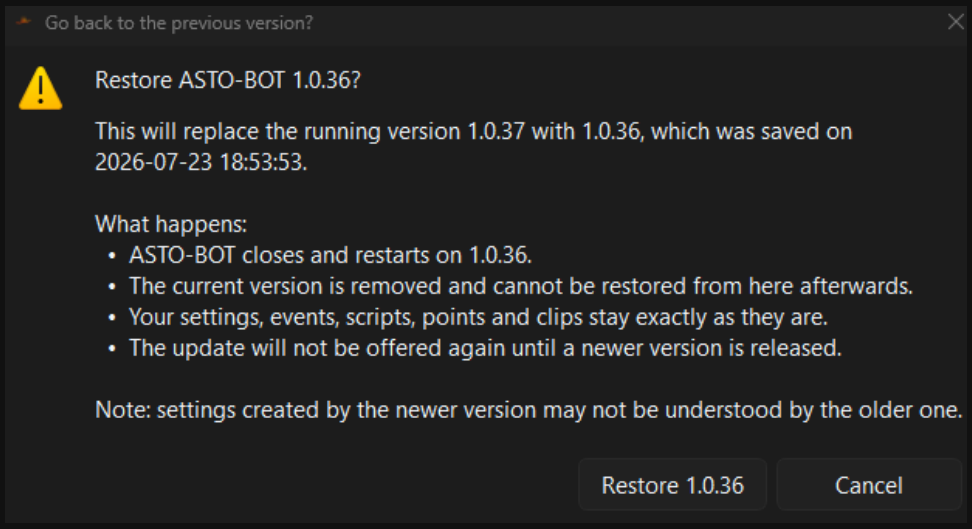
> Während das Update läuft, darf **nichts abgebrochen** werden. Warte, bis ASTO-BOT von sich aus neu startet.

> 💡 Wenn du den Ordner **runtime** und die EXE an einen anderen Ort verschoben hast, fragt ASTO-BOT beim nächsten Update nach, ob dieser neue Ort als Speicherverzeichnis beibehalten werden soll.

Zurück auf die vorherige Version

Vor jedem Update über **Check for Update** sichert ASTO-BOT den laufenden Programmstand. Zeigt sich danach, dass mit der neuen Version etwas nicht stimmt, brauchst du nichts von Hand herunterzuladen — der Weg zurück ist eingebaut.

Der Knopf □ **Back to …** sitzt direkt neben *Check for Update* und nennt die Version, auf die es zurückgeht. Er ist nur sichtbar, wenn wirklich eine Sicherung vorliegt — nach einer Neuinstallation und vor dem allerersten Update gibt es noch keine.



Die Rückfrage nennt beide Versionen und zählt auf, was passiert

Ein Klick öffnet zuerst diese Rückfrage. Bestätigst du sie, schließt ASTO-BOT, tauscht die Programmdateien und startet auf der älteren Version neu. Das dauert nur wenige Sekunden.

- Deine **Einstellungen, Events, Scripts, Punkte und Clips** bleiben unverändert — getauscht werden nur die Programmdateien.
- Die neuere Version wird dabei **entfernt**. Ein zweiter Schritt zurück ist also nicht möglich.
- Dieses Update wird **nicht erneut angeboten**, bis eine höhere Versionsnummer erscheint.

> Die Sicherung entsteht **nur bei Updates über den Knopf in ASTO-BOT**. Wer den Installer von Hand von GitHub lädt und drüberinstalliert, umgeht diesen Schritt — danach steht kein Rückweg zur Verfügung, beziehungsweise noch der aus dem letzten Update über den Knopf. Der Rollback geht dann entsprechend weiter zurück; welche Version es wird, steht immer auf dem Knopf und in der Rückfrage.

> Einstellungen, die eine **neuere** Version angelegt hat, kennt die ältere unter Umständen nicht. In aller Regel ignoriert sie diese einfach. Wenn du ganz sicher gehen willst, lege dir vorher über **Backup** (Abschnitt 4.13) einen Stand deiner Konfiguration weg.

> 💡 Die Sicherung liegt als ZIP im Programmordner unter **Backups\rollback** und belegt dort dauerhaft Platz. Bei jedem Update wird sie durch die jeweils aktuelle ersetzt — es gibt also immer nur genau eine.

4.13 Backup & Load Backup

Ganz unten in der Bereichsliste liegen zwei Knöpfe:

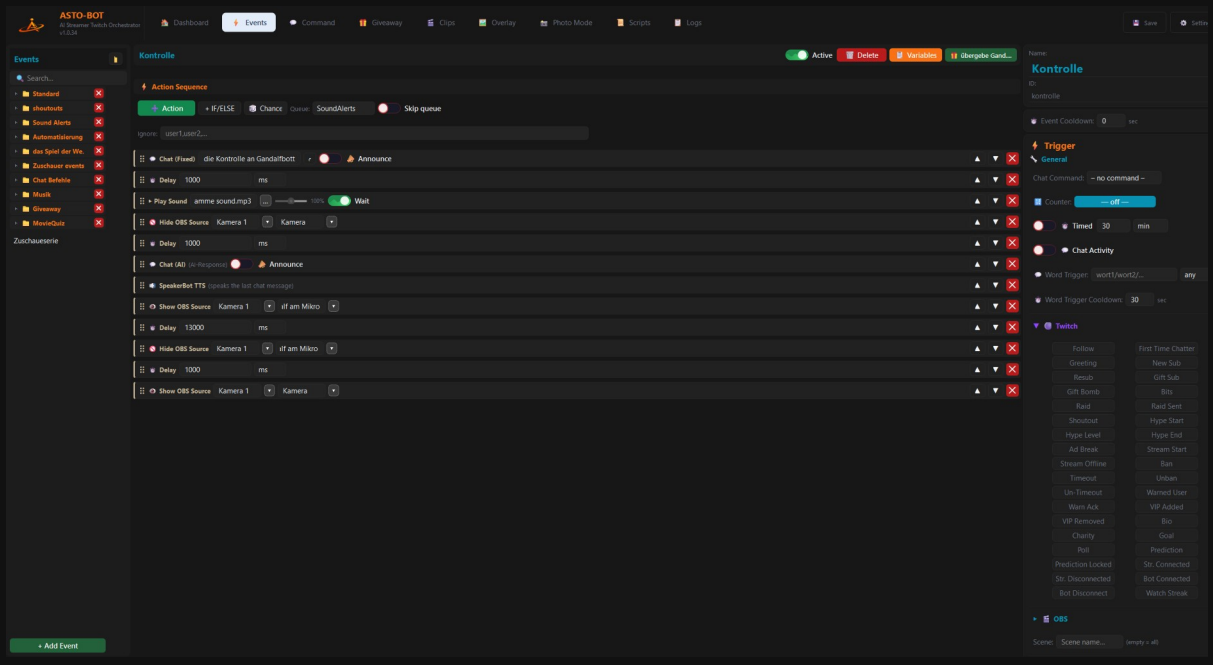
- **Backup** — erstellt sofort eine Sicherung aller Konfigurationsdateien.
- **Load Backup** — stellt einen früheren Stand wieder her.

Gesichert wird alles, was du dir eingerichtet hast: Events, Commands, Skripte samt ihrem Zustand, Overlays, Gewinnspiele, Punkte, globale Skript-Variablen, Statistiken und deine Discord-Webhooks. Um ältere Sicherungen musst du dich nicht kümmern, ASTO-BOT räumt sie von selbst weg.

> 💡 Mach ein Backup, bevor du größere Änderungen vornimmst. Es dauert einen Klick und erspart im Zweifel einen Abend Arbeit.

5. Events

Events sind das Herzstück von ASTO-BOT. Fast alles, was der Bot während deines Streams tut, wird von hier aus gesteuert. Ein Event besteht immer aus zwei Teilen: einem **Trigger** — der Auslöser, der bestimmt, **wann** etwas passiert — und einer **Action Sequence** — die Liste der Aktionen, die dann der Reihe nach abgearbeitet werden. Ein Follow kommt rein, ASTO-BOT spielt einen Sound, lässt die KI einen Dank formulieren, schickt ihn in den Chat und liest ihn vor: das ist ein Event.

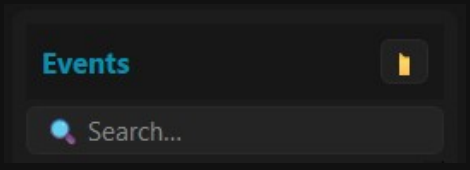


Die Events-Seite: links die Event-Liste, in der Mitte die Action Sequence, rechts die Trigger

> 💡 Nimm dir dieses Kapitel in Ruhe vor. Wenn du verstanden hast, wie ein Event aufgebaut ist, verstehst du ASTO-BOT.

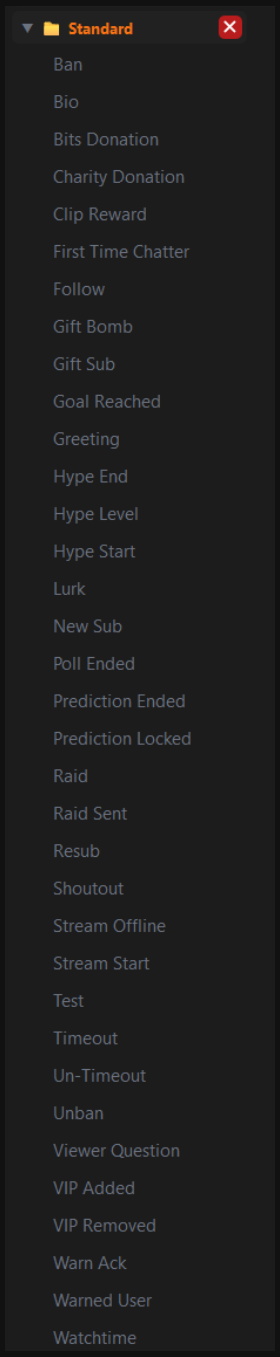
5.1 Aufbau der Events-Seite

Links liegt die **Event-Liste**. Ganz oben findest du den **Ordner-Knopf**, mit dem du eine neue Gruppe anlegst, darunter ein **Suchfeld**. Ganz unten sitzt **+ Add Event**.



Ordner-Knopf und Suchfeld über der Event-Liste

ASTO-BOT bringt eine **Standard-Gruppe** mit fertigen Beispiel-Events mit — Follow, Resub, Raid, Bits und so weiter. Das sind **Empfehlungen**, keine Pflicht. Du kannst sie benutzen, umbauen oder löschen.



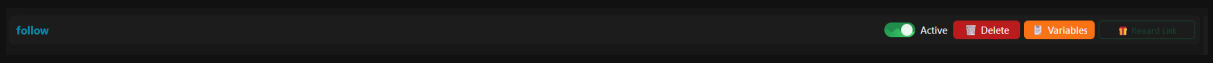
Die Standard-Gruppe mit den mitgelieferten Beispiel-Events

- Mit dem **Ordner-Knopf** legst du eigene **Gruppen** an — zum Beispiel *Sound Alerts*, *Shoutouts*, *Musik*.
- Ein Event verschiebst du per **Rechtsklick** in eine andere Gruppe — oder einfach per **Drag & Drop**.
- Per Drag & Drop ziehst du ein Event auch wieder **komplett aus einer Gruppe heraus**.
- **+ Add Event** legt ein neues, leeres Event an.

> 💡 Gruppen sind reine Ordnung — sie ändern nichts am Verhalten. Bei 50 Events willst du sie trotzdem haben.

5.2 Die Kopfzeile eines Events

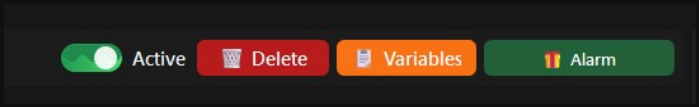
Wählst du ein Event aus, erscheint oben eine schmale Leiste mit dem Namen und vier Schaltern.



Die Kopfzeile eines Events — hier ist kein Reward verknüpft

- **Active** — schaltet das Event ein und aus. Ist es aus, passiert nichts, egal was der Trigger meldet.
- **Delete** — löscht das Event. Mit Sicherheitsabfrage.
- **Variables** — öffnet die Variablen-Übersicht (siehe unten).
- **Reward Link** — verknüpft das Event mit einem Kanalpunkte-Reward.

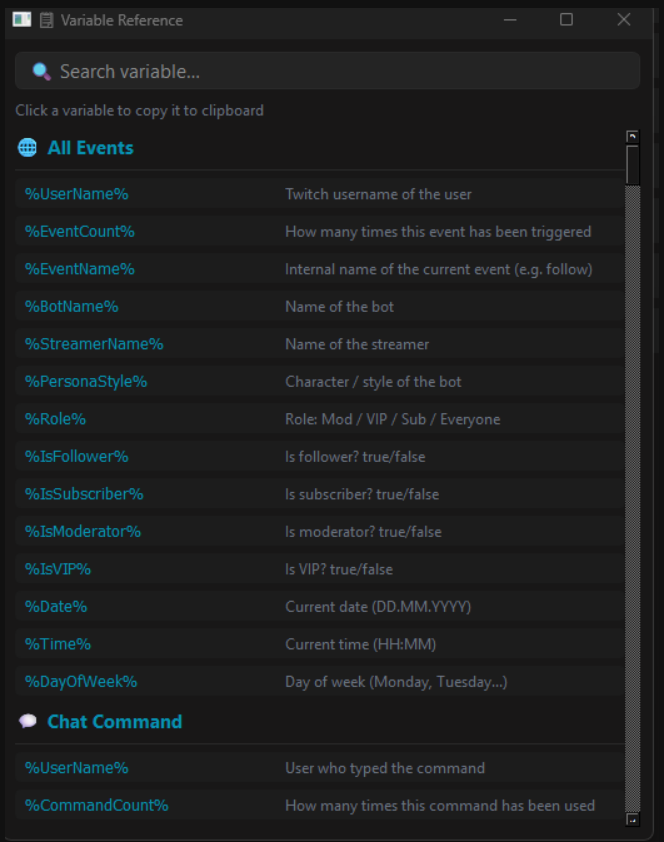
Am **Reward-Link**-Knopf erkennst du sofort, woran du bist: Steht dort dunkelgrün **Reward Link**, ist **kein** Reward verknüpft. Ist einer verknüpft, zeigt der Knopf stattdessen den **Namen des Rewards** an.



Hier hängt der Reward „Alarm“ am Event — der Knopf zeigt den Namen

Das Variables-Fenster

Variables öffnet eine Liste **aller** Variablen, die ASTO-BOT kennt — nach Kategorien sortiert, mit Suchfeld. Ein Klick auf eine Variable kopiert sie in die Zwischenablage, du kannst sie direkt in ein Feld oder in einen Prompt einfügen.

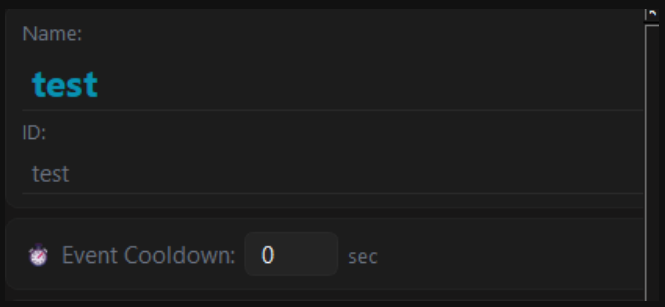


Die Variablen-Übersicht — Klick kopiert die Variable

> 💡 Manche Variablen tauchen mehrfach auf, zum Beispiel `%UserName%` oder `%UserInput%`. Das ist kein Fehler: Sie kommen in mehreren Kategorien vor und bedeuten dort jeweils dasselbe.

Name, ID und Event Cooldown

Rechts oben im Panel stehen **Name** und **ID** des Events. Die ID ist in der Regel der Name in Kleinbuchstaben, Leerzeichen werden zu _. Du brauchst sie, wenn du das Event von außen per **ASTOSCRIPT** oder **Websocket** auslösen willst (siehe Kapitel Settings → Websocket).



Name, ID und Event Cooldown — rechts oben im Trigger-Bereich

Darunter sitzt der **Event Cooldown** in Sekunden. Er sorgt dafür, dass genau **dieses** Event nach dem Auslösen eine Weile nicht wieder feuert. In den meisten Fällen brauchst du ihn nicht, `0` ist völlig in Ordnung.

> Jedes Event hat seinen **eigenen** Cooldown. Es gibt **keinen** übergreifenden Cooldown für alle Events — wenn du mehrere Events bremsen willst, musst du das bei jedem einzeln eintragen.

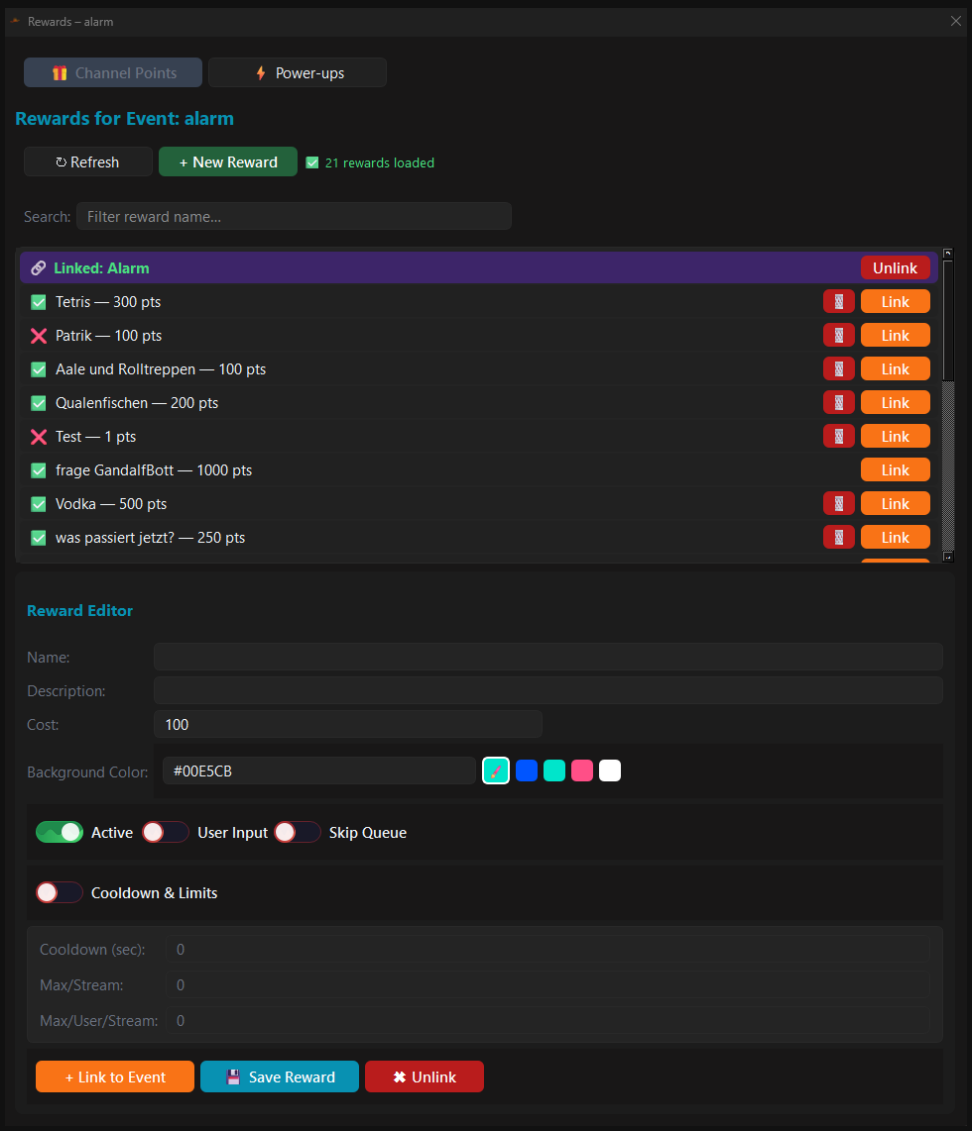
Benennst du **Name** oder **ID** um und lässt das andere stehen, erscheint neben dem Feld ein kleines ⚠. Fahr mit der Maus darüber, dann steht dort, was nicht zusammenpasst. Sobald beide wieder gleich heißen, verschwindet es von selbst.

> Änderst du die **ID**, wandert der **Prompt** des Events automatisch mit. Früher hing er an der alten ID und war nach dem Umbenennen scheinbar weg.

5.3 Rewards: Kanalpunkte und Power-ups

Der **Reward-Link**-Knopf öffnet ein eigenes Fenster mit zwei Reitern: **Channel Points** und **Power-ups**.

Channel Points



Das Reward-Fenster mit allen Kanalpunkte-Rewards des Kanals

- **Refresh** holt die Reward-Liste frisch von Twitch.
- **+ New Reward** legt einen neuen Reward an.
- Das **Suchfeld** filtert die Liste nach dem Namen.
- Der **grüne Haken** vor dem Namen bedeutet: Reward ist aktiv. Ein **rotes X** bedeutet: Reward ist deaktiviert.
- **Link** (der orange Knopf) verknüpft den Reward mit dem Event, **Unlink** löst die Verknüpfung wieder.

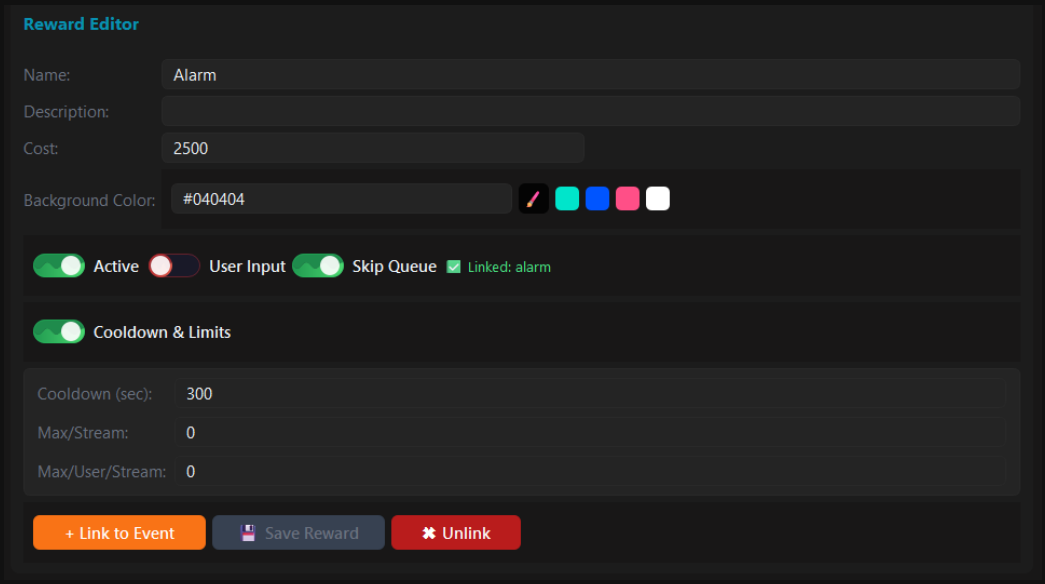
Der **Papierkorb** erscheint nur bei Rewards, die **ASTO-BOT selbst** erstellt hat. Rewards, die von Twitch oder von einem anderen Tool stammen, kannst du nur **verknüpfen** — bearbeiten oder löschen geht nicht. Das ist eine Einschränkung von Twitch, nicht von ASTO-BOT.

💡 **Die wichtigste Regel:** Ein Event kann **einen** Reward haben — aber ein Reward kann an **beliebig vielen** Events hängen. Ein Reward kann also mehrere Dinge gleichzeitig auslösen.

Der Reward Editor

Im unteren Teil des Fensters liegt der **Reward Editor**. Hier baust du einen Reward zusammen oder änderst einen bestehenden — vorausgesetzt, ASTO-BOT hat ihn erstellt.

Ist an dem Event bereits ein Reward verknüpft, steht er beim Öffnen des Fensters **schon im Editor** — du musst ihn nicht erst aus der Liste heraussuchen. Wer den Reward Editor aufmacht, will fast immer genau diesen einen anpassen.



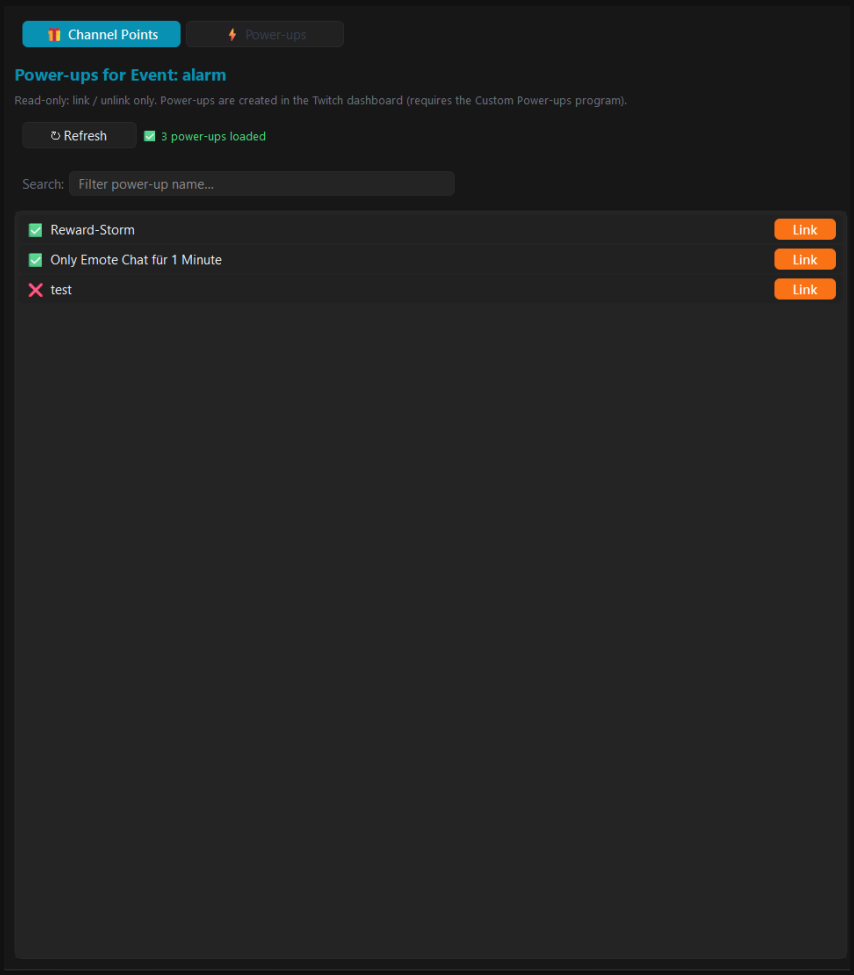
Der Reward Editor mit Kosten, Farbe, Cooldown und Limits

- **Name**, **Description** und **Cost** (die Kanalpunkte) — das, was der Zuschauer auf Twitch sieht.
- **Background Color** — als Hex-Wert oder über die Farbfelder daneben.
- **Active** — schaltet den Reward auf Twitch ein oder aus.
- **User Input** — der Zuschauer kann beim Einlösen einen Text mitgeben. Der landet in `%UserInput%`.
- **Skip Queue** — die Einlösung überspringt die Belohnungs-Warteschlange im Twitch-Dashboard und wird sofort bestätigt.
- **Cooldown & Limits** — Schalter, darunter: **Cooldown (sec)**, **Max/Stream** (wie oft insgesamt pro Stream) und **Max/User/Stream** (wie oft pro Zuschauer pro Stream). `0` heißt jeweils **unbegrenzt**.
- Unten: **+ Link to Event**, **Save Reward** und **Unlink**.

> Drücke **Save Reward**, **bevor** du das Fenster schließt. Sonst sind deine Änderungen weg.

Power-ups

Der zweite Reiter zeigt die **Power-ups** deines Kanals. Hier gibt es nur **Link** und **Unlink** — mehr nicht.

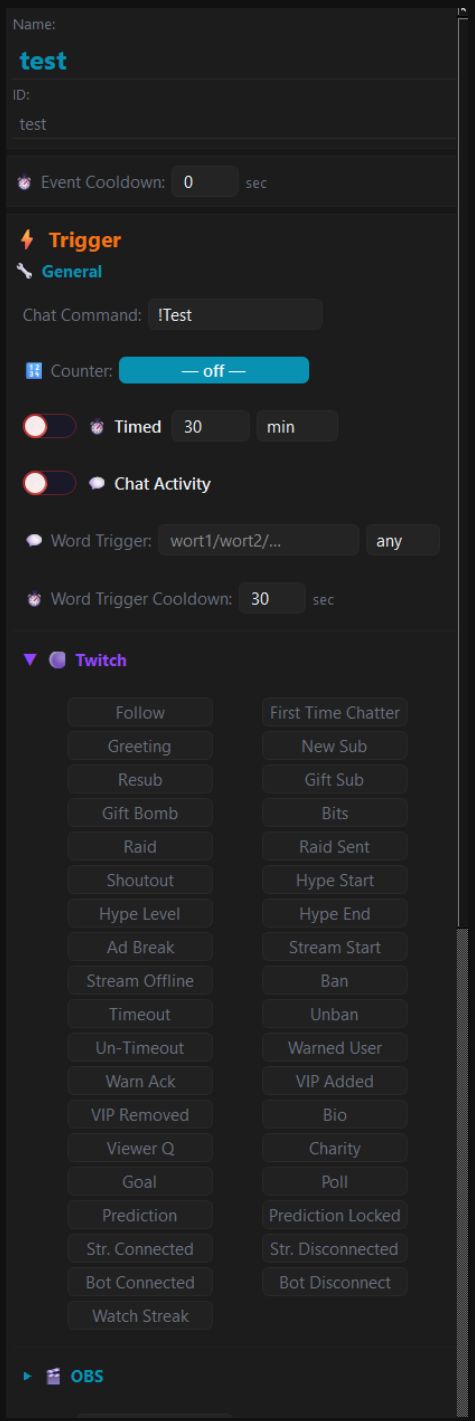


Power-ups lassen sich nur verknüpfen, nicht bearbeiten

> Power-ups werden im **Twitch-Dashboard** erstellt, ASTO-BOT kann sie nicht anlegen und nicht ändern — nur darauf reagieren.

5.4 Trigger — wann feuert das Event?

Der Trigger-Bereich liegt rechts im Panel, unter Name, ID und Cooldown. Er ist in Gruppen unterteilt: **General**, **Twitch**, **OBS**, **Giveaway** und **Music**. Du kannst mehrere Trigger gleichzeitig setzen — dann feuert das Event, sobald **einer** davon zutrifft.

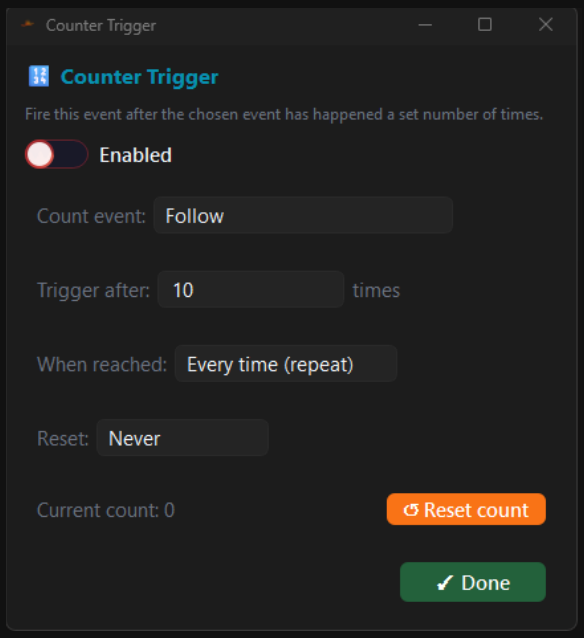


Der Trigger-Bereich eines Events

5.4.1 General

- **Chat Command** — das Event hängt an einem Chat-Befehl. Die Befehle selbst legst du im Bereich **Command** an (siehe Kapitel Command), hier wählst du nur aus.
- **Timed** — das Event feuert in einem festen Zeitabstand. Die Einheit ist umschaltbar: Sekunden, Minuten, Stunden oder Tage. Ideal für wiederkehrende Hinweise.

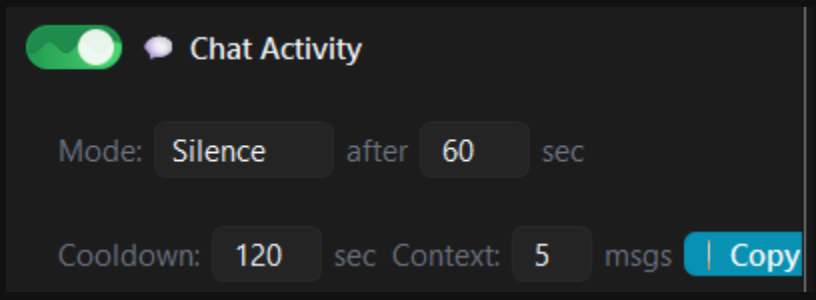
Counter — zählt, wie oft ein **anderes** Event ausgelöst wurde, und feuert dann. Ein Klick öffnet ein eigenes Fenster:



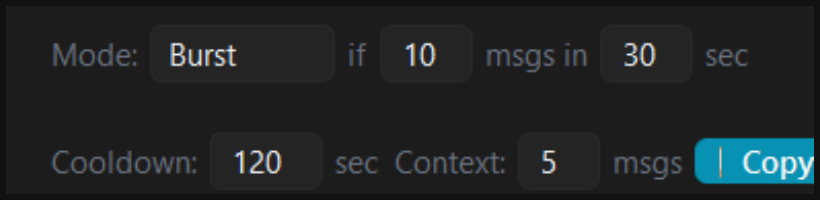
Der Counter-Trigger — hier feuert das Event nach jedem 10. Follow

- **Enabled** — Counter ein/aus.
- **Count event** — welches Event gezählt wird, zum Beispiel *Follow*.
- **Trigger after: N times** — nach wie vielen Malen es losgeht.
- **When reached:** `Every time (repeat)` feuert immer wieder — `Only once` feuert genau **ein einziges Mal** und nie wieder.
- **Reset:** `Never` behält den Zähler für immer — `per stream` setzt ihn nach jedem Stream zurück.
- **Current count** zeigt den aktuellen Stand, **Reset count** setzt ihn sofort auf 0.

Chat Activity — reagiert auf die Aktivität im Chat. Es gibt zwei Modi:



Silence — feuert, wenn 60 Sekunden lang niemand geschrieben hat



Burst — feuert, wenn 10 Nachrichten in 30 Sekunden kommen

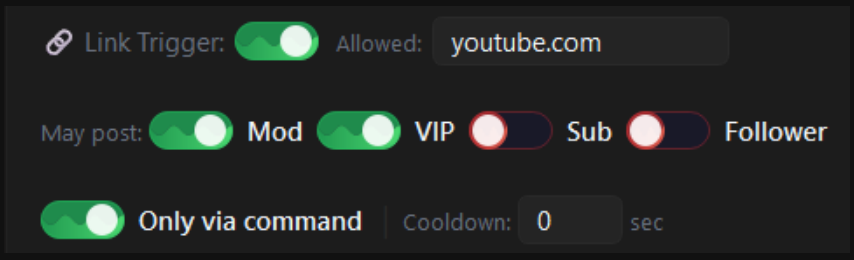
- **Silence** — feuert, wenn X Sekunden lang **niemand** geschrieben hat. Gut, um einen eingeschlafenen Chat aufzuwecken.
- **Burst** — feuert, wenn viele Nachrichten in kurzer Zeit kommen, zum Beispiel 10 Nachrichten in 30 Sekunden.

- **Cooldown** verhindert, dass der Trigger ständig nachfeuert. **Context** legt fest, wie viele der letzten Nachrichten ASTO-BOT der KI mitgibt.

Word Trigger — reagiert auf bestimmte Wörter im Chat. Mehrere Wörter trennst du mit `/`. Der Modus daneben entscheidet: **`any`** — eines der Wörter reicht. **`all`** — alle Wörter müssen in der Nachricht vorkommen. Der **Word Trigger Cooldown** darunter gilt nur für diesen Trigger.

> Trägst du nur ein `*` ein, werden **alle** Wörter akzeptiert — das Event feuert dann bei **jeder** Chat-Nachricht. Das ist mächtig, aber gefährlich: In Verbindung mit einer KI-Antwort oder einem Sound bedeutet das bei einem lebhaften Chat Dauerfeuer. Nur mit Bedacht und immer mit Cooldown verwenden.

Link Trigger — reagiert auf Adressen im Chat. Gedacht ist er gegen Werbung: Jemand schreibt einen Link, das Event feuert, und was dann passiert, entscheidest du in der Action Sequence — Nachricht löschen, verwarnen, Timeout, oder auch nur eine freundliche Ansage im Chat.



Der Link-Trigger mit Erlaubnisliste, Ausnahmen und dem strengen Modus

- **Link Trigger** — der Schalter ganz links schaltet den Trigger für dieses Event ein.
- **Allowed** — die Seiten, die erlaubt sind, kommagetrennt. Was hier steht, löst nicht aus.
- **May post** — Schalter für **Mod**, **VIP**, **Sub** und **Follower**. Wer hier an ist, darf Links posten, ohne dass das Event feuert. Du selbst und dein Bot lösen nie aus, dafür gibt es keinen Schalter.
- **Cooldown** — ein eigener Cooldown in Sekunden, unabhängig von dem des Events.
- **Only via command** — der strenge Modus, siehe unten.

Only via command dreht die Regel um: Erlaubt ist dann **nur noch** die Kombination aus dem Chat-Befehl dieses Events und einem Link von deiner Liste, also zum Beispiel `!link https://youtube.com/…`. Alles andere gilt als Verstoß und lässt das Event feuern — der erlaubte Link **ohne** Befehl genauso wie der Befehl **ohne** Link, ein Link von einer fremden Seite, oder gleich mehrere Links in einer Nachricht. Der Befehl ist in diesem Modus also die Erlaubnis, einen Link zu posten, und nicht mehr der Auslöser: Stimmt die Kombination, passiert nichts und der Link bleibt einfach stehen.

> Verglichen wird immer der **Domainname**, nie die ganze Adresse — sonst müsstest du jedes einzelne Video eintragen. Und aufgepasst: **youtu.be** ist eine andere Seite als **youtube.com**. Soll beides erlaubt sein, gehört beides in die Liste.

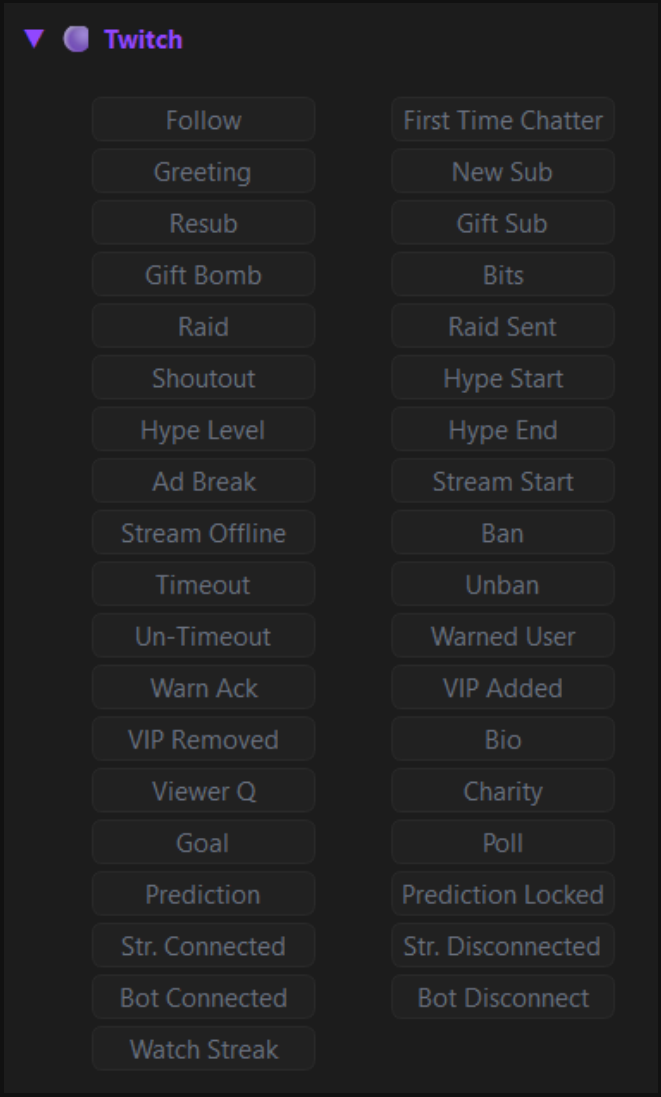
> 💡 Mit **%UserTriggerCount%** und einem IF/ELSE baust du dir eine Eskalation: beim ersten Mal eine Verwarnung, beim dritten Mal ein Timeout. Die Variable zählt, wie oft **dieser** Zuschauer **dieses** Event seit dem Streamstart ausgelöst hat.

Beim **Word Trigger** und beim **Link Trigger** sitzt rechts neben dem Eingabefeld ein □. Es öffnet dasselbe große Fenster wie bei den Prompts — dort hast du Platz für lange Listen, eine Zeile pro Eintrag, statt alles in ein schmales Feld zu quetschen.

Beide Felder verstehen außerdem **Variablen**. Du kannst also eine Wortliste oder eine Liste erlaubter Seiten an einer Stelle pflegen und in mehreren Events verwenden.

5.4.2 Twitch

Die größte Gruppe: **36 Trigger**, die direkt auf Twitch-Ereignisse hören. Ein Klick auf einen Knopf aktiviert ihn.



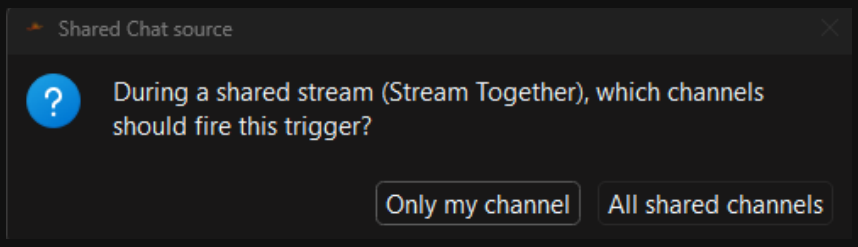
Die 36 Twitch-Trigger

Follow · First Time Chatter · Greeting · New Sub · Resub · Gift Sub · Gift Bomb · Bits · Raid · Raid Sent · Shoutout · Hype Start · Hype Level · Hype End · Ad Break · Stream Start · Stream Offline · Ban · Timeout · Unban · Un-Timeout · Warned User · Warn Ack · VIP Added · VIP Removed · Bio · Charity · Goal · Poll · Prediction · Prediction Locked · Str. Connected · Str. Disconnected · Bot Connected · Bot Disconnect · Watch Streak

Die meisten erklären sich selbst. Bei diesen lohnt sich ein zweiter Blick:

- **Shoutout** — jemand gibt **dir** einen Shoutout.
- **Raid** — du wirst geraidet. **Raid Sent** — du raidest jemanden.
- **Bio** — holt Informationen aus der Twitch-Biografie und den gespielten Spielen des Ziels. Die KI kann daraus eine Zusammenfassung schreiben.
- **Goal** — ein Twitch-Ziel wurde erreicht.
- **Watch Streak** — die Zuschauerserie, die ein Zuschauer erreicht hat.
- **Warn Ack** — eine Verwarnung wurde vom Zuschauer akzeptiert.
- **Str. Connected / Str. Disconnected** und **Bot Connected / Bot Disconnect** — die Verbindung des Streamer- bzw. des Bot-Kontos.

Shared Chat (Stream Together): Bei **Greeting**, **First Time Chatter**, **New Sub**, **Resub**, **Watch Streak** und den drei Hype-Train-Triggern (**Hype Start**, **Hype Level**, **Hype End**) fragt ASTO-BOT beim Anklicken nach, wie er sich bei einem geteilten Chat verhalten soll:

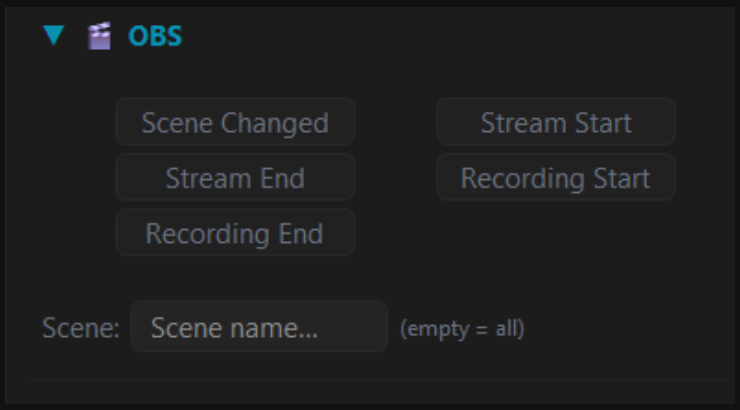


Bei geteiltem Chat fragt ASTO-BOT, welche Kanäle zählen sollen

- **Only my channel** — nur Zuschauer aus deinem eigenen Kanal lösen das Event aus.
- **All shared channels** — auch die Zuschauer aus den anderen Kanälen des geteilten Chats zählen.

5.4.3 OBS

Fünf Trigger, die auf OBS hören: **Scene Changed**, **Stream Start**, **Stream End**, **Recording Start** und **Recording End**.



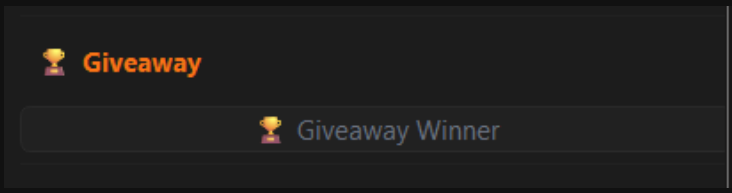
Die OBS-Trigger mit dem Scene-Feld

Im Feld **Scene:** darunter trägst du ein, bei welcher Szene der Trigger schalten soll. Lässt du es **leer**, gilt er für **jede** Szene.

Beim Trigger **Scene Changed** sitzt daneben ein Schalter **IN/OUT**. **IN** — die Voreinstellung — löst aus, wenn du **in** die Szene wechselst; **OUT** löst aus, wenn du sie wieder **verlässt**. Damit startest du etwas beim Betreten und beendest es beim Verlassen, ohne dafür ein zweites Event über Umwege zu bauen.

5.4.4 Giveaway

Giveaway Winner — feuert, sobald ein Giveaway abgeschlossen und ein Gewinner gezogen wurde. Details dazu findest du im Kapitel Giveaway.



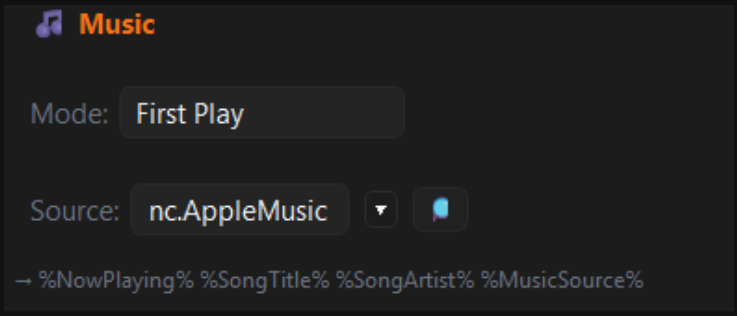
Der Giveaway-Trigger

Ausgelöst wird dieser Trigger vom **Draw**-Knopf auf der Giveaway-Seite, von der Aktion **Draw Giveaway** (Kapitel 5.6) und vom ASTOSCRIPT-Befehl `GIVEAWAY DRAW … ANNOUNCE`. Welches Giveaway gemeint ist, legst du direkt am Trigger fest.

> ⚠ Setz in **dasselbe** Event niemals zusätzlich die Aktion **Draw Giveaway** mit **demselben** Giveaway und eingeschaltetem **Announce** — das Event würde sich endlos selbst auslösen. Näheres in Kapitel 5.6, Gruppe *Giveaway*.

5.4.5 Music

Der Music-Trigger liest aus, was auf deinem PC gerade an Musik läuft — aus Spotify, Apple Music oder auch aus dem Browser.



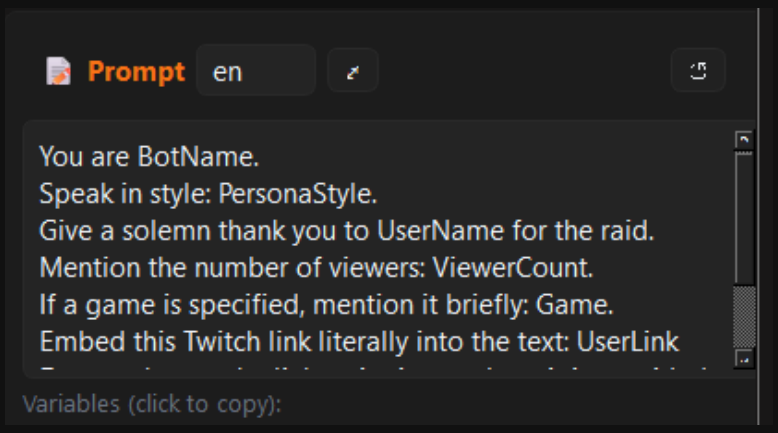
Der Music-Trigger mit Modus und Quelle

- **Mode** — `First Play` feuert beim ersten Abspielen, `On Song Change` bei jedem Songwechsel.
- **Source** — das Programm, das die Musik abspielt. Mit der **Lupe** sucht ASTO-BOT deinen PC nach Programmen ab, in denen gerade Musik läuft.
- Geliefert werden: `%NowPlaying%`, `%SongTitle%`, `%SongArtist%`, `%MusicSource%`.

> Es muss tatsächlich **Ton abgespielt werden**, sonst meldet Windows keine Informationen und ASTO-BOT findet das Programm nicht. Das ist mit Abstand die häufigste Fehlerquelle: Wer den Player nur geöffnet, aber pausiert hat, bekommt keine Daten.

5.4.6 AI Prompt

Ganz unten im Trigger-Bereich sitzt das **Prompt**-Feld. Das ist **kein Trigger**, sondern die Anweisung an die KI: Was soll sie sagen, wenn dieses Event feuert?

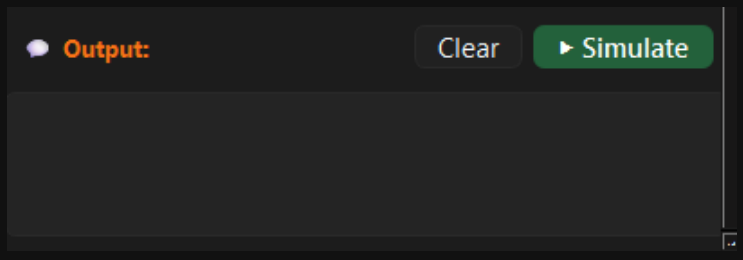


Der AI-Prompt eines Events, hier auf Englisch

- Bei den Standard-Events sind **deutsche und englische** Prompts hinterlegt. Mit dem kleinen Knopf (en) schaltest du die Sprache um.
- Der **kleine Pfeil** öffnet ein größeres Textfeld, in dem sich längere Prompts bequemer schreiben lassen.
- **Variablen sind auch im Prompt erlaubt** — `%UserName%`, `%CheerAmount%`, `%SongTitle%` und alle anderen.

5.4.7 Simulate

Ganz unten liegt das **Output**-Feld mit **Clear** und **Simulate**. **Simulate** löst das Event testweise aus, ohne dass ein Zuschauer etwas tun muss — die Antwort der KI erscheint direkt im Output-Feld.

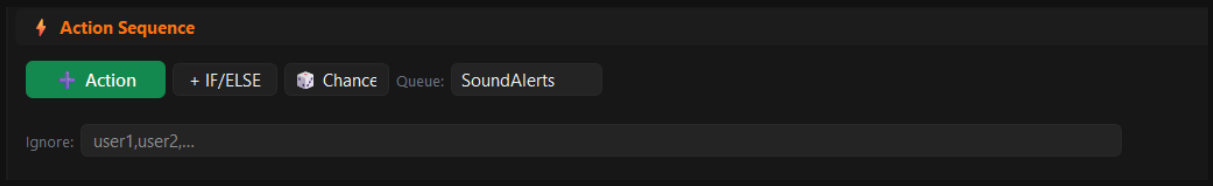


Mit Simulate testest du ein Event, ohne auf einen Zuschauer zu warten

> 💡 Baue ein Event, drücke **Simulate**, sieh dir das Ergebnis an, ändere den Prompt, drücke wieder Simulate. So bekommst du in fünf Minuten einen Ton hin, der zu deinem Kanal passt.

5.5 Die Action Sequence

Die Action Sequence ist die Liste der Aktionen, die abgearbeitet werden, wenn das Event feuert — von oben nach unten.



Die Kopfzeile der Action Sequence

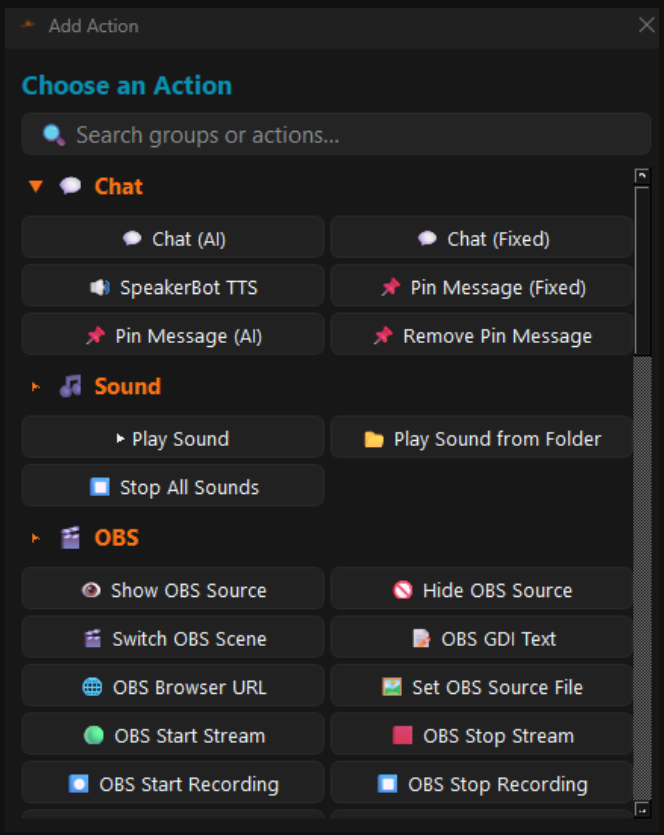
- **+ Action** — fügt eine Aktion hinzu.
- **+ IF/ELSE** — fügt eine Bedingung hinzu (Abschnitt 5.7).
- **Chance** — fügt einen Zufallsblock hinzu (Abschnitt 5.8).
- **Queue:** — in welcher Warteschlange das Event läuft. Die Queues legst du in den Settings an (Kapitel Settings → Queues).

- **Skip queue** — der Schalter daneben lässt das Event die Warteschlange **komplett überspringen**: Es startet sofort und belegt nie einen Platz. Ist er an, wird das Queue-Dropdown ausgegraut — die Zuordnung bleibt aber gespeichert und gilt wieder, sobald du den Schalter ausschaltest.
- **Ignore:** — Namen, kommagetrennt. Diese Zuschauer lösen das Event nicht aus. Gilt **nur für dieses eine Event**.

Skip queue ist für **lange** Events gedacht. Ein Event, das drei Minuten lang ein Bild einblendet, hält sonst drei Minuten lang jedes andere Event in seiner Queue auf. Mit dem Schalter brauchst du dafür keine eigene Warteschlange mehr.

> **Skip queue** hebt nur die Reihenfolge auf — **Cooldown** und **Ignore** gelten unverändert weiter. Bedenke aber: Feuert das Event erneut, während es noch läuft, laufen beide Durchgänge **gleichzeitig**. Bei Events mit Wartezeiten, Szenenwechseln oder Sounds kann das durcheinandergeraten — genau davor schützt sonst die Queue.

+ Action öffnet das Fenster **Choose an Action** mit allen verfügbaren Aktionen, nach Gruppen sortiert. Oben gibt es ein Suchfeld, und die Gruppen lassen sich ein- und ausklappen.



Das Fenster „Choose an Action“ mit allen Aktionsgruppen

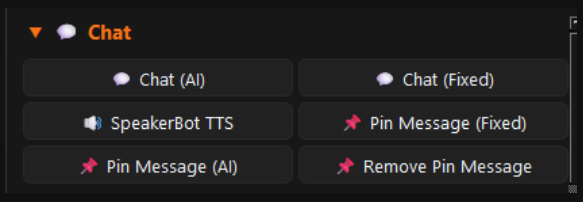
Eingefügte Aktionen sortierst du am **Griff** (⎇) per Drag & Drop, mit den Pfeilen ▲▼ verschiebst du sie um eine Position, und mit dem roten ✕ löschst du sie wieder.

> 💡 **Alle** Aktionen dürfen Variablen verwenden — auch die Prompts. Überall, wo ein Textfeld ist, kannst du `%UserName%`, `%UserInput%` und den Rest einsetzen.

5.6 Die Aktionsgruppen

ASTO-BOT bringt zwölf Aktionsgruppen mit: **Chat**, **Sound**, **OBS**, **Clips**, **System**, **Twitch**, **Moderation**, **Chat Mode**, **Discord**, **Points**, **Now Playing** und **Giveaway**.

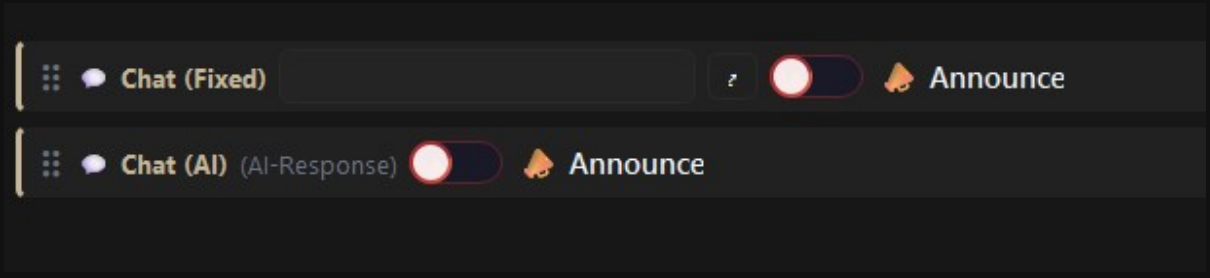
Chat



Die Gruppe Chat

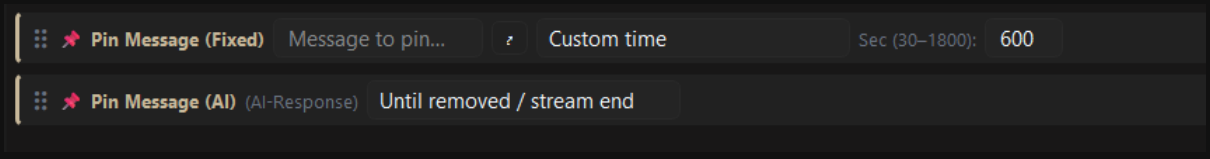
- **Chat (AI)** — schickt eine von der KI erzeugte Nachricht in den Chat. Es gibt **kein** eigenes Textfeld: Der Text kommt komplett aus dem **Prompt** des Events.
- **Chat (Fixed)** — eine feste Nachricht, die du selbst schreibst. Der kleine Pfeil öffnet ein größeres Textfeld.
- **SpeakerBot TTS** — schickt Text an SpeakerBot, damit er vorgelesen wird. Ohne eigenen Text liest er die letzte Chat-Aktion vor. Füllst du das **Textfeld** der Aktion aus, liest er stattdessen genau das — unabhängig davon, was vorher in den Chat ging. Mit dem **AI**-Schalter daneben bekommt die Aktion einen **eigenen** Prompt und kann damit etwas anderes formulieren als die Chat-Antwort desselben Events.
- **Pin Message (Fixed)** und **Pin Message (AI)** — pinnen die Nachricht bei Twitch an.
- **Remove Pin Message** — entfernt die angepinnte Nachricht wieder.

> Ist der **Prompt** des Events leer, kann **Chat (AI)** nichts erzeugen — dann kommt statt der Nachricht ein Hinweis der KI im Chat an. Der Prompt gehört also ausgefüllt.



Chat (Fixed) und Chat (AI) mit dem Announce-Schalter

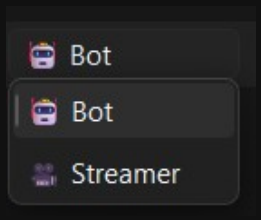
Beide Chat-Aktionen haben einen **Announce**-Schalter: Die Nachricht kommt bei Twitch dann als **Ankündigung** an — farbig hinterlegt und deutlich auffälliger als eine normale Chat-Zeile. Ist Announce aktiv, erscheinen daneben die **Farbfelder** für die Ankündigungsfarbe.



Pin Message (Fixed) mit fester Dauer, Pin Message (AI) bis zum Entfernen

Bei den Pin-Aktionen wählst du die Dauer: **Custom time** in Sekunden (mindestens 30, höchstens 1800) oder **Until removed / stream end** — dann bleibt die Nachricht hängen, bis du sie entfernst oder der Stream endet.

Wer schreibt: Bot oder Streamer? Alle vier Aktionen — **Chat (Fixed)**, **Chat (AI)**, **Pin Message (Fixed)** und **Pin Message (AI)** — haben eine eigene **Absender-Auswahl**. Damit legst du pro Aktion fest, unter welchem Namen die Nachricht im Chat erscheint. Du kannst innerhalb eines Events beliebig mischen: die Ansage vom Bot, die persönliche Reaktion vom Streamer-Konto.



Die Absender-Auswahl an einer Chat-Aktion

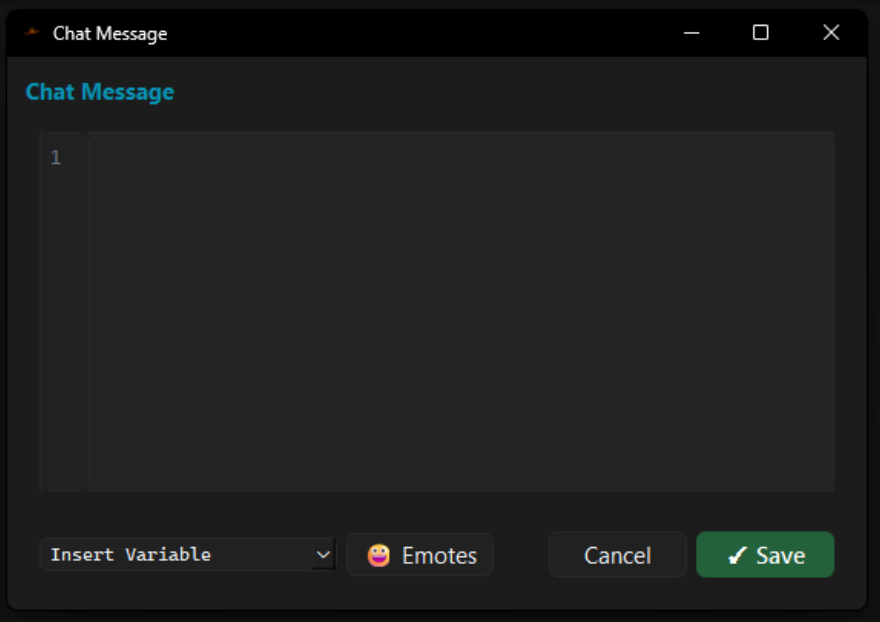
> Hast du **nur** dein Streamer-Konto verbunden, ist die Auswahl **ausgegraut** und alles wird vom Streamer-Konto geschickt. Ein Bot-Konto verbindest du in den **Settings** unter *Twitch*.

Die Wahl ist verbindlich: Steht sie auf **Streamer**, wird auch dann nicht heimlich auf den Bot ausgewichen, wenn das Senden fehlschlägt. Aktionen aus älteren Konfigurationen, an denen du die Auswahl nie angefasst hast, verhalten sich unverändert wie bisher.

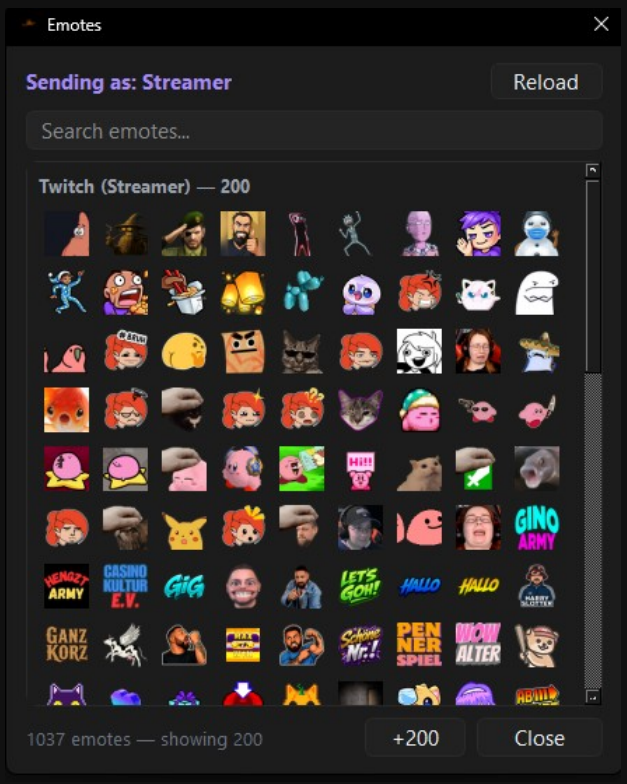
> ⚠ **Ankündigungen brauchen Moderatorenrechte.** Ist **Announce** eingeschaltet und der Absender ist der Bot, muss der Bot in deinem Kanal **Moderator** sein. Ist er es nicht, kommt die Ankündigung nicht an — in dem Fall stellst du die Aktion einfach auf **Streamer** um.

Emotes einfügen

In den Textfenstern von **Chat (Fixed)** und **Pin Message (Fixed)** liegt neben *Insert Variable* ein **Emotes**-Knopf. Er öffnet eine Auswahl aller Emotes, die das eingestellte Absender-Konto benutzen darf.



Der Emotes-Knopf neben Insert Variable



Der Emote-Picker mit Suchfeld und den Emotes des gewählten Kontos

- **Sending as** oben links zeigt, für welches Konto die Auswahl gilt. Schaltest du die Absender-Auswahl der Aktion um, ändert sich damit auch die angebotene Liste.
- Zuerst kommen die Emotes **deines eigenen Kanals**, dann Sub- und Follower-Emotes fremder Kanäle, dann die globalen Twitch-Emotes. Ganz unten stehen **BTTV / FFZ / 7TV**.
- Das **Suchfeld** filtert immer die vollständige Liste, nicht nur das gerade Sichtbare.
- Aus Geschwindigkeitsgründen werden 200 Emotes auf einmal gezeichnet. **+200** holt den nächsten Schwung.
- **Reload** lädt die Emotes neu — praktisch, wenn du gerade ein Abo abgeschlossen oder ein neues Kanal-Emote angelegt hast.
- Das Fenster bleibt nach einem Klick **offen**, damit du mehrere Emotes hintereinander einsetzen kannst.

> **Warum die Absender-Auswahl hier wichtig ist:** Twitch prüft beim Senden, ob das absendende Konto das Emote überhaupt besitzt. Schickt der Bot ein Sub-Emote deines Kanals, ohne dich abonniert zu haben, erscheint im Chat nur der **blanke Name** statt des Bildes. Der Picker zeigt deshalb von vornherein nur, was das gewählte Konto wirklich verwenden darf.
>
> **BTTV, FFZ und 7TV** sind keine Twitch-Emotes, sondern Ersetzungen im Browser des Zuschauers. Jedes Konto darf sie schreiben — sichtbar sind sie aber nur bei Zuschauern, die die passende Erweiterung installiert haben. Diese Liste füllt sich erst, wenn ASTO-BOT verbunden ist.

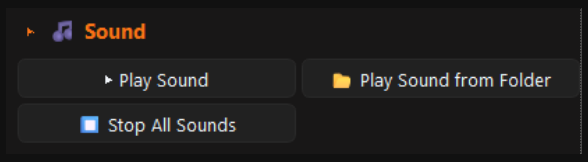
Auch der **Prompt Editor** hat den Emotes-Knopf. Dort werden die Namen nicht einzeln in den Text gestreut, sondern an eine Zeile *Only use these emotes:* angehängt. Als geschlossene Liste hält sich die KI deutlich häufiger daran, statt sich eigene Emotes auszudenken. Verbrauchen mehrere KI-Aktionen desselben Events den Prompt und stehen sie auf **verschiedenen** Absendern, zeigt der Picker nur die **Schnittmenge** — also das, was bei allen beteiligten Konten funktioniert.

> Für die Emote-Liste braucht ASTO-BOT die Twitch-Berechtigung `user:read:emotes`. Nach dem Update auf diese Version musst du deine Konten **einmal ab- und wieder anmelden**, sonst bleibt der Twitch-Teil der Auswahl leer. Ob etwas fehlt, steht beim Verbinden im **Log**.

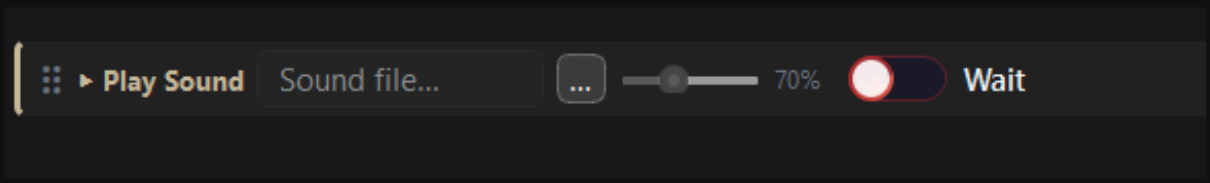
Ein Emote wandert nicht mit. Schaltest du die Absender-Auswahl einer Aktion nachträglich um, erscheint daneben ein ⚠. Es erinnert daran, dass die Emotes im Text zum neuen Konto vielleicht nicht mehr passen. Ein Klick darauf blendet es aus, spätestens beim nächsten Programmstart ist es weg.

> **Die wichtigste Regel bei TTS:** *SpeakerBot TTS* liest immer die **letzte Chat-Aktion** vor, die davor liegt — egal wie viele andere Aktionen dazwischenstehen. Zwei Chat-Aktionen direkt hintereinander, danach ein TTS: Es wird **nur die zweite** vorgelesen, die erste geht unter. Richtig ist es abwechselnd: Chat-Aktion → TTS → Chat-Aktion → TTS.

Sound

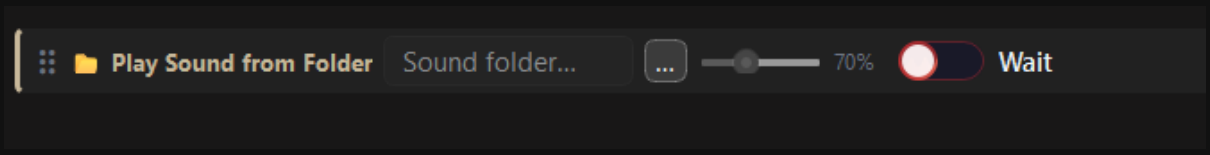


Die Gruppe Sound



Play Sound mit Dateiwahl, Lautstärke und Wait

- **Play Sound** — spielt eine Sounddatei von deinem PC. Die Datei wählst du über den **...**-Knopf, daneben liegen der **Lautstärke-Regler** und der **Wait**-Schalter.
- **Wait** — das Event **pausiert**, bis der Sound fertig ist, und macht erst dann mit der nächsten Aktion weiter.
- **Stop All Sounds** — unterbricht sofort **alle** laufenden Sounds, ohne Ausnahme.

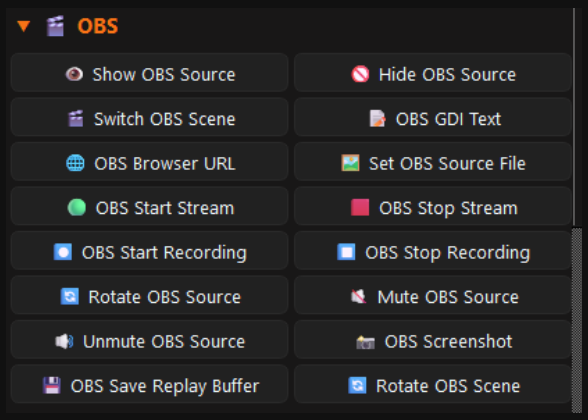


Play Sound from Folder — ein zufälliger Sound oder alle nacheinander

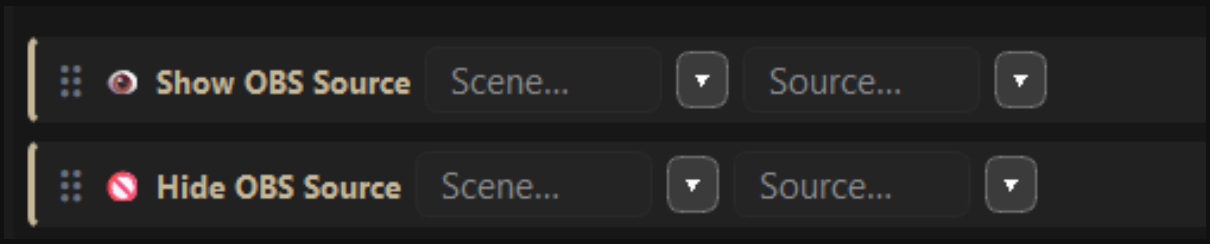
Play Sound from Folder spielt Sounds aus dem gewählten Ordner. Mit dem Schalter 🎲 **Random** — so ist es voreingestellt — kommt bei jedem Auslösen **ein** zufälliger Sound. Schaltest du ihn aus, laufen **alle** Sounds **nacheinander** in der gespeicherten Reihenfolge. Mit **Wait** geht es erst weiter, wenn der Sound beziehungsweise der letzte durch ist.

> 💡 Der **Lautstärke-Regler** wirkt erst beim **nächsten** Abspielen. Einen bereits laufenden Sound kannst du damit nicht leiser drehen.

OBS

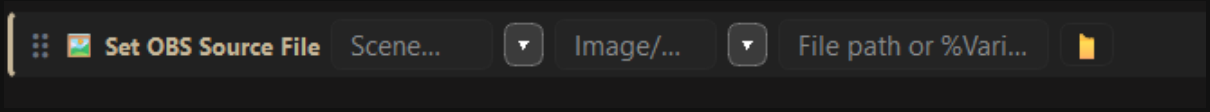


Die Gruppe OBS mit 14 Aktionen



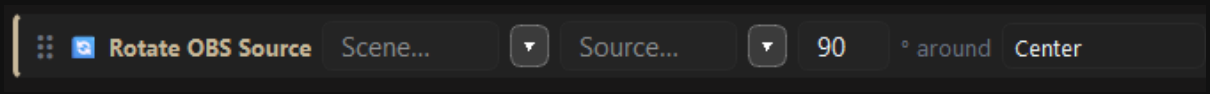
Show OBS Source und Hide OBS Source — Szene und Quelle auswählen

- **Show OBS Source** / **Hide OBS Source** — blendet eine Quelle ein oder aus. Du wählst Szene und Quelle.
- **Switch OBS Scene** — wechselt zur gewählten Szene.
- **OBS GDI Text** — schreibt Text in eine Text-(GDI+)-Quelle. Variablen sind erlaubt.
- **OBS Browser URL** — schreibt eine URL in eine Browser-Quelle.
- **OBS Start Stream** / **OBS Stop Stream** und **OBS Start Recording** / **OBS Stop Recording**.
- **Mute OBS Source** / **Unmute OBS Source** — stummschalten und wieder aufdrehen.



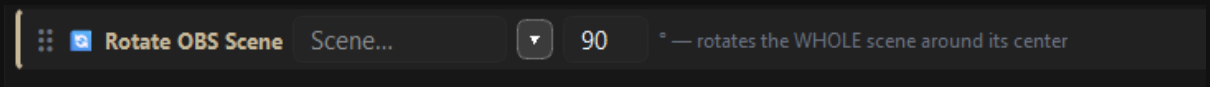
Set OBS Source File tauscht Bild oder Sound einer Quelle aus

Set OBS Source File setzt in einer **Image**-Quelle ein neues Bild oder in einer **Media**-Quelle einen neuen Sound von deinem PC. Du wählst Szene, Quelle und den Dateipfad — hier darf auch eine `%Variable%` stehen.



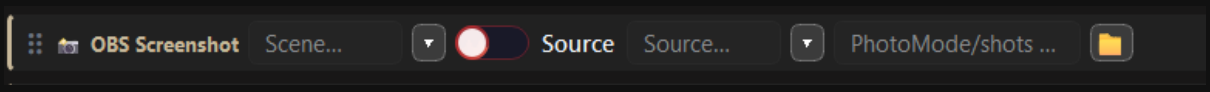
Rotate OBS Source dreht eine einzelne Quelle

> **Rotate OBS Source** mit Vorsicht verwenden: OBS dreht einzelne Quellen nicht sauber, das Ergebnis sieht oft nicht so aus, wie man es erwartet. Willst du auf Nummer sicher gehen, nimm **Rotate OBS Scene**.



Rotate OBS Scene dreht die ganze Szene um ihren Mittelpunkt

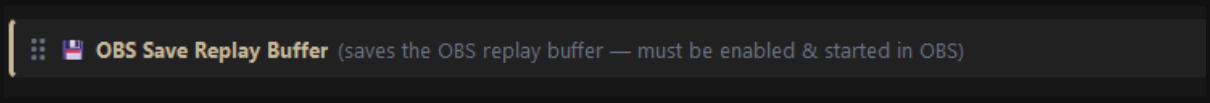
Rotate OBS Scene dreht die **komplette Szene** um ihren Mittelpunkt — und das zuverlässig.



OBS Screenshot — wahlweise die ganze Szene oder eine einzelne Quelle

OBS Screenshot macht ein Standbild. Du wählst die Szene, kannst mit dem Schalter aber auch auf eine einzelne **Source** umstellen. Rechts legst du den Zielordner fest.

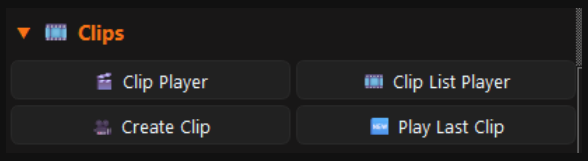
> **OBS Screenshot** hat nichts mit dem **Photo Mode** zu tun — das sind zwei verschiedene Dinge. Der Photo Mode hat ein eigenes Kapitel.



OBS Save Replay Buffer speichert die letzten Sekunden

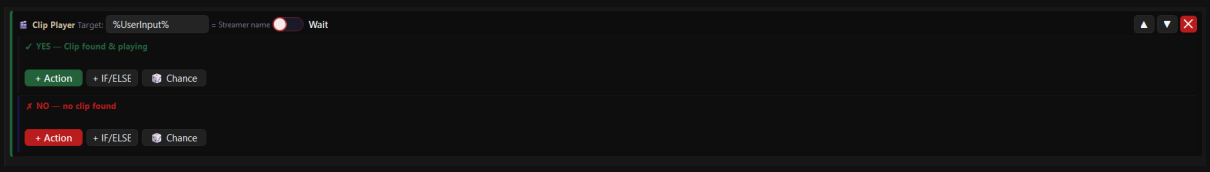
> **OBS Save Replay Buffer** funktioniert nur, wenn der Replay Buffer in den **OBS-Einstellungen aktiviert und gestartet** ist. Ist er nicht gestartet, passiert nichts.

Clips



Die Gruppe Clips

Drei der fünf Clip-Aktionen — **Clip Player**, **Clip List Player** und **Play Last Clip** — haben eine Besonderheit: Sie sind in zwei Zweige geteilt.

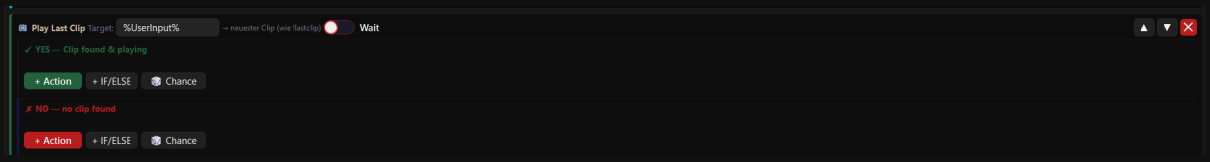


Der Clip Player mit YES- und NO-Zweig

- **YES — Clip found & playing** — hier kommen die Aktionen hinein, die laufen sollen, **wenn** ein Clip gefunden wurde.
- **NO — no clip found** — und hier die Aktionen für den Fall, dass **keiner** gefunden wurde.
- Beide Zweige haben ihre eigenen Knöpfe **+ Action**, **+ IF/ELSE** und **Chance**.

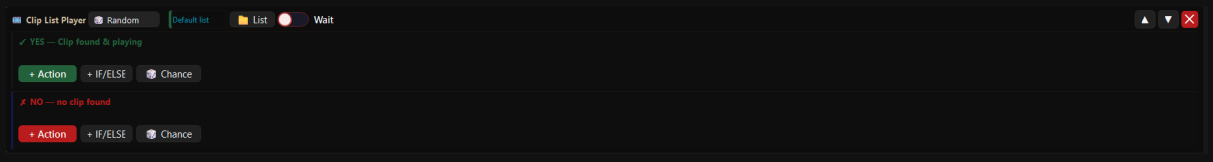
> 💡 Nutze den NO-Zweig. Ein Zuschauer löst einen Clip-Reward aus, es gibt aber keinen Clip — ohne NO-Zweig passiert dann einfach gar nichts, und der Zuschauer denkt, es sei kaputt. Eine kurze Chat-Nachricht reicht schon.

Clip Player spielt einen **zufälligen** Clip der Ziel-Person ab. Im Feld **Target** steht, wessen Clips gemeint sind: `%UserInput%` (der Zuschauer trägt beim Reward einen Streamer-Namen ein) oder `%UserName%` (die Person, die das Event auslöst). Der **Wait**-Schalter wartet, bis der Clip fertig ist — wird kein Clip gefunden, geht es sofort weiter.

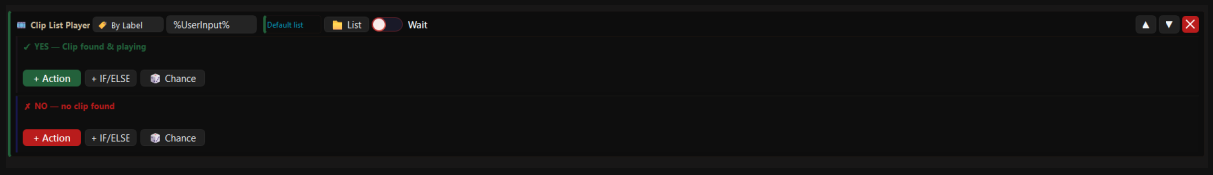


Play Last Clip spielt den neuesten Clip statt einen zufälligen

Play Last Clip funktioniert genauso, spielt aber immer den **neuesten** Clip — das, was `!lastclip` macht.

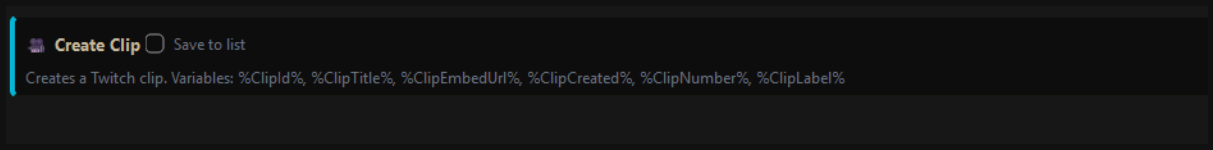


Clip List Player im Modus Random

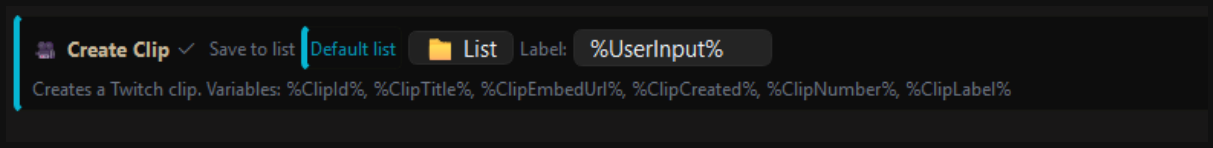


Clip List Player im Modus By Label

- **Clip List Player** spielt Clips aus einer intern angelegten **Liste**.
- Modus **Random** — ein zufälliger Clip aus der Liste.
- Modus **By Label** — ein **bestimmter** Clip. Entweder über eine Variable (zum Beispiel `%UserInput%`, wenn der Zuschauer den Clip-Namen in den Chat schreibt) oder fest eingetragen — dann läuft immer derselbe Clip.
- Der **List**-Knopf öffnet den **Clip List Manager**.



Create Clip — ohne Speichern in einer Liste

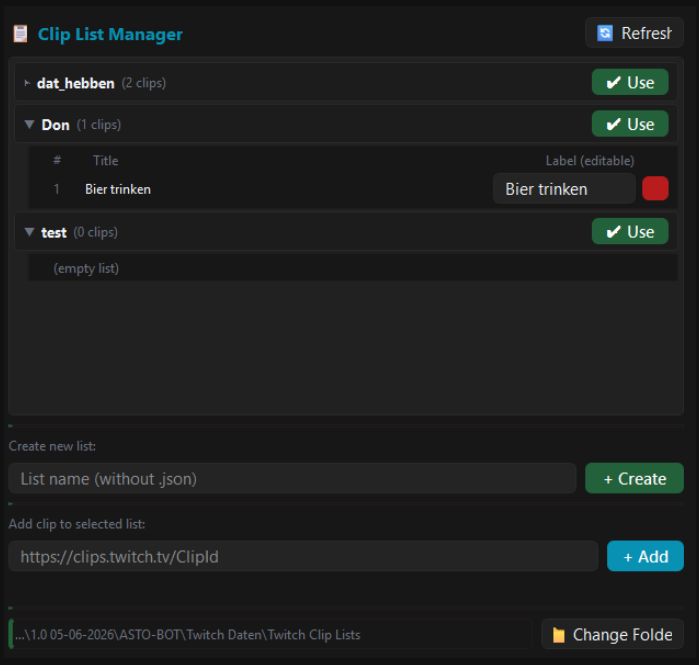


Create Clip mit „Save to list“ und einem Label aus %UserInput%

- **Create Clip** erstellt einen Clip auf Twitch.
- Mit dem Kästchen **Save to list** landet der Clip zusätzlich in einer Liste. Dann wählst du die Liste aus und vergibst ein **Label** — zum Beispiel `%UserInput%`, also den Namen, den der Zuschauer in den Chat schreibt.
- Geliefert werden: `%ClipId%`, `%ClipTitle%`, `%ClipEmbedUrl%`, `%ClipCreated%`, `%ClipNumber%` und `%ClipLabel%`.

Clip Playlist steuert die Playlist von der Clips-Seite (Kapitel 8.5): **Start**, **Stop** oder **Toggle**. Damit läuft sie zum Beispiel von selbst an, sobald du in eine bestimmte OBS-Szene wechselst — und hört wieder auf, wenn du sie verlässt.

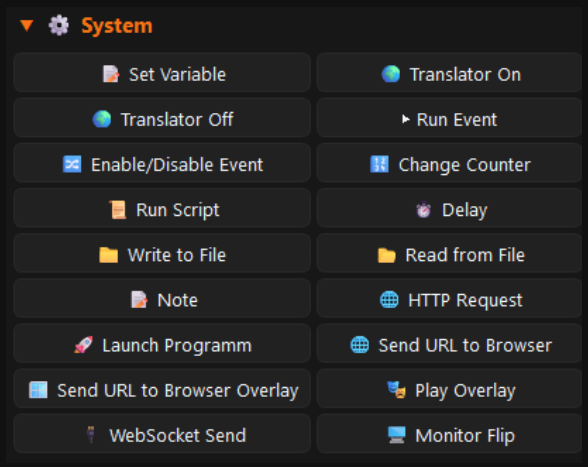
Der Clip List Manager



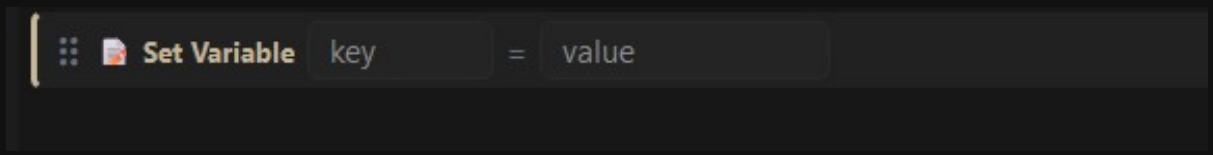
Der Clip List Manager mit aufgeklappter Liste und editierbaren Labels

- Du kannst **mehrere Listen** anlegen. **+ Create** legt eine neue an — der Name wird ohne `.json` eingetragen.
- **Use** wählt die Liste aus, mit der gearbeitet wird.
- Klappst du eine Liste auf, siehst du die Clips mit **#**, **Title** und **Label (editable)**. Das **Label ist jederzeit änderbar** — es ist der Name, über den *By Label* den Clip findet.
- Der **rote Knopf** löscht einen Clip aus der Liste.
- **Add clip to selected list:** Clip-URL (`https://clips.twitch.tv/ClipId`) einfügen und **+ Add** drücken. Alternativ trägt die Aktion **Create Clip** die Clips automatisch ein.
- **Change Folder** legt fest, wo die Listen gespeichert und woher sie geladen werden. **Refresh** lädt neu.

System

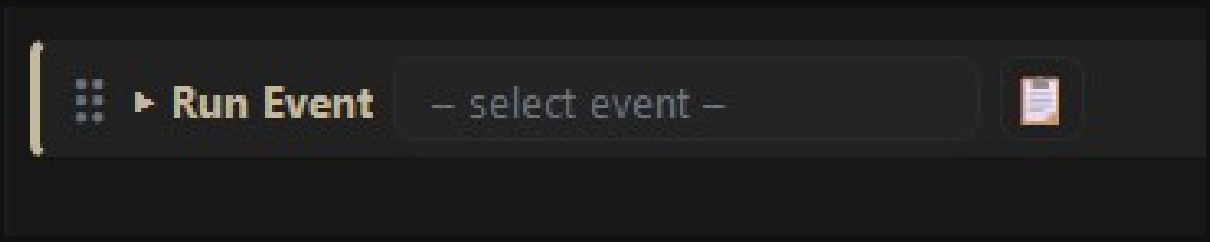


Die Gruppe System mit 12 Aktionen



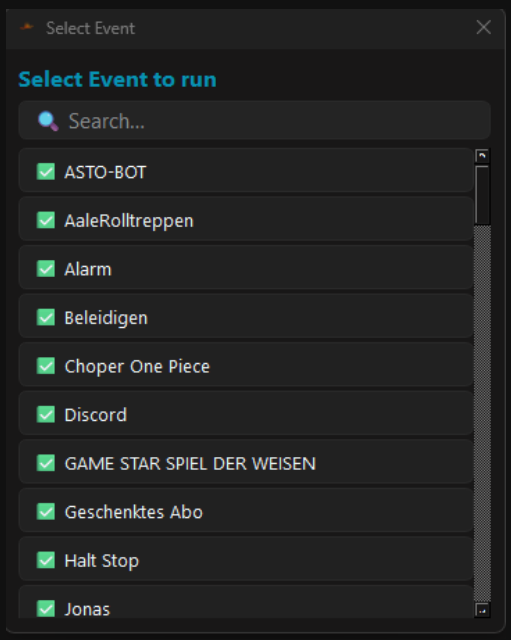
Set Variable — key = value

> **Set Variable** erstellt eine Variable, die **nur innerhalb dieses Events** gilt. Sie entsteht beim Start des Events und wird danach wieder gelöscht. Außerhalb des Events ist sie **nicht** aufrufbar.

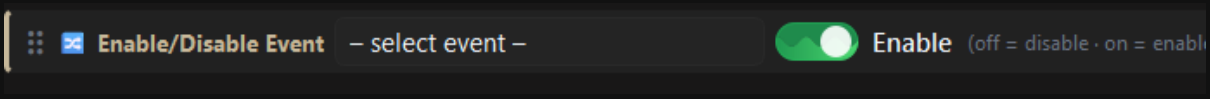


Run Event startet ein anderes Event

Run Event startet ein anderes Event. Der kleine Zettel-Knopf daneben öffnet eine Liste aller Events mit Suchfeld.

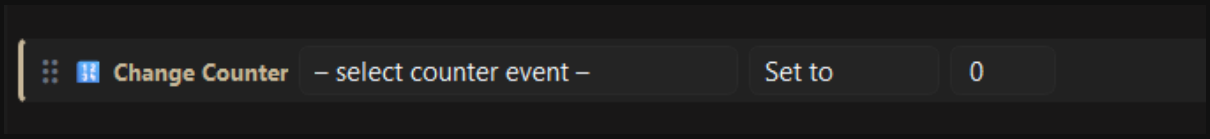


Die Event-Auswahl mit Suchfeld



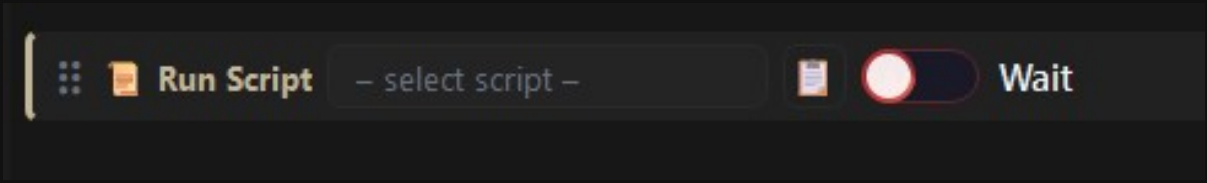
Enable/Disable Event — der Schalter entscheidet die Richtung

Enable/Disable Event schaltet ein Event ein oder aus. Der Schalter entscheidet die Richtung: grün (`on`) = einschalten, rot (`off`) = ausschalten.



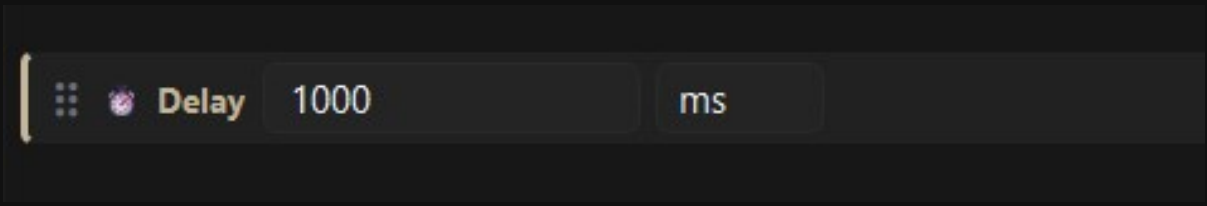
Change Counter verändert den Zähler eines anderen Events

Change Counter verändert den Counter-Trigger eines Events. Zur Wahl stehen **Set to**, **Add** und **Reset to 0**.



Run Script führt ein ASTOSCRIPT aus

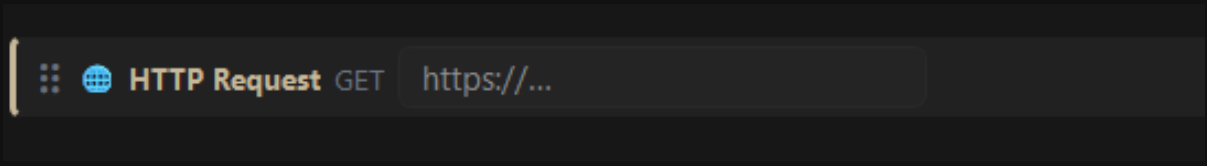
Run Script führt ein Script aus dem Scripts-Bereich aus (siehe Kapitel Scripts). Mit **Wait** wartet das Event, bis das Script fertig ist.



Delay — die Einheit ist umschaltbar

Delay lässt das Event warten. Die Einheit ist umschaltbar: Millisekunden, Sekunden, Minuten oder Stunden.

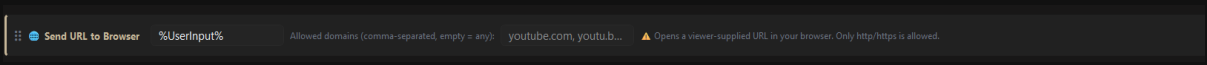
- **Write to File** — beschreibt eine Textdatei. Achtung: Der Inhalt wird **überschrieben**, nicht angehängt.
- **Read from File** — liest Text aus einer Datei. Das Ergebnis landet in `%line0%`, `%line1%` … beziehungsweise in `%FileContent%` und lässt sich zum Beispiel in einer Chat-Aktion weiterverwenden.



HTTP Request — nur GET

HTTP Request ruft eine URL auf. Es wird ausschließlich **GET** unterstützt.

🚀 **Launch Programm** startet ein Programm, das auf deinem PC liegt — zum Beispiel ein Spiel oder ein Tool.



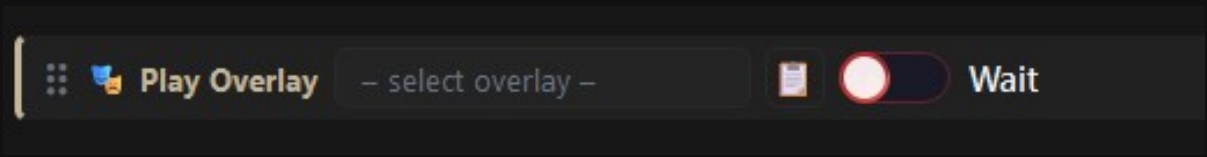
Send URL to Browser mit einer Liste erlaubter Domains

- **Send URL to Browser** öffnet eine URL in deinem Browser — typischerweise eine, die ein **Zuschauer** mitgeschickt hat, etwa per Kanalpunkte-Reward mit `%UserInput%` (ein YouTube-Link zum Beispiel).
- Im Feld **Allowed domains** trägst du kommagetrennt ein, welche Domains erlaubt sind — `youtube.com, google.com`. Es werden nur `http`- und `https`-Adressen akzeptiert.

> Lässt du **Allowed domains** leer, ist **jede** Domain erlaubt — dann kann ein Zuschauer jede beliebige Seite auf deinem Stream-PC öffnen, live vor Publikum. Trage eine Whitelist ein.

Send URL to Browser Overlay schickt eine URL an das **Browser Overlay** (Kapitel 9.8) — die Seite wird also direkt in deinem Stream eingeblendet, nicht im normalen Browser geöffnet.

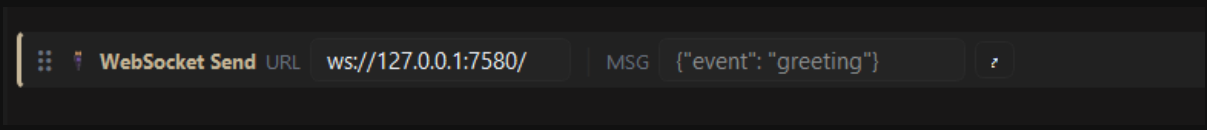
> Anders als bei **Send URL to Browser** gibt es hier **keine** eingebaute Domain-Liste. Sollen Zuschauer eine URL beisteuern dürfen, filtere **vorher per IF/ELSE**, welche Seiten erlaubt sind — sonst kann jeder jede beliebige Seite auf deinen Stream bringen.



Play Overlay startet ein Overlay aus dem Overlay-Bereich

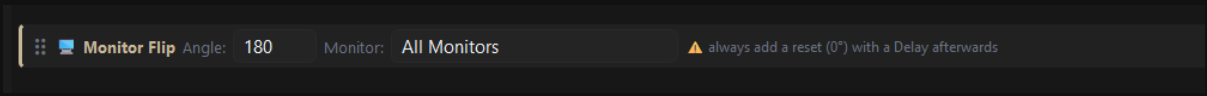
Play Overlay startet ein Overlay, das du im Overlay-Bereich gebaut hast (siehe Kapitel Overlay). Mit **Wait** geht es erst weiter, wenn das Overlay durchgelaufen ist.

🎲 **Play Random Overlay** spielt bei jedem Auslösen **eines** von mehreren Overlays — per Zufall. Über den Zettel-Knopf hakst du an, welche zur Auswahl stehen; jedes kommt gleich oft dran. Overlays, die inzwischen gelöscht oder umbenannt sind, werden übersprungen und in der Zeile mit ⚠ markiert. **Wait** funktioniert wie bei Play Overlay.

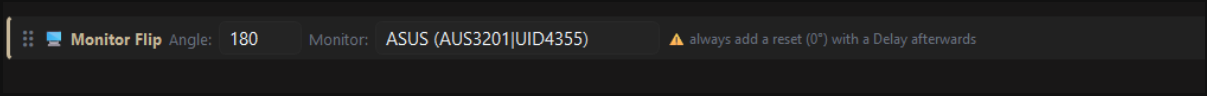


WebSocket Send schickt einen Befehl an ein anderes Programm

WebSocket Send schickt einen Websocket-Befehl an die angegebene **URL** — zum Beispiel an SpeakerBot oder Streamer.bot. Im Feld **MSG** steht die Nachricht, üblicherweise als JSON. Der kleine Pfeil öffnet ein größeres Textfeld.



Monitor Flip — hier werden alle Monitore gedreht



Monitor Flip auf einen bestimmten Monitor, erkannt an seiner Hardware-Kennung

Monitor Flip dreht einen Monitor. Bei **Angle** stellst du den Winkel ein, bei **Monitor** wählst du entweder einen bestimmten Bildschirm — er wird an seiner Hardware-Kennung erkannt, zum Beispiel `ASUS (AUS3201|UID4355)` — oder **All Monitors** für alle gleichzeitig.

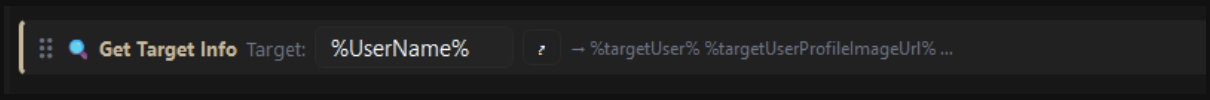
> **Monitor Flip gehört immer mindestens zweimal ins Event.** Die erste Aktion dreht den Monitor (zum Beispiel auf `180`), danach kommt ein **Delay**, und die letzte Aktion dreht mit `0` wieder zurück. Vergisst du das Zurückdrehen, bleibt dein Bild auf dem Kopf stehen — mitten im Stream, und du musst es kopfüber wieder geradebiegen.

📝 **Note** ist eine Notiz mitten in der Aktionskette. Sie tut nichts und stört nichts — sie hilft dir, in einem langen Event den Überblick zu behalten. Schaltest du 📋 **Log** ein, schreibt sie ihren Text ins Log und in den Event Inspector, mit allen Variablen bereits aufgelöst. So siehst du an genau dieser Stelle, was in einer Variablen wirklich steht. In den Chat geht sie nie.

Twitch

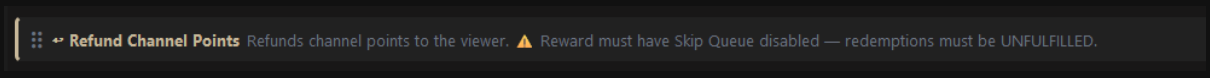


Die Gruppe Twitch mit ihren Aktionen



Get Target Info holt die Twitch-Daten einer Person

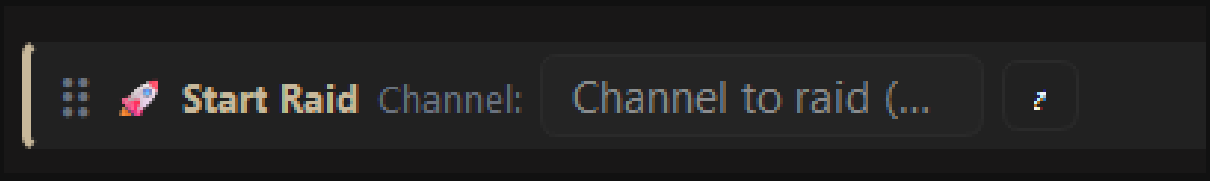
Get Target Info holt alle Twitch-Informationen zu einer Person: Biografie, Profilbild, gespielte Spiele und mehr. Im Feld **Target** steht, um wen es geht — `%UserName%` (wer das Event ausgelöst hat) oder `%UserInput%` (der Name, den jemand eingetippt hat). Die Daten landen in Variablen wie `%targetUser%` und `%targetUserProfileImageUrl%` und lassen sich danach im Prompt weiterverwenden.



Refund Channel Points erstattet die eingesetzten Kanalpunkte

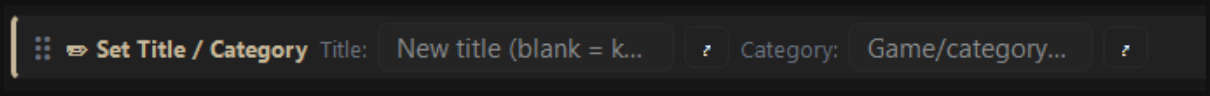
Refund Channel Points gibt dem Zuschauer die Kanalpunkte zurück, die er für das Event ausgegeben hat.

> **Refund Channel Points** funktioniert nur, wenn beim Reward **Skip Queue ausgeschaltet** ist. Die Einlösung muss auf Twitch noch **unfulfilled** sein — was bereits bestätigt wurde, lässt sich nicht mehr erstatten.



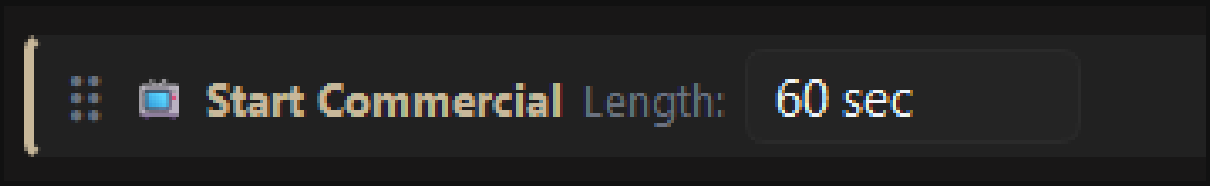
Start Raid — fester Kanalname oder Variable

Start Raid startet einen Raid. Du kannst den Kanal fest eintragen oder eine Variable verwenden: Mit `%UserInput%` und einem Chat-Befehl wie `!raid CaptainApolloo` wird genau der Kanal geraidet, der im Chat genannt wurde.

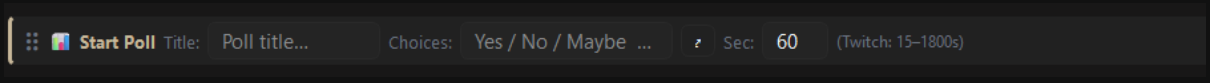


Set Title / Category — beides oder nur eines von beiden

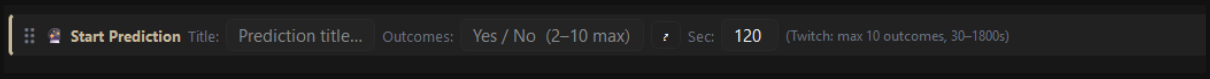
Set Title / Category ändert auf Twitch den Titel, die Kategorie oder beides. Ein leeres Feld bleibt unverändert. Variablen sind erlaubt — mit `%UserInput%` und einem Befehl wie `!titel Neuer Stream Titel` setzt der Chat den Titel.



Start Commercial — Werbedauer in Sekunden



Start Poll mit Titel, Antworten und Laufzeit



Start Prediction — zwei bis zehn Ausgänge

- **Start Commercial** — startet die Werbung für die eingestellte Dauer.
- **Start Poll** — startet eine Umfrage. **Title**, **Choices** (die Antworten) und **Sec** für die Laufzeit (Twitch erlaubt 15–1800 Sekunden).
- **Start Prediction** — startet eine Vorhersage. **Outcomes** sind die möglichen Ausgänge (2 bis 10), **Sec** die Laufzeit (30–1800 Sekunden).
- **End Poll** und **End Prediction** — beenden Umfrage bzw. Vorhersage vorzeitig.

Reward On/Off schaltet Kanalpunkte-Rewards an, aus oder auf Pause. Im Dropdown wählst du **Enable**, **Disable**, **Pause** oder **Resume**, darunter die betroffenen Rewards — mehrere auf einmal sind erlaubt.

Der Nutzen zeigt sich, sobald du mehr als ein Chat-Spiel hast: Läuft dein Labyrinth, soll niemand nebenher das andere Spiel starten können. Also am Anfang des Events das andere Reward **Disable**, am Ende wieder **Enable**. **Pause** ist der sanftere Weg — der Reward bleibt sichtbar, ist aber nicht einlösbar. **Enable** hebt eine Pause gleich mit auf.

> Twitch lässt über die Schnittstelle nur Rewards ändern, die **dieselbe Anwendung angelegt hat** — dieselbe Regel wie bei Refund Channel Points. Ein Reward, den du im Twitch-Dashboard von Hand erstellt hast, ist für ASTO-BOT unerreichbar; im Log steht dann *read-only (created by another app)*. Leg ihn in dem Fall über den Reward Editor neu an.

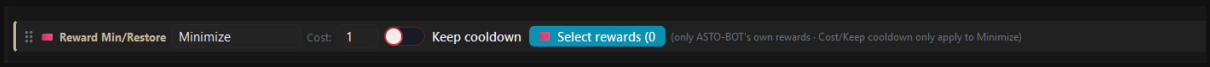
> 💡 Denk daran, wo das **Enable** steht. Liegt es im selben Event hinter einem **Run Script**, läuft es sofort los, weil Run Script nicht auf das Skript wartet — der Reward wäre wieder frei, bevor das Spiel überhaupt begonnen hat. Entweder beim Run Script **Wait** einschalten oder das Enable in ein eigenes Abschluss-Event legen, das dein Skript am Ende feuert.

Die restlichen Twitch-Aktionen erklären sich beim Öffnen von selbst:

- **Reset Reward Timer** — setzt den Cooldown-Timer eines Kanalpunkte-Rewards wieder auf seinen vollen Wert. Sind von fünf Minuten schon drei abgelaufen, stehen danach wieder fünf Minuten an.
- **Cancel Raid** — bricht einen laufenden Raid ab.
- **Shoutout** — gibt einen Twitch-Shoutout.
- **Stream Marker** — setzt einen Marker, damit du später im VOD schneller findest, wo etwas passiert ist.

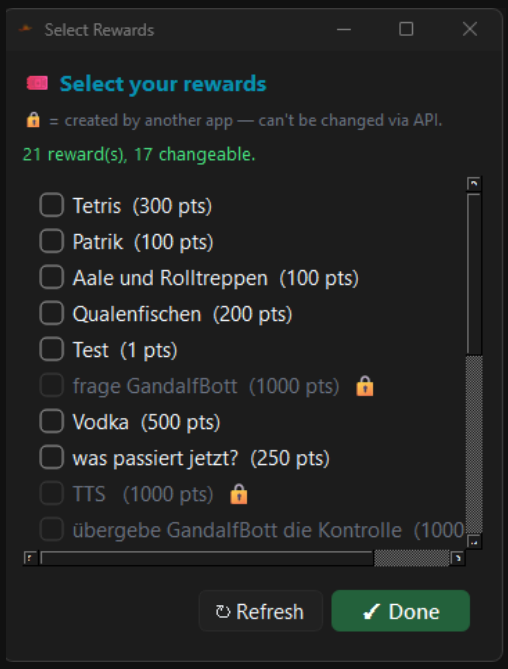
Reward Min/Restore

Diese Aktion setzt **Kosten** und **Cooldown** von Kanalpunkte-Rewards vorübergehend herab — und stellt sie danach wieder her.



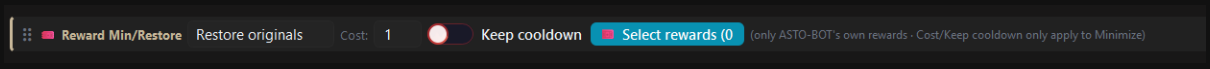
Minimize setzt die Kosten herunter, hier auf 1 Punkt

- Im Modus **Minimize** legst du die neuen **Cost** fest, zum Beispiel 1 Kanalpunkt.
- Der Schalter **Keep cooldown** behält einen vorhandenen Cooldown bei. Ist er aus, wird der Cooldown auf `0` gesetzt.
- **Cost** und **Keep cooldown** gelten **nur** für Minimize.
- Über **Select rewards** wählst du aus, welche Rewards betroffen sind — es dürfen mehrere sein.



Das Auswahlfenster — das Schloss markiert fremde Rewards

> Nur **ASTO-BOT-eigene** Rewards lassen sich ändern. Rewards, die von einer anderen App erstellt wurden, sind im Auswahlfenster mit einem **Schloss** markiert — Twitch erlaubt es nicht, sie über die API anzufassen.

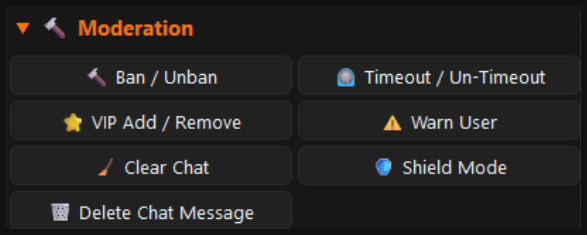


Restore originals stellt die ursprünglichen Werte wieder her

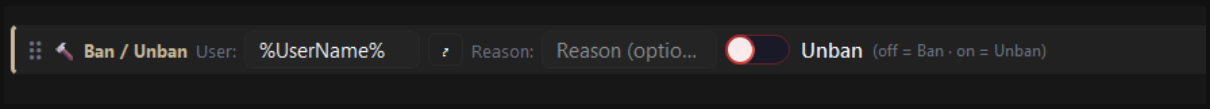
Im Modus **Restore originals** werden Kosten und Cooldown wieder auf ihre Ursprungswerte zurückgesetzt.

> 💡 Das übliche Muster: **Minimize** an den **Anfang** des Events, **Restore originals** ans **Ende**. Dazwischen liegt das, was passieren soll — zum Beispiel eine Minute, in der ein Reward nur einen einzigen Kanalpunkt kostet.

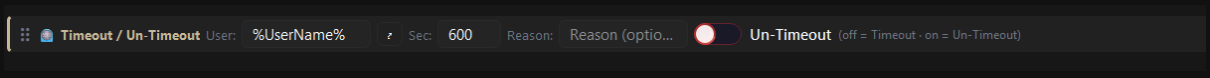
Moderation



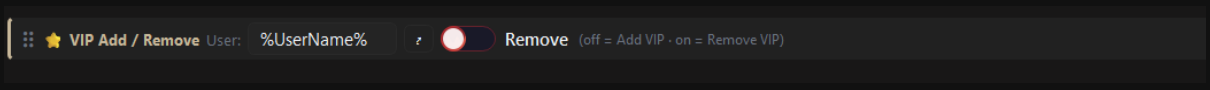
Die Gruppe Moderation



Ban / Unban — der Schalter entscheidet die Richtung



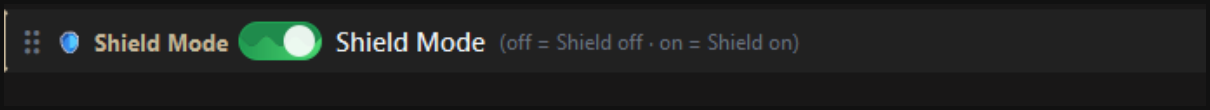
Timeout / Un-Timeout mit Dauer und Grund



VIP Add / Remove

Drei dieser Aktionen können jeweils **beides** — der Schalter rechts entscheidet, in welche Richtung es geht:

- **Ban / Unban** — Schalter aus = **Ban**, Schalter an = **Unban**. Im Feld **Reason** kannst du optional einen Grund angeben.
- **Timeout / Un-Timeout** — Schalter aus = **Timeout** für die eingestellten Sekunden, Schalter an = **Un-Timeout**. Auch hier ist ein Grund möglich.
- **VIP Add / Remove** — Schalter aus = VIP **vergeben**, Schalter an = VIP **entziehen**.
- Im Feld **User** steht, wen es trifft — meist `%UserName%`.



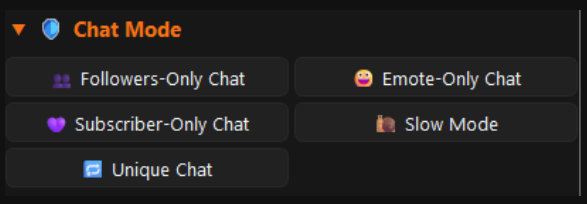
Shield Mode ein- und ausschalten

- **Warn User** — spricht eine Twitch-Verwarnung gegen den Zuschauer aus.
- **Clear Chat** — löscht den Chatverlauf für alle.
- **Shield Mode** — schaltet den Twitch-Shield-Mode ein oder aus.

Delete Chat Message löscht eine einzelne Nachricht. Im Modus **This message** trifft es die Nachricht, die das Event ausgelöst hat — der Normalfall, dafür musst du nichts weiter einstellen. Im Modus **Last message of** trifft es die letzte Nachricht der Person, die im Feld steht; das ist für Mod-Befehle gedacht, mit denen jemand gegen einen Dritten vorgeht.

> Twitch löscht keine Nachrichten von **Moderatoren** oder vom **Streamer**, und nichts, was älter als **sechs Stunden** ist. In diesen Fällen passiert nichts — warum, steht im Log.

Chat Mode

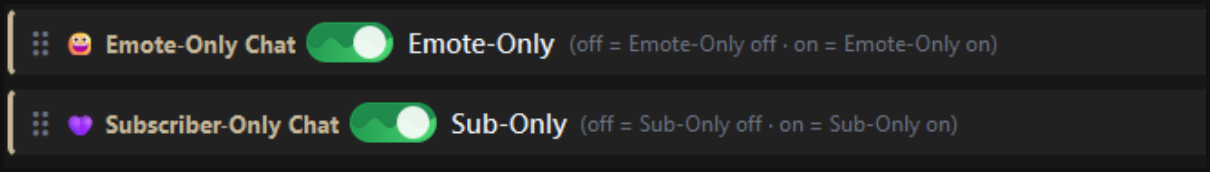


Die Gruppe Chat Mode

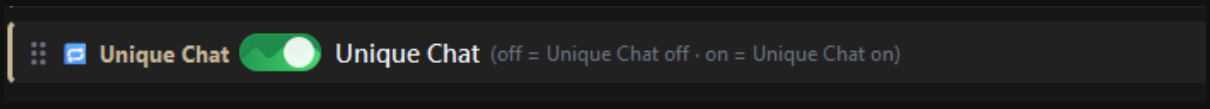


Followers-Only Chat — die Wartezeit kommt aus dem Dropdown

- **Followers-Only Chat** — stellt den Chat auf Follower um. Im Dropdown wählst du, wie lange jemand schon folgen muss: `Off`, `0 min (all followers)`, `10 min`, `30 min`, `1 hour`, `1 day`, `1 week`, `1 month` oder `3 months`. Mit `Off` schaltest du den Modus wieder ab.
- **Slow Mode** — schaltet den Slow Mode ein; die Wartezeit wählst du ebenfalls über ein Dropdown.



Emote-Only und Subscriber-Only — einfache Schalter

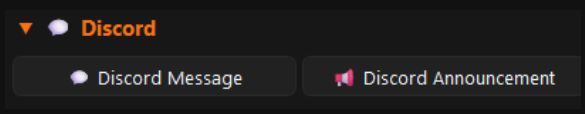


Unique Chat ein- und ausschalten

Emote-Only Chat, **Subscriber-Only Chat** und **Unique Chat** sind reine Schalter: an bedeutet einschalten, aus bedeutet ausschalten.

> 💡 Ein Chat-Mode-Event lässt sich gut mit dem Timed-Trigger kombinieren: Emote-Only für eine Minute einschalten — dazu ein zweites Event, das ihn wieder ausschaltet. Oder in einem Event mit einem **Delay** dazwischen.

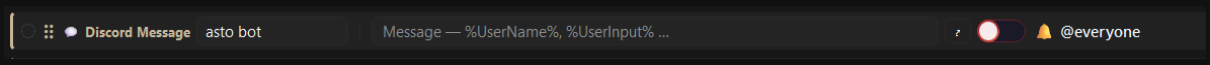
Discord



Die Gruppe Discord

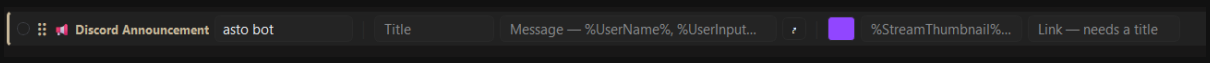
Zwei Aktionen schicken eine Nachricht in deinen Discord. Beide brauchen einen Webhook, den du vorher in den Settings angelegt hast (Kapitel 4.4); ausgewählt wird er über seinen Namen.

Discord Message schickt eine gewöhnliche Nachricht. Variablen funktionieren darin wie überall sonst.



Discord Message — Webhook, Text und der @everyone-Schalter

📢 **Discord Announcement** schickt eine Karte — in Discord heißt das Embed. Dazu gehören ein **Titel**, ein Text, eine **Farbe** über den Farbwähler, ein **Bild** und ein **Link**.



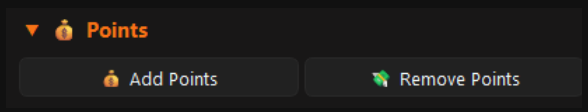
Discord Announcement — Titel, Text, Farbe, Bild und Link

- **Webhook** — welcher deiner Webhooks die Nachricht bekommt.
- 🔔 **@everyone** — benachrichtigt alle im Kanal. Standardmäßig **aus**, und das mit Absicht: Ohne diesen Schalter löst keine Erwähnung eine Benachrichtigung aus, auch wenn @everyone im Text steht.

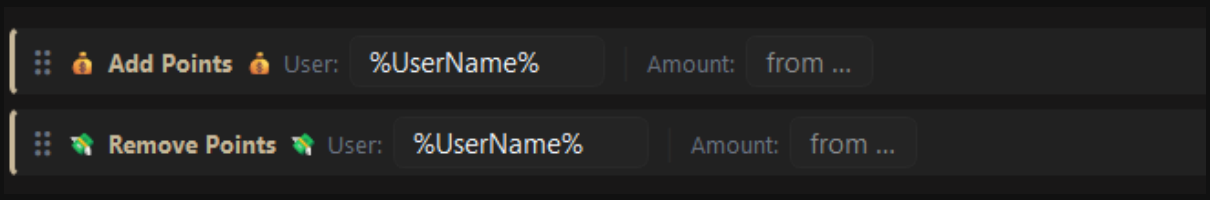
> Der **Link** einer Ankündigung hängt am **Titel**. Ohne Titel zeigt Discord ihn gar nicht erst an.

> 💡 Für die Stream-Ankündigung: **%StreamTitle%** und **%GameName%** stehen inzwischen in **jedem** Event bereit, und **%StreamThumbnail%** liefert das Vorschaubild deines Streams als Bild für die Karte.

Points



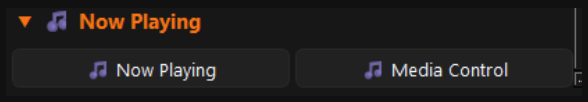
Die Gruppe Points



Add Points und Remove Points

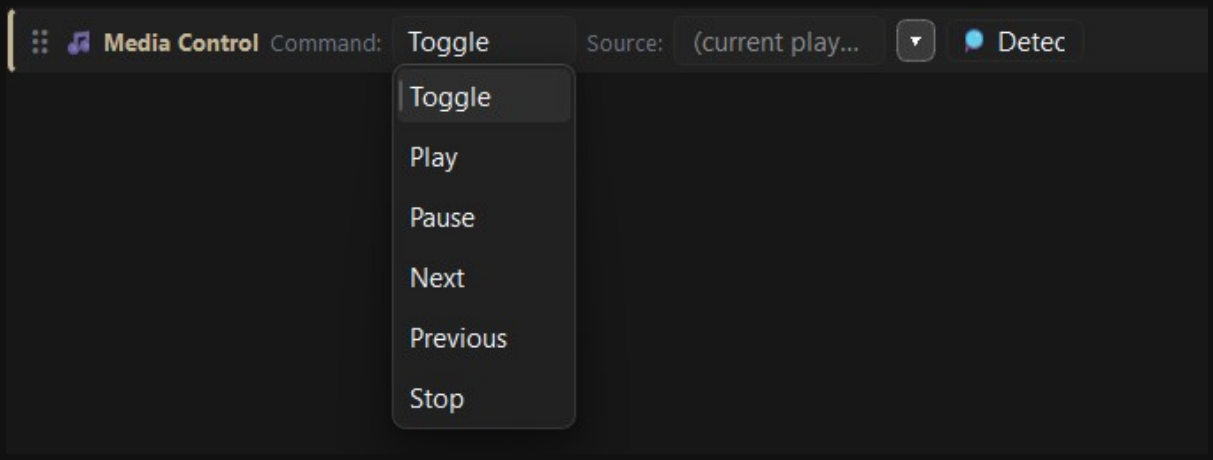
ASTO-BOT hat ein eigenes **Punkte-System**, das im Giveaway-Bereich verwaltet wird (siehe Kapitel Giveaway). Mit **Add Points** gibst du einem Zuschauer Punkte, mit **Remove Points** nimmst du sie ihm wieder weg. Im Feld **User** steht, wer gemeint ist, im Feld **Amount** wie viele Punkte. Bei **Amount** ist beides erlaubt: eine feste Zahl oder eine **Variable**.

Now Playing



Die Gruppe Now Playing

Now Playing holt sich vom Musikprogramm — Spotify, Apple Music oder einem Browser — die Information, was gerade läuft, und legt sie in Variablen ab: `%NowPlaying%`, `%SongTitle%`, `%SongArtist%`, `%SongAlbum%`, `%MusicSource%` und `%MusicStatus%`. Die Aktion ist aufgebaut wie der Music-Trigger.



Media Control steuert den Player direkt

Media Control steuert das Programm, das du im Dropdown auswählst. Unter **Command** stehen **Toggle**, **Play**, **Pause**, **Next**, **Previous** und **Stop**. Mit **Detect** sucht ASTO-BOT den laufenden Player.

> Auch hier gilt: Das Programm muss **gerade Ton abspielen**, sonst liefert Windows keine Informationen — weder für **Now Playing** noch für **Detect**. Ein pausierter Player wird nicht gefunden.

Giveaway

Mit der Aktion **Draw Giveaway** löst ASTO-BOT ein Giveaway automatisch auf — genau so, als hättest du auf der Giveaway-Seite selbst auf **Draw** gedrückt. Damit kann ein Reward, ein Chat-Befehl oder ein Timer die Ziehung starten, ohne dass du die Hand am Rechner hast.

Über den Knopf 📋 öffnest du die Auswahl. Dort stehen **alle** Giveaways — aktive wie inaktive, jeweils mit Status, Teilnehmerzahl und, falls schon gezogen, dem bisherigen Gewinner. Der Schalter **aktiv** entscheidet nur darüber, ob Zuschauer noch beitreten können; ziehen lässt sich ein Giveaway in jedem Fall.

Der Schalter **Announce** (standardmäßig an) bestimmt, ob zusätzlich die Events feuern, die als **Giveaway Winner** an dieses Giveaway gebunden sind. Schaltest du ihn aus, wird nur gezogen — die Ansage machst du dann selbst in den folgenden Aktionen desselben Events.

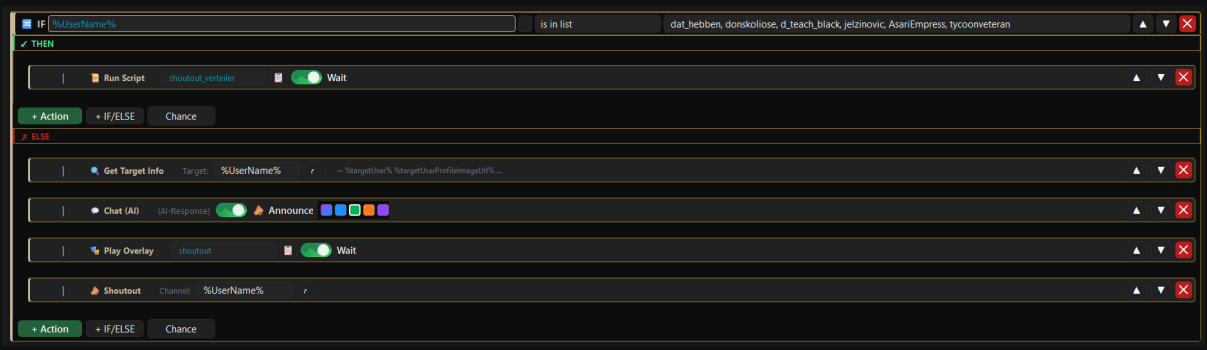
Nach der Ziehung stehen die Gewinner in den Variablen bereit und können in **jeder nachfolgenden Aktion** desselben Events benutzt werden: `%GiveawayWinner%`, `%GiveawaySecond%`, `%GiveawayThird%`, `%GiveawayName%`, `%GiveawayEntryCount%` (Teilnehmerzahl), `%GiveawayResult%` (`drawn`, `not_found`, `no_entries`) und `%GiveawaySuccess%` (`true` / `false`).

> 💡 Bau eine **IF/ELSE**-Abfrage auf `%GiveawaySuccess%` **gleich hinter** die Aktion. Ist ein Giveaway leer oder falsch geschrieben, ruft dein Event sonst einen leeren Gewinner aus. Im ELSE-Zweig verrät dir `%GiveawayResult%`, woran es lag.

> ⚠ **Achtung — Endlosschleife!** Verwende in **einem** Event niemals den Trigger **Giveaway Winner** und die Aktion **Draw Giveaway** für **dasselbe** Giveaway gleichzeitig, solange **Announce** eingeschaltet ist. Die Ziehung feuert dann das Gewinner-Event, dieses zieht erneut, feuert wieder — das Event ruft sich endlos selbst auf und flutet Chat und Log. **Richtig ist eine von diesen drei Varianten:** Trigger und Aktion auf **zwei getrennte Events** verteilen (ein Event zieht, ein zweites reagiert auf Giveaway Winner) · oder im ziehenden Event **Announce ausschalten** und die Ansage direkt dahinter setzen · oder in der Aktion ein **anderes Giveaway** wählen als im Trigger.

5.7 IF/ELSE — Bedingungen

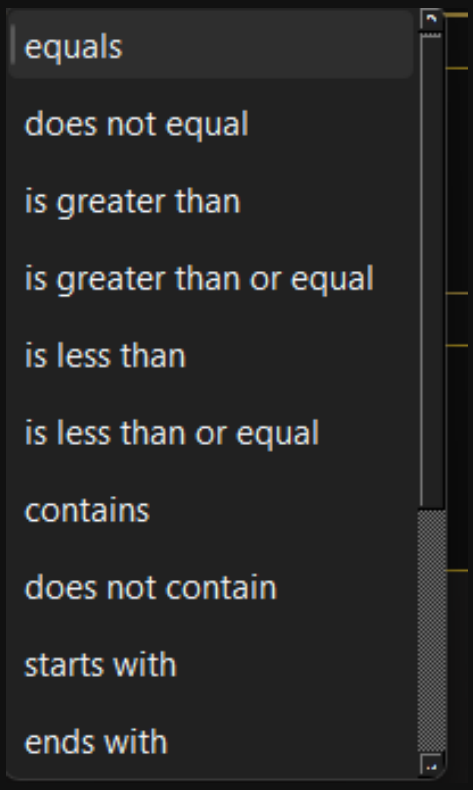
Mit **+ IF/ELSE** trifft dein Event Entscheidungen. Eine Bedingung besteht immer aus drei Teilen: einer **Variablen**, einem **Operator** und einem **Wert**. Trifft sie zu, läuft der **THEN**-Zweig — sonst der **ELSE**-Zweig.



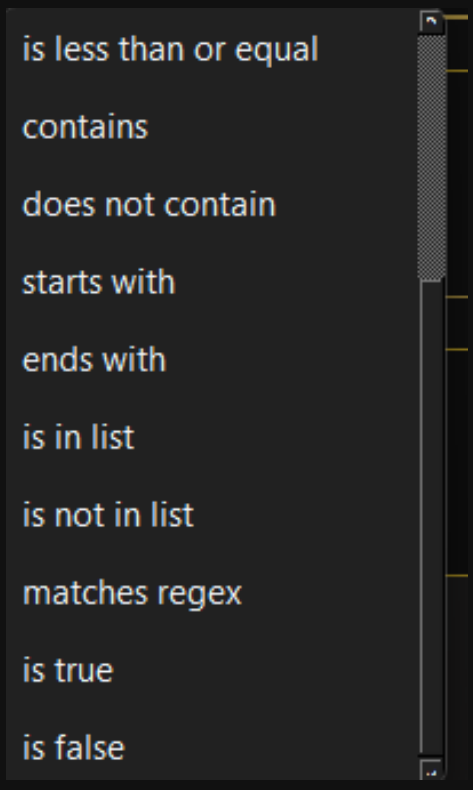
Ein fertiges IF/ELSE: Stammgäste bekommen das Script, alle anderen den Standard-Shoutout

Links trägst du die **Variable** ein — `%UserName%`, `%CheerAmount%`, `%IsFollower%`, `%Role%` und alles andere, was das Variables-Fenster hergibt. In der Mitte wählst du den **Operator**, rechts steht der **Wert**, gegen den geprüft wird.

Die Operatoren



Die Operatoren, erster Teil



Die Operatoren, zweiter Teil

- **equals** / **does not equal** — ist genau gleich / ist nicht gleich.
- **is greater than** / **is greater than or equal** — größer als / größer oder gleich. Für Zahlen, zum Beispiel Bits.

- **is less than** / **is less than or equal** — kleiner als / kleiner oder gleich.
- **contains** / **does not contain** — enthält / enthält nicht.
- **starts with** / **ends with** — fängt an mit / hört auf mit.
- **is in list** / **is not in list** — steht in einer Liste / steht nicht darin. Die Liste trägst du kommagetrennt in das Wertfeld ein.
- **matches regex** — prüft gegen einen regulären Ausdruck. Für alle, die wissen, was sie tun.
- **is true** / **is false** — für Variablen, die nur wahr oder falsch sein können, etwa `%IsFollower%` oder `%IsGift%`. Hier bleibt das Wertfeld leer.

Variable gegen Variable

Im Wertfeld rechts muss keine feste Zahl stehen — du kannst dort **auch eine Variable** eintragen. Die Bedingung `%FollowDays%` **is greater than or equal** `%GiveawayMinFollowDays%` vergleicht also, wie lange jemand folgt, mit dem, was du beim Giveaway eingestellt hast.

Der Vorteil: Die Zahl steht nur noch an **einer** Stelle. Änderst du die Mindest-Follow-Dauer im Giveaway von 10 auf 30 Tage, zieht die Bedingung von selbst mit. Trägst du stattdessen eine feste `10` ein, musst du daran denken, sie an beiden Orten zu ändern — sonst lehnt ASTO-BOT ab 29 Tagen ab, deine Meldung spricht aber weiter von 10.

> Aufgelöst wird nur, was ASTO-BOT als Variablennamen **kennt**. Ein `contains` auf `50%` bleibt damit ein normaler Textvergleich, und ein regulärer Ausdruck mit Prozentzeichen verhält sich wie bisher. Vertippst du dich im Namen, bleibt er stehen — so fällt der Fehler auf, statt still ins Leere zu laufen.

THEN und ELSE

In **beide** Zweige kannst du beliebig viele Aktionen einfügen — jeder Zweig hat unten seine eigenen Knöpfe **+ Action**, **+ IF/ELSE** und **Chance**.

> Verschachteln geht **drei Ebenen tief**, und pro Ebene darfst du **mehrere** IF/ELSE- und Chance-Blöcke nebeneinander setzen — die Knöpfe verschwinden nach dem Platzieren nicht mehr.
>
> Was **nicht** in einen Zweig gehört, sind die Clip-Aktionen: **Clip Player**, **Clip List Player**, **Play Last Clip** und **Create Clip** bleiben auf der obersten Ebene. Verschachtelt lief ihre Wiedergabe unzuverlässig, deshalb lassen sie sich dort weder anlegen noch hineinziehen.

Drei Beispiele

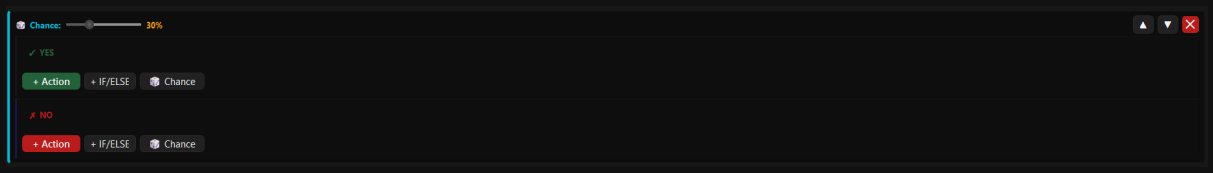
Beispiel 1 — Stammgäste anders behandeln. Genau das zeigt der Screenshot oben: `%UserName%` **is in list** `dat_hebben, donskoliose, d_teach_black, ...` → im **THEN**-Zweig läuft ein Script (`shoutout_verteiler`), das für diese Leute etwas Eigenes macht. Alle anderen landen im **ELSE**-Zweig und bekommen den Standardweg: **Get Target Info** → **Chat (AI)** → **Play Overlay** → **Shoutout**.

Beispiel 2 — großzügige Bits belohnen. Bedingung: `%CheerAmount%` **is greater than or equal** `1000`. Im **THEN**-Zweig: ein lauterer Sound, eine **Pin Message (AI)** und ein Overlay. Im **ELSE**-Zweig: eine normale **Chat (AI)**-Nachricht. So bekommt der große Cheer seinen Moment, ohne dass jeder kleine Cheer den ganzen Zirkus auslöst.

Beispiel 3 — Nicht-Follower freundlich abholen. Bedingung: `%IsFollower%` **is false**. Im **THEN**-Zweig eine Chat-Nachricht, die freundlich auf den Follow-Knopf hinweist. Im **ELSE**-Zweig die normale Begrüßung. Wichtig: Das Wertfeld bleibt bei `is false` leer.

5.8 Chance — der Zufall

Der Knopf **Chance** fügt einen Zufallsblock ein. Oben sitzt ein **Prozent-Regler**: Er bestimmt, wie wahrscheinlich es ist, dass der **YES**-Zweig läuft. Bei 30 % passiert das im Schnitt bei drei von zehn Auslösungen — in den anderen sieben Fällen läuft der **NO**-Zweig.



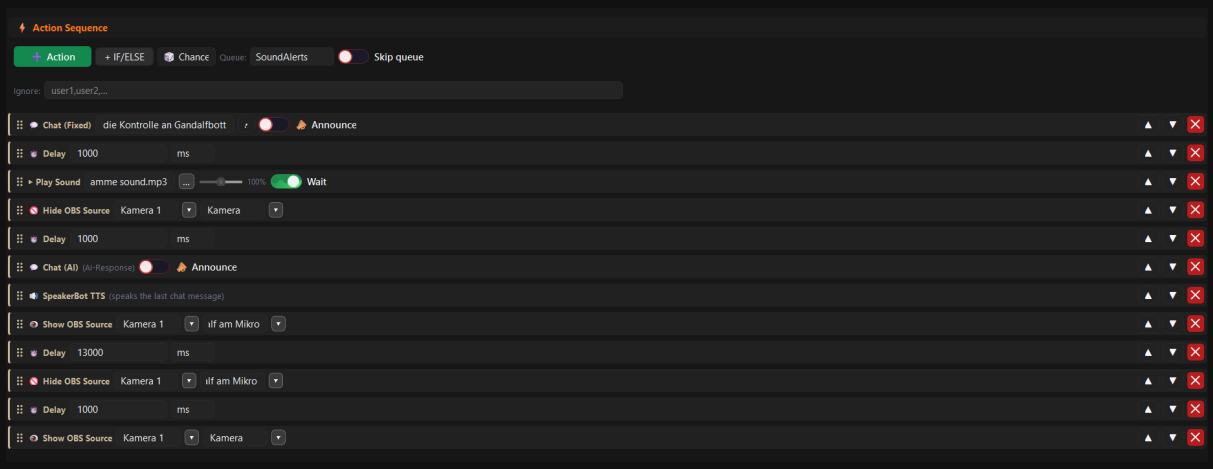
Der Chance-Block mit Prozent-Regler, YES- und NO-Zweig

Wie beim IF/ELSE kannst du in beide Zweige Aktionen einfügen — und dort auch wieder IF/ELSE-Blöcke und weitere Chance-Blöcke.

> 💡 Chance macht Events lebendig. Ein Follow-Sound, der zu 10 % ein anderer ist, sorgt für genau die kleinen Überraschungen, über die dein Chat redet.

5.9 Ein Event von vorne bis hinten

Zum Schluss ein komplettes Event, wie es in der Praxis aussieht. Der Zuschauer löst es aus, ASTO-BOT kündigt es im Chat an, spielt einen Sound, blendet die Kamera aus, lässt die KI etwas sagen, liest es vor, zeigt eine andere Kameraeinstellung und räumt am Ende wieder auf.



Ein fertig gebautes Event — zwölf Aktionen der Reihe nach

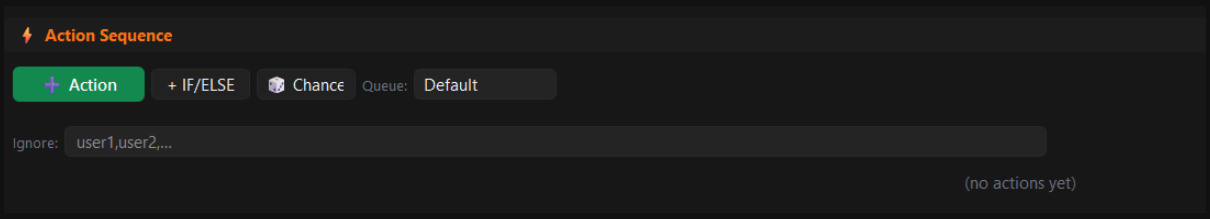
Von oben nach unten gelesen:

- **Chat (Fixed)** — eine feste Ansage in den Chat.
- **Delay 1000 ms** — eine Sekunde Pause, damit die Nachricht ankommt, bevor es weitergeht.
- **Play Sound** mit **Wait** — der Sound läuft, und das Event wartet, bis er durch ist.
- **Hide OBS Source** — die Kamera wird ausgeblendet.
- **Delay 1000 ms**
- **Chat (AI)** — die KI schreibt ihre Nachricht, gesteuert vom Prompt des Events.
- **SpeakerBot TTS** — liest genau diese Nachricht vor. Sie steht direkt darüber, deshalb funktioniert es.
- **Show OBS Source** — eine andere Quelle wird eingeblendet.
- **Delay 13000 ms** — dreizehn Sekunden, in denen man sie sieht.
- **Hide OBS Source** — und wieder aus.
- **Delay 1000 ms**
- **Show OBS Source** — die Kamera kommt zurück. Der Ausgangszustand ist wiederhergestellt.

Rechts an jeder Zeile siehst du ▲▼ zum Verschieben und das rote ✕ zum Löschen. Oben steht die **Queue**, in der das Event läuft — hier *SoundAlerts*.

> 💡 Zwei Dinge, die dieses Beispiel gut zeigt: Erstens räumt ein sauberes Event am Ende wieder auf — was du ausblendest, blendest du auch wieder ein. Zweitens stehen **Chat (AI)** und **SpeakerBot TTS** direkt hintereinander, damit auch wirklich das vorgelesen wird, was die KI gerade geschrieben hat.

Ein leeres Event sieht am Anfang so aus — und von hier baust du dich Aktion für Aktion nach unten:

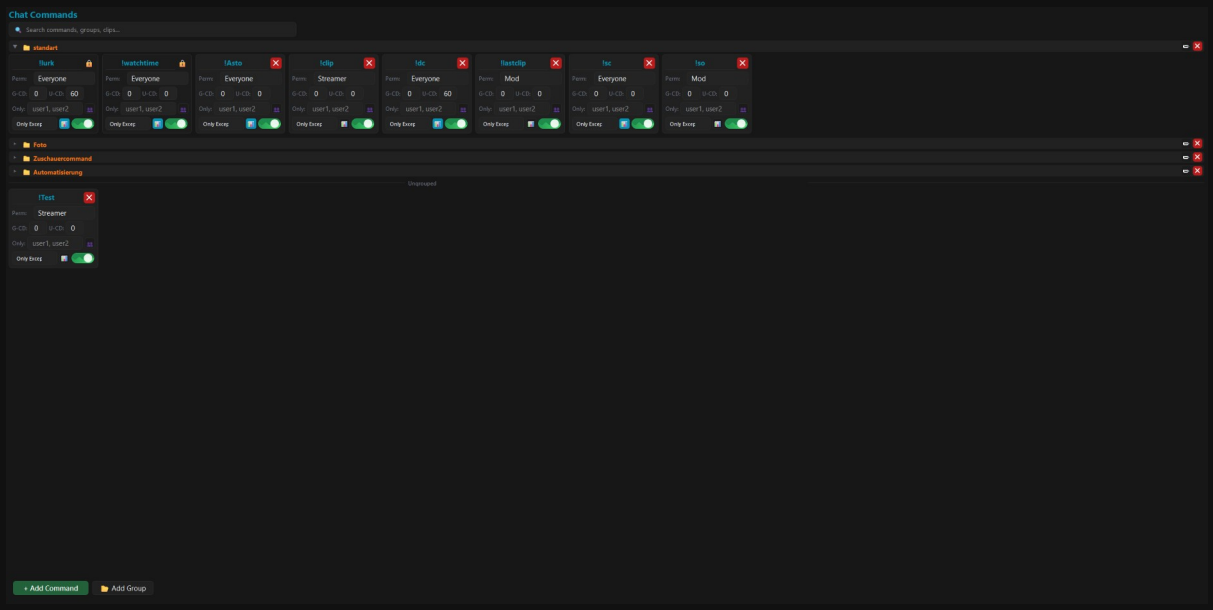


Eine leere Action Sequence — hier fängt jedes Event an

6. Command

Im Bereich **Command** legst du die Chat-Befehle deines Kanals an — `!clip`, `!lurk`, `!discord`, was immer du brauchst. Wichtig ist, was ein Command **nicht** ist: Er führt selbst **nichts** aus.

> 💡 **Ein Command ist nur ein Auslöser.** Was passieren soll, wenn ihn jemand im Chat tippt, baust du im **Event** — dort wählst du den Command beim Trigger **Chat Command** aus (siehe Kapitel 5.4.1). Hier legst du nur fest: Wie heißt der Befehl, wer darf ihn benutzen, und wie oft?



Der Command-Bereich mit Gruppen, Karten und dem Bereich „Ungrouped“

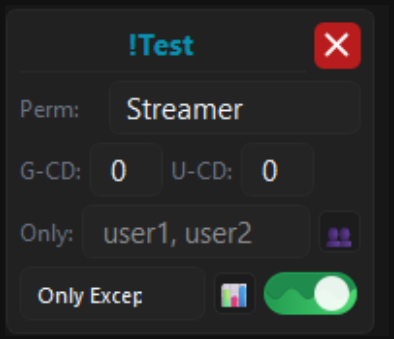
6.1 Aufbau der Seite

- Ganz oben liegt das **Suchfeld** — es durchsucht Commands, Gruppen und Clips.
- Darunter stehen die **Gruppen**. Jede Gruppe lässt sich auf- und zuklappen, umbenennen und löschen.
- Commands ohne Gruppe sammeln sich unten unter **Ungrouped**.
- Unten links: **+ Add Command** und **Add Group**.

+ Add Command fragt zuerst nach dem **Namen** — das `!` steht schon drin, Leerzeichen sind nicht erlaubt. Danach erscheint die neue Karte. Verschieben geht per **Drag & Drop**: einfach die Karte in eine andere Gruppe ziehen.

6.2 Die Command-Karte

Jeder Command ist eine kleine Karte. Alles, was du einstellen kannst, steht direkt darauf.



Eine Command-Karte mit allen Feldern

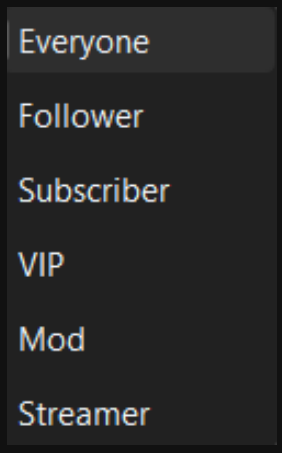
Name

Oben in Blau steht der **Name** des Commands — und dort änderst du ihn auch. Ein Command hat genau **einen** Namen; **Aliase gibt es nicht**, `!so` und `!shoutout` wären also zwei getrennte Commands.

Ist ein Name schon vergeben, hängt ASTO-BOT eine Zahl an — aus `!sound` wird `!sound2` — und färbt den Namen **rot**. Zwei Commands mit demselben Namen würden sich im Chat in die Quere kommen: Nur einer von beiden würde je antworten. Klick den roten Namen an und gib ihm einen eigenen.

Who — wer darf?

Das Dropdown **Who** legt fest, wer den Command auslösen darf — in älteren Versionen hieß es *Perm*. Die Stufen bauen aufeinander auf: Wer weiter oben steht, darf immer auch das, was weiter unten erlaubt ist. Fährst du mit der Maus über die Beschriftung oder das Dropdown, zeigt dir eine Kurzhilfe die ganze Rangfolge.



Die Berechtigungsstufen

- **Everyone** — jeder im Chat.
- **Follower** — Follower, Subscriber, VIPs, Moderatoren und du. Also alle außer den Zuschauern, die dir noch nicht folgen.
- **Subscriber** — Subscriber, VIPs, Moderatoren und du.
- **VIP** — VIPs, Moderatoren und du.
- **Mod** — Moderatoren und du.
- **Streamer** — nur du.

> Ob jemand Follower ist, fragt ASTO-BOT bei Twitch ab und merkt es sich kurz. Die Prüfung läuft nur dann, wenn ein Command wirklich **Follower** verlangt und der Zuschauer nicht ohnehin Subscriber, VIP oder Moderator ist.

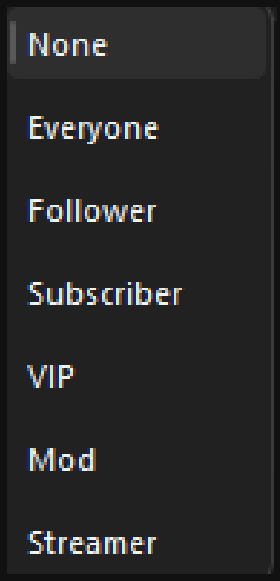
G-CD und U-CD — die Cooldowns

- **G-CD** — der **globale Cooldown**: Nach dem Auslösen ist der Command für **alle** gesperrt.
- **U-CD** — der **User-Cooldown**: Er gilt für **jeden Zuschauer einzeln**. Der eine kann den Befehl also benutzen, während ein anderer noch wartet.

> 💡 Beide zusammen ergeben die übliche Kombination: ein kleiner globaler Cooldown gegen Spam, ein größerer User-Cooldown gegen den einen Zuschauer, der den Befehl zwanzigmal hintereinander tippt.

Only und Only Exception

Im Feld **Only** trägst du Namen ein — kommagetrennt, wenn es mehrere sind. Dann darf **ausschließlich** dieser Personenkreis den Command benutzen. Der Knopf mit den zwei Köpfen daneben öffnet ein größeres Textfeld, wenn die Liste länger wird.



Only Exception — eine Rolle zusätzlich zu den Namen

Only Exception nimmt zu diesen Namen noch eine **Rolle** dazu. Die Voreinstellung ist **None**: Dann gilt die Only-Liste **strikt** — es darf wirklich nur, wer dort eingetragen ist. Wählst du dagegen *Subscriber*, dürfen den Command die eingetragenen Namen **und** alle Subscriber benutzen; mit *Follower* kommen alle Follower dazu, mit *Streamer* nur du selbst. Auch hier erklärt eine Kurzhilfe unter dem Mauszeiger die ganze Liste.

Statistik und Ein-/Ausschalter

Der kleine **Tabellen-Knopf** schaltet die Statistik ein: ASTO-BOT zählt dann mit, wie oft der Command ausgelöst wurde. Die Zahlen landen in der Karte **Command Ranking** auf dem Dashboard (Kapitel 3). Der **Schalter** ganz rechts aktiviert und deaktiviert den Command.

Das Warnsymbol



Ist ein Command an **mehreren Stellen** verknüpft, erscheint auf der Karte ein **Warnsymbol**. Ein Klick darauf zeigt dir, womit er überall verbunden ist. Das ist kein Fehler — es ist ein Hinweis. Wenn ein Befehl gleich drei Events startet, willst du das wissen, bevor du dich wunderst.

6.3 Gruppen

Commands lassen sich wie Events in **Gruppen** einteilen — zum Beispiel *Zuschauercommands*, *Foto* oder *Automatisierung*. **Add Group** legt eine neue Gruppe an, Gruppen lassen sich umbenennen und löschen, und Karten ziehst du per Drag & Drop von einer Gruppe in die nächste. Was in keiner Gruppe liegt, steht unter **Ungrouped**.

6.4 Die mitgelieferten Commands

Beim ersten Start bringt ASTO-BOT vier Commands mit: **!lurk**, **!watchtime**, **!clip** und **!lastclip**.

- **!lurk** — ein ganz normaler Command. Du kannst ihn ignorieren oder mit einem Lurk-Event verknüpfen, das sich bei Zuschauern bedankt, die im Hintergrund mitlaufen lassen.
- **!watchtime** — zeichnet auf, wie lange jeder Zuschauer dir schon zusieht. In Events gibst du den Wert mit der Variable `%Watchtime%` aus.
- **!clip** und **!lastclip** — hängen am Clip-System. Was sie tun und wie du sie einsetzt, steht im Kapitel Clips.

> **!lurk** und **!watchtime** tragen ein **Schloss**: Sie lassen sich **nicht löschen**. **Umbenennen** kannst du sie aber jederzeit — wenn dir `!zuschauzeit` besser gefällt, nimm `!zuschauzeit`.
>
> Die Watchtime wird ab dem Zeitpunkt gezählt, an dem ASTO-BOT das **erste Mal** in einem Stream mitgelaufen ist. Alles davor kann er nicht wissen — die Zahlen wachsen also erst mit der Zeit in etwas Aussagekräftiges hinein.

6.5 Vom Command zum Event

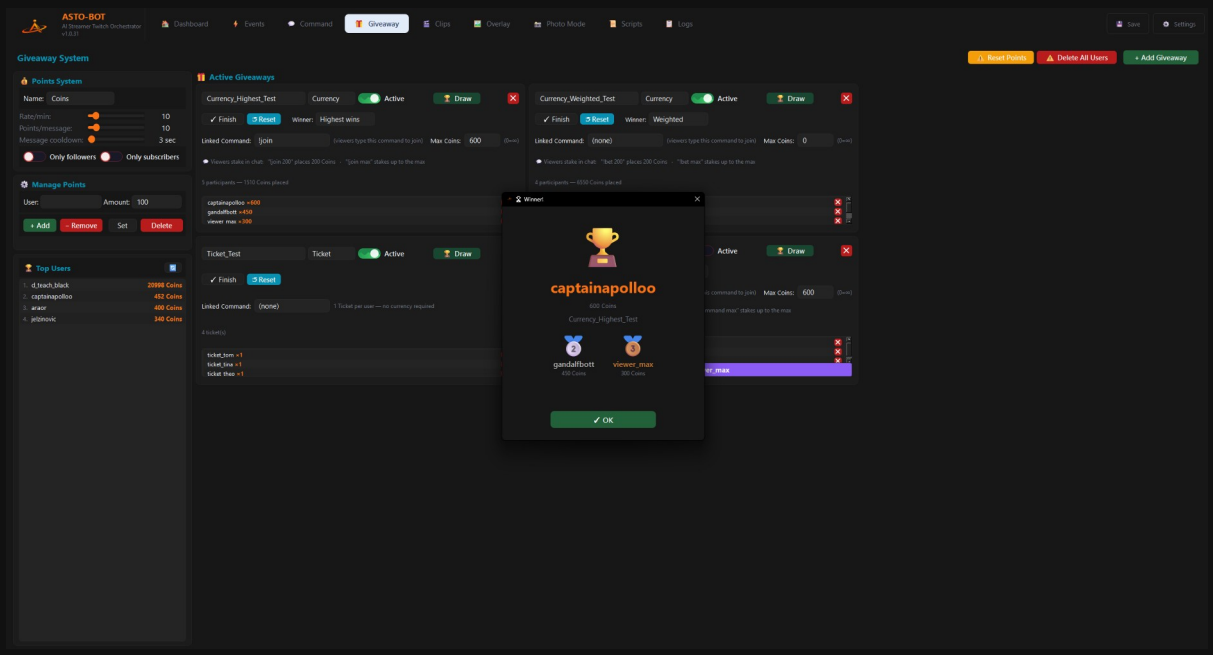
Ein neuer Chat-Befehl entsteht immer in zwei Schritten:

- **Erstens, hier: + Add Command**, Name vergeben, **Perm** setzen, Cooldowns eintragen.
- **Zweitens, im Event:** Ein Event anlegen (oder ein bestehendes nehmen), rechts im Trigger-Bereich unter **General** bei **Chat Command** den neuen Befehl auswählen — und in der Action Sequence festlegen, was passieren soll.

> 💡 Tippt der Command im Chat und es passiert nichts, ist fast immer eines von drei Dingen schuld: Der Command ist **ausgeschaltet**, es hängt **kein Event** daran — oder ein **Cooldown** läuft noch.

7. Giveaway & Punkte

Der Giveaway-Bereich besteht aus zwei Teilen, die zusammenhängen: einem **Punkte-System**, in dem deine Zuschauer fürs Zuschauen Währung sammeln — und den **Giveaways**, in denen sie diese Währung wieder einsetzen können.



Der Giveaway-Bereich: links das Punkte-System, rechts die aktiven Giveaways

7.1 Das Punkte-System

- **Name** — wie deine Währung heißen soll. Standard ist **Coins**, aber nichts hält dich davon ab, sie *Taler*, *Sterne* oder *Bierdeckel* zu nennen.
- **Rate/min** — der Regler bestimmt, wie viele Punkte jeder Zuschauer **pro Minute Zusehen** bekommt.
- **Points/message** — zusätzliche Punkte für **jede Chat-Nachricht**. So belohnst du auch aktive Schreiber, nicht nur stille Zuschauer. `0` schaltet es ab.
- **Message cooldown** — die Sperre in Sekunden zwischen zwei Nachrichten, die Punkte bringen. Sie zählt **pro Zuschauer**: Steht dort `30`, zählt für denselben Zuschauer höchstens alle 30 Sekunden eine Nachricht. `0` heißt, jede Nachricht zählt.
- **Only followers** — ist der Schalter an, sammeln nur Follower Punkte — sowohl pro Minute als auch pro Nachricht.
- **Only subscribers** — ist der Schalter an, sammeln nur Abonnenten Punkte. Sind **beide** Schalter an, muss ein Zuschauer Follower **und** Abonnent sein.

> Damit **Points/message** nicht per Spam ausgenutzt wird, zahlt eine Nachricht nur, wenn sie mindestens **10 Zeichen** lang ist und mindestens **4 verschiedene** Buchstaben oder Ziffern enthält — aaaaaaaaaa bringt also nichts. Chat-Befehle (!…) und eine direkt wiederholte Nachricht zählen ebenfalls nicht.
>
> Punkte bekommt nur, wer **gerade zusieht** — und nur, solange dein **Stream läuft**. Ist der Stream offline, sammelt niemand Punkte, weder pro Minute noch pro Nachricht, und auch die **Watchtime** zählt nicht weiter. Das ist der Sinn der Sache: Die Punkte sind eine Belohnung für Anwesenheit im Stream.

Manage Points

Hier greifst du von Hand ein. Du trägst einen **User** und einen **Amount** ein und wählst dann:

- **+ Add** — gibt dem Zuschauer Punkte dazu.
- **– Remove** — zieht ihm Punkte ab.
- **Set** — setzt seinen Punktestand auf genau diesen Wert.
- **Delete** — löscht den Zuschauer **komplett** aus dem Punkte-System.

> 💡 Du musst den Namen nicht abtippen: Ein **Klick auf einen Namen** in der Top-Users-Liste übernimmt ihn sofort ins User-Feld.

Top Users

Die Rangliste zeigt, wer die meisten Punkte hat. Der kleine Knopf oben rechts in der Karte **aktualisiert** die Liste — nötig ist er selten, das passiert auch von allein.

Reset Points und Delete All Users

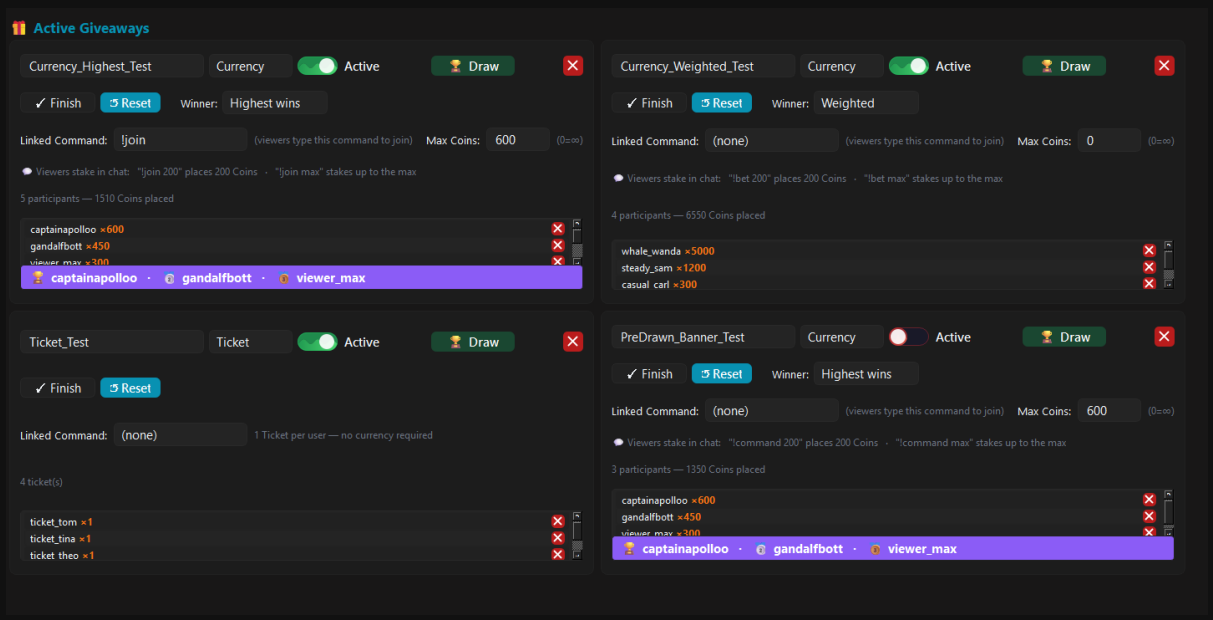
Oben rechts auf der Seite liegen zwei Knöpfe, die das **gesamte** Punkte-System betreffen:

- **Reset Points** — setzt die Punkte **aller** Zuschauer auf 0. Die Zuschauer selbst bleiben in der Liste.
- **Delete All Users** — löscht **alle** Zuschauer aus dem Punkte-System.

> Beide Knöpfe wirken auf **jeden** Zuschauer gleichzeitig. Wenn deine Community monatelang Punkte gesammelt hat, ist das mit einem Klick weg. Zweimal hinsehen, bevor du drückst.

7.2 Giveaways anlegen

+ Add Giveaway oben rechts legt ein neues Giveaway an. Du kannst **beliebig viele** anlegen — jedes wird als eigene Karte dargestellt, lässt sich umbenennen, aktivieren, deaktivieren und mit dem roten ✕ wieder löschen.



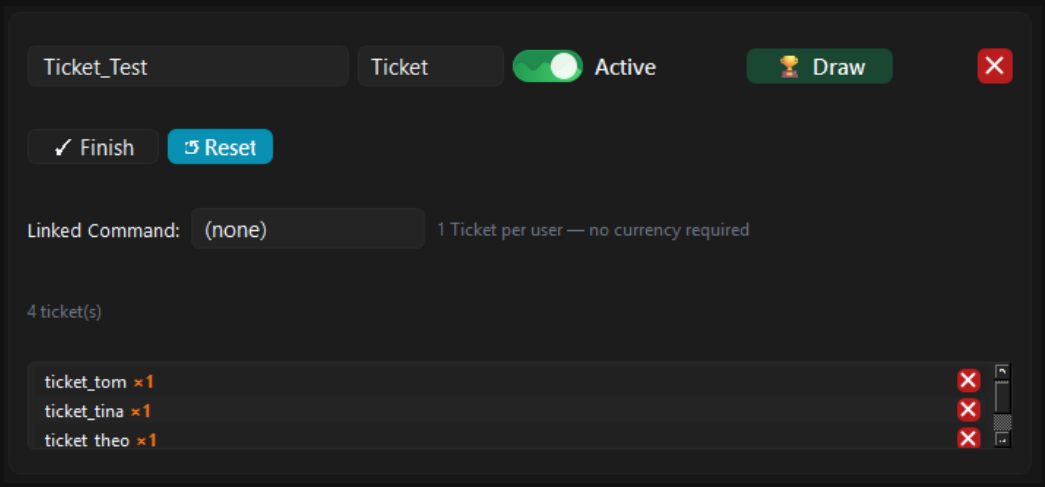
Mehrere Giveaways nebeneinander — hier im Modus Ticket und im Modus Currency

> Alle Giveaways greifen auf **dasselbe** Punkte-System zu. Es gibt nicht pro Giveaway eine eigene Währung — die Coins, die ein Zuschauer im einen Giveaway einsetzt, fehlen ihm im anderen.

7.3 Ticket oder Currency — der wichtigste Schalter

Jedes Giveaway läuft in einem von zwei Modi. Der Unterschied ist groß, größer als er aussieht.

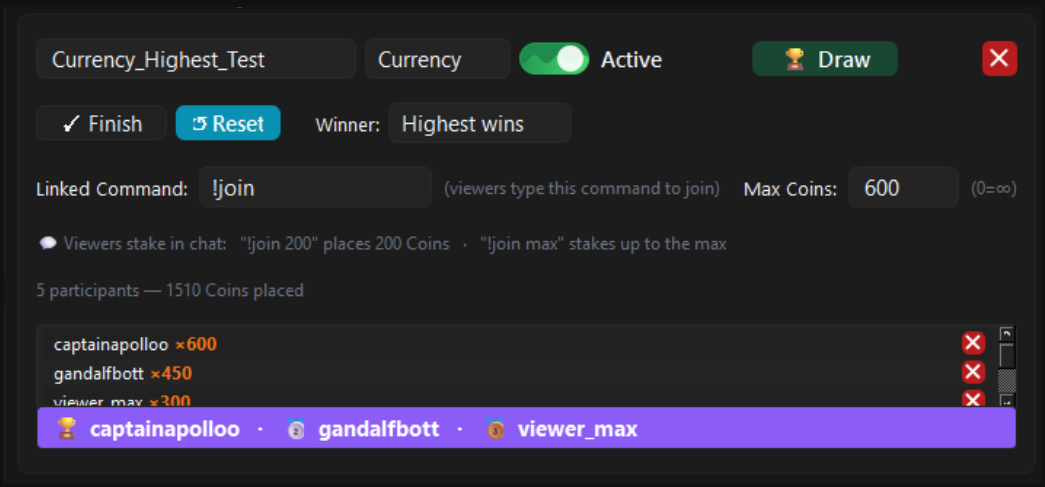
Ticket — die echte Verlosung



Ein Giveaway im Modus Ticket

Im Modus **Ticket** hat jeder Teilnehmer genau **ein** Ticket. Es kostet keine Währung, und beim Ziehen entscheidet der **Zufall**. Der Zuschauer, der zum ersten Mal reinschaut, hat dieselbe Chance wie dein treuester Stammgast. Zum Beitreten reicht der Command allein.

Currency — der Einsatz zählt



Ein Giveaway im Modus Currency — der Teilnehmer hat 500 Coins gesetzt

Im Modus **Currency** setzen die Zuschauer ihre gesammelten Punkte ein. **Max Coins** legt fest, wie viel ein einzelner Zuschauer höchstens investieren darf — `0` bedeutet unbegrenzt. Unter der Karte siehst du, wer mitmacht und wie viel er gesetzt hat.

Rechts oben in der Karte, neben **Reset**, sitzt das Dropdown **Winner** — es entscheidet, **wie** im Currency-Modus gezogen wird:

- **Highest wins** — der höchste Einsatz gewinnt **immer**, ohne Zufall. Setzt A 500 Coins und B 499, gewinnt A. Jedes Mal — eine Versteigerung, keine Verlosung.
- **Weighted** — der Zufall entscheidet, aber **gewichtet nach Einsatz**: Wer mehr setzt, hat bessere Chancen, aber nie 100 %. Wer 10 setzt, behält gegen jemanden mit 100000 eine winzige Chance. Es gewinnt trotzdem genau **einer**.

> 💡 **Highest wins** belohnt den, der am meisten riskiert — spannend, aber vorhersehbar. **Weighted** hält es offen: Auch kleine Einsätze können gewinnen, große aber deutlich wahrscheinlicher. Wer eine echte Gleichverteilung will, bei der jeder dieselbe Chance hat, nimmt **Ticket**.

> Die eingesetzten Punkte werden dem Zuschauer **abgezogen**. Sie sind ausgegeben — auch dann, wenn er nicht gewinnt.

7.4 Linked Command — wie Zuschauer beitreten

Im Feld **Linked Command** trägst du den Chat-Befehl ein, mit dem man mitmacht. `!join` ist nur ein **Beispiel** — du kannst jeden beliebigen Command nehmen.

- Im Modus **Ticket** reicht der Befehl allein: `!join`.
- Im Modus **Currency** kommt der Einsatz dahinter: `!join 500` setzt 500 Punkte.
- `!join max` setzt **alles**, was der Zuschauer hat — begrenzt durch **Max Coins**.

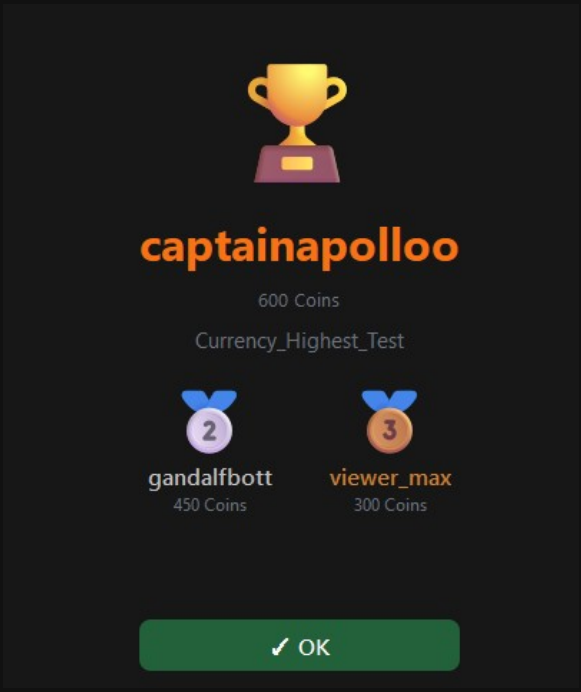
Unter der Karte läuft die Teilnehmerliste mit. Der **rote Knopf** an einer Zeile nimmt **einen einzelnen** Teilnehmer wieder heraus.

Wird die Liste lang, öffnet 🔍 **Open list** neben der Teilnehmerzahl ein eigenes Fenster mit allen Teilnehmern. Oben sitzt ein **Suchfeld**: Tipp einen Namen ein, und die Liste filtert sofort mit. Löschen funktioniert dort genauso wie auf der Karte, und die Änderungen sind sofort gespeichert.

7.5 Draw, Finish und Reset

- **Draw** — zieht den Gewinner. Er erscheint in einem eigenen Fenster. Wurde schon einmal gezogen, fragt ASTO-BOT vorher nach, ob wirklich **neu** gezogen werden soll — ein Fehlklick überschreibt das Ergebnis also nicht.
- **Finish** — beendet das Giveaway und **deaktiviert** es gleich mit.
- **Reset** — setzt Gewinner und **alle** Teilnehmer zurück. Das Giveaway steht dann wieder bei null.

Das Gewinner-Fenster



Das Gewinner-Fenster mit Podium: Gold, Silber und Bronze

Ganz oben steht der **Gewinner** (1. Platz) mit seinem Einsatz. Darunter — sofern genug Teilnehmer da waren — das **Podium**: links **Silber** (2. Platz), rechts **Bronze** (3. Platz), jeweils mit Name und Einsatz.

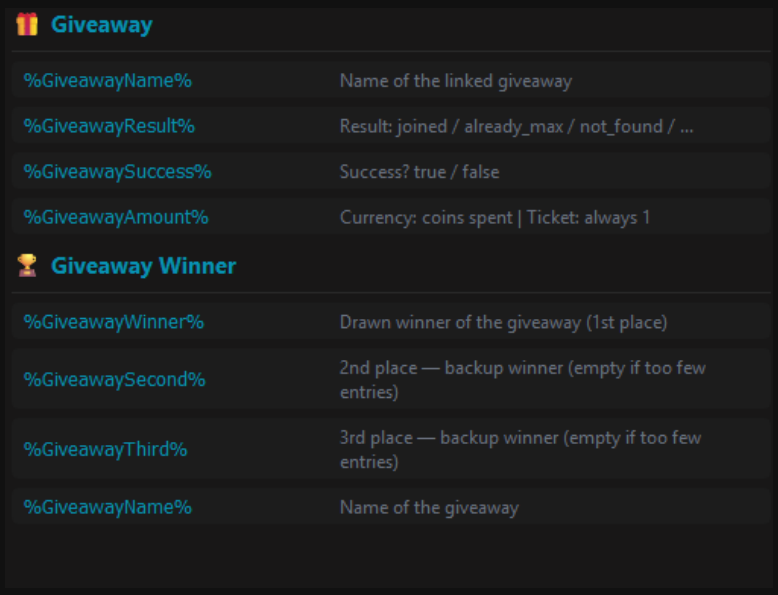
> 💡 Das Podium ist deine **Absicherung**: Ist der Gewinner nicht mehr im Chat oder nicht erreichbar, hast du mit Platz 2 und 3 sofort einen Ersatz. **Ein Klick auf einen Namen** — egal ob Gold, Silber oder Bronze — kopiert ihn in die Zwischenablage.

> Im Modus **Ticket** zeigt das Fenster nur die Namen, keine Einsätze — dort hat ja jeder genau ein Ticket.

Auch auf der Karte selbst bleibt das Podium sichtbar: Nach dem Ziehen steht dort ein Banner in der Form 🏆 **Gewinner** · 🥈 **Zweiter** · 🥉 **Dritter**.

7.6 Den Gewinner im Stream feiern

Das Gewinner-Fenster sieht nur **du**. Damit dein Chat etwas davon hat, verknüpfst du das Giveaway mit einem Event: Im Event wählst du den Trigger **Giveaway Winner** und dort das Giveaway aus. Sobald du **Draw** drückst, feuert das Event — und ASTO-BOT kann den Gewinner ausrufen, einen Sound spielen und ein Overlay zeigen.



Die Giveaway-Variablen im Variables-Fenster

Zwei Kategorien, die man nicht verwechseln sollte:

- **Giveaway** — diese Variablen beschreiben den **Beitritt**, also den Moment, in dem jemand `!join` tippt: `%GiveawayName%` (das verknüpfte Giveaway), `%GiveawayResult%` (`joined`, `already_max`, `not_found` …), `%GiveawaySuccess%` (`true` / `false`) und `%GiveawayAmount%` (bei Currency die eingesetzten Coins, bei Ticket immer `1`).
- **Giveaway Winner** — diese beschreiben die **Ziehung**: `%GiveawayWinner%` ist der gezogene Gewinner, `%GiveawayName%` das Giveaway.
- Neu sind **`%GiveawaySecond%`** und **`%GiveawayThird%`** — der **zweite** und **dritte** Platz des Podiums. Damit kannst du im Event auch die Ersatz-Gewinner ausrufen, falls dein eigentlicher Gewinner nicht erreichbar ist. Gab es zu wenige Teilnehmer, bleiben sie leer.

> 💡 Der Gewinnername steht in **`%GiveawayWinner%`** — nicht in `%UserName%`. Wenn dein Gewinner-Event den falschen Namen ausruft, ist fast immer das die Ursache.

Automatisch ziehen lassen

Du musst **Draw** nicht selbst drücken. Die Aktion **Draw Giveaway** (Kapitel 5.6, Gruppe *Giveaway*) zieht das Giveaway aus einem Event heraus — angestoßen von einem Reward, einem Chat-Befehl oder einem Timer. Gewinner und Podium landen dabei genauso in der Giveaway-Seite, als hättest du den Knopf gedrückt.

Neben `%GiveawayWinner%`, `%GiveawaySecond%` und `%GiveawayThird%` liefert die Aktion auch `%GiveawayEntryCount%` — die Zahl der Teilnehmer. Damit wird aus einer nüchternen Ansage eine kleine Show: *„🎉 Gewonnen hat %GiveawayWinner% von %GiveawayEntryCount% Teilnehmern!"*

> ⚠ Ein Event, das auf **Giveaway Winner** hört, darf nicht gleichzeitig dasselbe Giveaway per **Draw Giveaway** mit **Announce** ziehen — sonst löst es sich endlos selbst aus. Trenne beides auf zwei Events, oder schalte im ziehenden Event **Announce** aus.

Wer lieber schreibt als klickt, macht dasselbe in ASTOSCRIPT: `GIVEAWAY DRAW $gewinner "Name"` zieht und legt den Gewinner in einer eigenen Variablen ab, `GIVEAWAY ENTRIES`, `GIVEAWAY WINNER` und `GIVEAWAY ACTIVE` lesen ein Giveaway aus, ohne zu ziehen. Mit dem Zusatz `ANNOUNCE` feuern zusätzlich die Gewinner-Events. Alle Befehle mit Beispielen findest du im Tutorial der Scripts-Seite und in Kapitel 11.

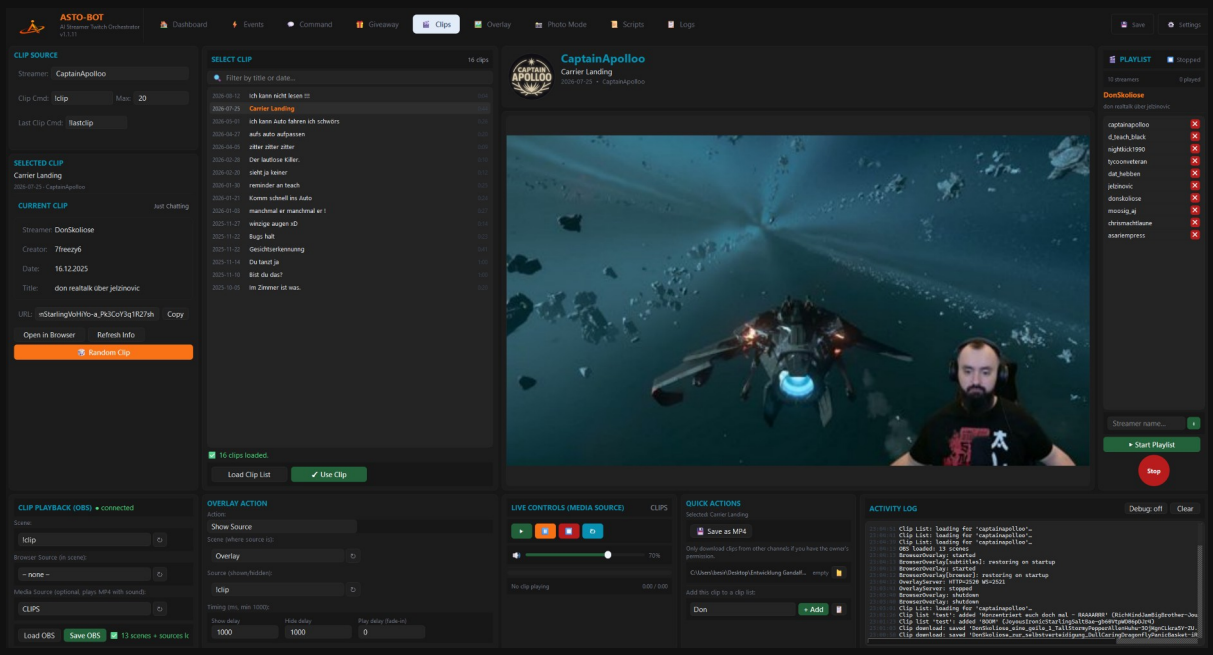
Punkte aus Events heraus vergeben

Das Punkte-System hängt nicht nur an der Uhr. In jedem Event kannst du mit den Aktionen **Add Points** und **Remove Points** (Gruppe *Points*, Kapitel 5.6) Punkte verschenken oder abziehen — zum Beispiel als Bonus für ein Sub, für einen Raid oder als Preis in einem Chat-Spiel.

> 💡 Eine runde Sache: Ein Event vergibt bei jedem Follow 500 Coins, ein zweites Event ruft nach dem Draw den Gewinner aus. Zwischen beidem liegt nur das Punkte-System — und dein Chat hat einen Grund, dabeizubleiben.

8. Clips

Der Clips-Bereich ist Kommandozentrale und Einrichtungswerkstatt in einem: Hier suchst du Clips heraus und spielst sie ab — und hier legst du fest, **wie** Clips überhaupt in deinem Stream erscheinen.



Der Clips-Bereich mit Clip-Liste, Vorschau, Playback-Einstellungen und Playlist

> 💡 Die Einstellungen unter **Clip Playback** und **Overlay Action** gelten für **alle** Wege, auf denen ein Clip abgespielt werden kann — für diese Seite, für `!clip` und `!lastclip`, für die Aktionen **Clip Player**, **Clip List Player** und **Play Last Clip** aus dem Events-Kapitel und für die Playlist. Alles läuft über **dieselbe** Quelle und wird gleich ein- und ausgeblendet. Einmal sauber einrichten, und du hast Ruhe.

8.1 Streamer und Chat-Befehle

Oben links trägst du bei **Streamer:** einen Namen ein — jeden beliebigen, Hauptsache die Person hat Clips. Dieses Feld gilt **nur für diese Seite**: Es bestimmt, wessen Clips du hier lädst und abspielst.

> Das **Streamer**-Feld hat **nichts** mit den beiden Chat-Befehlen zu tun. Bei denen schreibt der Zuschauer den Namen selbst hinter den Befehl in den Twitch-Chat.

- **Clip Cmd** — der Befehl für einen **zufälligen** Clip. Tippt jemand `!clip CaptainApolloo` in den Chat, läuft ein zufälliger Clip dieses Kanals.
- **Max Clips** — aus wie vielen Clips dabei gewühlt wird. Steht dort `20`, würfelt ASTO-BOT unter den **20 neuesten** Clips.
- **Last Clip Cmd** — der Befehl für den **neuesten** Clip: `!lastclip CaptainApolloo`.

Beide Befehle lassen sich jederzeit umbenennen. `!clip` und `!lastclip` sind nur die Vorgabe, keine Vorschrift.

Ein Clip-Befehl kann zusätzlich ein **Event** auslösen — für eine Ansage im Chat zum Beispiel. Dafür verknüpfst du den Befehl im **Command**-Bereich mit einem Event wie jeden anderen auch; der Clip läuft trotzdem. Das Event feuert erst, wenn der Clip **feststeht**: Nur so beschreiben `%ClipStreamer%`, `%ClipTitle%`, `%ClipGame%` und die übrigen `%Clip…%` den Clip, der gerade startet, und nicht den davor. Wird keiner gefunden, läuft das Event trotzdem — dann mit `%ClipFound%` = `false`, womit sich eine Ersatzansage bauen lässt.

8.2 Clips laden und auswählen

Zwei Knöpfe laden die Clips des oben eingetragenen Streamers, sortiert nach Datum. **Load max 300 Clips** holt die neuesten 300 — der schnelle Weg für den Alltag. **All Clips** holt das ganze Archiv; bei Kanälen mit vielen Clips dauert das einen Moment, rechne mit 5 bis 20 Sekunden.

> Die Liste zeigt höchstens **300 Zeilen** an. Steht darunter ein Hinweis wie *Showing 300 of 1200 clips*, sind die übrigen trotzdem geladen — die **Suche** findet sie alle.

- Ein **Klick** auf einen Clip **markiert** ihn — er erscheint links unter **Selected Clip**.
- Ein **Doppelklick** — oder der Knopf **Use Clip** — spielt ihn in OBS ab.
- Ein Klick auf das große **Thumbnail** öffnet den Clip im Browser.

Über der Liste liegt ein **Suchfeld**. Es filtert die bereits geladenen Clips nach Titel, Datum oder Ersteller — es lädt nichts nach. Rechts oben in der Karte steht, wie viele Clips gerade angezeigt werden, und in jeder Zeile rechts die Länge des Clips. Während **Load Clip List** läuft, zeigt ein Balken unter der Liste, wie weit ASTO-BOT ist.

> Twitch nennt keine Gesamtzahl an Clips. Der Balken zeigt deshalb nicht „347 von 594“, sondern wie weit ASTO-BOT den Zeitraum bis 2016 rückwärts abgesucht hat. Daneben steht mit, wie viele Clips bisher zusammengekommen sind.

Selected Clip und Current Clip

Diese beiden Blöcke sehen ähnlich aus, meinen aber Verschiedenes:

- **Selected Clip** — der Clip, den du in der Liste **angeklickt** hast. Ändert sich bei jedem Klick.
- **Current Clip** — der Clip, der zuletzt **abgespielt** wurde. Streamer, Spiel, Ersteller, Datum und Titel, darunter die **Clip URL** mit **Copy**-Knopf.

> **Current Clip** ist mehr als eine Anzeige: Diese Daten stehen in den Dateien im Ordner **Twitch Daten** und speisen `%ClipTitle%` und die anderen Clip-Variablen in Events und Overlays — und `!lastclip` spielt genau diesen Clip. Deshalb ändert ein bloßer Klick in der Liste daran **nichts**.

- **Open in Browser** — spielt den Clip in deinem Standardbrowser ab.
- **Random Clip** — spielt einen zufälligen Clip ab.
- **Refresh Info** — lädt die Informationen neu, falls das nicht von allein passiert ist.

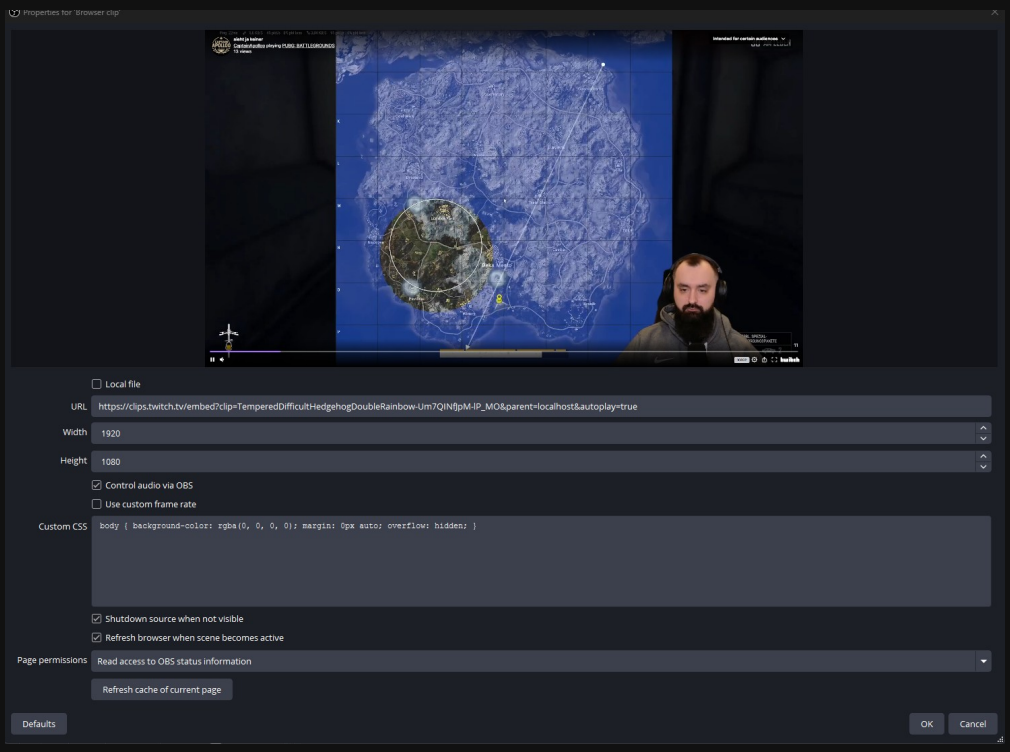
8.3 Clip Playback — die OBS-Quelle einrichten

Damit ein Clip im Stream zu sehen ist, braucht ASTO-BOT eine Quelle in OBS. Unten links wählst du zuerst die **Scene**, in der die Quelle liegt, und dann eine von zwei Möglichkeiten:

- eine **Browser Source** — spielt den Clip über den Twitch-Player,
- oder eine **Media Source** — spielt die MP4-Datei direkt ab, mit Ton.

> Setzt du **beide**, wird die **Media Source bevorzugt**. Wenn du das nicht willst, stell die andere Quelle auf **– none –**.

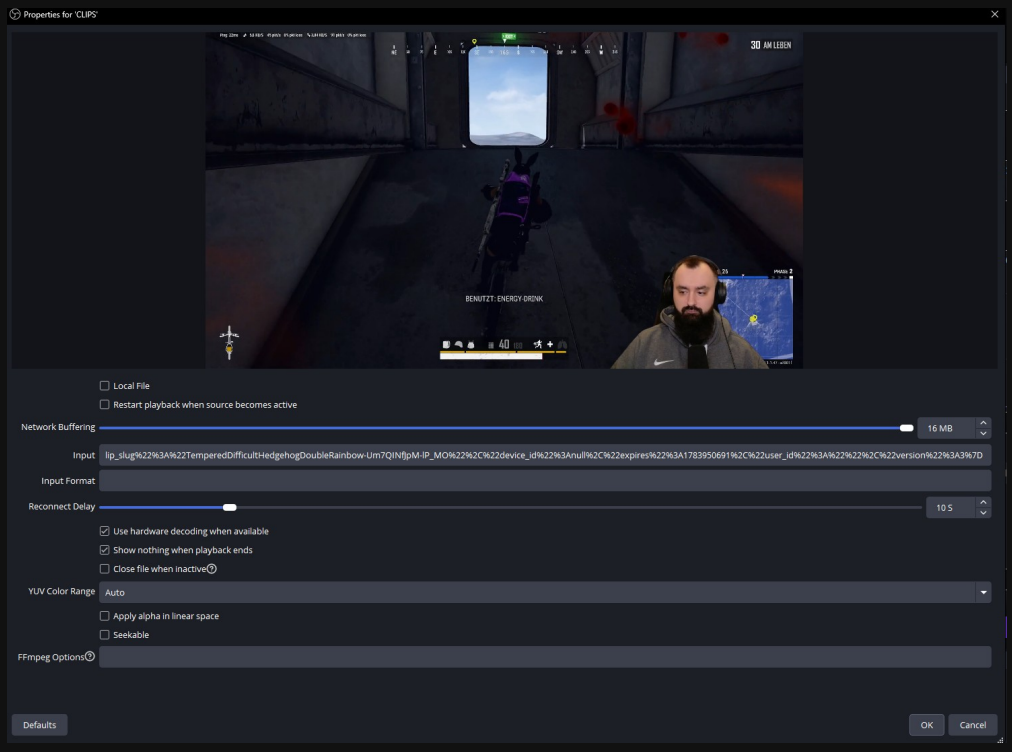
Die Browser-Quelle in OBS



Die Eigenschaften der Browser-Quelle in OBS

- **Local file — aus**.
- **Control audio via OBS — an**.
- **Shutdown source when not visible — an**.
- **Refresh browser when scene becomes active — an**.

Die Media-Quelle in OBS



Die Eigenschaften der Media-Quelle in OBS

- **Local File** — **aus**.
- **Network Buffering** — mindestens **4 MB**.
- **Use hardware decoding when available** — **an**.
- **Show nothing when playback ends** — **an**.
- **YUV Color Range** — **Auto**.

> Die URL beziehungsweise den Input trägt **ASTO-BOT selbst** ein. Du musst dort nichts eingeben — in den Screenshots steht nur deshalb etwas drin, weil gerade ein Clip läuft.

Live Controls

Unter **Live Controls (Media Source)** steuerst du den laufenden Clip von Hand: **Play**, **Pause**, **Stop** und **Reset** (spielt den letzten Clip noch einmal). Daneben liegt der **Lautstärke-Regler**. Rechts oben in der Karte steht der Name der Media Source, die gerade gesteuert wird.

Darunter läuft ein **Fortschrittsbalken** mit Titel und Zeit mit. Er gilt für **jeden** Clip — egal ob du ihn hier startest, ob ihn `!clip` auslöst, ein Kanalpunkt-Reward oder die Playlist.

Ist OBS erreichbar?

Im Kopf der Karte **Clip Playback (OBS)** steht ein Punkt: **grün** heißt verbunden, **rot** heißt nicht erreichbar, **grau** heißt noch nicht geprüft. ASTO-BOT fragt jede Minute nach und außerdem bei jedem echten Zugriff auf OBS.

8.4 Overlay Action — das Drumherum

Die Szene, in der der Clip **abgespielt** wird, muss nicht die Szene sein, die dein Publikum **sieht**. Genau dafür ist die **Overlay Action** da: Sie blendet beim Start eines Clips das Richtige ein — und danach wieder aus.

- **Action** — was passieren soll: **Show Source**, **Switch Scene** oder **nothing**.
- **Scene** und **Source** — was genau eingeblendet beziehungsweise gewechselt wird.

Timing

- **Show delay** — wie lange **vor** dem Start des Clips eingeblendet wird.
- **Hide delay** — wie lange **nach** dem Ende des Clips gewartet wird, bevor wieder ausgeblendet wird.
- **Play delay (fade-in)** — wie lange nach dem Einblenden gewartet wird, bis automatisch **Play** gedrückt wird.
- Für **Show delay** und **Hide delay** sind mindestens **1000 ms** möglich.

> 💡 Stell **Show delay** und **Hide delay** genauso ein wie die **Show- und Hide-Transition** deiner Quelle in OBS. Dann passt das Einblenden zum Start des Clips, und nichts ruckelt gegeneinander.

Load OBS lädt alle Szenen und Quellen aus OBS, **Save OBS** speichert deine Einstellungen.

8.5 Die Clip Playlist

Rechts am Rand liegt die **Clip Playlist** — ein Dauerprogramm aus fremden Clips, ideal für die Pause, den Startabend oder einfach als Hintergrund.

- Trage unten beliebig viele **Streamernamen** ein. Der grüne Knopf fügt einen hinzu, das rote ✕ entfernt ihn wieder.
- **Start Playlist** startet das Programm, **Stop** beendet es.
- ASTO-BOT zieht dann einen **zufälligen Namen** aus der Liste und davon einen **zufälligen Clip** aus den **100 neuesten** dieses Kanals.
- Während ein Clip läuft, wählt ASTO-BOT bereits den nächsten aus — so gibt es keine Lücke dazwischen.

> Die Playlist läuft **endlos**, bis du **Stop** drückst. Sie hört nicht von allein auf.

> 💡 Eine schöne Geste für die Community: Trag die Namen der Leute ein, die in deinem Kanal unterwegs sind, und lass die Playlist in der Pause laufen. Dann sieht dein Chat die Clips der eigenen Leute.

8.6 Quick Actions — Clips speichern und einsortieren

Die Karte **Quick Actions** arbeitet immer mit **einem** Clip. Welcher gemeint ist, steht oben in der Karte: Ist in der Liste ein Clip markiert, gilt dieser — sonst der unter **Current Clip**. Damit kannst du dich durch die Liste klicken und Clips wegspeichern, ohne jeden erst abspielen zu müssen.

Save as MP4

Speichert den Clip als Videodatei auf deine Festplatte.

- Der Zielordner ist zunächst **Downloaded Clips** neben der ASTO-BOT.exe. Über das **Ordnersymbol** wählst du einen beliebigen anderen — die Wahl bleibt gespeichert.
- Daneben steht laufend, wie viele Dateien im Ordner liegen und wie viel Platz sie belegen.
- Während des Ladens siehst du den Fortschritt in Prozent.
- Ein Clip, der schon im Ordner liegt, wird **nicht** noch einmal geladen.

> Lade Clips **fremder** Kanäle nur herunter, wenn du die Erlaubnis der Rechteinhaber hast. Dass ein Clip öffentlich abspielbar ist, heißt nicht, dass er dir gehört.

> 💡 Ein abgebrochener Download hinterlässt keine halbe Datei: Solange der Clip noch lädt, heißt die Datei `.part` und bekommt ihren richtigen Namen erst, wenn sie vollständig ist.

Add this clip to a clip list

Das Dropdown zeigt alle vorhandenen **Clip-Listen**. **+ Add** trägt den aktuellen Clip in die gewählte Liste ein.

- Der Knopf mit dem 📋 öffnet den **Clip List Manager**. Dort legst du neue Listen an, benennst Clips um oder löschst sie.
- Ein ✔ **Use** im Manager übernimmt die Liste — das Dropdown in den Quick Actions springt dann auf sie.
- Ist der Clip schon in der Liste, sagt ASTO-BOT das und trägt ihn kein zweites Mal ein.

> Es sind dieselben Listen, die der **Clip List Player** in den Events abspielt. Was du hier einsortierst, steht dort sofort zur Verfügung.

8.7 Activity Log

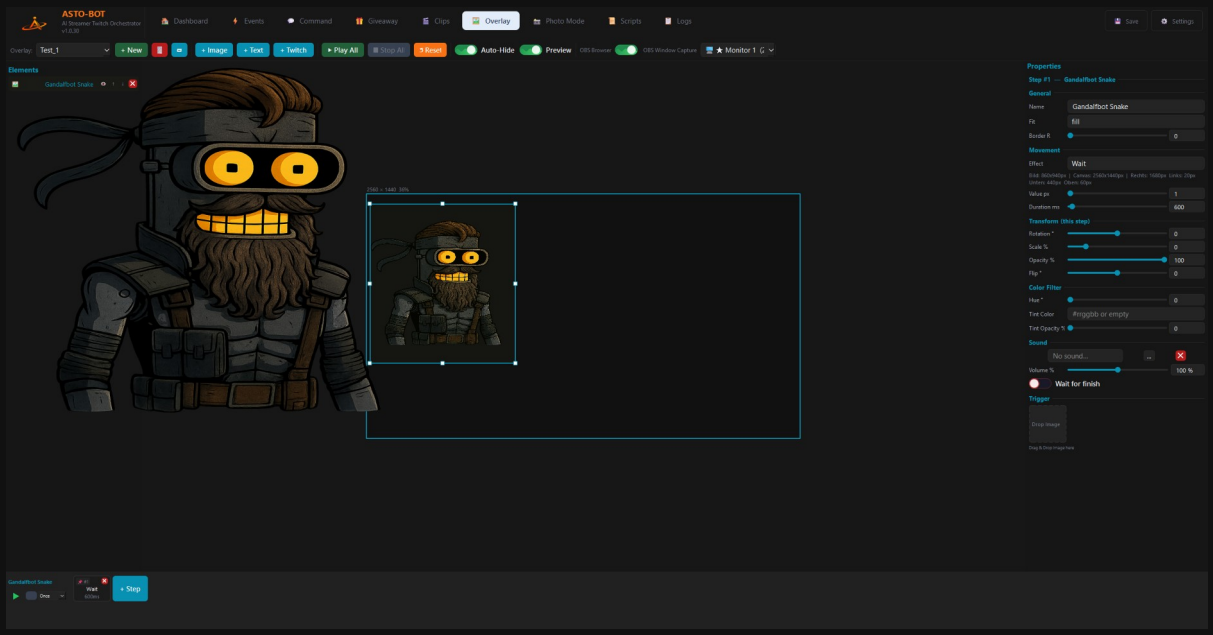
Rechts unten läuft das **Activity Log** mit — die letzten Meldungen rund um Clips, OBS und Playlist, **neueste Zeile oben**. Es zeigt auch Clips, die durch ein **Event** ausgelöst wurden, nicht nur die von dieser Seite.

- **Debug: off / on** blendet zusätzlich die technischen Zeilen ein. Für die Fehlersuche nützlich, im Alltag zu viel.
- **Clear** leert nur die **Anzeige**. Die Logdatei selbst bleibt vollständig erhalten.

> 💡 Wenn ein Clip nicht kommt, steht hier meistens schon der Grund — eine fehlende Quelle, ein nicht erreichbares OBS oder eine MP4, die sich nicht auflösen ließ.

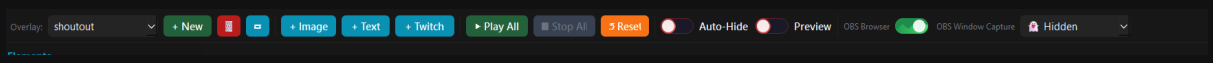
9. Overlay

Im Overlay-Bereich baust du die Einblendungen, die dein Stream zeigt: ein Bild, das hereinfliegt, ein Text mit dem Namen des Spenders, das Profilbild des Zuschauers, dazu ein Sound. Gestartet wird ein Overlay später aus einem Event heraus — mit der Aktion **Play Overlay** (Kapitel 5.6).



Der Overlay-Editor: links die Elemente, in der Mitte das Canvas, rechts die Eigenschaften, unten die Steps

9.1 Die Kopfleiste



Die Kopfleiste des Overlay-Editors

- **Overlay:** — das Dropdown wählt aus, welches Overlay du gerade bearbeitest.
- **+ New** legt ein neues an, der rote **Mülleimer** löscht das gewählte, der Knopf daneben **benennt es um**.
- **+ Image** fügt ein Bild oder GIF hinzu, **+ Text** ein Textfeld, **+ Twitch** das Profilbild eines Zuschauers.
- **Play All** spielt alle Animationen des Overlays ab, **Stop All** stoppt sie, **Reset** setzt alle Elemente auf ihre Startposition zurück.
- **Auto-Hide** — die Elemente sind nur zu sehen, solange das Overlay abspielt: Beim Start werden sie eingeblendet, am Ende wieder ausgeblendet.
- **Preview** — blendet das Overlay zum Ansehen ein.
- 🔒 **Schloss** — sperrt die Elemente gegen die Maus. Auswählen und Eintippen geht weiter, tot sind nur Verschieben, Skalieren und Drehen. Gilt pro Overlay, übersteht einen Neustart und wandert beim Umbenennen mit.

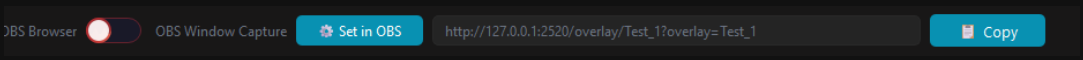
> 💡 Bilder und GIFs kannst du auch einfach per **Drag & Drop** direkt ins Canvas ziehen. Das ist der schnellste Weg.
>
> 💡 Fährst du mit der Maus über einen Knopf der Kopfleiste, erklärt dir eine kurze Einblendung, was er tut.

> Benennst du ein Overlay um, ziehen Events, die es mit **Play Overlay** abspielen, **nicht** mit. Trag dort danach den neuen Namen ein.

9.2 Wie das Overlay nach OBS kommt

Rechts in der Kopfleiste entscheidest du, **wie** OBS das Overlay aufnimmt. Der Schalter steht zwischen **OBS Browser** und **OBS Window Capture** — und zwar **pro Overlay**: Jedes Overlay hat seine eigene Einstellung.

OBS Browser



Im Browser-Modus liefert ASTO-BOT die URL gleich mit

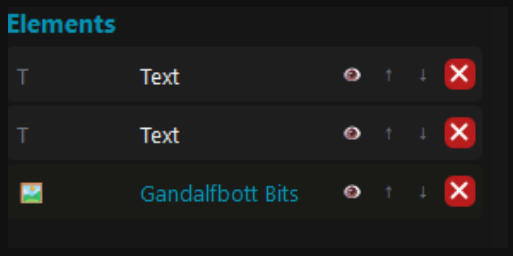
Im Browser-Modus erscheint der Knopf **Set in OBS**: Er trägt die URL selbständig in die Browser-Quelle in OBS ein. Daneben steht dieselbe URL zum Nachlesen, mit einem **Copy**-Knopf für den Fall, dass du sie von Hand einsetzen willst.

OBS Window Capture

Im Window-Capture-Modus zeigt ASTO-BOT das Overlay als eigenes Fenster an. Das Dropdown daneben legt fest, auf **welchem Monitor** dieses Fenster erscheint — oder du stellst es auf **Hidden**, dann bleibt es unsichtbar und wird nur von OBS aufgenommen.

> 💡 Welcher Modus der richtige ist, hängt an deinem Setup. Die Einrichtung beider Quellen in OBS ist im Kapitel Settings → OBS beschrieben.

9.3 Elemente



Die Elementliste — Auge, Pfeile, Löschen

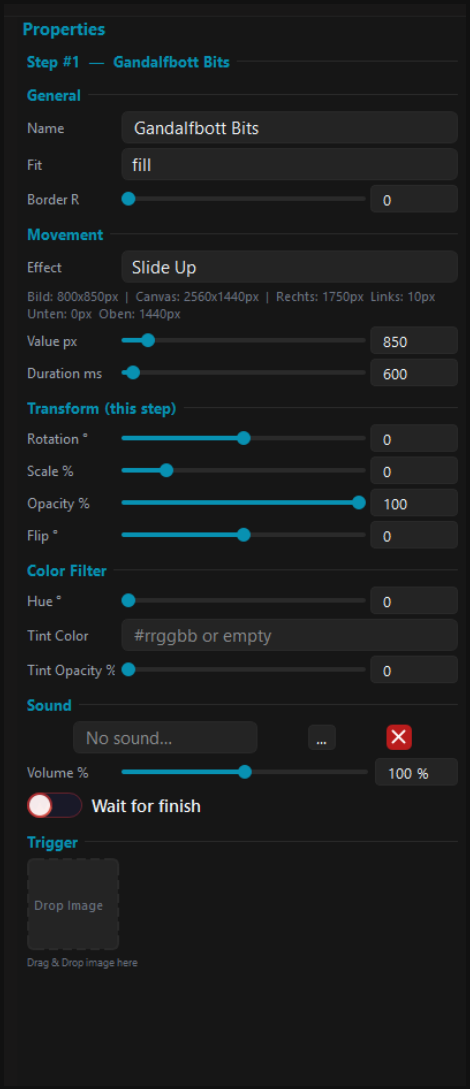
- Das **Auge** blendet ein Element im Editor aus, ohne es zu löschen.
- Die **Pfeile** ⬆ ⬇ sortieren die Elemente übereinander.
- Das rote ✕ löscht das Element.

Das Twitch-Element

+ Twitch setzt einen Platzhalter für das **Profilbild eines Zuschauers** ins Canvas. Beim Einfügen siehst du zunächst nur ein Standardbild — das ist normal.

> Damit im Stream wirklich das Bild des Zuschauers erscheint, muss im Event die Aktion **Get Target Info vor** der Aktion **Play Overlay** stehen. Get Target Info holt das Profilbild und schickt es an das Overlay. Fehlt diese Reihenfolge, bleibt das Standardbild stehen.

9.4 Die Eigenschaften eines Bildes

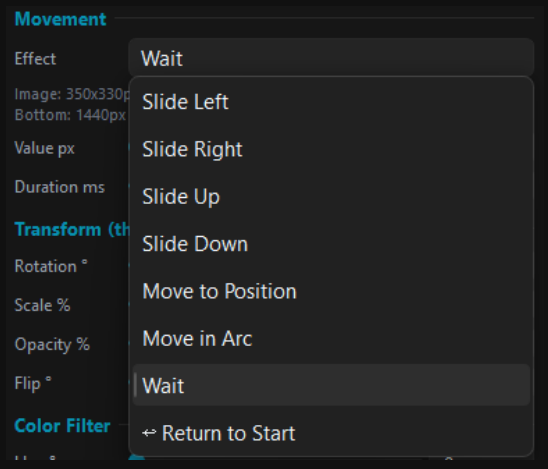


Die Eigenschaften eines Bild-Elements

General

- **Name** — wie das Element heißt.
- **Fit** — wie das Bild in seinen Rahmen passt: `fill`, `contain`, `cover` oder none. In der Praxis kannst du das meist auf **fill** stehen lassen, der Unterschied fällt kaum ins Gewicht.
- **Border R** — rundet die Ecken ab.

Movement — die Bewegung



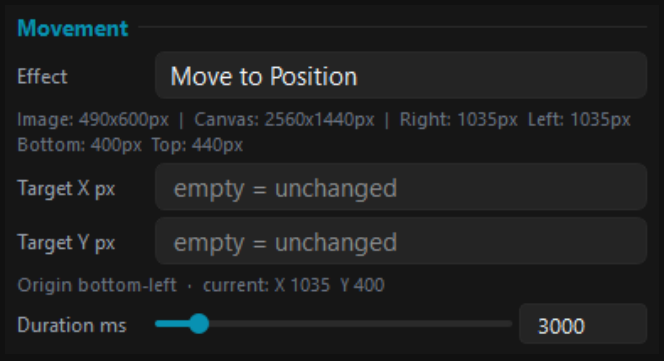
Die Bewegungseffekte im Dropdown

- **Slide Left**, **Slide Right**, **Slide Up**, **Slide Down** — das Element fährt in die jeweilige Richtung.
- **Move to Position** — das Element fährt an eine **feste Stelle** im Canvas (siehe unten).
- **Move in Arc** — das Element fährt einen **Bogen** statt einer Geraden (siehe unten).
- **Wait** — es passiert nichts, das Element bleibt stehen. Damit baust du Pausen.
- **Return to Start** — das Element springt an seinen Ausgangspunkt zurück.
- **Value px** — wie weit es sich bewegt, **Duration ms** — wie lange die Bewegung dauert.

> Die Eingabefelder wechseln mit dem Effekt: Bei **Move to Position** und **Move in Arc** verschwindet **Value px** und es erscheinen die passenden Felder. Was du dort einträgst, bleibt gespeichert — schaltest du den Effekt zurück, sind die Werte beim nächsten Mal wieder da.

Move to Position — an eine feste Stelle fahren

Statt „wie weit" gibst du hier „wohin" an. Das ist praktisch, wenn ein Element **immer an derselben Stelle** landen soll — unabhängig davon, wo es gerade steht oder was vorherige Steps mit ihm gemacht haben.



Move to Position — Ziel X und Ziel Y

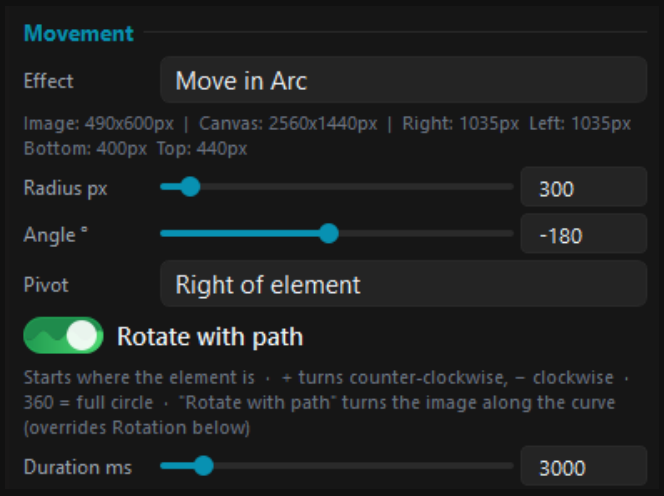
- **Der Nullpunkt liegt unten links** im Canvas: X zählt nach **rechts**, Y nach **oben**. Im Canvas siehst du dort ein türkises Achsenkreuz mit zwei Pfeilen, damit du dich nicht verirrst.
- Bezugspunkt am Element ist ebenfalls die **linke untere Ecke**. `X 0` und `Y 0` legen das Element also genau in die untere linke Canvas-Ecke.
- **Ein Feld leer lassen** heißt: diese Achse bleibt, wie sie ist. Trägst du nur Y ein, bewegt sich das Element **senkrecht** — nur X eingetragen, bewegt es sich **waagrecht**.
- **Negative Werte sind erlaubt.** So schickst du ein Element gezielt aus dem Bild heraus, zum Beispiel `X -400`, damit es links aus dem Stream fährt.

- Die kleine Zeile unter den Feldern zeigt die **aktuelle Position** im selben System — du kannst also direkt ablesen, welcher Zielwert sinnvoll ist.

> 💡 Weil X und Y gleichzeitig verändert werden, entsteht eine **schräge Bewegung**, sobald du beide Felder ausfüllst. Das Element läuft dann in einer geraden Diagonale zum Ziel — nicht erst zur Seite und dann nach oben.

Move in Arc — im Bogen fahren

Hier fährt das Element keine Gerade, sondern eine **Kurve**. Es startet dort, wo es gerade steht, läuft den Bogen ab und bleibt am Ende stehen — von da aus kannst du im nächsten Step ganz normal weitermachen, zum Beispiel mit einer Geraden.



Move in Arc — Radius, Winkel, Drehpunkt und Mitdrehen

- **Radius px** — wie weit der Drehpunkt vom Element entfernt liegt. Kleiner Radius = enge Kurve, großer Radius = weiter, sanfter Schwung.
- **Angle °** — wie weit herum es geht. **Positiv** dreht **gegen** den Uhrzeigersinn, **negativ mit** dem Uhrzeigersinn. `90` ist ein Viertelkreis, `360` eine volle Runde, `720` zwei Runden.
- **Pivot** — auf welcher **Seite** des Elements der Drehpunkt sitzt: rechts, links, oben oder unten davon.
- **Rotate with path** — das Element **dreht sich mit der Kurve mit**, so wie ein Auto in einer Kurve. Ohne den Schalter behält es seine Ausrichtung bei.

> Warum es das Feld **Pivot** braucht: Radius und Winkel allein legen den Bogen noch nicht fest. Um einen Punkt herum gibt es unendlich viele Kreise mit demselben Radius — erst die Seite, auf der der Drehpunkt liegt, macht daraus eine eindeutige Bahn. Zusammen mit dem Vorzeichen beim Winkel bekommst du damit jede Richtung hin.

> 💡 **Rotate with path** nimmt dir das Rechnen ab: Die Drehung wird automatisch aus dem Bogen abgeleitet und läuft gleichmäßig mit. Achtung — der Schalter **überschreibt** eine feste **Rotation** im selben Step. Wenn du die Drehung selbst bestimmen willst, lass ihn aus.
>
> 💡 Ein schwebender Effekt gelingt gut mit zwei Bögen hintereinander: erst `Angle 60` mit Pivot **oben**, dann `Angle -60` mit Pivot **unten**. Das ergibt eine sanfte Wellenbewegung statt eines starren Kreises.
>
> 💡 Setze **Return to Start** möglichst immer als **letzten Step**. Dann landen Bild, Text und Twitch-Bild wieder an ihrem Ausgangspunkt — und das Overlay sieht beim nächsten Mal genauso aus wie beim ersten Mal.

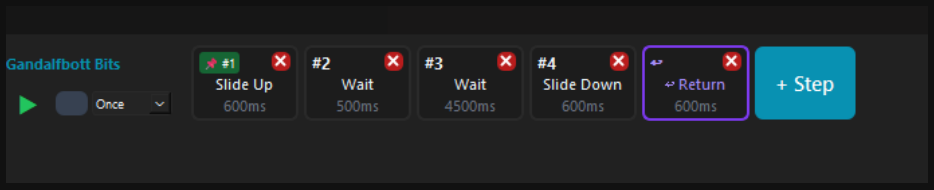
Transform, Color Filter und Sound

- **Transform (this step)** — gilt **für diesen einen Step**: **Rotation**, **Scale %**, **Opacity %** und **Flip**.
- **Color Filter** — **Hue**, **Tint Color** (als Hex) und **Tint Opacity**.
- **Sound** — jeder Step kann seinen **eigenen Sound** abspielen, mit **Volume**-Regler.

- **Wait for finish** — der nächste Step startet erst, wenn der Sound fertig ist.

9.5 Die Step-Timeline

Unten liegt die **Timeline**. Jeder **Step** ist ein Effekt mit einer Dauer, und die Steps laufen von links nach rechts nacheinander ab. **+ Step** hängt einen weiteren an.

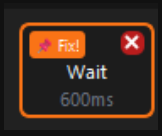


Eine Timeline mit fünf Steps — Slide Up, zwei Pausen, Slide Down, Return

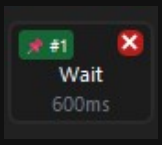
- Der **grüne Play-Knopf** links startet **nur dieses eine Element** — praktisch beim Feinschliff. Daneben liegt der Stop-Knopf.
- Das Dropdown daneben kennt zwei Werte: **Once** (einmal durchlaufen) und **Loop** (endlos wiederholen).

Der Pin — die wichtigste Kleinigkeit

Am **ersten Step** sitzt ein kleiner **Pin**. Seine Farbe sagt dir, ob die Position deines Elements gespeichert ist.



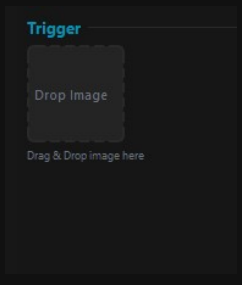
*Orange: die Position ist **nicht** gespeichert*



Grün: die Position ist gespeichert

> Immer wenn du ein Element im Canvas **von Hand verschiebst**, wird der Pin **orange**. Das heißt: Die neue Position ist **nicht** gespeichert — und beim nächsten Abspielen verschiebt sich das Element. Klick den Pin an, bis er **grün** ist. Dann sitzt die Position fest.

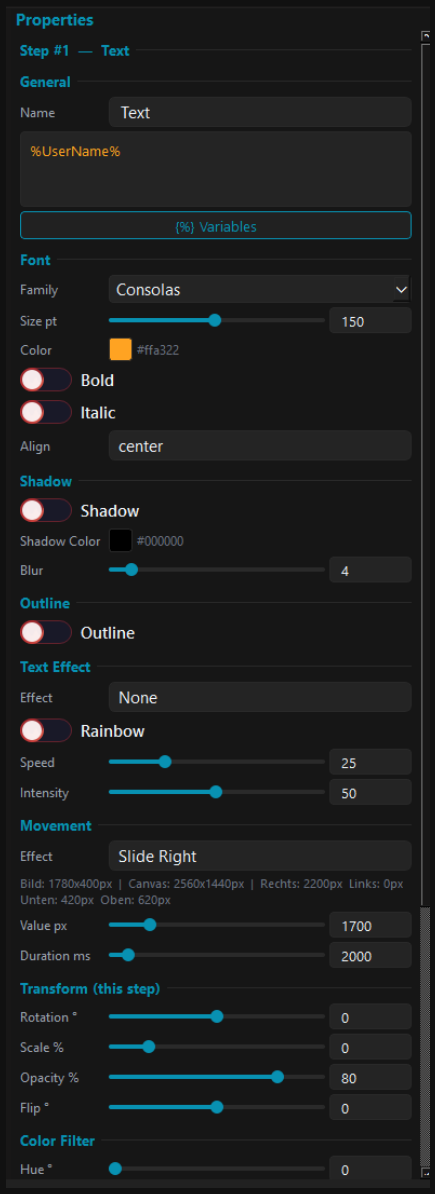
Mehrere Timelines hintereinander



Der Trigger-Bereich

Ganz unten in den Eigenschaften liegt der **Trigger**-Bereich. Ziehst du dort per Drag & Drop ein **neues Bild** hinein, entsteht **ab dem gerade gewählten Step** eine **neue Timeline** darunter. So kettest du beliebig viele Abläufe aneinander: Bild 1 fährt herein, und an einer bestimmten Stelle seiner Bewegung startet Bild 2 — und von dort aus das nächste.

9.6 Text-Elemente



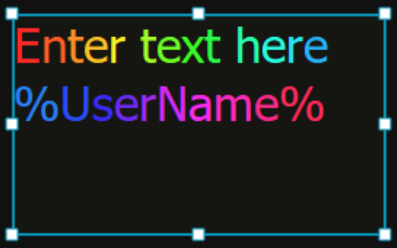
Die Eigenschaften eines Text-Elements

- Oben steht das **Textfeld**. Der Knopf **{%} Variables** öffnet die Variablen-Übersicht.
- **Font** — **Family**, **Size pt**, **Color**, **Bold**, **Italic** und **Align**.
- **Shadow** — Schatten mit Farbe und **Blur**. **Outline** — eine Kontur um den Text.
- **Movement**, **Transform** und **Color Filter** funktionieren wie beim Bild — auch Text kann sich also bewegen.

Variablen im Text

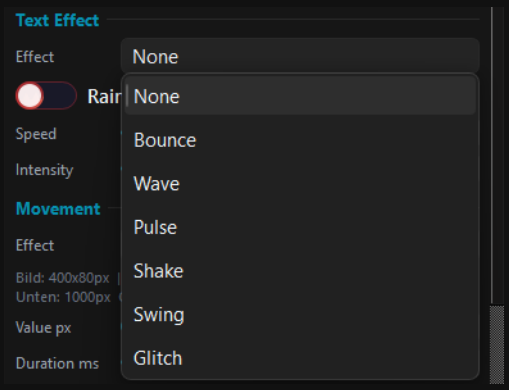
Im Text funktionieren **Variablen**: `%UserName%`, `%CheerAmount%` und alle anderen. Im Canvas siehst du weiterhin die Variable selbst stehen — das ist Absicht. Ausgewertet wird sie erst dort, wo es darauf ankommt: im Fenster und in der Browser-Quelle, also im Stream.

Farben und Text-Effekte



Einzelne Wörter lassen sich getrennt einfärben

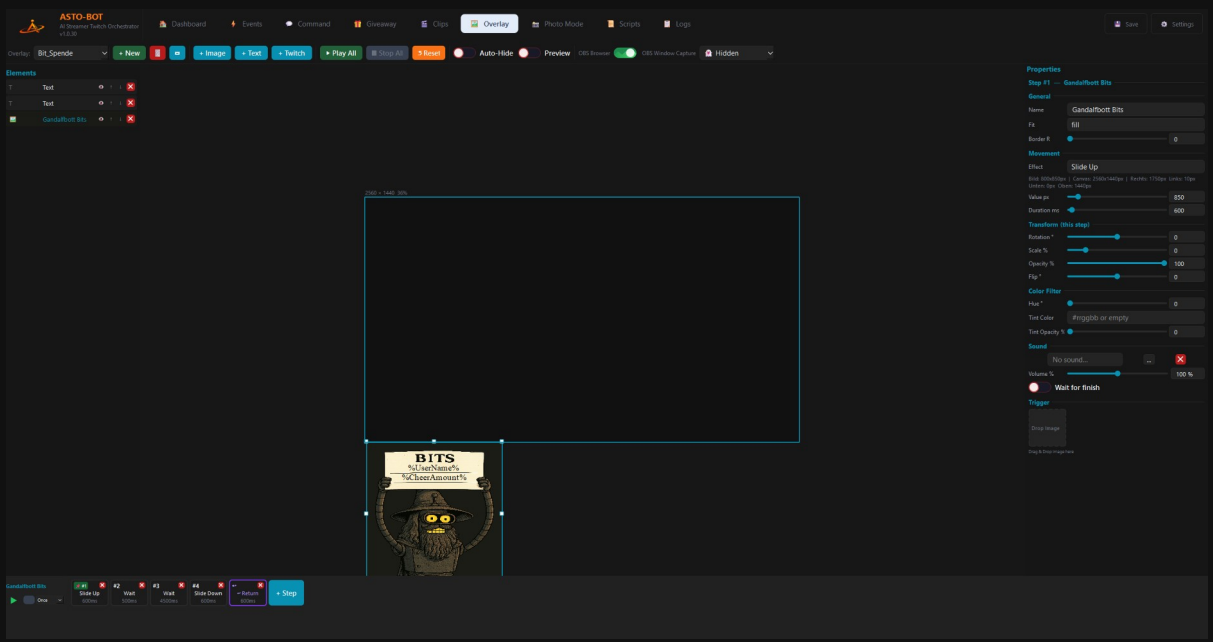
Du kannst nicht nur den ganzen Text einfärben, sondern auch **einzelne Wörter**: Wort markieren, Farbe wählen — fertig.



Die Text-Effekte

- **Text Effect** — **None**, **Bounce**, **Wave**, **Pulse**, **Shake**, **Swing** oder **Glitch**.
- **Rainbow** — ist der Schalter an, läuft der Text in wechselnden Farben. **Speed** und **Intensity** regeln, wie schnell und wie kräftig.

9.7 Ein Overlay in Aktion



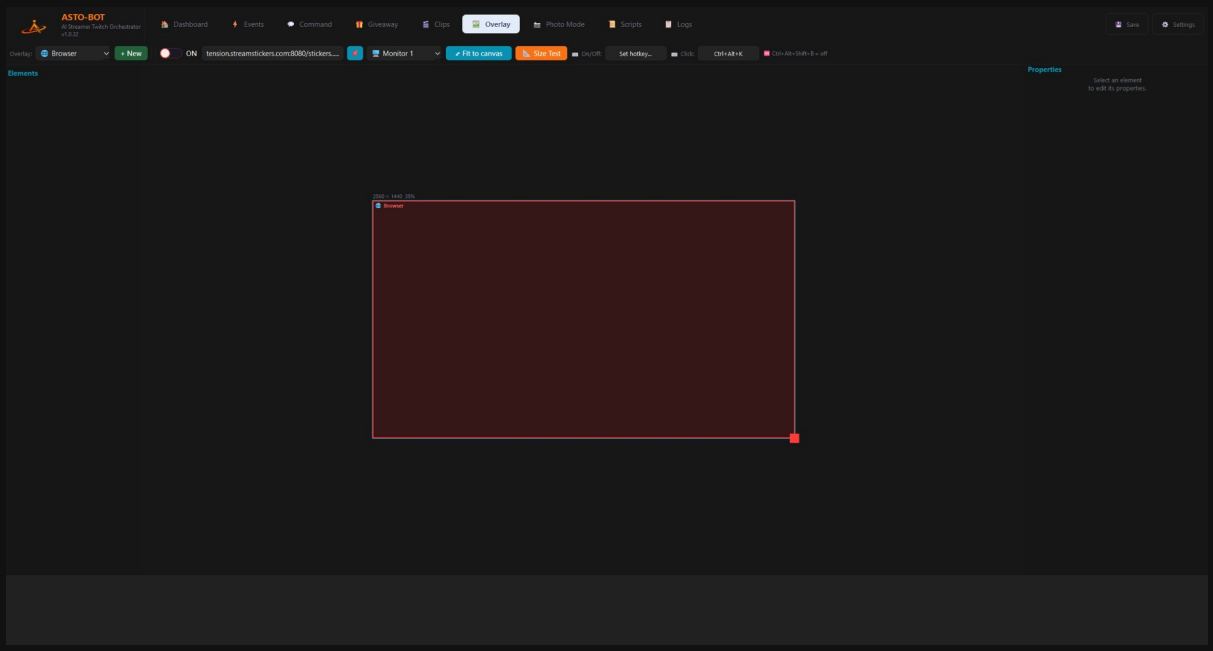
Das Overlay „Bit_Spende“: zwei Texte, ein Bild, fünf Steps

Das Beispiel zeigt, wie die Teile zusammenkommen: Ein Bild mit einem Schild, darauf zwei Text-Elemente mit `%UserName%` und `%CheerAmount%`. Die Timeline dazu ist fünf Steps lang — **Slide Up** (600 ms) fährt das Schild ins Bild, zwei **Wait**-Steps lassen es stehen (zusammen fünf Sekunden), **Slide Down** fährt es wieder hinaus, und **Return** setzt alles zurück auf Anfang.

Im Event hängt daran nur noch eine einzige Aktion: **Play Overlay** mit diesem Overlay — und, wenn ein Twitch-Bild vorkommt, davor ein **Get Target Info**.

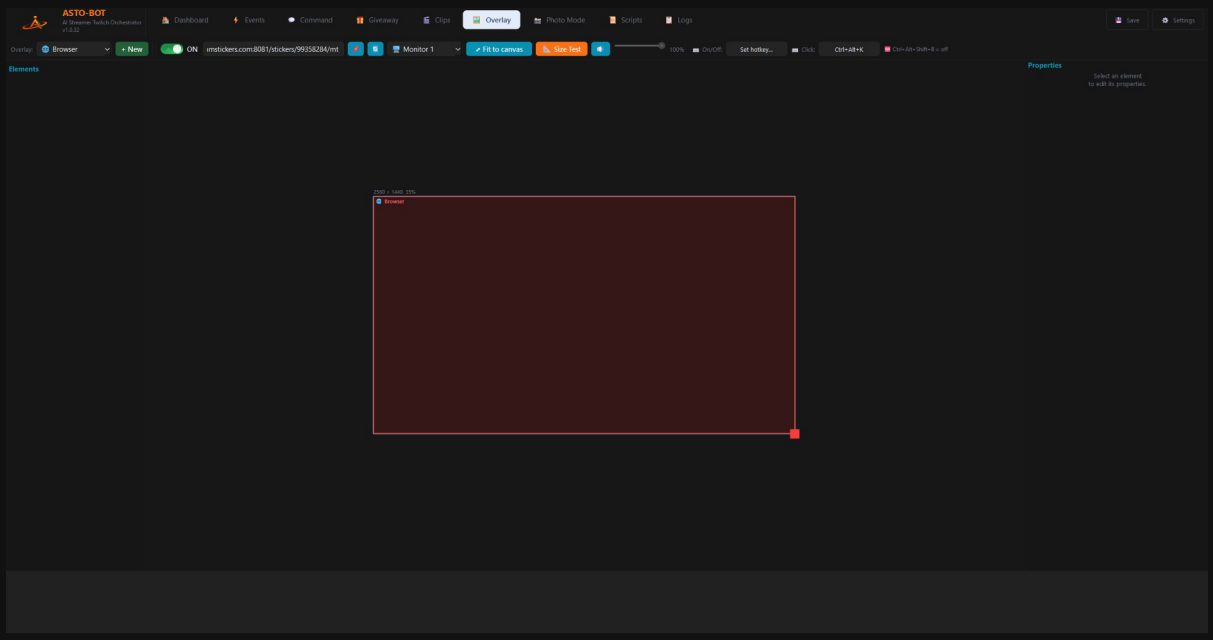
9.8 Das Browser Overlay

Das **Browser Overlay** ist ein besonderes, fest eingebautes Overlay mit dem Namen **Browser** — du findest es oben im Dropdown. Anders als bei den übrigen Overlays baust du hier **keine** Bilder oder Texte zusammen: Es lädt eine **Webseite über ihre URL** und zeigt sie an — zum Beispiel StreamStickers. Der Clou: Die Seite läuft in einem **eigenen Fenster direkt auf deinem Monitor**, nicht nur in OBS.



Das Browser Overlay im Editor — das rote Rechteck ist der Bereich, den die Webseite einnimmt

Die Kopfleiste



Die Kopfleiste des Browser Overlays

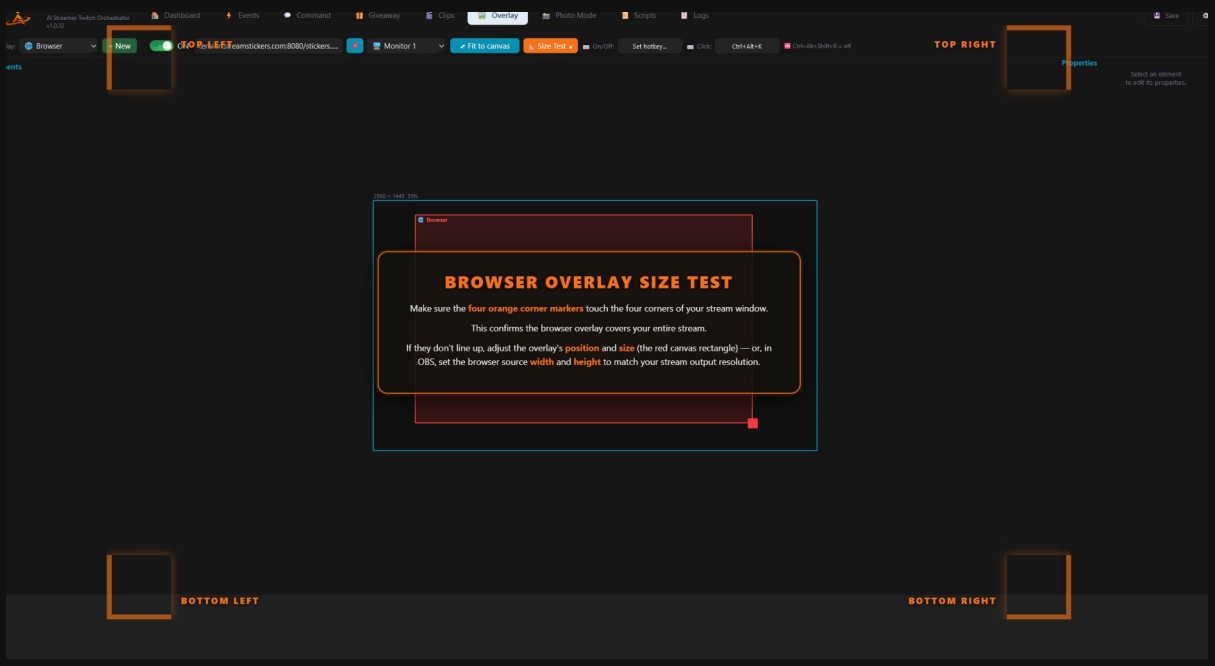
Von links nach rechts:

- **ON / OFF** — schaltet das Browser-Fenster ein und aus.
- **URL-Feld** — hier trägst du die Adresse ein, die angezeigt werden soll.
- **URL-Liste** (der blaue Knopf mit dem roten Symbol) — öffnet eine Liste, in der du **mehrere URLs** hinterlegen kannst. Aktiv ist aber immer nur **eine** gleichzeitig, nie mehrere.
- **Refresh** — lädt die aktuelle Seite neu.
- **Monitor** — auf welchem Bildschirm das Fenster erscheint. Wählst du **Hidden**, bleibt es unsichtbar und wird nur von OBS aufgenommen.
- **Fit to canvas** — legt das Fenster automatisch über den ganzen Bildschirm. Alternativ ziehst du es von Hand auf die Größe und Position, die du brauchst.
- **Size Test** — blendet eine Hilfe zum Ausrichten ein (siehe unten).
- **Mute** und der **Lautstärke-Regler** — schalten den Ton der Seite stumm bzw. regeln die Lautstärke.
- **On/Off-Hotkey** und **Click-Hotkey** — Tastenkürzel zum Ein-/Ausschalten und zum Freischalten von Klicks (dazu gleich mehr).

> Das Fenster ist standardmäßig **klick-durchlässig** — du klickst also hindurch, als wäre es nicht da. Über den **Click-Hotkey** machst du es kurz klickbar, um auf der Seite etwas zu bedienen.
> **Notausschalter:** Lädst du eine Seite, die **nicht durchsichtig** ist (z. B. YouTube), und sie legt sich genau über das ASTO-BOT-Fenster, kommst du mit **`Ctrl+Alt+Shift+B`** sofort wieder heraus — das schaltet das Overlay hart ab.

Size Test — sauber ausrichten

Ein Druck auf **Size Test** blendet im Browser-Fenster eine Anzeige mit vier orangen Eckmarkern und einem Hinweistext ein. Schieb Position und Größe des roten Rechtecks (oder in OBS die Breite und Höhe der Quelle) so, dass die vier Marker **genau in den Ecken deines Streambildes** sitzen. Dann deckt das Overlay deinen Stream lückenlos ab.

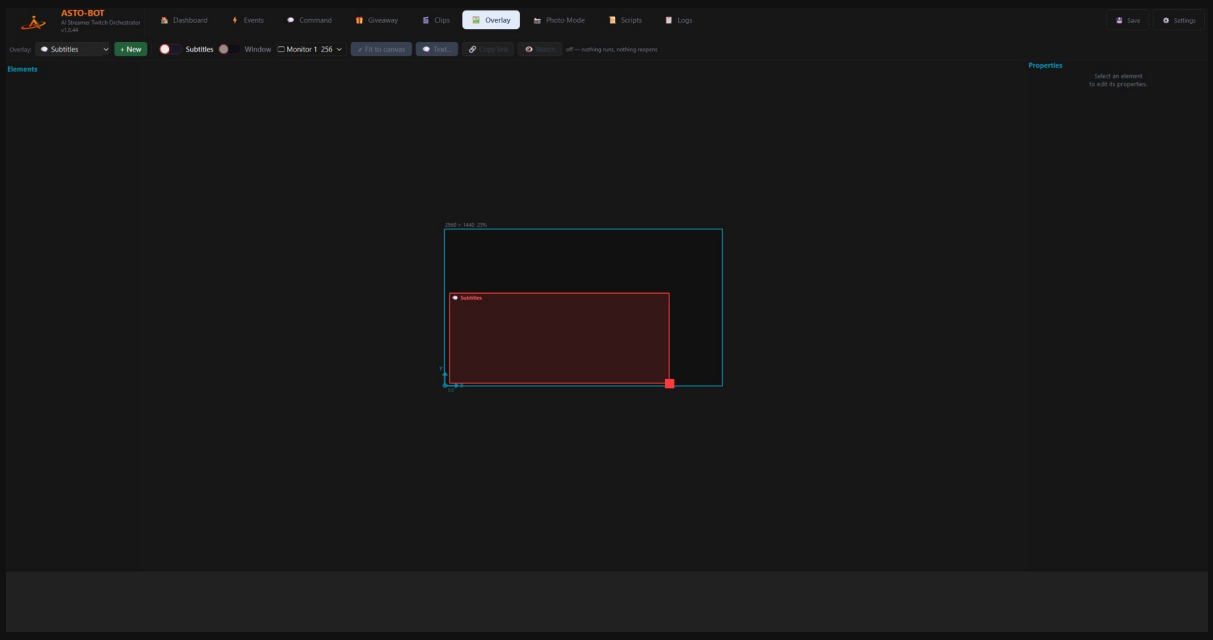


Der Size Test mit den vier Eckmarkern und dem Hinweistext

> Das Browser Overlay läuft in einem **eigenen Fenster** und braucht deshalb eine **eigene OBS-Quelle**. Wie du sie als Fensteraufnahme einrichtest, steht in Kapitel 4.3.4.

9.9 Untertitel-Overlay

Das Untertitel-Overlay heißt 💬 **Subtitles** und steht wie das Browser Overlay fest im Dropdown oben links. Hier landen die Zeilen, die der Live Translator über den Schalter **To Subtitles** verschickt.



Die Kopfleiste des Untertitel-Overlays mit dem roten Textbereich im Editor

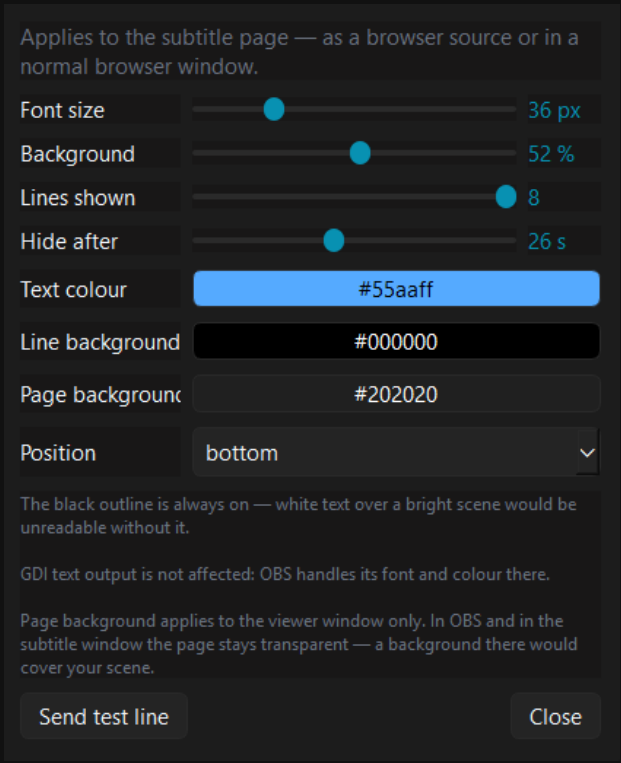
Das **rote Rechteck** auf der Arbeitsfläche ist der Bereich, in dem der Text erscheint. Verschiebst oder vergrößerst du es, wandert der Text sofort mit — im eigenen Fenster genauso wie in einer Browserquelle in OBS. Die Lage wird in Prozent gespeichert und funktioniert damit auf 1080p, 1440p und 4K gleichermaßen.

Die Kopfleiste

- **Subtitles** — schaltet das Overlay scharf.
- **Window** — öffnet das Untertitelfenster auf deinem Bildschirm. Es ist durchsichtig und klick-durchlässig, du kannst also normal weiterarbeiten.
- **Monitor** — auf welchem Bildschirm das Fenster liegt.
- **Fit to canvas** — setzt den roten Bereich auf die volle Fläche zurück.
- **Text…** — Schrift, Farben und Position, siehe unten.
- **Copy link** — die Adresse für eine **Browserquelle in OBS**. Diese Seite ist durchsichtig: Sie zeigt den Text und sonst nichts.
- **Watch** — öffnet die Untertitel in deinem normalen Browser. Diese Variante malt einen Hintergrund, weil ein Browser Durchsichtigkeit als Weiß darstellt.

> **Watch** ist der Weg für alle, die weder OBS noch ein Twitch-Konto haben — zum Beispiel bei einem Videogespräch oder wenn du dir ein Video mit Untertiteln ansehen willst.

Text… — das Aussehen



Die Einstellungen für Schrift, Farben und Position der Untertitel

- **Font size** — die Schriftgröße. Sie skaliert automatisch mit: 36 px auf deiner Arbeitsfläche sehen bei einer 4K-Quelle genauso groß aus.
- **Background** — wie deckend der Kasten hinter jeder Zeile ist.
- **Lines shown** — wie viele Zeilen gleichzeitig stehen bleiben.
- **Hide after** — nach wie vielen Sekunden eine Zeile verschwindet. Auf **0** bleibt sie stehen.
- **Text colour** — die Schriftfarbe.
- **Line background** — die Farbe des Kastens hinter jeder Zeile.
- **Page background** — der Hintergrund **nur** für das Ansichtsfenster aus **Watch**. In OBS und im Untertitelfenster bleibt die Seite durchsichtig; ein Hintergrund würde dort deine Szene zudecken.
- **Position** — oben, mittig oder unten innerhalb des roten Bereichs.

Send test line schickt eine Beispielzeile los, ohne dass du sprechen musst. Damit prüfst du Größe und Farbe gegen deine echte Szene — und siehst sofort, ob die Verbindung überhaupt steht.

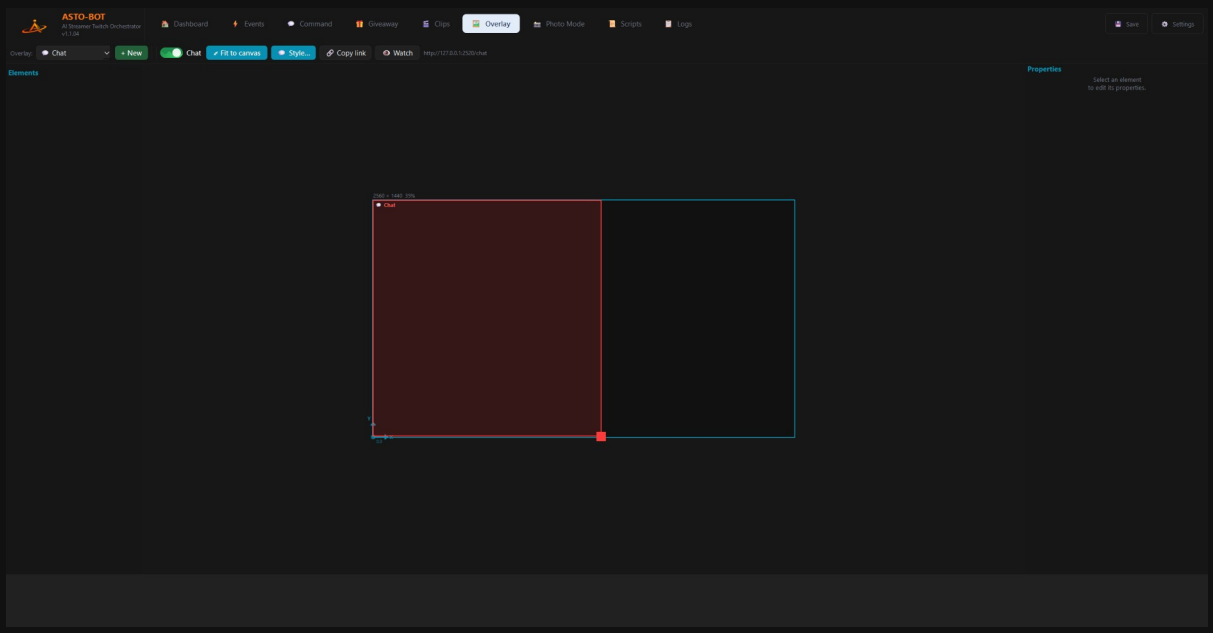
> 💡 Der schwarze Umriss um die Schrift ist immer an. Weißer Text über einer hellen Szene wäre ohne ihn nicht zu lesen.

> Die Einstellungen hier gelten **nicht** für die Ausgabe **To GDI Text**. Dort bestimmt OBS Schrift und Farbe.

9.10 Chat-Overlay

Das Chat-Overlay heißt 💬 **Chat** und steht wie das Browser Overlay und die Untertitel fest im Dropdown oben links — löschen lässt es sich nicht. Es zeigt deinen **Twitch-Chat direkt im Stream**: mit den echten Farben der Zuschauernamen, ihren Badges und ihren Emotes. Und zwar nicht nur den Twitch-Emotes, sondern auch denen von **7TV**, **BTTV** und **FrankerFaceZ**.

Du brauchst dafür also keinen fremden Dienst und keine zweite Seite im Browser. ASTO-BOT liefert die Chatseite selbst aus — es hängt ohnehin schon an deinem Chat und weiß als einziges, welches Konto dein Bot ist und was ein Befehl war. Genau das kann ein externes Overlay nur raten.



Das Chat-Overlay im Overlay-Bereich: die Kopfleiste und der rote Bereich auf der Arbeitsfläche

Das **rote Rechteck** auf der Arbeitsfläche ist der Bereich, in dem der Chat läuft — genau wie bei den Untertiteln. Verschiebst oder vergrößerst du es, wandert der Chat sofort mit. Die Lage wird in Prozent gespeichert und sitzt damit auf 1080p, 1440p und 4K an derselben Stelle.

> Chat-Overlay, Untertitel-Overlay und Browser Overlay sind **voneinander unabhängig**. Jedes hat sein eigenes Rechteck, seine eigenen Einstellungen und seine eigene Seite. Du kannst alle drei gleichzeitig laufen lassen.

Die Kopfleiste

- **Chat** — der Hauptschalter. Aus bedeutet: Es werden keine Nachrichten mehr geschickt **und** die Zeilen, die noch stehen, werden gelöscht. Sonst würde der Chat nach dem Ausschalten stehen bleiben und seinen Zweck verfehlen.
- **Fit to canvas** — setzt den roten Bereich auf die volle Fläche zurück.
- **Style…** — Schrift, Farben, Designs und Filter, siehe unten.
- **Show area** — zeichnet den Chat-Bereich als gestrichelten Rahmen mit Beschriftung, auch in OBS. Damit richtest du ihn aus, ohne darauf zu warten, dass jemand schreibt. Nochmal drücken schaltet ihn wieder aus — der Knopf heißt dann **Hide area**, du siehst also immer, woran du bist.
- **Copy link** — kopiert die Adresse der Chatseite. Genau die brauchst du gleich in OBS.
- **Watch** — öffnet den Chat in deinem normalen Browser. Diese Variante malt einen Hintergrund, weil ein Browser Durchsichtigkeit als Weiß darstellt.
- Rechts daneben steht die aktuelle Adresse — oder *off*, wenn der Schalter aus ist.

So kommt der Chat nach OBS

Das Chat-Overlay ist eine **Internetseite**, die ASTO-BOT selbst ausliefert — kein Fenster und keine Datei auf deiner Festplatte. In OBS richtest du dafür eine **Browser-Quelle** ein und trägst die Adresse ein:

- In ASTO-BOT auf 🔗 **Copy link** drücken. In der Zwischenablage steht jetzt `http://127.0.0.1:2520/chat`.
- In OBS unter **Quellen** auf das **+** klicken und **Browser** wählen. Gib ihr einen Namen, zum Beispiel *ASTO Chat*.
- Im Feld **URL** die kopierte Adresse einfügen.
- **Breite** und **Höhe** auf die Auflösung deiner Szene stellen — in der Regel `1920 × 1080`. Die Seite rechnet die Schriftgröße selbst um, du musst nichts nachstellen.
- Mit **OK** bestätigen. Sobald jemand etwas in den Chat schreibt, steht es im Bild.

> **Wichtig:** Der Haken **Lokale Datei** muss **aus** bleiben. Es ist keine HTML-Datei, die du dir irgendwo heraussuchst, sondern eine Adresse — sie gehört ausschließlich in das Feld **URL**. Mit *Lokale Datei* sucht OBS auf deiner Festplatte nach einer Datei, findet nichts und zeigt eine leere Quelle.

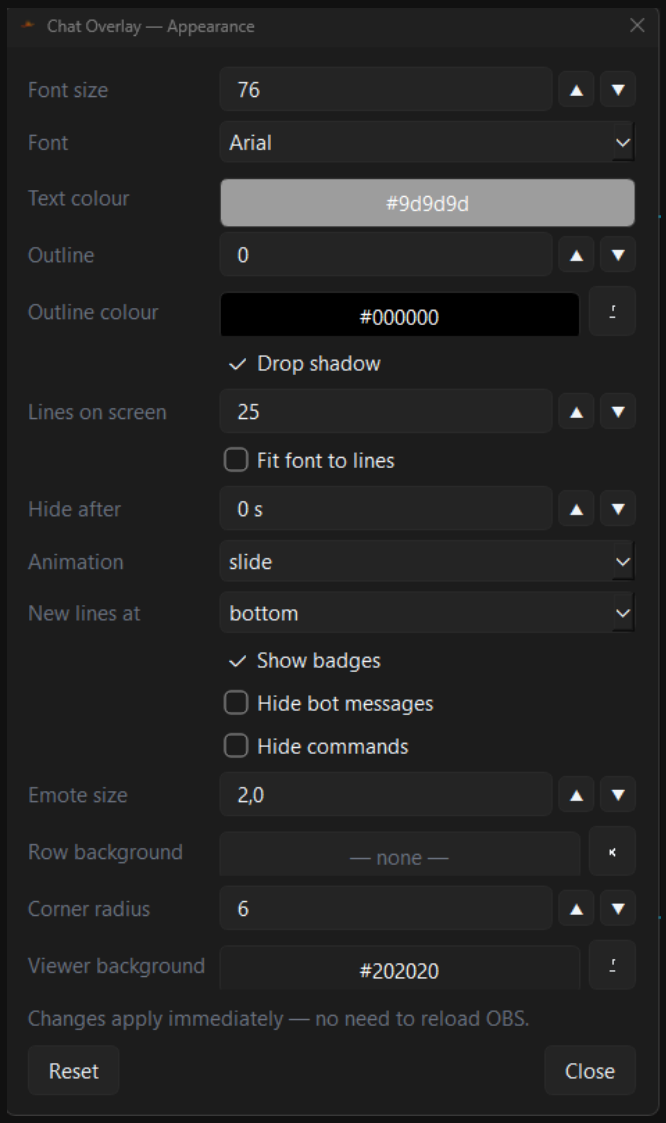
> 💡 Die Größe und Position im Bild stellst du **nicht** in OBS ein, sondern über das rote Rechteck in ASTO-BOT. Lass die Quelle in OBS einfach auf voller Größe liegen — dann passt beides immer zusammen.

Gelöschte Nachrichten verschwinden mit

Wird eine Nachricht in deinem Chat gelöscht, verschwindet sie jetzt auch aus dem Overlay — egal ob du sie einzeln löschst, jemanden in den Timeout schickst oder den ganzen Chat leerst. Vorher blieb sie im Stream stehen, obwohl sie auf Twitch längst weg war.

> **Nach dem Update einmal nötig:** In OBS **Rechtsklick** auf die Chat-Quelle → **Cache des aktuellen Seiteninhalts leeren**. OBS behält sonst die alte Seite, und die kennt das Löschen noch nicht.
>
> ASTO-BOT muss laufen, damit die Seite ausgeliefert wird. Ist ASTO-BOT zu, bleibt die Browser-Quelle leer. Beim Beenden räumt ASTO-BOT den Chat von selbst aus dem Bild — es bleibt also nichts stehen.

Style… — das Aussehen



Die Einstellungen für Schrift, Farben, Zeilenzahl und Filter

Jede Änderung wirkt **sofort** — du musst weder speichern noch die Quelle in OBS neu laden. Stell einfach ASTO-BOT und OBS nebeneinander und dreh so lange, bis es passt.

- **Design** — das Aussehen des Chats. Zur Auswahl stehen neun Designs, von **Classic** — dem bisherigen Bild — bis zu Blasen, Leuchtrand, Papier und Sofortbild. Unter der Auswahl steht jeweils, was ein Design ausmacht. Es ist immer genau **eines** aktiv; beim Umschalten lädt die Browser-Quelle sich selbst neu, in OBS musst du nichts anfassen.
- **Font size** — die Schriftgröße, gewählt gegen die Breite deiner Arbeitsfläche. Die Seite rechnet sie um, damit 40 px auf einem 2560er Canvas auch dann wie 40 px wirken, wenn die Quelle in 4K rendert.
- **Font** — die Schriftart. Das Dropdown zeigt jede Schrift gleich in ihrer eigenen Schrift an, du siehst also vorher, was du bekommst. Leer bedeutet: die eingebaute Standardkette.
- **Text colour** — die Schriftfarbe der Nachrichten. Die Namen behalten immer ihre echte Twitch-Farbe.
- **Outline** — die Kontur um die Schrift in Pixeln. **0** schaltet sie aus.
- **Outline colour** — die Farbe dieser Kontur. Schwarz ist auf fast jeder Szene die sichere Wahl.
- **Drop shadow** — ein weicher Schatten unter der Schrift, zusätzlich zur Kontur.

- **Limit by** — wodurch entschieden wird, wann eine alte Nachricht verschwindet. **Fit to area** nimmt so viele, wie in den roten Bereich passen: Die älteste geht in dem Moment, in dem die nächste oben über die Kante laufen würde. Da ist nichts einzustellen — das Rechteck, das du ziehst, **ist** die Grenze. **Messages on screen** nimmt stattdessen eine feste Anzahl.
- **Messages on screen** — die feste Anzahl für den zweiten Modus. Ältere Nachrichten fallen oben heraus. Beachte dabei: 25 kurze Nachrichten füllen ein Drittel des Kastens, 25 lange laufen unten heraus — dieselbe Einstellung, zwei völlig verschiedene Bilder. Genau deshalb ist **Fit to area** die Voreinstellung.
- **Fit font to lines** — dreht das Verhältnis um: Statt dass die Schriftgröße bestimmt, wie viele Zeilen in den Kasten passen, bestimmt die Zeilenzahl die Schriftgröße. 25 Zeilen heißen dann wirklich 25 sichtbare Zeilen. Gehört zu **Messages on screen** und ist bei **Fit to area** ausgegraut — dort entscheidet die Höhe des Rechtecks.
- **Hide after** — nach wie vielen Sekunden eine Zeile ausblendet. **0** heißt: nie.
- **Animation** — wie neue Zeilen hereinkommen: *slide*, *fade*, *zoom* oder gar nicht.
- **Slide from** — aus welcher Richtung Zeilen seitlich hereinkommen und wieder hinausgehen: von links oder von rechts. Betrifft nur seitliche Bewegung.
- **New lines at** — ob neue Zeilen unten ankommen (wie im echten Twitch-Chat) oder oben.
- **Show badges** — zeigt Mod-, Sub- und VIP-Abzeichen vor dem Namen.
- **Emote size** — die Größe der Emotes als Vielfaches der Zeilenhöhe. **1.5** entspricht ungefähr der Größe im Twitch-Chat.
- **GIFs at once** — wie viele GIFs sich gleichzeitig bewegen dürfen. Ältere werden durch ihren Titel ersetzt, damit der Chat flüssig bleibt. **0** bedeutet: gar keine GIFs — im Feld steht dann *0 — no GIFs*.
- **Row background** — ein Kasten hinter jeder Zeile. Meist besser leer; nimm lieber die Kontur. Mit ✕ wirst du ihn wieder los.
- **Corner radius** — wie rund die Ecken dieses Kastens sind.
- **Viewer background** — der Hintergrund **nur** für das Ansehen über 👁 **Watch**. In OBS bleibt die Seite durchsichtig; ein Hintergrund würde dort deine Szene zudecken.
- **Reset** — setzt alles auf die Voreinstellungen zurück.

> Deine Farben und Größen bleiben beim Wechseln erhalten — ein Design auszuprobieren kostet dich also nichts. Was ein Design **selbst** festlegt, ist im Dialog **ausgegraut** statt überschrieben: Ein Terminal ohne feste Schriftbreite wäre kein Terminal mehr. Schaltest du zurück, stehen deine eigenen Werte sofort wieder da.

> 💡 Siehst du in OBS nur wenige Zeilen, obwohl du 25 eingestellt hast? Dann ist die Schrift für den roten Bereich zu groß — die übrigen Zeilen sind da, liegen aber außerhalb. Das ist genau der Fall, für den es **Fit to area** gibt: Dort kann nichts herauslaufen, weil gemessen statt gezählt wird. Bleibst du bei **Messages on screen**, hilft nur das Rechteck höher ziehen, die Schrift kleiner machen oder **Fit font to lines** einschalten.

Eigene Designs

Die neun mitgelieferten Designs sind nicht das Ende. Unter der Liste stehen drei Knöpfe, mit denen du eigene hinzufügst — und, was mehr wert ist als jeder eingebaute Editor: mit denen Designs **zwischen Nutzern wandern** können. Einer baut eines, legt es in den Discord, alle anderen ziehen es sich herein.

- **Export current** — schreibt das oben gewählte Design als `.css`-Datei heraus. Das ist der Weg, den ich empfehle: Bei einer leeren Datei anzufangen schafft kaum jemand, mit Polaroid als Vorlage und drei geänderten Farben dagegen schon.
- **Import design…** — lädt eine solche Datei. Du kannst sie stattdessen auch einfach **auf das Fenster ziehen**, das ist derselbe Weg.

- **Remove design** — löscht ein importiertes Design wieder. Nur eigene; die eingebauten sind geschützt, dann ist der Knopf ausgegraut.

Importierte Designs landen in `Overlays/ChatDesigns/` und stehen sofort in der Liste, mit **(own)** dahinter. Der Kopf der Datei sind CSS-Kommentare, damit sie eine gültige CSS-Datei bleibt und sich in jedem Editor mit Farbhervorhebung bearbeiten lässt:

```
/* @label  Retro Terminal   */
/* @info   Grün auf Schwarz */
/* @owns   font, color      */
/* @leave  300              */

.line { color: #0f0; }
```

@owns nennt die Einstellungen, die das Design selbst bestimmt — die werden im Dialog ausgegraut, genau wie bei den eingebauten. **@leave** ist die Ausblenddauer in Millisekunden.

> **Fremdes CSS wird beim Ausliefern entschärft.** Eine Design-Datei wird in deine Overlay-Seite hineingeschrieben, und sie kommt aus dem Discord, von einem Zuschauer, von irgendwo — der Absender muss es nicht einmal böse meinen. Entfernt werden: alles, was aus dem Stylesheet ausbrechen und eigenes JavaScript in dein Overlay bringen könnte; Nachladen weiterer Dateien aus dem Netz; und Bildadressen, die auf fremde Server zeigen — die sähen sonst bei jedem Start deines Streams deine IP-Adresse. Eingebettete Bilder in der Datei selbst bleiben erlaubt. Was entfernt wurde, steht im Log. Deine Datei auf der Festplatte bleibt dabei unverändert, du kannst sie also weiter bearbeiten.
>
> Fehlt eine Design-Datei später — gelöscht, verschoben, umbenannt —, fällt der Chat auf **Classic** zurück und schreibt eine Zeile ins Log. Ein schwarzes Rechteck im Stream gibt es dadurch nicht.

Was nicht im Bild landet

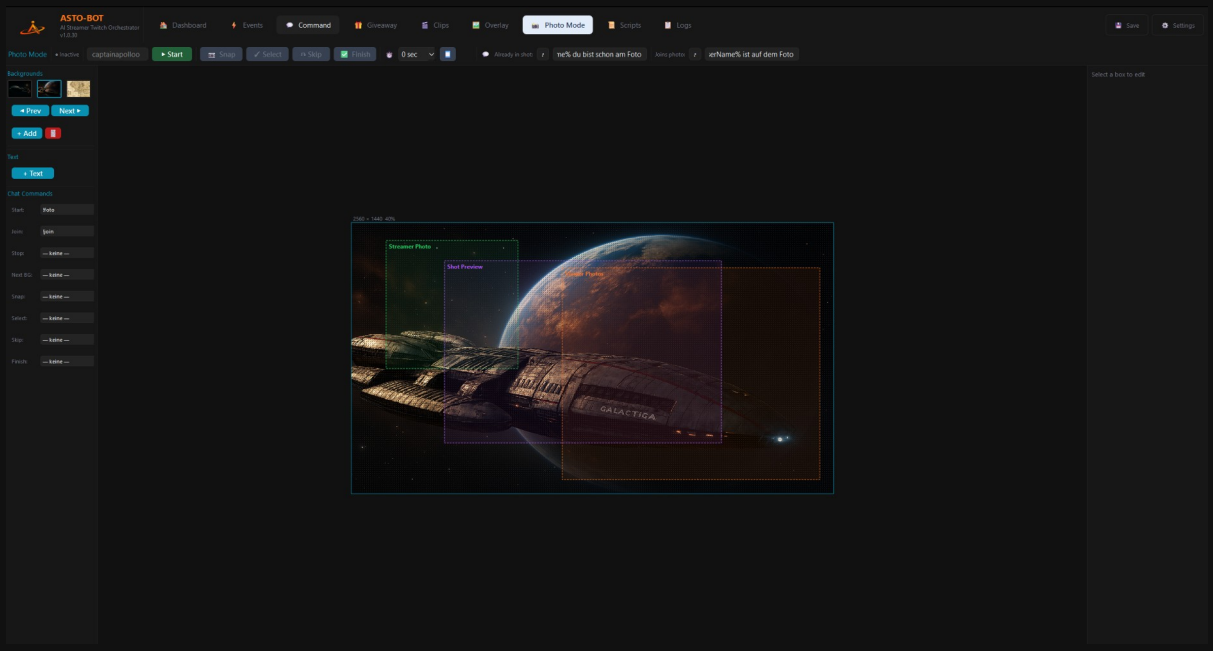
- **Hide bot messages** — blendet alles aus, was dein **Bot-Konto** und die Einträge deiner **Ignore-Liste** schreiben. Praktisch, wenn deine automatischen Ansagen nicht doppelt im Bild stehen sollen.
- **Hide commands** — blendet jede Nachricht aus, die mit `!` beginnt. Damit tauchen `!timer` oder `!join` gar nicht erst auf, wohl aber die Antwort deines Bots darauf.

> **Ankündigungen** aus dem Chat (`/announce`) erscheinen mit einem farbigen Balken an der linken Seite — in derselben Farbe, die du beim Absenden gewählt hast. Sie kommen bei Twitch über einen anderen Weg herein als normale Nachrichten und werden deshalb auch anders dargestellt.

> 💡 Den Chat musst du nicht am Rechner ein- und ausschalten: Es gibt einen **Websocket-Befehl** dafür (Kapitel 4.7), und im **Stream-Deck-Plugin** von ASTO-BOT gibt es eine eigene Taste **Chat Overlay**. Ein Druck, und der Chat ist im Bild oder wieder weg.

10. Photo Mode

Der Photo Mode holt deine Zuschauer buchstäblich mit aufs Bild: Du startest den Modus, dein Twitch-Profilbild erscheint vor einem Hintergrund, die Zuschauer treten per Chat-Befehl dazu — und dann wird abgedrückt. Das fertige Foto landet in deiner Galerie.



Der Photo Mode: links Hintergründe und Befehle, in der Mitte die Bildaufteilung

> Nicht zu verwechseln mit der Aktion **OBS Screenshot** aus dem Events-Kapitel — die macht nur ein Standbild deiner Szene. Der Photo Mode ist etwas anderes: eine kleine Fotosession mit deiner Community.

10.1 Die Kopfleiste

- **Active** — zeigt an, dass der Modus läuft, mitsamt dem Namen, um den es geht.
- **Snap** — macht das Foto.
- **Select** — übernimmt das gemachte Foto.
- **Skip** — verwirft es. Dann muss ein neues gemacht werden.
- **Finish** — beendet den Photo Mode.
- Das **Uhr-Dropdown** ist der Selbstauslöser: **0**, **3**, **5** oder **10 Sekunden** nach dem Drücken von Snap.

Die beiden Chat-Texte

- **Joins photo** — die Nachricht, die in den Chat geht, wenn jemand dem Foto beitritt.
- **Already in shot** — die Nachricht für den Fall, dass jemand schon auf dem Bild ist und noch einmal beitreten will.

In beiden Texten sind **Variablen** erlaubt — `%UserName%` gehört praktisch immer hinein.

10.2 Hintergründe und Texte

- Unter **Backgrounds** liegen die Hintergrundbilder. Mit ◀ **Prev** und **Next** ▶ blätterst du durch.
- **+ Add** fügt einen neuen Hintergrund hinzu, der **Papierkorb** löscht den gewählten.
- **+ Text** setzt ein Textfeld ins Bild. Die Texte funktionieren genau wie im Overlay-Editor — dieselben Effekte, dieselben Variablen (Kapitel 9.6).

10.3 Die drei Bereiche im Bild

Im Canvas siehst du drei farbig umrandete Kästen. Jeder hat seine Aufgabe, und jeden kannst du frei verschieben und in der Größe anpassen.

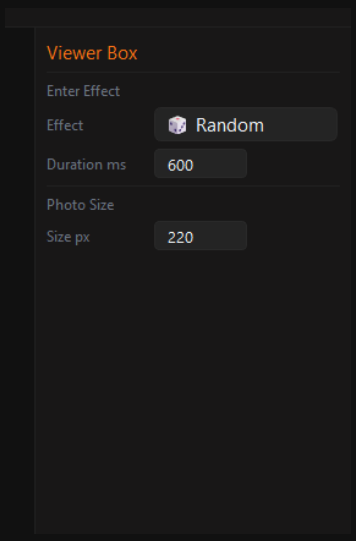
Grün — Streamer Photo



Die Streamer Box

Hier erscheint **dein** Twitch-Profilbild — die Person, um die es beim Foto geht. Unter **Enter Effect** wählst du, aus welcher Richtung es hereinkommt, **Duration ms** bestimmt, wie lange das dauert.

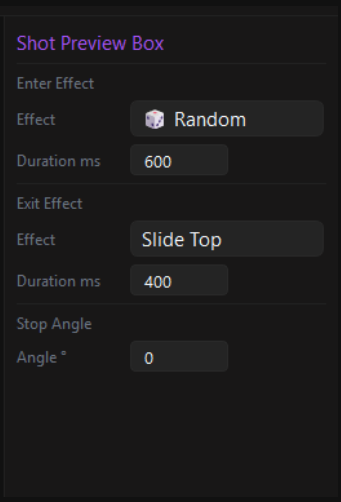
Orange — Viewer Photos



Die Viewer Box

Hier sammeln sich die **Twitch-Profilbilder der Zuschauer**, die beigetreten sind. Neben dem Enter-Effekt gibt es hier zusätzlich die **Photo Size** in Pixeln — wie groß die einzelnen Bilder dargestellt werden.

Lila — Shot Preview



Die Shot Preview Box

In diesem Feld erscheint das **geschossene Foto**. Es steht im **Vordergrund**, vor allen anderen Bildern. Diese Box hat als einzige auch einen **Exit Effect** — wie das Foto wieder verschwindet — und einen **Stop Angle**, mit dem du es leicht schräg stehen lassen kannst, wie ein hingeworfenes Polaroid.

10.4 Die Chat-Befehle

Links unten liegt die komplette Befehlsliste. Jeder Schritt des Photo Mode lässt sich über den Chat steuern:

- **Start** — startet den Modus, zum Beispiel `!foto`. Mit Namen dahinter: `!foto CaptainApolloo`.
- **Join** — mit diesem Befehl treten Zuschauer bei. **Ohne** weiteren Zusatz — also einfach nur `!join` oder `!beitreten`.
- **Stop**, **Next BG**, **Snap**, **Select**, **Skip** und **Finish** — dieselben Funktionen wie die Knöpfe oben, nur eben aus dem Chat heraus.

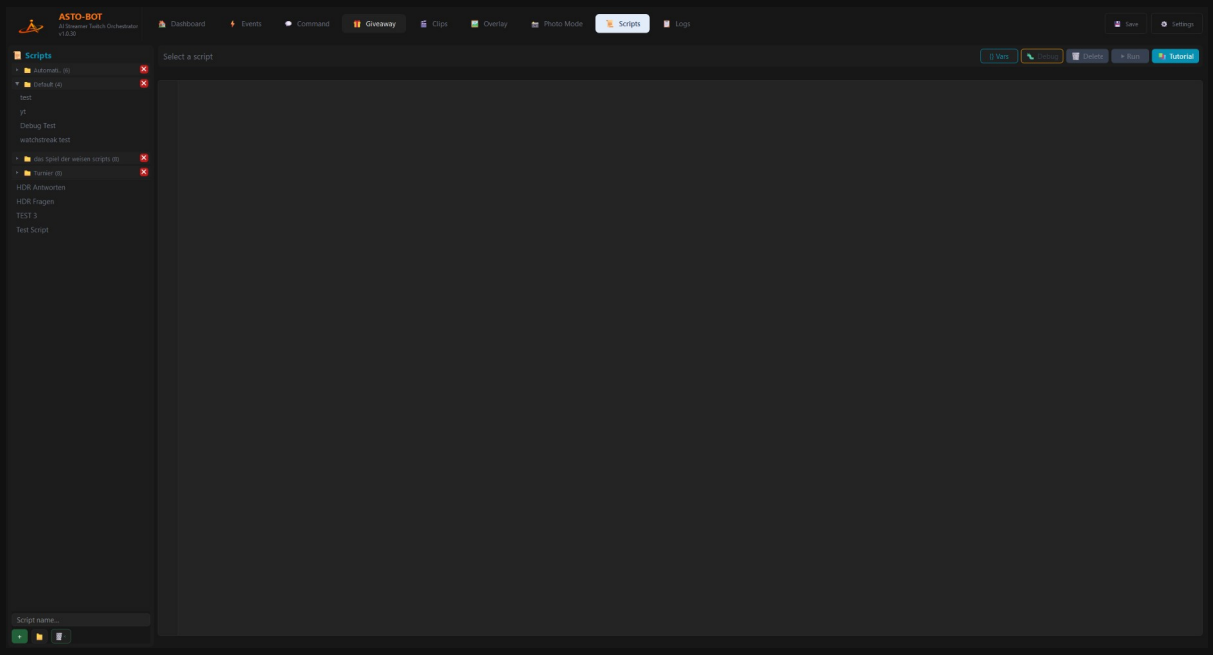
> 💡 Du musst nicht am Rechner sitzen: Die Befehle lassen sich auch per **Websocket** auslösen — zum Beispiel über eine Taste am Stream Deck (Kapitel Settings → Websocket). Snap, Select und Finish auf drei Tasten, und du hast beide Hände frei fürs Posieren.

10.5 Wo die Fotos landen

Die fertigen Fotos speichert ASTO-BOT im Ordner **PhotoMode**, der neben der EXE liegt. Genau auf diesen Ordner greift auch die Karte **Photo Gallery** auf dem Dashboard zu (Kapitel 3) — die beiden gehören zusammen. Was du hier aufnimmst, kannst du dort direkt durchblättern.

11. Scripts — ASTOSCRIPT

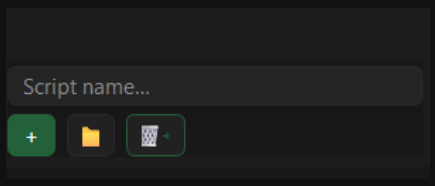
Alles, was du bisher gesehen hast, lässt sich zusammenklicken. Irgendwann kommt aber der Punkt, an dem das nicht mehr reicht: Du willst eine Wetter-Abfrage im Chat, deine Elgato-Lampe zum Blinken bringen oder ein kleines Spiel mit deiner Community spielen. Dafür gibt es **ASTOSCRIPT** — die eingebaute Skriptsprache von ASTO-BOT.



Der Scripts-Bereich: links die Skriptliste, rechts der Editor

> 💡 Die Sprache selbst wird in diesem Handbuch **nicht** vollständig erklärt — das erledigt das **Tutorial-Fenster** direkt in ASTO-BOT, und zwar besser: Es ist immer aktuell, durchsuchbar und auf Deutsch und Englisch verfügbar. Dieses Kapitel zeigt dir alles **drumherum**: wie du Skripte anlegst, testest, Fehler findest und aus Events heraus startest.

11.1 Skripte anlegen und ordnen

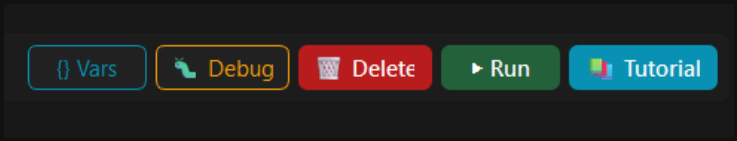


Unten links entstehen neue Skripte und Ordner

- Namen unten ins Feld **Script name...** eintippen und auf das grüne **+** drücken — fertig ist das neue Skript.
- Der **Ordner-Knopf** daneben legt einen neuen Ordner an. Skripte ziehst du per **Drag & Drop** in den Ordner, in den sie gehören.
- Ein **Rechtsklick** auf ein Skript in der Liste bietet **Umbenennen** und **Kopieren** an.

> 💡 Der **Papierkorb** neben dem Ordner-Knopf ist kein Lösch-Knopf, sondern das Gegenteil: Er stellt **gelöschte Skripte wieder her**. Wenn dir also ein Skript abhandengekommen ist — hier schaust du zuerst nach.

11.2 Die Knöpfe oben rechts



Vars, Debug, Delete, Run und Tutorial

- **Vars** — öffnet dasselbe Variablen-Fenster, das du schon aus den Events kennst. Klick kopiert.
- **Debug** — startet den Debug-Modus (Abschnitt 11.5).
- **Delete** — löscht das Skript.
- **Run** — führt es aus.
- **Tutorial** — öffnet das Tutorial- und Nachschlagewerk (Abschnitt 11.6).

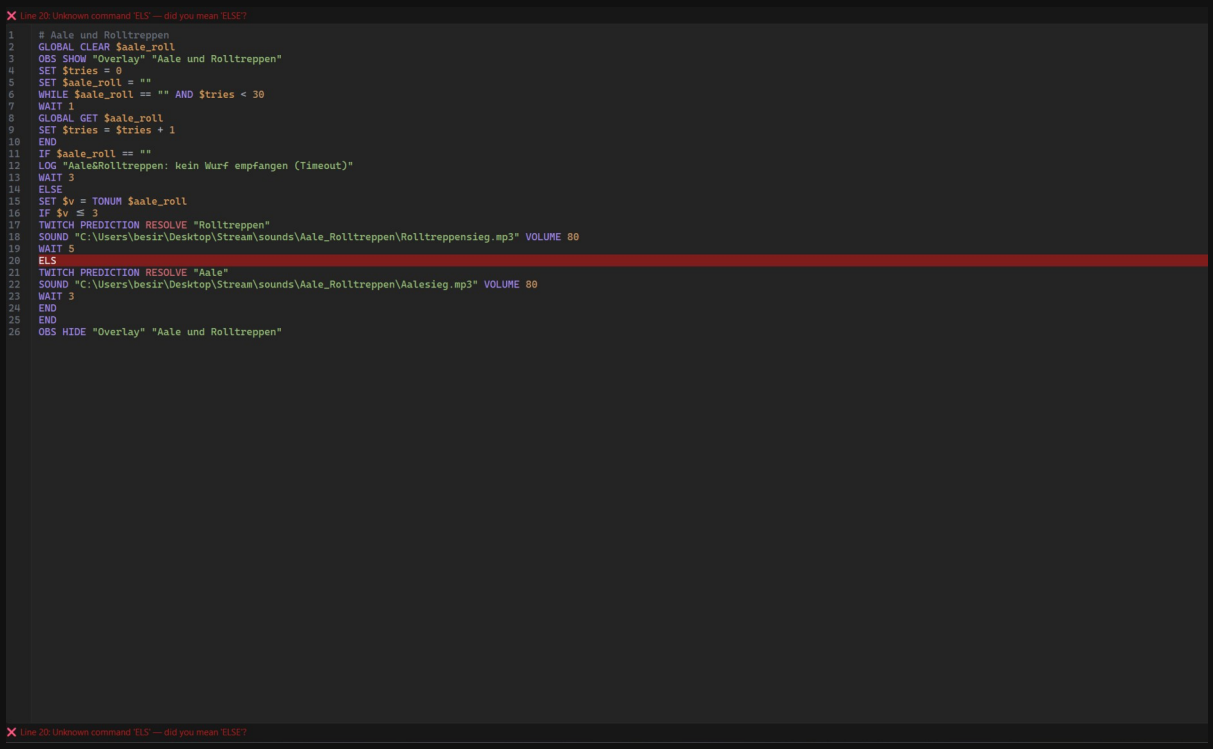
11.3 Der Editor hilft mit

Beim Tippen nimmt dir der Editor Arbeit ab:

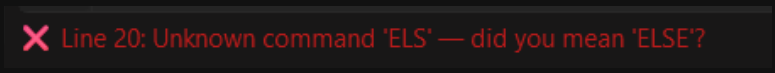
- Er schlägt dir **Befehle, Bedingungen und Variablen** vor, während du schreibst.
- Tippst du ein `"`, setzt er sofort das **zweite Anführungszeichen** dahinter — du schreibst einfach dazwischen weiter. Dasselbe gilt für Klammern `()`.
- Nach einer `IF`-Zeile rückt er die nächste Zeile automatisch ein.

11.4 Fehler finden

Drückst du **Run** und im Skript steckt ein Fehler, musst du nicht raten: ASTO-BOT markiert die betroffene Zeile **rot** und schreibt unten hin, was nicht stimmt — und meistens gleich dazu, was wahrscheinlich gemeint war.



Ein Tippfehler in Zeile 20 — die Zeile wird rot markiert

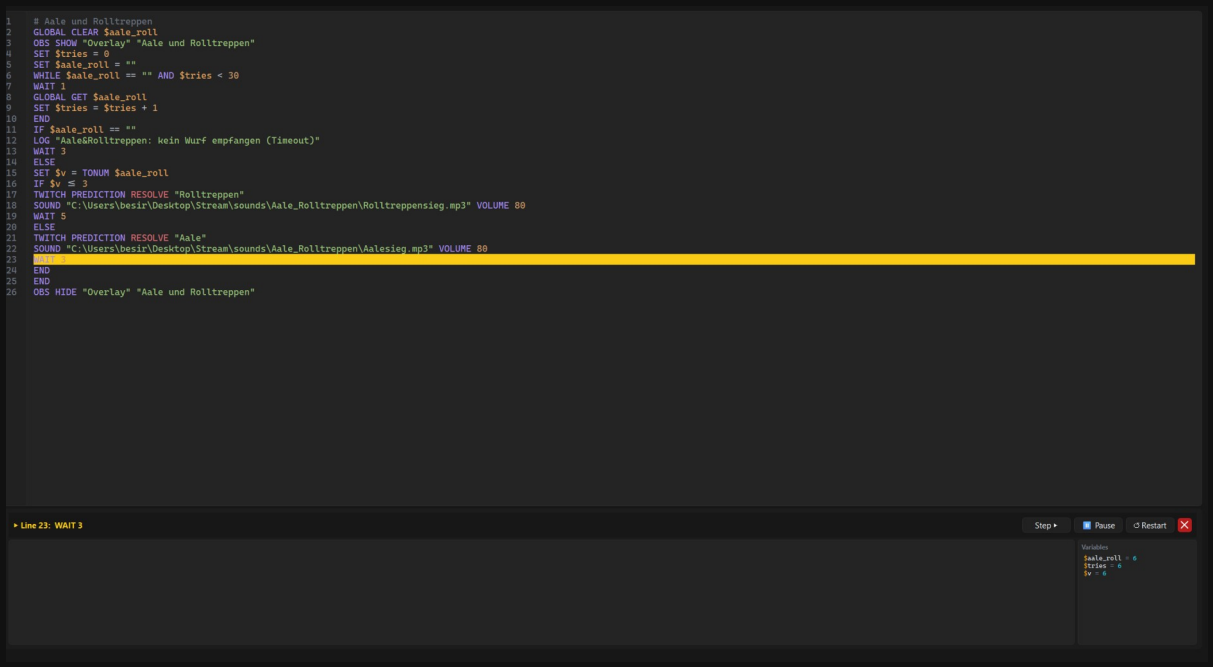


Die Fehlermeldung nennt Zeile, Problem und Vorschlag

Fehler beheben, noch einmal **Run** drücken — die Markierung verschwindet, und das Skript läuft.

11.5 Der Debug-Modus

Debug lässt dich beim Skript über die Schulter schauen: Es läuft dann Zeile für Zeile, und du siehst jederzeit, wo es gerade steht und was in den Variablen drinsteht.



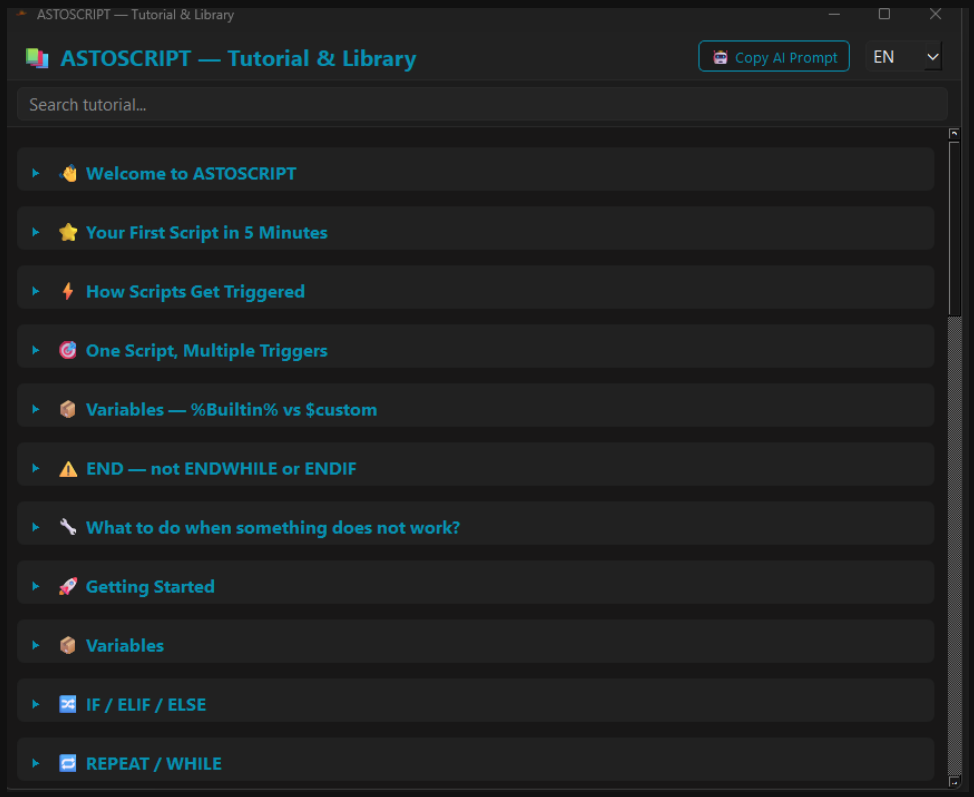
Der Debug-Modus: aktuelle Zeile markiert, unten die Steuerung und die Variablen

- Die gerade laufende Zeile wird **gelb** hervorgehoben, unten links steht sie noch einmal im Klartext.
- **Step** — geht eine Zeile weiter.
- **Pause** — hält an.
- **Restart** — fängt von vorne an.
- Rechts unter **Variables** siehst du **live**, welchen Wert jede Variable gerade hat.

> 💡 Wenn ein Skript zwar läuft, aber etwas anderes tut als gedacht, ist der Debug-Modus der kürzeste Weg zur Antwort. Meist sieht man schon nach drei Steps, dass eine Variable etwas anderes enthält, als man angenommen hat.

11.6 Das Tutorial-Fenster

Der Knopf **Tutorial** öffnet das eingebaute Handbuch zu ASTOSCRIPT — mit Suchfeld, nach Themen sortiert, von den ersten Schritten bis zu den Feinheiten.



Das Tutorial — Nachschlagewerk und Einführung in einem

- Oben rechts stellst du die Sprache um: **EN** oder **DE**.
- Die Kapitel reichen von *Welcome to ASTOSCRIPT* und *Your First Script in 5 Minutes* über *How Scripts Get Triggered* bis zu den einzelnen Sprachbausteinen — Variablen, `IF / ELIF / ELSE`, `REPEAT / WHILE` und so weiter.
- Das Suchfeld oben findet jeden Befehl.

> **Die vollständige Befehlsliste steht im Tutorial-Fenster, im Bereich Glossar / Library.** Dort sind alle ASTOSCRIPT-Keywords nach Kategorien sortiert und einzeln erklärt — auf Deutsch und Englisch. Dieses Handbuch wiederholt sie bewusst nicht: Das Glossar im Programm wächst mit jeder Version mit und ist damit immer aktueller als jedes gedruckte Verzeichnis.

Copy AI Prompt

Der Knopf **Copy AI Prompt** ist der heimliche Star dieses Fensters. Er kopiert einen fertigen Prompt in die Zwischenablage, den du jeder KI geben kannst, die programmieren kann — Claude, Gemini oder was immer du nutzt. Die KI weiß danach, wie ASTOSCRIPT funktioniert, und kann dir Skripte schreiben oder deine reparieren.

> 💡 Das ist der schnellste Einstieg, wenn du noch nie programmiert hast: Prompt kopieren, der KI geben, beschreiben was du willst — und das Ergebnis hier einfügen und mit **Run** testen.

> Neu dazugekommen sind vier Befehle: eine Chat-Nachricht **löschen**, den Chat **leeren**, deine Chat-**Farbe** setzen und eine Nachricht an deinen **Discord** schicken. Erklärt und mit Beispielen stehen sie im Tutorial-Fenster.

11.7 Skripte aus Events starten

Ein Skript, das niemand startet, tut nichts. Der Weg dorthin ist derselbe wie bei allem anderen: über ein **Event**. In der Action Sequence fügst du die Aktion **Run Script** ein (Gruppe *System*, Kapitel 5.6), wählst dein Skript aus — fertig.

Der **Wait**-Schalter dort entscheidet, ob das Event wartet, bis das Skript durchgelaufen ist, oder sofort mit der nächsten Aktion weitermacht.

Ein Skript, viele Events

Ein häufiges Missverständnis: Es braucht **nicht** ein Skript pro Event. Dasselbe Skript kann von **beliebig vielen** Events gestartet werden — von einem Chat-Befehl, von einem Kanalpunkte-Reward, von einem Timer, alles gleichzeitig. Du schreibst es **einmal** und hängst es überall dort ein, wo du es brauchst.

Damit das Skript weiß, **wer** es gerade gestartet hat, gibt es die Variable **`%EventName%`** — sie enthält die ID des auslösenden Events. So kann ein einziges Skript mehrere Aufgaben übernehmen und intern verzweigen. Wie das in der Praxis aussieht, zeigt das Beispiel in Abschnitt 11.9.

Und noch ein Weg führt zum Skript: Events lassen sich auch per **Websocket** von außen auslösen — zum Beispiel mit einer Taste am Stream Deck (Kapitel Settings → Websocket). Damit startest du ein Skript, ohne überhaupt den Chat zu berühren.

11.8 Zwei Skripte aus der Praxis

Zum Abschluss zwei Skripte, die zeigen, wozu ASTOSCRIPT taugt — nicht zum Abtippen, sondern zum Anschauen.

Wetter aus dem Chat

Ein Zuschauer tippt einen Stadtnamen, das Skript holt die Koordinaten, dann die Wetterdaten — und lässt die KI daraus einen lesbaren Satz für den Chat machen. Ohne API-Schlüssel, nur mit öffentlichen Schnittstellen.

```
# Stadt aus dem Chat (Leerzeichen URL-sicher machen)
SET $city = %UserInput%
SET $city = REPLACE $city " " "+"

# 1) Stadt -> Koordinaten (Open-Meteo Geocoding, kein API-Key)
SET $geourl = "https://geocoding-api.open-meteo.com/v1/search?name=" + $city +
"&count=1&language=de&format=json"
HTTP GET $geourl $geo

SET $lat = JSON $geo "results[0].latitude"
SET $lon = JSON $geo "results[0].longitude"
SET $ort = JSON $geo "results[0].name"
SET $land = JSON $geo "results[0].country"

IF $lat == ""
CHAT "Stadt nicht gefunden: " + %UserInput%
STOP
END

# 2) Wetter fuer 3 Tage (Open-Meteo Forecast, kein API-Key)
SET $wurl = "https://api.open-meteo.com/v1/forecast?latitude=" + $lat + "&longitude=" + $lon
SET $wurl += "&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max"
SET $wurl += "&forecast_days=3&timezone=auto"
HTTP GET $wurl $weather

# Plausibilitaet: kam eine Temperatur zurueck?
SET $t0 = JSON $weather "daily.temperature_2m_max[0]"
IF $t0 == ""
CHAT "Wetterdaten konnten nicht abgerufen werden."
STOP
END

# 3) Von der KI formatieren lassen (Zahlen sind exakt, nur Aufbereitung)
SET $p = "Du bist %BotName% und antwortest im Stil: %PersonaStyle%."
SET $p += " Echte 3-Tage-Wetterdaten fuer " + $ort + " (" + $land + "): " + $weather
SET $p += " Gib Temperaturen und Regenwahrscheinlichkeit GENAU aus den Zahlen wieder, erfinde NICHTS."
SET $p += " Kurze klare Antwort fuer Twitch-Chat auf Deutsch. Maximal 250 Zeichen."
AI PROMPT $p $antwort
CHAT $antwort
```

Interessant daran ist die Arbeitsteilung: Die **Zahlen** kommen aus der Schnittstelle und sind exakt, die **KI** darf sie nur noch in einen schönen Satz verpacken — und wird ausdrücklich angewiesen, nichts zu erfinden.

Die Elgato-Lampe zum Blinken bringen

Dieses Skript spricht eine Elgato-Lampe direkt im Netzwerk an: eine zufällige Anzahl Blinks in zufälligem Takt, dann ein Finale — ein perfekter Kanalpunkte-Reward.

```
# Random Flash Script - zufaelliges Blinkmuster
SET $url = "http://192.168.0.242:9123/elgato/lights"
SET $on  = "{\"numberOfLights\":1,\"lights\":[{\"on\":1,\"brightness\":100,\"temperature\":143}]}"
SET $off = "{\"numberOfLights\":1,\"lights\":[{\"on\":0,\"brightness\":100,\"temperature\":143}]}"

# Zufaellige Anzahl Blinks: 5-30
SET $blinks = RANDOM 5 30

REPEAT $blinks
HTTP PUT $url $on
SET $delay_on = RANDOM 50 1000
WAIT $delay_on MS

HTTP PUT $url $off
SET $delay_off = RANDOM 50 500
WAIT $delay_off MS
END

# Finale nach zufaelliger Pause (1-10 Sekunden)
SET $finale_pause = RANDOM 1 10
WAIT $finale_pause

# 3x schnell blinken zum Abschluss
REPEAT 3
HTTP PUT $url $on
WAIT 100 MS
HTTP PUT $url $off
WAIT 100 MS
END

HTTP PUT $url $on
```

> 💡 Beide Skripte zeigen dasselbe Muster: ASTOSCRIPT redet über `HTTP` mit der Welt — mit Wetterdiensten, mit Lampen, mit allem, was eine Schnittstelle hat. Was du damit anstellst, ist deine Sache.

11.9 Praxisbeispiel: die YouTube-Song-Warteschlange

Zum Schluss ein vollständiges System, das zeigt, wie die Teile zusammenspielen: Zuschauer schicken YouTube-Links in den Chat, ASTO-BOT stellt sie in eine Warteschlange, spielt sie der Reihe nach ab und sagt jederzeit, was gerade läuft und von wem es stammt.

Das Besondere daran: Es ist **ein einziges Skript** — aber **vier Events** starten es. Welches davon es war, erfährt das Skript über `%EventName%`, und danach verzweigt es intern.

Was du dafür anlegst

- Vier **Events** mit den IDs, die im Skript oben eingetragen sind — im Beispiel `test1` bis `test4`. Jedes bekommt einen **Chat Command** als Trigger und **eine** Aktion: **Run Script** mit diesem Skript.
- `test1` — **Song hinzufügen**. Der Zuschauer schickt den Link mit, er landet in `%UserInput%`.
- `test2` — **nächsten Song abspielen**.
- `test3` — **Warteschlange zurücksetzen**.
- `test4` — **was läuft gerade?**

> Trag im Skript oben bei `$evAdd`, `$evNext`, `$evReset` und `$evNp` die **echten IDs deiner Events** ein — die ID steht rechts im Event-Panel unter dem Namen (Kapitel 5.2). Stimmen sie nicht überein, passiert nichts, weil keine der vier Bedingungen zutrifft.

Wie es funktioniert

- Die Warteschlange liegt in **zwei Textdateien** — eine mit den Links, eine mit den Namen der Zuschauer. Beide Listen sind mit `|` getrennt und gehen zeilenweise Hand in Hand: Der dritte Link gehört zum dritten Namen.
- `FILE READ` holt sie herein, `LIST SPLIT` macht daraus Listen, `LIST APPEND` hängt an, `LIST POP` nimmt den nächsten heraus, `LIST JOIN` schreibt sie wieder zusammen — und `FILE WRITE` speichert.
- Das `TRY` / `CATCH` drumherum fängt den Fall ab, dass die Dateien noch gar nicht existieren — beim allerersten Start ist das so.
- `CLOSE_TASK` schließt den Browser, `OPEN` öffnet den nächsten Link. Dazwischen ein `WAIT`, damit der Browser Zeit hat, sich zu beenden.
- `GLOBAL SET` merkt sich, **wer** den laufenden Song eingeschickt hat — über das Skript-Ende hinaus. Nur so kann Event 4 später noch sagen, von wem der Song stammt.
- `MUSIC NOW` liest aus, was gerade tatsächlich läuft, und füllt `%SongTitle%` und `%SongArtist%`.

```
# ═══════════════════════════════════════
# SCRIPT: YT Song Queue (All-in-One, Datei-basiert)
# Wird von allen 4 Events aufgerufen.
# Verzweigt nach %EventName% (= Signal-ID des Events).
# Queue wird in zwei Textdateien gespeichert, getrennt mit "|".
# ═══════════════════════════════════════

# --- Konfiguration: hier deine echten Event-IDs eintragen ---
SET $evAdd   = "test1"   # Video hinzufuegen
SET $evNext  = "test2"   # naechster Song
SET $evReset = "test3"   # Queue reset
SET $evNp    = "test4"   # Song-Info

# --- Weitere Variablen ---
SET $browserProcess = "brave.exe"
SET $waitSeconds = 2
SET $domain1 = "youtube.com"
SET $domain2 = "youtu.be"
SET $sep = "|"

# --- Dateipfade fuer die Queue ---
SET $fileUrls  = "Config/song_queue_urls.txt"
SET $fileUsers = "Config/song_queue_users.txt"



# --------------- 1) ADD ---------------
IF %EventName% == $evAdd
IF CONTAINS %UserInput% $domain1 OR CONTAINS %UserInput% $domain2
SET $urlsRaw = ""
SET $usersRaw = ""
TRY
FILE READ $urlsRaw $fileUrls
CATCH
SET $urlsRaw = ""
END
TRY
FILE READ $usersRaw $fileUsers
CATCH
SET $usersRaw = ""
END
LIST SPLIT $urls $urlsRaw $sep
LIST SPLIT $users $usersRaw $sep
LIST APPEND $urls %UserInput%
LIST APPEND $users %UserName%
LIST JOIN $urls $sep $urlsOut
LIST JOIN $users $sep $usersOut
FILE WRITE $fileUrls $urlsOut
FILE WRITE $fileUsers $usersOut
```

```
LIST SIZE $urls $pos
CHAT "@" + %UserName% + " dein Song wurde zur Warteschlange hinzugefuegt! Position: " + TOSTR $pos
ELSE
CHAT "@" + %UserName% + " das ist kein gueltiger YouTube-Link!"
END
END



# --------------- 2) PLAY NEXT ---------------
IF %EventName% == $evNext
SET $urlsRaw = ""
SET $usersRaw = ""
TRY
FILE READ $urlsRaw $fileUrls
CATCH
SET $urlsRaw = ""
END
TRY
FILE READ $usersRaw $fileUsers
CATCH
SET $usersRaw = ""
END
LIST SPLIT $urls $urlsRaw $sep
LIST SPLIT $users $usersRaw $sep
LIST SIZE $urls $qsize

IF $qsize == 0
CHAT "Die Warteschlange ist leer! Keine weiteren Songs."
ELSE
LIST POP $urls $nextUrl
LIST POP $users $nextUser
LIST JOIN $urls $sep $urlsOut
LIST JOIN $users $sep $usersOut
FILE WRITE $fileUrls $urlsOut
FILE WRITE $fileUsers $usersOut
GLOBAL SET $currentSongUser $nextUser
LIST SIZE $urls $remaining

CLOSE_TASK $browserProcess
WAIT $waitSeconds
OPEN $nextUrl

CHAT "Spiele jetzt: " + $nextUrl + " (eingespielt von " + $nextUser + ", noch " + TOSTR $remaining + " in
der Warteschlange)"
END
END



# --------------- 3) RESET ---------------
IF %EventName% == $evReset
FILE WRITE $fileUrls ""
FILE WRITE $fileUsers ""
CHAT "Die Song-Warteschlange wurde zurueckgesetzt."
END



# --------------- 4) NOW PLAYING ---------------
IF %EventName% == $evNp
MUSIC NOW
GLOBAL GET $currentSongUser
IF %SongTitle% == ""
CHAT "Aktuell laeuft kein erkennbarer Song."
ELSE
IF $currentSongUser == ""
CHAT "Laeuft gerade: " + %SongArtist% + " - " + %SongTitle%
ELSE
CHAT "Laeuft gerade: " + %SongArtist% + " - " + %SongTitle% + " (eingespielt von " + $currentSongUser + ")"
END
END
END
```

> 💡 Der Prozessname bei `$browserProcess` muss zu **deinem** Browser passen — im Beispiel steht dort `brave.exe`. Für Chrome wäre es `chrome.exe`, für Firefox `firefox.exe`. Und bedenke: `CLOSE_TASK` schließt den Browser **ganz** — nimm dafür am besten einen Browser, in dem sonst nichts Wichtiges offen ist.

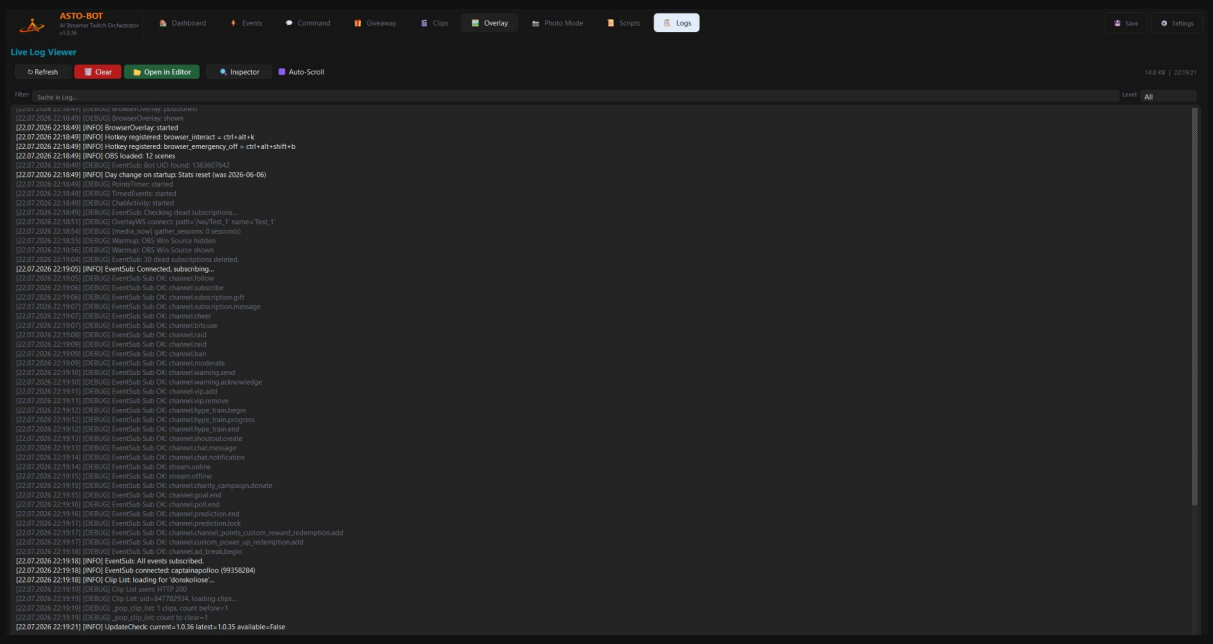
Das ist der Punkt, an dem ASTOSCRIPT seinen Sinn zeigt: Vier Chat-Befehle, ein Skript, zwei Textdateien — und deine Community hat einen funktionierenden Song-Request. Zusammengeklickt bekommst du das nicht.

12. Logs

Die Logs-Seite hat **zwei Ansichten**, zwischen denen der Knopf oben umschaltet: den **Live Log Viewer** — das laufende Protokoll — und den **Event Inspector**, der zeigt, warum ein Event oder Command *nicht* gefeuert hat.

12.1 Der Live Log Viewer

Der **Live Log Viewer** ist das Protokoll von ASTO-BOT: Jede Verbindung, jedes ausgelöste Event, jede gespeicherte Einstellung und jeder Fehler landet hier — mit Datum, Uhrzeit und Stufe.



Der Live Log Viewer

- **Refresh** — lädt das Protokoll neu.
- **Clear** — leert die Ansicht.
- **Open in Editor** — öffnet die Log-Datei in deinem Texteditor.
- **Auto-Scroll** — die Ansicht springt automatisch zur neuesten Zeile.
- **Filter** — durchsucht das Protokoll nach einem Begriff, zum Beispiel dem Namen eines Events.
- **Level** — filtert nach Stufe. Rechts oben stehen außerdem die Größe der Log-Datei und die Uhrzeit der letzten Aktualisierung.

> 💡 Lass **Level** am besten immer auf **All** stehen — dann siehst du am meisten. Die grauen `[DEBUG]`-Zeilen wirken zwar unwichtig, aber genau in ihnen steht oft, **warum** ein Event nicht gefeuert hat.

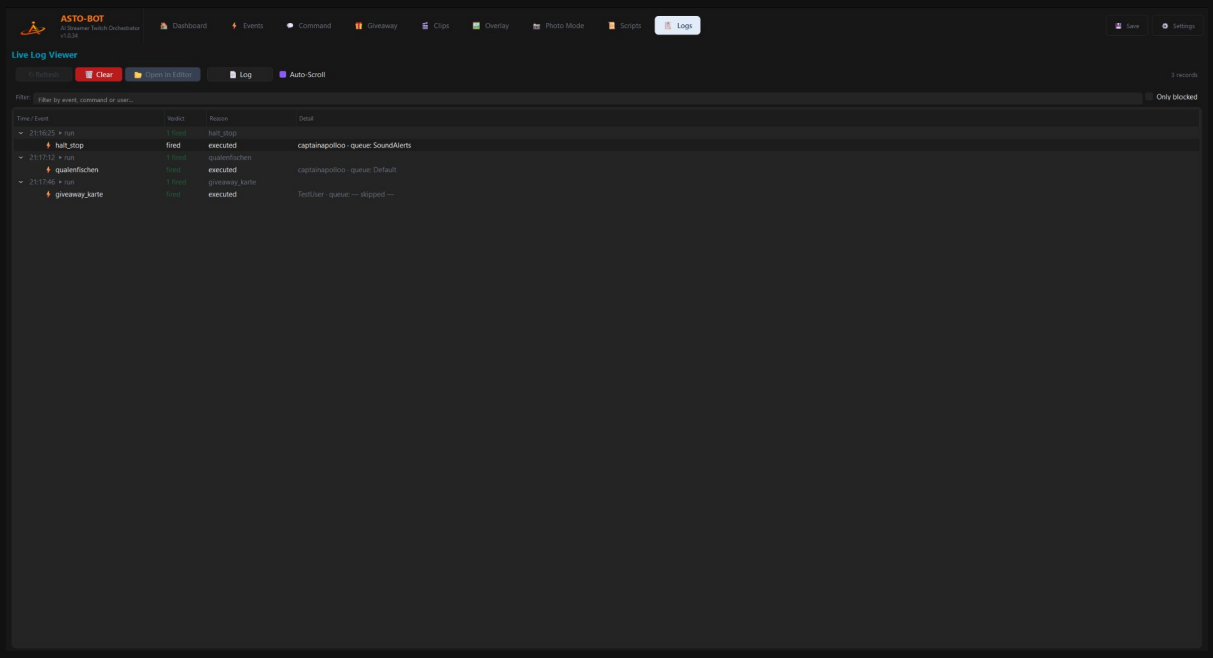
Und genau dafür ist dieser Bereich da: Wenn etwas nicht funktioniert, ist das Log die erste Adresse. Tipp den Namen des Events in den Filter und sieh nach, was ASTO-BOT in dem Moment gemacht hat. Meistens steht die Antwort dort im Klartext.

> Wenn du im Discord um Hilfe bittest, hängt am besten gleich den passenden Ausschnitt aus dem Log an. Das erspart eine Menge Hin und Her.

12.2 Der Event Inspector

Das Log beantwortet die Frage **„was ist passiert?“**. Der Event Inspector beantwortet die andere, viel häufigere: **„warum ist nichts passiert?“** — der Command wurde getippt, aber

der Bot schweigt; das Event ist eingerichtet, aber es geschieht nichts. Genau diese Fälle stehen hier im Klartext.



Der Event Inspector — jede Zeile zeigt, was ausgelöst wurde und was damit geschah

Der Knopf 🔍 **Inspector** oben schaltet um; aus ihm wird dann 📄 **Log** für den Weg zurück. **Clear** leert immer nur die Ansicht, die gerade offen ist — nie beide.

Wie der Baum zu lesen ist

Jede **oberste Zeile** ist ein Auslöser: eine Chat-Nachricht, ein Trigger oder ein Event-Lauf. Darunter hängen die **Kandidaten** — alles, was darauf reagiert hat oder reagiert hätte. Bei IF/ELSE lässt sich eine Ebene tiefer aufklappen: Dort stehen die einzelnen **Bedingungen**.

- 💬 **chat** — eine Chat-Nachricht kam herein. Daneben steht, wer sie geschrieben hat und was drin stand.
- ▶ **run** — ein Event wurde ausgeführt.
- ❯ **Command**, 🔤 **Word-Trigger**, ⚡ **Event** — die Art des Kandidaten.
- 🔀 **IF/ELSE** und 🎲 **Chance** — Bedingungen und Zufallsblöcke aus der Action Sequence.

Die Spalte **Detail** zeigt je nach Zeile den auslösenden Zuschauer und die **Queue**, in der das Event gelaufen ist. Steht dort *— skipped —*, hat das Event die Warteschlange übersprungen (Abschnitt 5.5).

Die Spalte Verdict

- Grünes **fired** — hat ausgelöst.
- Rotes **blocked** — wurde gestoppt. Warum, steht in der Spalte **Reason**.
- Oranges **waiting** — weder ausgeführt noch verworfen: Das Event liegt in einer **pausierten** Queue und wartet dort, bis du sie wieder einschaltest.

Die häufigsten Gründe

- **disabled** — Event oder Command ist ausgeschaltet. Mit Abstand der häufigste Fall.
- **permission** — die Rolle reicht nicht. Daneben steht, was verlangt war, zum Beispiel *needs Follower*.
- **cooldown** und **user cooldown** — die Sperre läuft noch; die Restzeit steht daneben.
- **only-list** — der Command ist auf bestimmte Namen beschränkt.

- **event cooldown** — der Cooldown des Events selbst.
- **ignore list** — dieser Zuschauer steht auf der Ignore-Liste des Events.
- **shared-chat scope** — der Trigger ist auf den eigenen Kanal begrenzt, kam aber aus einem geteilten Chat.
- **queue paused** — die Warteschlange ist angehalten.
- **condition false** — eine Bedingung im IF/ELSE war nicht erfüllt. Klapp die Zeile auf: Darunter steht Feld für Feld, welcher **Ist-Wert** mit welchem **Soll-Wert** verglichen wurde.

> 💡 Der Filter unten durchsucht Event-Namen, Commands und Zuschauernamen gleichzeitig. Und der Schalter **Only blocked** blendet alles aus, was funktioniert hat — dann bleibt genau das übrig, wonach du suchst.

Was der Inspector nicht zeigt

Aufgezeichnet wird nur, was **überhaupt in Frage kam**. Schreibt jemand einen Satz, der auf keinen einzigen Command passt, taucht das hier nicht auf — sonst stünden bei jeder Nachricht Dutzende Zeilen „passte nicht“. Was du siehst, sind die **Beinahe-Treffer**: Fälle, in denen der Auslöser gepasst hat und erst danach etwas dazwischenkam. Und natürlich die Erfolge.

> Der Inspector hält die letzten **200** Einträge im Arbeitsspeicher und schreibt **nichts** auf die Festplatte — nach einem Neustart von ASTO-BOT ist die Liste leer. Für das dauerhafte Protokoll ist der Live Log Viewer zuständig.

13. Wenn etwas nicht funktioniert

Fast alle Probleme, die im Discord landen, sind eine Handvoll immer gleicher Stolpersteine. Hier stehen sie — mit der Lösung daneben.

13.1 Chat, KI und TTS

SpeakerBot liest die falsche Nachricht vor

TTS liest immer die **letzte Chat-Aktion vor sich**. Stehen zwei Chat-Aktionen hintereinander und danach ein TTS, wird nur die zweite vorgelesen. Bau es abwechselnd: Chat → TTS → Chat → TTS.

Chat (AI) schreibt nichts in den Chat

Fehler der KI landen **nicht im Chat** — deine Zuschauer sollen keine Fehlermeldung sehen —, sondern im **Log**, als Zeile, die mit `chat_ai:` beginnt. Dort steht der Grund:

- **No prompt configured** oder **Prompt … is not set up yet** — der **Prompt des Events ist leer**. **Chat (AI)** hat kein eigenes Textfeld; der Text kommt vollständig aus dem Prompt im Trigger-Bereich.
- **FEHLER: Kein API-Key konfiguriert** oder eine Fehlermeldung von OpenAI bzw. Gemini — dann sind **alle** hinterlegten Schlüssel fehlgeschlagen oder es ist keiner eingetragen. Prüfe sie unter **Settings** → **AI** (Feld **API Key** und optional **Backup API Key**). Ist ein zweiter Schlüssel hinterlegt, springt er automatisch ein.

13.2 Events und Commands

Der Chat-Befehl tut nichts

Drei Ursachen, in dieser Reihenfolge prüfen: Der Command ist **ausgeschaltet**. Es hängt **kein Event** daran (Trigger *Chat Command*). Oder ein **Cooldown** läuft noch — G-CD gilt für alle, U-CD pro Zuschauer.

Das Event feuert gar nicht — und ich weiß nicht, warum

Ins **Log** sehen (Kapitel 12), **Level: All**, den Event-Namen in den Filter tippen. Dort steht im Klartext, was ASTO-BOT in dem Moment gemacht hat.

Nach einer Scope-Änderung geht nichts mehr

Kommen neue Berechtigungen dazu, verlangt Twitch eine **komplette Neuanmeldung beider Konten** — Streamer und Bot.

13.3 Rewards und Giveaways

Ich kann einen Reward nicht bearbeiten oder löschen

Dann hat ihn nicht ASTO-BOT erstellt, sondern Twitch selbst oder ein anderes Tool. Solche Rewards lassen sich nur **verknüpfen**. Im Auswahlfenster tragen sie ein **Schloss**. Das ist eine Regel von Twitch.

Meine Änderungen am Reward sind weg

Save Reward drücken, **bevor** du das Fenster schließt.

Refund Channel Points funktioniert nicht

Der Reward muss **Skip Queue ausgeschaltet** haben. Eine bereits bestätigte Einlösung lässt sich nicht mehr erstatten.

Beim Giveaway gewinnt immer derselbe

Dann steht es auf **Currency** mit **Winner: Highest wins** — da gewinnt **immer der höchste Einsatz**, ohne Zufall. Stell **Winner** auf **Weighted**, wenn auch kleinere Einsätze eine Chance haben sollen, oder auf **Ticket** für eine echte Verlosung mit gleicher Chance für alle.

Mein Gewinner-Event ruft den falschen Namen aus

Der Gewinner steht in **`%GiveawayWinner%`** — nicht in `%UserName%`.

13.4 OBS, Clips und Overlays

Der Clip wird nicht abgespielt

Prüfe die Quelle unter **Clip Playback**: Sind **beide** Quellen gesetzt, gewinnt die **Media Source**. Stell die andere auf **– none –**. Und geh die Checkliste für die OBS-Quelle durch (Kapitel 8.3) — **Local file** muss **aus** sein.

OBS Save Replay Buffer speichert nichts

Der Replay Buffer muss in OBS **aktiviert und gestartet** sein. Aktiviert allein reicht nicht.

Mein Overlay-Element verschiebt sich bei jedem Abspielen

Der **Pin** am ersten Step ist **orange** — die Position ist nicht gespeichert. Klick ihn an, bis er **grün** ist.

Im Overlay erscheint immer nur das Standard-Profilbild

Im Event muss **Get Target Info vor Play Overlay** stehen. Nur dann wird das Twitch-Bild des Zuschauers geholt und ins Overlay geschickt.

Nach dem Event steht mein Bildschirm auf dem Kopf

Monitor Flip gehört mindestens **zweimal** ins Event: einmal zum Drehen, dann ein **Delay**, und am Ende noch einmal mit `0` zum Zurückdrehen.

Rotate OBS Source sieht kaputt aus

OBS dreht einzelne Quellen nicht sauber. Nimm **Rotate OBS Scene**, das funktioniert zuverlässig.

Der Stream ruckelt oder stockt

Blende die Karte **Stream Health** ein (Kapitel 3.11). Sie zeigt, ob die Leitung, der Encoder oder das Rendering schuld ist, nennt den wahrscheinlichen Grund und hält jedes Problem mit Uhrzeit fest. Über **OBS Setup Info** siehst du dazu, ob deine OBS-Einstellungen zu Twitch passen.

13.5 Musik und Skripte

ASTO-BOT findet meinen Musik-Player nicht

Der Player muss in dem Moment **tatsächlich Ton abspielen**. Ist er nur geöffnet oder pausiert, meldet Windows keine Informationen — und ASTO-BOT sieht nichts. Das gilt für den Music-Trigger, für **Now Playing** und für **Detect** bei Media Control.

Ich habe ein Skript versehentlich gelöscht

Der **Papierkorb** neben dem Ordner-Knopf im Scripts-Bereich stellt gelöschte Skripte wieder her.

Mein Skript läuft, macht aber etwas anderes als gedacht

Debug-Modus starten und mit **Step** durchgehen. Rechts siehst du live, was in den Variablen steht — meistens ist nach drei Schritten klar, wo es klemmt.

13.6 Und wenn nichts hilft?

Dann komm in den **Discord** — https://discord.gg/n2nstVwspk. Häng am besten gleich den passenden Ausschnitt aus dem **Log** an (Kapitel 12) und schreib dazu, was du erwartet hast und was stattdessen passiert ist. Das erspart eine Menge Hin und Her.

ASTO-BOT — AI Streamer Twitch Orchestrator

Version 1.1.22 | Entwickelt von CaptainApolloo | 2026

Join Apolloo's Discord Server für Support und Updates